



Institut Supérieur des Études Technologiques de Mahdia

Département : Technologies de l'Informatique

Développement des Systèmes Informatiques — DSI3.2

Rapport de Projet de Fin d'Études

**Koda — Robot compagnon intelligent et émotionnel connecté,
piloté par une application cross-platform**

**Système Embarqué • Intelligence Artificielle • Application Mobile
Cross-Plateforme**

Réalisé par :

Oussema KARIA

Salahedin JENDOUBI

Encadré par :

M.Wahid BANOUR — (Encadrant)

M.Hamza SASSI (Encadrant Entreprise)

Organisme d'accueil : —

Période : Février 2026 – Juin 2026

Résumé

Ce rapport présente KODA, un système embarqué intelligent conçu pour être un assistant de vie et un ami virtuel doté de sa propre personnalité et nature. KODA est capable de répondre de manière réfléchie à n'importe quelle personne, de raisonner comme un être humain et de reconnaître les individus grâce à ses modes de sécurité et de configuration.

L'architecture du système repose sur plusieurs composants complémentaires : un système embarqué Raspberry Pi constituant le cœur physique du robot ; un moteur de workflows n8n formant le cerveau de KODA, responsable de la préparation et de la réflexion des réponses ; une couche de services Python agissant comme distributeur universel, assurant la communication entre n8n, la base de données, l'application mobile et le Raspberry Pi tout en exposant les endpoints de tout l'écosystème ; une application mobile cross-plateforme pour la configuration et le suivi de vie du robot ; et une base de données persistante servant de mémoire à KODA.

Mots-clés : Système Embarqué, Assistant Virtuel, Raspberry Pi, n8n, Python, Cross-Plateforme, Intelligence Artificielle, Assistant de Vie, KODA.

DÉDICACE

Je dédie ce travail à ma famille, pilier silencieux de ma force et source inépuisable de soutien.

*Merci pour vos sacrifices immenses et pour avoir cru en moi tout au long de mon parcours
académique.*

*À mes parents, pour leur amour inconditionnel, leurs encouragements et leurs prières qui ont été la
lumière guidant chacun de mes pas. Ce succès est le vôtre autant que le mien.*

*À mes amis et collègues, pour leur soutien moral et pour avoir partagé avec moi cette aventure
inoubliable.*

*À mon binôme Salahedin, pour sa persévérance, sa créativité et sa complicité tout au long de ce
projet.*

*À tous ceux qui ont contribué, même par un simple mot d'encouragement, à l'accomplissement de
ce travail.*

Oussema KARIA

DÉDICACE

À ma famille,
pour son soutien inconditionnel et ses sacrifices immenses
tout au long de mon parcours académique.

À mon binôme Oussema, pour son enthousiasme, sa rigueur
et sa précieuse collaboration dans la réalisation de ce projet.

À mes encadrants,
pour leur guidance, leur patience et leur expertise technique.

Salahedin JENDOUBI

Remerciements

À notre encadrant académique, M. Wahid Banour,

nous adressons nos remerciements les plus sincères pour la qualité de l'accompagnement pédagogique, la disponibilité dont il a fait preuve à chaque étape, ainsi que pour ses conseils exigeants qui ont permis de structurer notre démarche et de renforcer la rigueur scientifique et technique de ce travail.

À notre encadrant en entreprise, M. Hamza Sassi,

nous exprimons toute notre gratitude pour l'accueil au sein de l'organisme d'accueil, pour le temps accordé aux échanges techniques et pour les orientations concrètes qui ont éclairé nos choix d'architecture et d'intégration tout au long du projet.

À la direction et à l'ensemble du corps enseignant de l'ISSET de Mahdia,

et en particulier au département *Développement des systèmes informatiques* (DSI), nous témoignons notre reconnaissance pour la formation dispensée, pour l'exigence professionnelle transmise et pour l'environnement d'apprentissage qui a rendu possible la réalisation de ce projet de fin d'études.

À nos proches, amis et camarades,

nous disons merci pour la patience, les encouragements et la confiance manifestés durant cette période intensive ; leur soutien moral a compté autant que le travail collectif mené au sein du binôme.

Nous saluons enfin toute personne ayant contribué, directement ou indirectement, à l'aboutissement de ce rapport.

Oussema Karia

Salahedin Jendoubi

SOMMAIRE

Introduction Générale	1
I Cadre Général du Projet et Présentation de l'Organisme	2
I.1 Introduction du chapitre	3
I.2 Contexte Général du Projet	3
I.3 Présentation de l'Organisme d'Accueil (MicroEdition)	3
I.3.1 Profil de l'Entreprise	3
I.3.2 Services	4
I.3.3 Recherche & Développement (R&D)	4
I.3.4 Structure Organisationnelle	4
I.4 Problématique	5
I.5 Analyse des Solutions Existantes	5
I.5.1 Étude de l'existant	6
I.5.2 Synthèse et Critique : Le "Manque d'Humanité"	6
I.5.3 La Différenciation de Koda	7
I.6 Solution Proposée	7
I.7 Méthodologie du Projet	8
I.7.1 Modèle en spirale	8
I.8 Conclusion	8
II Préparation du Projet	10
II.1 Introduction du chapitre	11
II.2 Spécification des besoins	11
II.2.1 Besoins fonctionnels	11
II.2.2 Besoins non fonctionnels	13
II.3 Architecture générale et acteurs	14
II.4 Méthodologie en spirale et planification	16
II.5 Environnements de développement	18
II.5.1 Matériel et logiciel	18

II.5.2	Langages et frameworks utilisés	19
II.6	Conclusion du chapitre	20
III	Cycle 1 : ANALYSEUR ET RÉFLEXION	21
III.1	Introduction	22
III.2	Fondements théoriques	22
III.2.1	Théorie du double traitement	22
III.2.2	De la perception à la formulation	22
III.2.3	Conclusion — fondements théoriques	23
III.3	Définitions fondamentales	23
III.3.1	Grand modèle de langage (LLM)	23
III.3.2	Conclusion — définitions fondamentales	24
III.4	Modélisation globale du Cycle 1	24
III.4.1	Vue d'ensemble des trois flux n8n	25
III.4.2	Diagramme de cas d'utilisation général	26
III.4.3	Schéma de classes général	27
III.5	Fonctionnalités Techniques — les trois flux	27
III.5.1	Premier Flux — Webhook 1 : Analyse et Réponse	27
III.5.1.1	Principe général et découpage en six parties	27
III.5.1.2	Diagramme de séquence du Webhook 1	27
III.5.1.3	Format de la Requête Entrante	28
III.5.1.4	Les six parties du Webhook 1	28
III.5.1.4.1	Étape 1 — Entrée fonctionnelle	28
III.5.1.4.1.1	Branche musique — SerpAPI (moteur YouTube)	31
III.5.1.4.2	Étape 2 — Confrontation à l'historique	32
III.5.1.4.3	Étape 3 — Classification sémantique	32
III.5.1.4.3.1	Basic LLM Chain plutôt qu'AI Agent	34
III.5.1.4.3.2	OpenRouter (Gemini flash)	34
III.5.1.4.4	Étape 4 — Gestion utilisateur (reconnaissance et enre-	
	gistrement)	35
III.5.1.4.4.1	Scénario 1 — presentation (nouvel utilisateur)	36
III.5.1.4.4.2	Scénario 2 — unkonu (invitation à se présenter)	36
III.5.1.4.4.3	Scénario 3 — personne connue	39
III.5.1.4.5	Étape 5 — Réponse contextualisée	39

III.5.1.4.6	Étape 6 — Notes et mémoire persistante	42
III.5.1.4.6.1	Partie 1 — Extraction des notes (conditionnelle).	42
III.5.1.4.6.2	Partie 2 — Persistance et clôture du webhook.	44
III.5.1.5	Conclusion du Webhook 1	46
III.5.2	Deuxième Flux — Webhook 2 : Filtre et Synthèse Journalière	47
III.5.2.1	Enchaînement fonctionnel	47
III.5.2.2	Contenu de la fiche journalière	48
III.5.2.3	Persistance et impact système	49
III.5.2.4	Conclusion du Webhook 2	50
III.5.3	Troisième Flux — Webhook 3 : Spontanéité et Interaction Proactive	50
III.5.3.1	Principe : la spontanéité comme dimension humaine	50
III.5.3.2	Diagrammes du Webhook 3	51
III.5.3.3	Déclenchement et entrées	52
III.5.3.4	Préparation des données (nœud Code11)	53
III.5.3.5	Génération de l'introduction et sortie JSON	53
III.5.3.6	Conclusion du Webhook 3	54
III.6	Synthèse cognitive du Cycle 1	54
III.7	Conclusion	55
IV	Cycle 2 : Distributeur de Services	56
IV.1	Introduction	57
IV.1.0.0.1	Rôle global du Distributeur.	57
IV.1.0.0.2	Objectifs et résultats attendus.	57
IV.1.0.0.3	Organisation du chapitre.	58
IV.2	Fondements théoriques et positionnement du Distributeur	58
IV.2.1	Le principe du point de liaison unique	58
IV.2.2	Services backend et fonctionnalité globale du système	59
IV.2.3	Communication vocale et applicative : deux faces, un backend	59
IV.3	Modélisation et place dans l'architecture	59
IV.3.1	Diagramme de cas d'utilisation général	60
IV.3.2	Architecture globale : pivot de flexibilité et de disponibilité	61
IV.4	Architecture interne du backend	63
IV.4.1	Choix du langage : Python	63
IV.4.2	Clean Architecture : principes et application	64

IV.4.3	Structure des répertoires du projet	65
IV.5	Services exposés et flux de communication	66
IV.5.1	Service dédié vers le système embarqué (robot)	66
IV.5.1.1	Cas d'utilisation robot ↔ backend	67
IV.5.1.2	Action 1 — Demander une réponse vocale (Flux 1)	67
IV.5.1.2.1	Lien avec le Cycle 1.	69
IV.5.1.3	Action 2 — Demander une introduction de parole (Flux 3)	69
IV.5.1.3.1	Lien avec le Cycle 1.	71
IV.5.1.4	Canal WebSocket (temps réel)	71
IV.5.1.5	Chaîne vocale complète (Azure STT / TTS)	72
IV.5.1.6	Reconnaissance des personnes	74
IV.5.1.6.1	Principe et modèle de données.	74
IV.5.1.6.2	Pipeline d'identification.	74
IV.5.1.6.3	Cas particuliers.	76
IV.5.1.6.4	Contraintes et intégration.	77
IV.5.2	Service dédié vers l'application mobile (KodaMate)	77
IV.5.2.1	Contexte architectural	77
IV.5.2.2	Principe de communication	77
IV.5.2.3	Flux fonctionnels principaux	78
IV.5.2.3.1	Authentification.	78
IV.5.2.3.2	Configuration et contrôle du robot.	78
IV.5.2.3.3	Chat intelligent (délégation n8n).	79
IV.5.2.3.4	Historique des conversations.	79
IV.5.2.3.5	Données complémentaires (hors backend).	80
IV.5.3	Service dédié vers la base de données (PostgreSQL / Supabase)	81
IV.5.3.1	Modèle relationnel et rôle des tables	81
IV.5.4	Service dédié vers le workflow n8n (Cycle 1)	82
IV.6	Catalogue des endpoints REST du Distributeur	84
IV.6.1	Infrastructure et documentation (hors /api)	84
IV.6.2	Communication avec PostgreSQL (persistance Supabase)	85
IV.6.2.1	Santé de la base	85
IV.6.2.2	Entités persistées (CRUD)	85
IV.6.3	Communication avec le système embarqué (Raspberry Pi)	85
IV.6.4	Communication avec le workflow n8n (Cycle 1)	85

IV.6.5	Communication avec l'application mobile (KodaMate)	85
IV.6.5.1	Endpoints consommés à l'écran	88
IV.6.5.2	Endpoints codés mais non branchés à l'interface	88
IV.6.5.3	API externe et services locaux	88
IV.6.5.4	Cartographie écran → endpoints	88
IV.7	Problèmes rencontrés et solutions apportées	88
IV.7.1	Conflit d'adressage Raspberry Pi / backend	89
IV.7.2	Problème CORS entre backend et application mobile	90
IV.8	Perspectives et évolutions futures	90
IV.9	Conclusion	91
V	Cycle 3 : Système Embarqué — Partie Matérielle	92
V.1	Introduction	93
V.2	Architecture Matérielle Globale	94
V.3	Conception mécanique et châssis 3D (Blender)	95
V.4	Description Détaillée des Composants	96
V.4.1	Unité Centrale : Raspberry Pi 4 Model B	96
V.4.2	Unité de Commande Moteur : Arduino Uno + Shield L293D	97
V.4.3	Système d'Actionneurs	99
V.4.3.1	Moteurs DC — Mobilité	99
V.4.3.2	Servomoteurs — Articulations	99
V.4.4	Système Audio	100
V.4.4.1	Entrée Audio : ReSpeaker 4-Mic Array USB	100
V.4.4.2	Sortie Audio : double chaîne (jack filaire et Bluetooth)	101
V.4.5	Système de Vision : Caméra Raspberry Pi	102
V.4.6	Interface d'Expression Faciale : Nextion NX4832T035_011	103
V.4.7	Système d'Alimentation	104
V.5	Schéma de Câblage Global	105
V.6	Assemblage et points d'attention	106
V.7	Validation matérielle — tests par composant	106
V.7.1	Commandes de test par composant	106
V.7.1.0.1	Power Bank + Raspberry Pi.	107
V.7.1.0.2	ReSpeaker 4-Mic Array USB.	107
V.7.1.0.3	Sortie jack + amplificateur PAM8403.	108

V.7.1.0.4	Sortie Bluetooth.	108
V.7.1.0.5	Caméra Raspberry Pi (CSI).	108
V.7.1.0.6	Écran Nextion (UART).	109
V.7.1.0.7	Arduino + Shield L293D (commande série USB).	109
V.7.1.0.8	Moteurs DC (M1, M3).	110
V.7.1.0.9	Servomoteurs (S1, S2, S3).	110
V.7.1.0.10	Alimentation 3 piles 3,7 V (Vin shield).	110
V.7.2	Test d'intégration matérielle	111
V.8	Conclusion	111
VI	Cycle 4 : Système Embarqué — Partie Logicielle Raspberry Pi	112
VI.1	Introduction	113
VI.2	Architecture logicielle globale	113
VI.3	Architecture du code	114
VI.4	Configuration et lancement	115
VI.5	Démarrage et diagnostic matériel	116
VI.6	Boucle principale et machine d'états	117
VI.7	Pipeline vocal : mot de réveil, écoute et réponse	119
VI.8	Gestion des actions retournées par le backend	119
VI.9	Affichage Nextion et expressions	119
VI.10	Mouvement, Arduino USB et orientation DOA	122
VI.11	Robustesse et arrêt propre	122
VI.12	Tests et validation logicielle	123
VI.13	Limites et perspectives	124
VI.14	Conclusion	125
VII	Cycle 5 : Application Mobile Cross-Plateforme KodaMate	126
VII.1	Introduction : pourquoi une application compagnon ?	127
VII.2	Place de KodaMate dans l'écosystème	128
VII.3	Parcours et principes d'expérience	129
VII.4	Les interfaces utilisateur	131
VII.4.1	Interface de chargement	131
VII.4.2	Interface de connexion	132
VII.4.3	Interface de configuration du robot	134

VII.4.4 Interface d'accueil (Home)	136
VII.4.5 Interface de dialogue (Chat)	138
VII.4.6 Interface d'historique	139
VII.4.7 Interface de paramètres	141
VII.5 Organisation technique du projet	142
VII.6 Fiabilité et session	143
VII.7 Tests et validation	143
VII.8 Limites et perspectives	144
VII.9 Conclusion	145
VIII Conclusion générale et synthèse du projet	146
VIII.1 Du constat initial à la solution réalisée	147
VIII.2 Vue d'ensemble : les cinq cycles et leurs rôles	147
VIII.3 Les cinq piliers du système KODA	148
VIII.3.1 Le cerveau conversationnel (Cycle 1)	148
VIII.3.2 Le système nerveux central (Cycle 2)	149
VIII.3.3 Le corps du compagnon (Cycles 3 et 4)	149
VIII.3.4 La relation à distance (Cycle 5)	149
VIII.4 Comment lire l'ensemble : un scénario type	149
VIII.5 Apports du projet	150
VIII.6 Limites et enseignements	151
VIII.7 Perspectives	151
VIII.8 Conclusion finale	151

LISTE DES FIGURES

I.1	Modèle en spirale — enchaînement des six cycles de développement KODA	9
II.1	Synthèse des besoins fonctionnels KODA	13
II.2	Architecture générale du système KODA	15
II.3	Acteurs principaux et contexte d'intégration du système KODA	15
II.4	Modèle en spirale et six cycles thématiques KODA	16
II.5	Traçabilité besoins → cycles de réalisation	17
II.6	Planification — dépendances entre les six cycles	18
II.7	Environnement matériel de test	18
II.8	Environnement logiciel — stack technique KODA	19
III.1	De la question à la réponse : réflexion rapide ou réflexion lente	23
III.2	Architecture cognitive de KODA inspirée de la théorie du double traitement	24
III.3	Cycle 1 — vue d'ensemble des trois workflows n8n	25
III.4	Cycle 1 — diagramme de cas d'utilisation général	26
III.5	Cycle 1 — schéma de classes général (persistance)	27
III.6	Webhook 1 — diagramme de séquence (six parties du pipeline)	28
III.7	Webhook 1 — entrée fonctionnelle (nœud <i>Test, Switch</i>)	29
III.9	Webhook 1 — cas d'utilisation de l'entrée fonctionnelle	29
III.8	Webhook 1 — séquences entrée fonctionnelle (partie 1 : réception et intention ; partie 2 : routage <i>Switch</i>)	30
III.10	Branche musique — credential SerpAPI et nœud <i>Search Music</i>	31
III.11	Exemple <code>search_query, youtube_url</code>	31
III.12	Confrontation à l'historique — workflow n8n	32
III.13	Confrontation à l'historique — diagramme de séquence	32
III.14	Classification sémantique — chaîne n8n	33
III.15	Vue synthétique OpenRouter / Gemini	33
III.16	Classification sémantique — cas d'utilisation	34
III.17	Credential OpenRouter et modèle Gemini	35
III.18	Suivi consommation API workflow n8n	35

III.19	Gestion utilisateur — <i>Switch1</i>	36
III.20	Séquence 1 — presentation → API + PostgreSQL → étape 5	37
III.21	Séquence 2 — unkonu → accueil LLM	37
III.22	Cas unkonu — entrée <i>Switch1</i> et sortie LLM	38
III.23	Séquence 3 — personne connue → étape 5	39
III.24	Gestion utilisateur — séquence complète détaillée	39
III.25	Réponse contextualisée — vue des deux branches	40
III.26	Séquence simplifiée étape 5	41
III.27	Branche mémorielle — récupération profil et génération	41
III.28	Branche générale — contexte conversationnel	41
III.29	Diagramme de séquence détaillé — étape 5	42
III.30	Extraction des notes — test <code>questiontype</code> et structuration LLM	43
III.31	Sous-workflow <code>n8n</code> — branche <code>note</code> (LLM Chain 2)	43
III.32	Entrée <code>n8n</code> — <code>question</code> , <code>output</code> , <code>questiontype</code> = <code>note</code>	44
III.33	Sortie LLM — <code>date_evenement</code> et <code>evenement</code>	44
III.34	Persistance et clôture — écritures PostgreSQL et réponse webhook	45
III.35	Schéma de classes — tables <code>conversations</code> , <code>historique</code> , <code>notes</code>	46
III.36	Webhook 2 — workflow <code>n8n</code> (collecte, synthèse, insertion)	47
III.37	Webhook 2 — séquence de synthèse journalière	48
III.38	Exemple de sortie — fiche journalière (quatre dimensions)	49
III.39	Schéma utilisateur → accompagnement	50
III.40	Webhook 3 — workflow <code>n8n</code>	51
III.41	Webhook 3 — diagramme de séquence	52
III.42	Webhook 3 — entrée du nœud <i>Code11</i>	53
III.43	Webhook 3 — sortie JSON (<code>parole_du_robot</code> , <code>emotion_visuelle</code>)	53
III.44	Architecture cognitive globale de KODA — interaction entre les trois webhooks	54
IV.1	Cycle 2 — diagramme de cas d'utilisation général du Distributeur de services	60
IV.2	Cycle 2 — schéma de classes général du backend (persistance PostgreSQL)	61
IV.3	Architecture multicouche de KODA — place du Distributeur de services (Cycle 2)	62
IV.4	Cycle 2 — Distributeur pivot central (composants et flux)	63
IV.5	Architecture des dossiers du projet backend Distributeur (<code>src/</code>)	65
IV.6	Exécution du programme backend (Distributeur)	66
IV.7	Cycle 2 — cas d'utilisation du système embarqué vis-à-vis du Distributeur de services	67

IV.8	Action 1 — vue compacte du pipeline vocal (Flux 1)	68
IV.9	Action 1 (a) — entrée : capture, identification faciale et transcription STT	69
IV.10	Action 1 (b) — sortie : Flux 1, synthèse TTS et restitution au robot	69
IV.11	Action 2 — vue compacte de l'introduction de parole (Flux 3)	70
IV.12	Action 2 (a) — entrée : détection de présence et requête au Distributeur	71
IV.13	Action 2 (b) — sortie : Flux 3, synthèse TTS et restitution au robot	71
IV.14	Cycle 2 — échanges WebSocket robot ↔ Distributeur (temps réel)	72
IV.15	Configuration Azure Speech (STT/TTS)	73
IV.16	Test Text-to-Speech	73
IV.17	Test Speech-to-Text	73
IV.18	Pipeline de reconnaissance faciale — de la capture caméra à l'identité utilisateur	74
IV.19	Diagramme de séquence — identification faciale (robot, Distributeur, base de référence)	75
IV.20	Exemple d'images de référence en mémoire (profils utilisateurs)	76
IV.21	Authentification mobile — séquence POST /api/auth/login	78
IV.22	Chat mobile — séquence POST /api/trigger-n8n	79
IV.23	Paramètres, power et historique — séquences REST KodaMate	80
IV.24	Exécution de quelques endpoints (tests manuels)	81
IV.25	Interface Supabase — administration PostgreSQL	81
IV.26	Délégation au workflow n8n — séquence robot ou mobile → Distributeur → n8n	83
IV.27	Communication avec n8n — exemple de payload en entrée de webhook	83
IV.28	Conflit d'adressage et CORS — topologie réseau et solutions	90
V.1	Prototype physique KODA après impression 3D et montage	94
V.2	Conception 3D du châssis KODA sous Blender (vue éclatée des pièces)	96
V.3	Raspberry Pi 4 Model B utilisé comme unité centrale de KODA	97
V.4	Carte Arduino Uno (microcontrôleur ATmega328P à 16 MHz)	98
V.5	Shield L293D empilé sur l'Arduino Uno (commande des moteurs DC et servomoteurs)	98
V.6	Moteur DC utilisé pour la mobilité du robot (bornes M1 et M3 du shield)	99
V.7	Principe de traction différentielle — moteurs M1 et M3	99
V.8	Servomoteur utilisé pour les articulations (bras S1/S2 et rotation Nextion S3)	100
V.9	ReSpeaker 4-Mic Array USB : quatre microphones MEMS pour capture omnidirectionnelle et détection de direction (DOA)	101

V.10 Enchaînement conceptuel : DOA (micro) → rotation du robot (M1, M3) → caméra (torse)	101
V.11 Amplificateur audio PAM8403 (chaîne filaire entre la sortie jack du Pi et les haut-parleurs)	102
V.12 Paire de haut-parleurs gauche/droite alimentés par le PAM8403	102
V.13 Caméra Raspberry Pi connectée au port CSI (acquisition d'images pour l'identification visuelle)	103
V.14 Écran Nextion NX4832T035_011 (3,5") affichant le visage animé de KODA	103
V.15 Power Bank 5 V / 3 A alimentant le Raspberry Pi (chaîne logique)	104
V.16 3 piles lithium 3,7 V en série (11,1 V total) alimentant le Shield L293D et les moteurs	105
V.17 Schéma de montage matériel de KODA — style Fritzing (Cycle 3)	105
VI.1 Cycle 4 — architecture logique du logiciel Raspberry Pi	114
VI.2 Cycle 4 — le Raspberry Pi comme corps opérationnel de KODA	114
VI.3 Cycle 4 — architecture du code codeRaspberry	115
VI.4 Cycle 4 — classes principales, services et adaptateurs	115
VI.5 Cycle 4 — cas d'utilisation du logiciel embarqué Raspberry Pi	116
VI.6 Cycle 4 — séquence de démarrage et diagnostic matériel	117
VI.7 Cycle 4 — machine d'états logicielle du robot	118
VI.8 Cycle 4 — pipeline audio logiciel du Raspberry Pi	120
VI.9 Cycle 4 — séquence mot de réveil, écoute et réponse	121
VI.10 Cycle 4 — dispatch des actions retournées par le backend	121
VII.1 Cycle 5 — cas d'utilisation de KodaMate (vue utilisateur)	128
VII.2 Cycle 5 — place de KodaMate entre l'utilisateur, le Distributeur, la cognition n8n et le corps du robot	129
VII.3 Cycle 5 — parcours utilisateur de KodaMate	130
VII.4 Cycle 5 — séquence de démarrage et orientation de l'utilisateur	132
VII.5 Cycle 5 — connexion et test du serveur Distributeur	133
VII.6 Cycle 5 — séquence d'authentification utilisateur	134
VII.7 Cycle 5 — configuration initiale : identité du robot (gauche) et voix, langue et état d'allumage (droite)	135
VII.8 Cycle 5 — séquence de lecture et mise à jour de la configuration	136
VII.9 Cycle 5 — accueil : veille et environnement (gauche), réseau déconnecté (centre), robot actif (droite)	137

VII.1	Cycle 5 — séquence de supervision : état KODA, météo et marche/arrêt	137
VII.1	Cycle 5 — dialogue texte avec KODA	138
VII.1	Cycle 5 — séquence d'envoi d'une question et affichage de la réponse	139
VII.1	Cycle 5 — historique des conversations avec filtre et recherche	140
VII.1	Cycle 5 — séquence de consultation de l'historique	140
VII.1	Cycle 5 — paramètres : profil robot (gauche), Wi-Fi et préférences (centre), informations et déconnexion (droite)	141
VII.1	Cycle 5 — séquence de sélection d'un réseau Wi-Fi (simulation locale)	142
VIII.	Vue synthétique de l'écosystème KODA au terme du projet	148

LISTE DES TABLEAUX

II.1	Catalogue des besoins fonctionnels KODA	12
II.2	Catalogue des besoins non fonctionnels KODA	14
II.3	Acteurs et rôles dans l'écosystème KODA	14
II.4	Cycles, chapitres du rapport et livrables	16
III.1	Priorité des intentions — nœud <i>Test</i>	31
III.2	Catégories de la classification sémantique	33
III.3	Les trois scénarios du <i>Switch1</i>	36
III.4	Les deux branches de l'étape 5	40
III.5	Opérations de persistance — étape 6 (partie 2)	45
III.6	Phases du Webhook 2	47
III.7	Quatre dimensions de la synthèse Webhook 2	48
III.8	Correspondance entre mécanismes cognitifs humains et implémentations de KODA	54
IV.1	Comparaison des technologies backend envisagées	64
IV.2	Répartition des 65 endpoints par axe de communication	84
IV.3	Endpoints racine — exploitation et documentation	84
IV.4	Endpoint de diagnostic base de données	85
IV.5	Endpoints CRUD — communication avec PostgreSQL par entité	86
IV.6	Endpoints orientés robot — audio, vision et mot-clé	87
IV.7	Endpoints de délégation au moteur n8n	87
IV.8	Endpoints backend effectivement utilisés par KodaMate (UI)	87
IV.9	Appels réseau complémentaires de KodaMate (hors Distributeur)	88
IV.10	Mapping des écrans KodaMate vers les endpoints backend	88
IV.11	Problèmes techniques rencontrés et solutions apportées	89
V.1	Inventaire des composants matériels de KODA	95
V.2	Spécifications techniques du Raspberry Pi 4 Model B	97
V.3	Architecture d'alimentation de KODA	104
V.4	Récapitulatif des connexions entre composants	106

V.5 Tests unitaires par composant (Cycle 3)	107
VI.1 Expressions logicielles envoyées à l'écran Nextion	122
VI.2 Tests de validation du logiciel Raspberry Pi	124
VII.1 Interfaces KodaMate et rôles utilisateur	131
VII.2 Correspondance services principaux et besoins utilisateur	142
VII.3 Tests de validation par interface	144
VIII.1 Synthèse des cycles KODA — besoin couvert et composant principal	147

INTRODUCTION GÉNÉRALE

Dans un monde en constante évolution technologique, la demande en solutions intelligentes capables d'améliorer la qualité de vie des individus n'a jamais été aussi forte. Les assistants virtuels et les systèmes embarqués intelligents représentent aujourd'hui une réponse concrète à ce besoin, en combinant les avancées de l'intelligence artificielle, des systèmes embarqués et du développement logiciel moderne.

Ce projet de fin d'études, réalisé dans le cadre de la formation en Développement des Systèmes Informatiques (DSI3.2) à l'ISET Mahdia, porte sur la conception et le développement de **KODA** : un assistant de vie et ami virtuel embarqué. KODA est un système robotique doté de sa propre personnalité et nature, capable de répondre intelligemment à toute personne, de raisonner à la manière d'un être humain, de reconnaître les individus à travers ses modes de sécurité, et d'être configuré et suivi via une application mobile cross-plateforme dédiée.

La problématique centrale de ce projet est : **Comment concevoir un système embarqué intelligent capable d'assurer un rôle d'assistant de vie personnalisé, intégrant reconnaissance des personnes, réflexion autonome et pilotage à distance via une application mobile ?**

Ce rapport est organisé comme suit :

- **Chapitre I** présente le cadre général du projet, l'organisme d'accueil, la problématique, l'analyse de l'existant et la solution proposée.
- **Chapitre II** expose la préparation du projet : spécification des besoins, identification des acteurs, méthodologie de pilotage *en spirale* par cycles, planification, architecture cible et environnements de travail.
- **Chapitre III** correspond au **Cycle 1 — Analyseur et réflexion** : workflows n8n (webhooks, six blocs de traitement, intégration LLM et persistance).
- **Chapitre IV** correspond au **Cycle 2 — backend Distributeur de Services** (FastAPI, persistance, services cloud).
- **Chapitre V — Cycle 3** : système embarqué, partie matérielle (plateforme robot, câblage, validation).
- Les **cycles 4 à 6** (logiciel Raspberry Pi, application cross-plateforme, tests finaux) sont présentés dans la suite du document.

———— CHAPITRE I ————

CADRE GÉNÉRAL DU PROJET ET PRÉSENTATION DE L'ORGANISME

I.1 Introduction du chapitre

Ce chapitre esquisse le cadre fondamental de notre étude en définissant l’environnement contextuel et organisationnel du projet. Nous commençons par introduire l’organisme d’accueil, en mettant en évidence son expertise technologique et sa structure interne dédiée à l’innovation.

Ensuite, nous exposons la problématique centrale, justifiant le besoin critique d’un système intelligent. Une analyse rigoureuse des solutions existantes nous permet d’identifier les limites actuelles et de mieux positionner notre système proposé. Enfin, nous détaillons la méthodologie de projet adoptée, assurant une approche structurée et efficace tout au long du cycle de développement.

I.2 Contexte Général du Projet

L’évolution technologique actuelle transforme notre interaction avec l’environnement. La robotique sociale et les assistants personnels virtuels s’imposent comme des éléments clés pour améliorer la qualité de vie, offrant des solutions innovantes pour l’assistance quotidienne.

Le projet KODA s’inscrit dans cette dynamique en proposant un assistant de vie virtuel, conçu sous la forme d’un système embarqué intelligent. Contrairement aux solutions traditionnelles qui se limitent à l’exécution de commandes, KODA vise à offrir une véritable présence interactive, dotée d’une personnalité propre et d’une capacité de raisonnement.

La complexité de ce système réside dans l’intégration harmonieuse de multiples composants hétérogènes. Il nécessite la combinaison d’un environnement matériel embarqué (Raspberry Pi), d’un moteur de workflow intelligent (n8n) agissant comme cerveau, d’une couche logicielle orchestrant les communications (Python), et d’interfaces de contrôle intuitives (application mobile cross-plateforme). L’objectif est de créer un écosystème cohérent capable de reconnaître les utilisateurs, de traiter les requêtes de manière naturelle, et de fournir un accompagnement personnalisé.

I.3 Présentation de l’Organisme d’Accueil (MicroEdition)

I.3.1 Profil de l’Entreprise

MicroEdition est une entreprise pionnière spécialisée dans les Technologies de l’Information et de la Communication (TIC). Fondée en 2025, la société est stratégiquement implantée au sein de la Pépinière d’Entreprises de Mahdia. Cet environnement offre un écosystème robuste dédié à l’innovation, fournissant l’infrastructure nécessaire pour accompagner les startups technologiques à fort potentiel. MicroEdition se consacre à la conception et au déploiement de solutions numériques intelligentes, évolutives et sécurisées, visant à combler le fossé entre la recherche académique et

l'application industrielle.

I.3.2 Services

Le portefeuille de l'entreprise repose sur trois piliers technologiques fondamentaux :

- **Développement logiciel** : Conception d'applications web et mobiles haute performance, adaptées à des logiques métier spécifiques.
- **Internet des Objets (IoT)** : Conception d'écosystèmes connectés de bout en bout pour l'automatisation industrielle et agricole.
- **Cybersécurité** : Mise en œuvre de protocoles de sécurité rigoureux pour protéger les actifs numériques et garantir l'intégrité des données dans les secteurs public et privé.

I.3.3 Recherche & Développement (R&D)

Un trait distinctif de MicroEdition est sa division spécialisée en Recherche et Développement. Ce département se concentre fortement sur l'IA embarquée (Edge AI) et l'intelligence embarquée. En explorant des méthodologies d'exécution de réseaux de neurones complexes directement sur des microcontrôleurs, l'équipe R&D cherche à éliminer la dépendance à une connectivité cloud permanente. Cet axe stratégique est central pour le projet KODA, car il permet la création de systèmes autonomes et résilients capables de fonctionner dans des environnements où l'accès à Internet peut être intermittent.

I.3.4 Structure Organisationnelle

MicroEdition fonctionne selon un cadre Agile qui met l'accent sur l'itération rapide et la collaboration interdisciplinaire. La structure interne de l'entreprise est composée d'unités spécialisées, notamment :

- **Architectes logiciels** : Conception de backends systèmes évolutifs.
- **Ingénieurs en systèmes embarqués** : Pont entre le matériel et le logiciel.
- **Spécialistes en IA et Data Scientists** : Développement et optimisation de modèles d'apprentissage automatique pour le déploiement embarqué.

Cette synergie permet à MicroEdition de maintenir un cycle d'innovation soutenu, garantissant que chaque projet — tel que KODA — repose sur des fondations d'excellence technique et de standards architecturaux modernes.

I.4 Problématique

L'évolution constante de notre société moderne, marquée par un rythme de vie accéléré et une digitalisation croissante, a paradoxalement accentué certains besoins fondamentaux de l'être humain, notamment celui de l'accompagnement et de l'interaction sociale.

Le problème central auquel nous tentons de répondre s'articule autour de trois axes principaux :

A. Le besoin de présence et d'interaction naturelle

Malgré la prolifération des appareils connectés, la plupart des outils actuels (smartphones, enceintes connectées classiques) restent des outils purement fonctionnels. Ils exécutent des ordres mais n'offrent pas de réelle interaction "humaine". Il existe un manque flagrant de systèmes capables d'afficher une personnalité propre et de simuler une forme de réflexion empathique capable de briser la solitude ou d'assister l'utilisateur au quotidien.

B. La complexité de la gestion de vie autonome

Dans un environnement domestique, la gestion des tâches, de la sécurité et du suivi technique des objets connectés devient de plus en plus complexe pour l'utilisateur moyen. Il n'existe pas de solution centralisée qui combine à la fois un assistant de vie intelligent (capable de raisonner) et un système de sécurité actif (capable de reconnaître les personnes et de protéger l'habitat).

C. Les limites de l'intelligence statique

La majorité des assistants actuels dépendent de réponses préprogrammées et rigides. Le défi est de passer d'une machine qui "répond" à une machine qui "réfléchit" et "apprend". Le problème est donc de concevoir une architecture capable de :

- Identifier l'interlocuteur de manière sécurisée.
- Traiter des demandes complexes de manière fluide et humaine.
- Offrir une interface de contrôle simple (application mobile) pour superviser cette "vie artificielle".

En résumé, la question fondamentale de notre projet est la suivante : **Comment concevoir un compagnon robotique intelligent, doté d'une mémoire et d'une personnalité, capable d'offrir un accompagnement humain authentique tout en assurant une gestion technique et sécurisée de l'environnement de l'utilisateur ?**

I.5 Analyse des Solutions Existantes

I.5.1 Étude de l'existant

Dans cette section, nous examinons les assistants domestiques et les robots compagnons les plus populaires sur le marché afin d'identifier leurs forces et leurs lacunes.

- **A. Les Assistants Vocaux Statiques (Amazon Alexa, Google Home)**

Ces dispositifs sont les plus répandus. Ils excellent dans l'exécution de tâches simples (domotique, rappels, recherches internet).

Limites : Ils n'ont pas de présence physique mobile. Surtout, ils restent des outils purement réactifs : ils attendent un "mot déclencheur" et répondent de manière standardisée. Il n'y a aucune notion de "personnalité" évolutive ou de sentiment d'appartenance.

- **B. Les Robots Domestiques (ex : Amazon Astro, Vector/Cozmo)**

Certains robots essaient d'occuper l'espace physique. Amazon Astro, par exemple, surveille la maison et suit l'utilisateur.

Limites : Bien que mobiles, ces produits restent perçus comme des gadgets technologiques. Leur interaction est limitée par des scripts pré-enregistrés. Ils ne "réfléchissent" pas de manière humaine et leur capacité à reconnaître et s'adapter à l'humeur ou à l'identité spécifique de chaque membre de la famille de façon approfondie reste superficielle.

- **C. Les Pepper & Nao (Robotique de service)**

Ce sont des robots haut de gamme capables de simuler des émotions et de reconnaître des visages.

Limites : Ces solutions sont extrêmement coûteuses (plusieurs milliers d'euros) et sont destinées aux entreprises plutôt qu'au grand public. Leur configuration est complexe et ils ne sont pas conçus comme des "amis" personnels intégrés à un écosystème mobile fluide pour l'utilisateur lambda.

I.5.2 Synthèse et Critique : Le "Manque d'Humanité"

Le point commun de tous ces concurrents est leur nature générique. Ce sont des machines produites en série qui traitent chaque utilisateur de la même manière.

- **Absence de personnalité** : La réponse d'une Alexa est la même pour tout le monde. Elle n'a pas de "caractère" propre qui pourrait la faire passer pour un compagnon de vie.
- **Froideur algorithmique** : Les interactions sont basées sur des arbres de décision rigides. L'utilisateur sent qu'il parle à une base de données, pas à une entité qui "réfléchit".

- **Dépendance au Cloud propriétaire** : L'utilisateur a peu de contrôle sur la "mémoire" ou la logique interne de l'assistant.

I.5.3 La Différenciation de Koda

C'est ici que notre projet se distingue. Contrairement aux machines industrielles, Koda est conçu pour être :

- **Unique** : Grâce à l'intégration de n8n (le cœur de réflexion), Koda peut avoir une logique de réponse personnalisée et évolutive.
- **Multimodal** : Il combine la vision (Raspberry Pi), la réflexion (IA via n8n) et la mobilité.
- **Accessible** : Une application mobile dédiée permet de configurer sa "personnalité", rendant l'utilisateur maître de son compagnon.

I.6 Solution Proposée

Pour surmonter ces limitations, nous proposons KODA, une architecture résiliente et multi-couche conçue pour l'autonomie et l'intelligence. La solution comprend :

- **Contrôleur Embarqué (Raspberry Pi)** : Le cœur physique du système, gérant les composants matériels et assurant le fonctionnement du robot.
- **Moteur de Réflexion (n8n)** : Le cerveau de KODA, responsable de la préparation des réponses et du traitement intelligent des requêtes complexes.
- **Couche de Distribution (Python)** : Le système nerveux responsable de la communication entre n8n, la base de données, l'application mobile et le Raspberry Pi, offrant les points d'accès (endpoints) de tout l'écosystème.
- **Expérience Utilisateur Mobile** : Une application cross-plateforme permettant à l'utilisateur de configurer, contrôler et suivre la vie de son compagnon robotique en temps réel.
- **Mémoire Persistante (Base de données)** : Un système de stockage permettant à KODA d'enregistrer des informations contextuelles et d'apprendre de ses interactions.

En intégrant ces composants, la solution proposée garantit une résilience opérationnelle, des interactions humaines authentiques et une expérience de gestion unifiée.

I.7 Méthodologie du Projet

Pour garantir un haut degré de fiabilité et une intégration harmonieuse des différentes couches logicielles et matérielles, ce projet suit un cadre d'ingénierie rigoureux.

I.7.1 Modèle en spirale

Le pilotage du projet KODA repose sur le **modèle en spirale**, et non sur un cycle en V séquentiel ni sur une succession de sprints Scrum. À chaque tour de spirale, une partie du système est **précisée, construite, testée** et **raccordée** au reste ; avant d'engager le tour suivant, l'équipe **réévalue les risques** (techniques, d'intégration, de délai) et ajuste les priorités. Cette démarche convient à un compagnon robotique multicouche : on ne livre pas tout d'un coup, on fait monter en puissance le moteur de réflexion, le lien avec le robot, l'application mobile, puis la validation d'ensemble.

Le périmètre est découpé en **six cycles thématiques**, ordonnés de façon logique :

1. analyse et réflexion (moteur conversationnel) ;
2. couche de distribution et persistance des données ;
3. partie matérielle embarquée ;
4. partie logicielle sur le robot ;
5. application mobile de pilotage ;
6. tests et validation globale.

Chaque cycle se clôt par un **incrément démontrable** (fonctionnalité branchée, scénario utilisateur, critères d'acceptation) et par un point de synchronisation avec l'encadrement. La planification détaillée, les rôles et le calendrier des jalons sont formalisés au chapitre II ; la figure ci-dessous en résume le principe.

Artefacts et traçabilité

À chaque tour, des livrables assurent la continuité entre les cycles :

- **Besoins et périmètre** mis à jour (utilisateur, sécurité, mémoire).
- **Architecture et intégration** (schémas, interfaces entre robot, logiciel de réflexion, application).
- **Réalisation et recette** du cycle en cours (tests, démonstration, correctifs avant le tour suivant).

I.8 Conclusion

Ce chapitre a établi le contexte du projet KODA. En identifiant les lacunes critiques des assistants virtuels actuels — notamment le manque de personnalité, de réflexion autonome et d'intégration

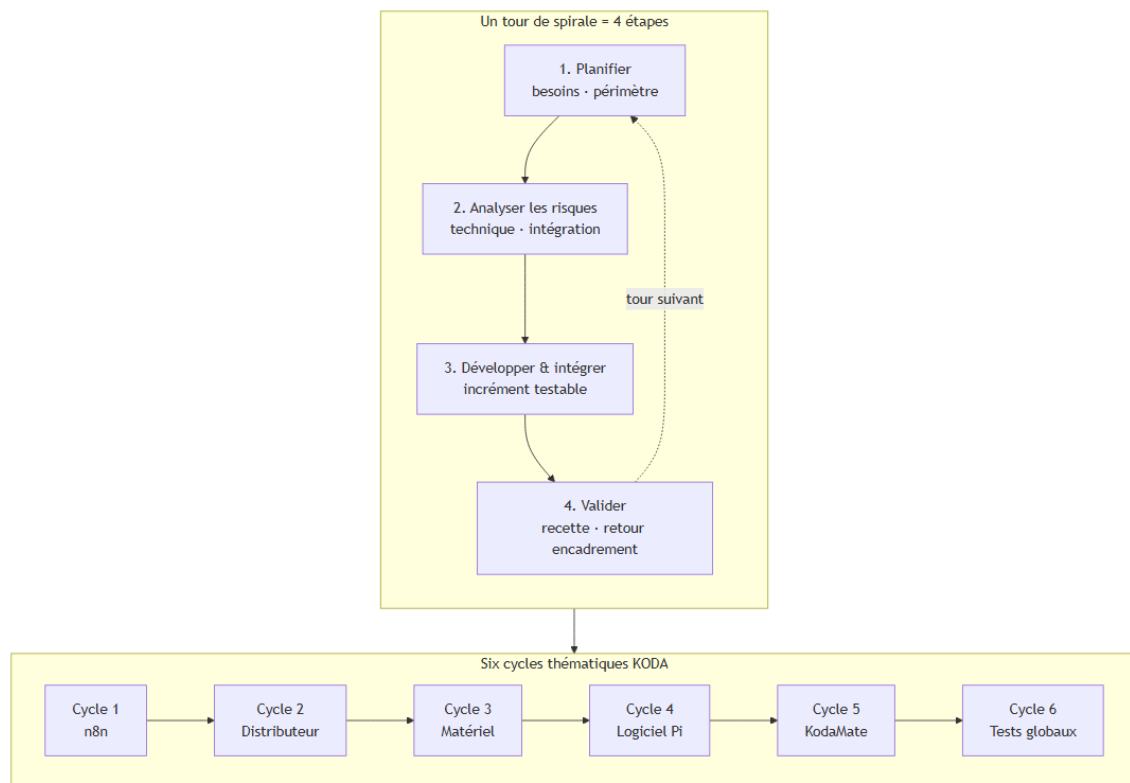


FIGURE I.1 – Modèle en spirale — enchaînement des six cycles de développement KODA

embarquée — nous avons défini notre solution innovante. Le **modèle en spirale** et les six cycles de développement posent une feuille de route incrémentale, adaptée à la complexité d'un système distribué entre compagnon physique, réflexion et pilotage mobile.

Le chapitre suivant, «**Préparation du Projet**», formalise les besoins, la méthodologie de développement **en spirale** par cycles successifs et les environnements techniques retenus pour mener à bien la réalisation de KODA.

_____ CHAPITRE II _____

PRÉPARATION DU PROJET

II.1 Introduction du chapitre

Le chapitre I a posé le **contexte**, la **problématique** et la **solution KODA** sous forme de vision. Le présent chapitre II en est la **traduction ingénierie** : il formalise ce que le système doit faire (besoins), comment il doit se tenir dans le temps (exigences non fonctionnelles), comment il est structuré (architecture générale), et comment il a été construit progressivement (modèle en spirale et cycles). Chaque besoin identifié ici est **traçable** vers un cycle de réalisation et vers un chapitre technique du rapport, afin d'éviter l'écart entre l'intention du début de stage et le prototype livré.

II.2 Spécification des besoins

II.2.1 Besoins fonctionnels

Les besoins fonctionnels (BF) décrivent *ce que* KODA doit offrir à l'utilisateur. Ils sont dérivés du chapitre I, enrichis par les capacités réellement implémentées dans les cycles 1 à 5 (workflows n8n, Distributeur, robot, KodaMate).

TABLE II.1 – Catalogue des besoins fonctionnels KODA

Id.	Besoin	Description	Cycle(s)
BF01	Dialogue naturel	Comprendre une question (oral ou texte), répondre de façon claire ; réponses immédiates si commande directe (arrêt, mot de passe, musique, déplacement), sinon analyse approfondie.	1, 4, 5
BF02	Mémoire & personnalisation	Retenir profils, préférences, notes et historique ; répondre en s'appuyant sur le passé ; consolidation mémorielle périodique.	1, 2
BF03	Accueil & identification	Reconnaître un visiteur connu ou inviter un inconnu à se présenter (unkonu) ; accueil personnalisé.	1, 2, 4
BF04	Présence physique	Matérialiser l'interaction par voix, expression (Nextion), mouvement (Arduino) et capteurs, relié au système central.	3, 4
BF05	Pilotage mobile	Configurer KODA (nom, langue, caractère), superviser l'état, dialoguer par texte, consulter l'historique.	2, 5
BF06	Sécurité d'usage	Modes ouvert / protégé par mot de passe ; limiter les fonctions sensibles en mode sécurisé.	1, 2, 5
BF07	Commandes fonctionnelles	Musique (recherche), notes/rappels, informations personnelles, agenda (selon branches workflow).	1, 2
BF08	Spontanéité	Prendre l'initiative de parler (Webhook 3) en s'appuyant sur les synthèses d'accompagnement.	1
BF09	Chaîne vocale embarquée	Mot de réveil, capture VAD, STT/TTS via backend, restitution audio sur le robot.	2, 4
BF10	Reconnaissance faciale	Identifier un interlocuteur via caméra et service backend dédié.	2
BF11	Supervision état robot	Lire et commander l'état marche/arrêt ; afficher état réseau et contexte (météo) sur mobile.	2, 5
BF12	Authentification mobile	Connexion utilisateur, session persistante, test de joignabilité du serveur.	2, 5

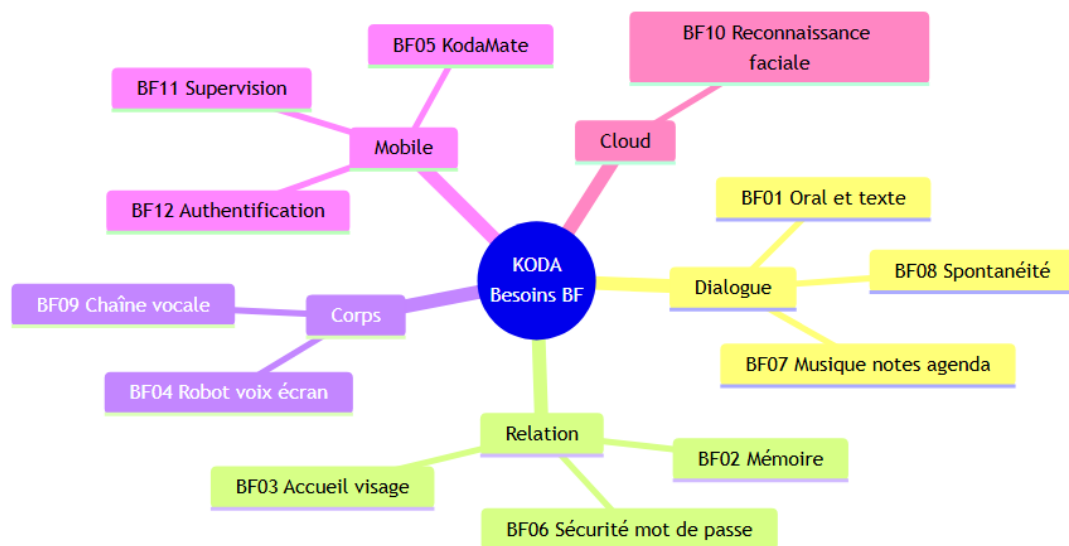


FIGURE II.1 – Synthèse des besoins fonctionnels KODA

II.2.2 Besoins non fonctionnels

Les besoins non fonctionnels (BNF) précisent les **qualités** attendues du système (tableau II.2).

TABLE II.2 – Catalogue des besoins non fonctionnels KODA

Id.	Qualité	Exigence	Contrôle
BNF01	Réactivité perçue	Réponses fluides ; réutilisation de la mémoire sans retraitement inutile.	Tests démo
BNF02	Disponibilité des données	Profils, historiques et paramètres conservés malgré coupures partielles de services cloud.	PostgreSQL
BNF03	Sécurité & confidentialité	Authentification, protection des données personnelles, accès restreint aux fonctions sensibles.	Auth JWT, modes
BNF04	Maintenabilité	Découpage par couches (mobile, backend, n8n, embarqué) ; interfaces stables (REST, webhooks).	Architecture
BNF05	Évolutivité	Ajout de capacités (nouvelle intention, écran, capteur) sans refonte globale.	Distributeur pivot
BNF06	Accessibilité	Interfaces compréhensibles pour un utilisateur non expert (mobile, modes robot).	UX KodaMate
BNF07	Intégration incrémentale	Chaque cycle livre un incrément testable raccordé au précédent.	Spirale
BNF08	Robustesse embarquée	Mode dégradé si matériel ou réseau indisponible ; arrêt propre des actionneurs.	Cycle 4

II.3 Architecture générale et acteurs

L'architecture est **multicouche** avec le **Distributeur FastAPI** comme hub unique : KodaMate et le robot passent par lui pour atteindre n8n, PostgreSQL et les API cloud (Azure, OpenRouter, SerpAPI, Open-Meteo).

TABLE II.3 – Acteurs et rôles dans l'écosystème KODA

Acteur	Rôle
Utilisateur final	Dialogue, configuration et supervision via KodaMate.
Équipe projet / encadrant	Conception, intégration, validation (PFE).
Distributeur	Médiation technique entre tous les composants.
Services cloud	IA, voix, vision, recherche, base de données.
Organisme d'accueil	Cadre professionnel (MicroEdition).

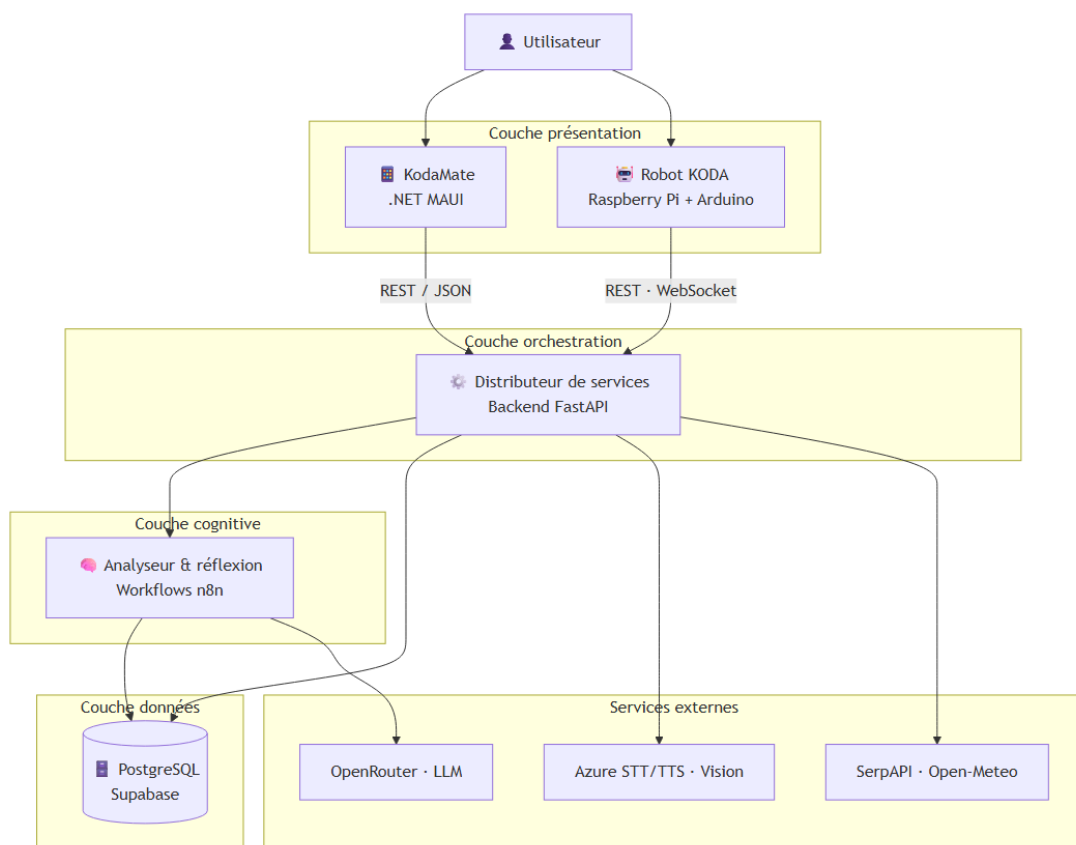


FIGURE II.2 – Architecture générale du système KODA

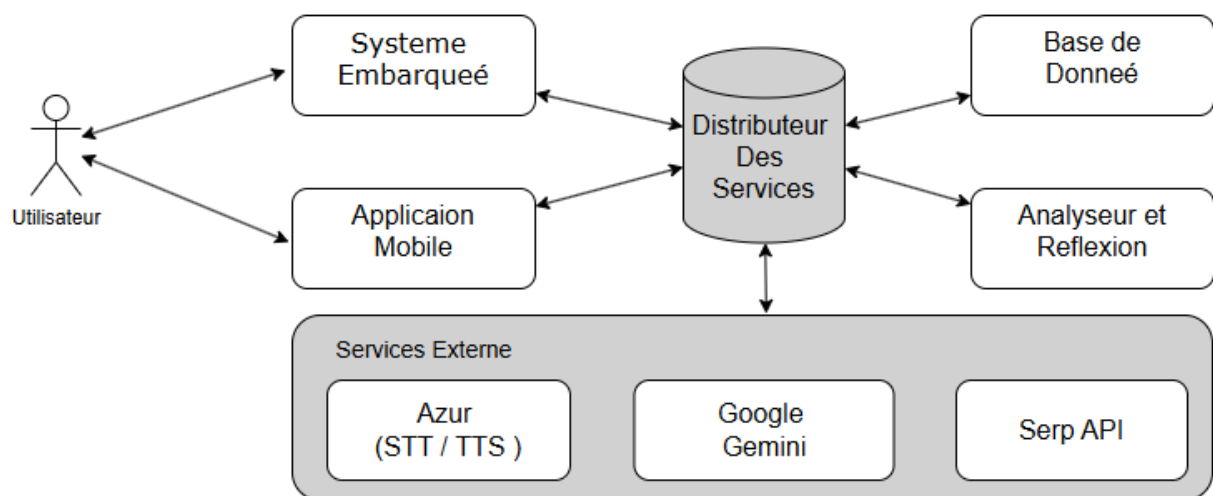


FIGURE II.3 – Acteurs principaux et contexte d'intégration du système KODA

II.4 Méthodologie en spirale et planification

Le pilotage repose sur le **modèle en spirale** (Boehm), et non sur Scrum ni un cycle en V strict : à chaque tour, on **planifie**, **analyse les risques**, **développe** un incrément testable, puis **valide** avant le cycle suivant. Cette approche convient à un prototype IA multicouche (n8n, backend, robot, mobile).

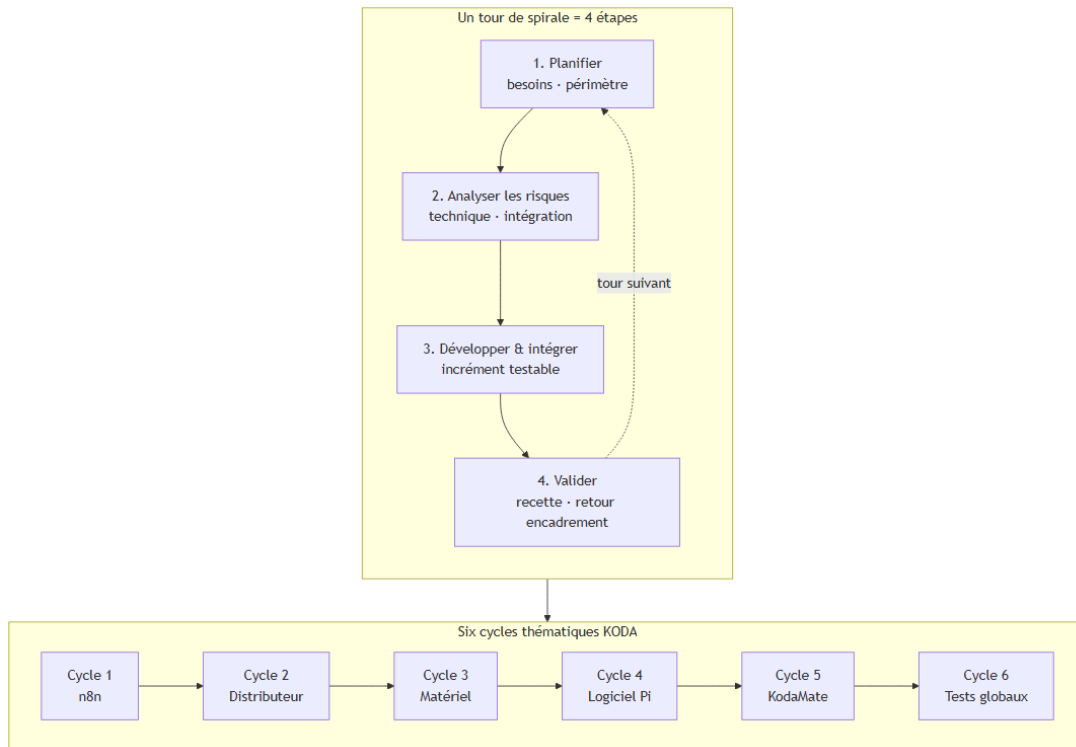


FIGURE II.4 – Modèle en spirale et six cycles thématiques KODA

Le tableau II.4 relie chaque **cycle** au **chapitre** du rapport (chapitre IV = Cycle 2 backend).

TABLE II.4 – Cycles, chapitres du rapport et livrables

Cy.	Thème	Ch.	Livrable	Besoins
1	n8n / cognition	III	3 webhooks, mémoire	BF01–03, 07–08
2	Distributeur	IV	API, BDD, hub cloud	BF02, 05–12
3	Matériel robot	V	Pi, Arduino, Nextion	BF04
4	Logiciel Pi	VI	Audio, VAD, mouvement	BF04, 09
5	KodaMate MAUI	VII	App mobile	BF05, 11–12
6	Validation globale	—	Recette BNF	BNF01–08

Équipe PFE (DSI3.2, ISET Mahadia), binôme, encadrement et lien MicroEdition (ch. I). Dépendances : cycles 1–2 en parallèle partiel puis intégration ; cycle 3 tôt ; cycles 4–5 après backend ; cycle 6 en recette globale.

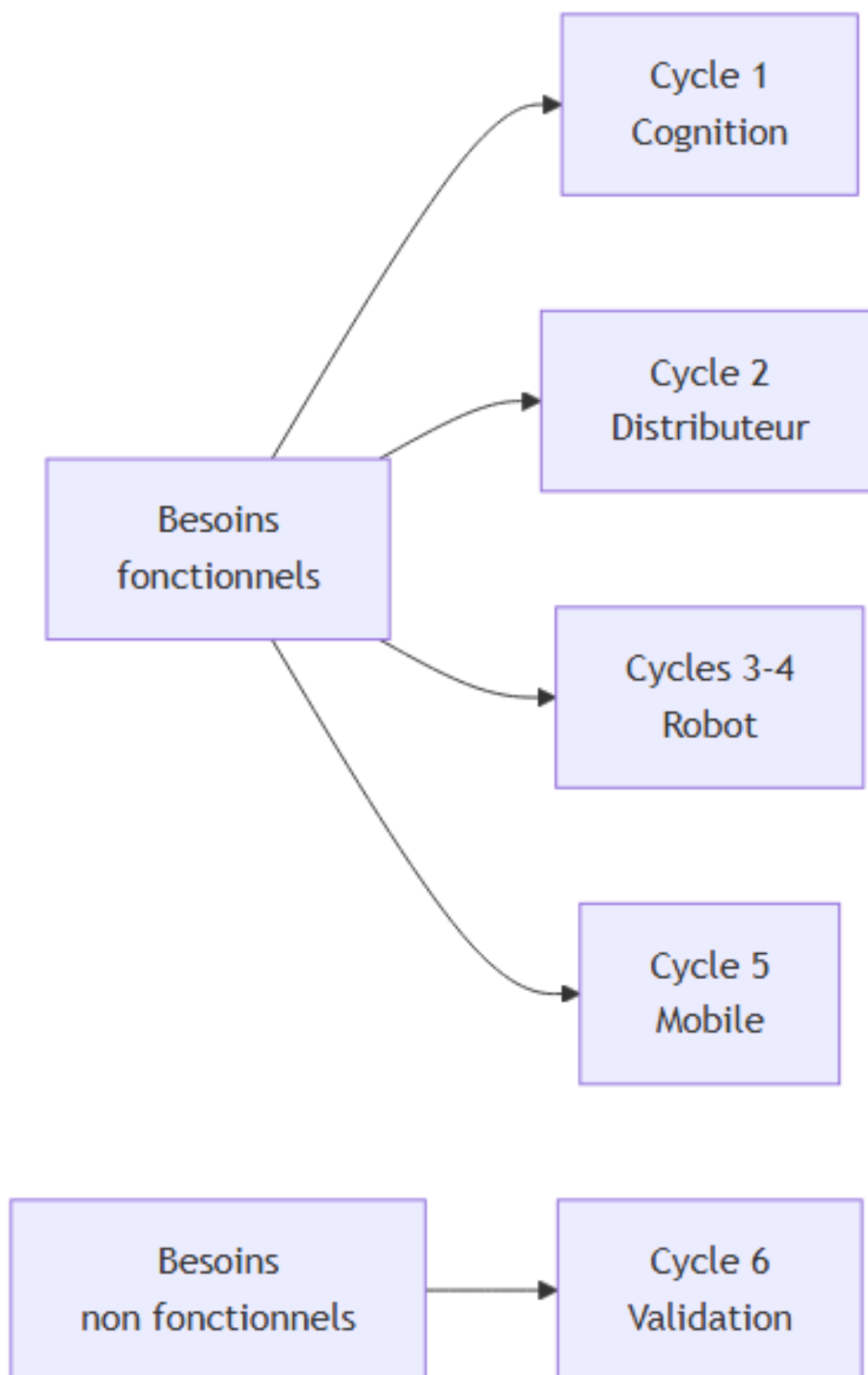


FIGURE II.5 – Traçabilité besoins → cycles de réalisation

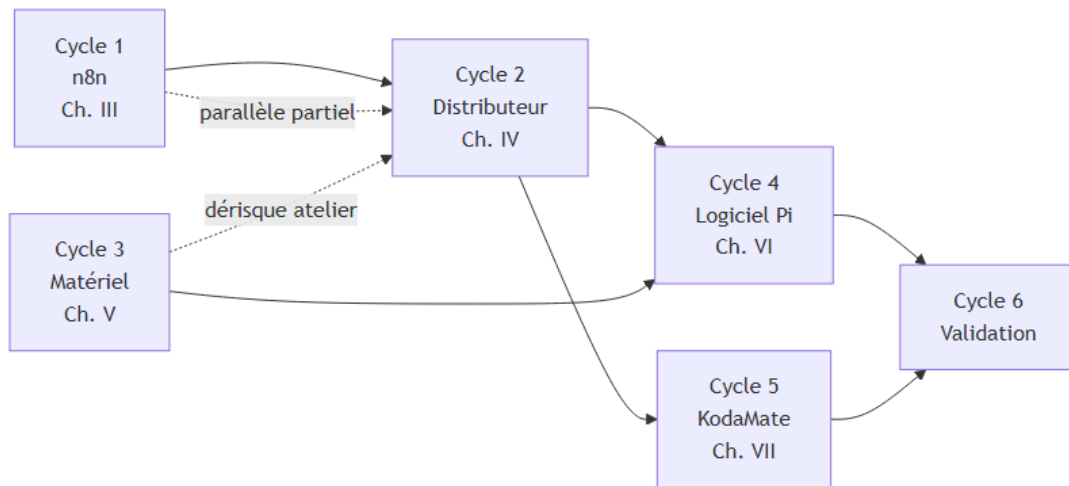


FIGURE II.6 – Planification — dépendances entre les six cycles

II.5 Environnements de développement

II.5.1 Matériel et logiciel

Matériel : PC de développement (8 Go+ RAM), Raspberry Pi, prototype robot (audio, caméra, Nextion), smartphone ou émulateur pour KodaMate, accès Internet (Supabase, OpenRouter, SerpAPI, Azure).

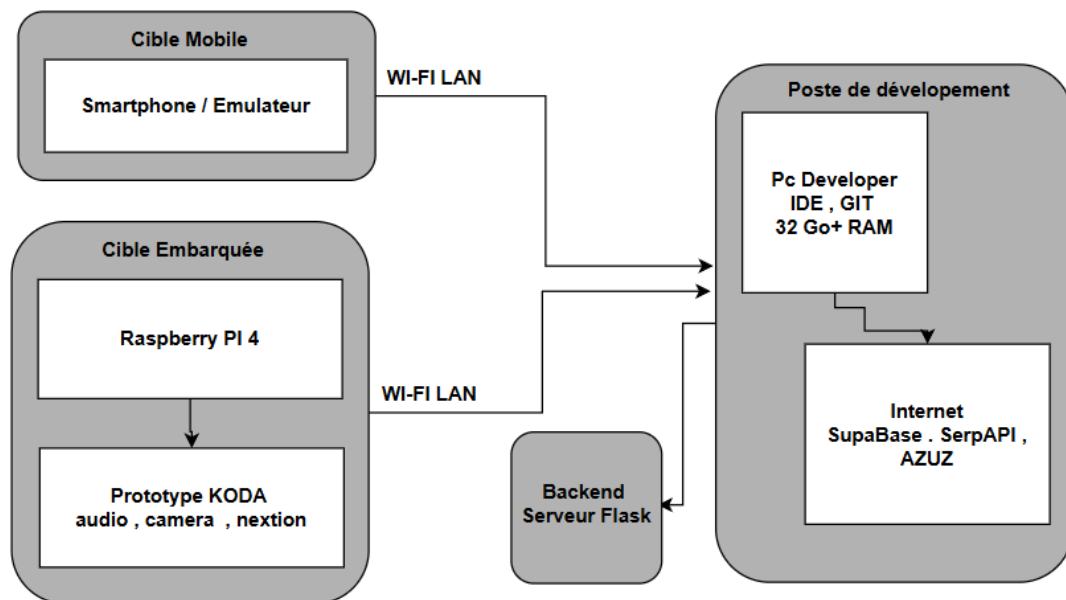


FIGURE II.7 – Environnement matériel de test

Logiciel :

- **n8n** (self-hosted ou cloud) pour l'édition des workflows ;
- **Visual Studio Code** ou Visual Studio pour Python, C# et LaTeX ;
- **Git** pour le versioning ;

- **draw.io / Mermaid** / captures d'écran pour les diagrammes de séquence et de cas d'utilisation ;
- **Supabase Studio** pour l'administration PostgreSQL ;
- **Docker** (le cas échéant) pour le déploiement du backend et la reproductibilité des environnements.

II.5.2 Langages et frameworks utilisés

- **JavaScript** (expressions n8n, nœuds Code) ;
- **Python 3** avec **FastAPI**, **SQLAlchemy**, **asyncpg**, **SDK Azure** et intégrations **Gemini/OpenRouter** côté Distributeur ;
- **SQL** (PostgreSQL) ;
- **C# / .NET MAUI 8** pour l'application cross-plateforme **KodaMate** ;
- **Bash** et utilitaires système sur **Raspberry Pi OS**.

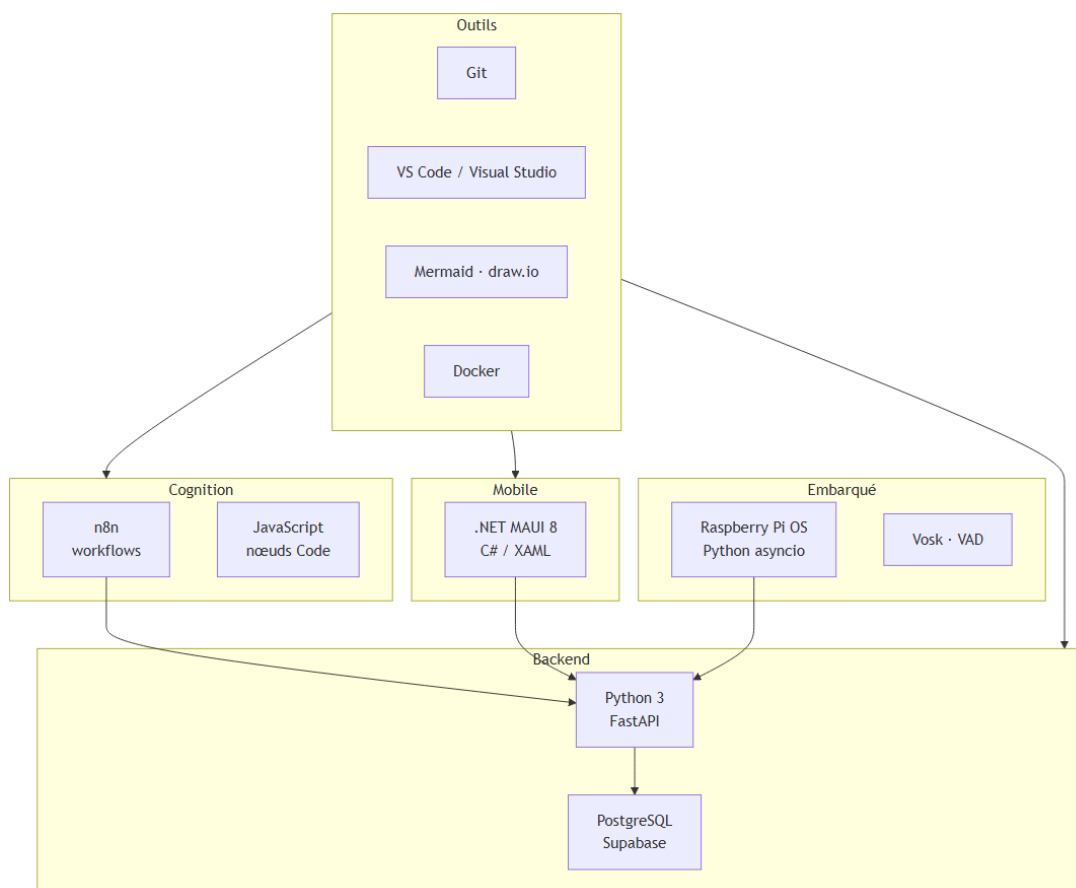


FIGURE II.8 – Environnement logiciel — stack technique KODA

II.6 Conclusion du chapitre

Ce chapitre traduit la vision du chapitre I en ingénierie exploitable : **12 besoins fonctionnels** et **8 besoins non fonctionnels** identifiés, une **architecture générale** centrée sur le Distributeur, une **méthodologie en spirale** avec six cycles tracés vers les chapitres III à VII, et les **environnements** matériel et logiciel du prototype. Les éléments qui manquaient initialement dans ce chapitre — diagramme d’architecture à jour, figure de spirale, tableaux de besoins numérotés, traçabilité explicite — sont désormais formalisés ici pour servir de référence unique avant l’entrée dans le détail du **Cycle 1** (chapitre III, workflows n8n).

_____ CHAPITRE III _____

CYCLE 1 : ANALYSEUR ET RÉFLEXION

III.1 Introduction

Le **Cycle 1 — Analyseur et réflexion** constitue le **moteur cognitif** de KODA. Son rôle est double : assurer la **génération des réponses** lorsque l'utilisateur pose une question, et alimenter la **spontanéité** du robot lorsqu'il prend l'initiative de parler. L'objectif poursuivi est de développer un système capable d'**accueillir une question**, de la **comprendre dans son contexte**, d'effectuer les **recherches et consultations** nécessaires (mémoire, profil, services externes), puis de **formuler et restituer une réponse** adaptée.

La chaîne de traitement — de l'entrée utilisateur à la réponse — forme une **ligne de processus** successifs. Les **outils d'automatisation de flux** offrent une meilleure lisibilité et un débogage par étape que du code monolithique ; la **bonne pratique** retenue est la plateforme **n8n**, auto-hébergée et adaptée aux pipelines mêlant règles métier et intelligence artificielle.

Trois **webhooks** n8n structurent ce cycle. Après les fondements théoriques, la modélisation globale puis l'analyse détaillée de chaque flux, le lecteur dispose d'une vue complète du moteur d'analyse et de réflexion.

III.2 Fondements théoriques

Cette section pose le **cadre cognitif** du Cycle 1 : comment une question est comprise et transformée en réponse, *avant* l'outillage n8n (section suivante).

III.2.1 Théorie du double traitement

La **théorie du double traitement** (*dual-process theory*, Kahneman, 2011) rappelle qu'une même question peut emprunter deux chemins : une **réflexion rapide (cognition intuitive)** qui réactive un schéma ou un souvenir déjà présent (échange récent, habitude, fait mémorisé) ; et une **réflexion lente (cognition analytique)** qui croise plusieurs souvenirs pour **construire** une réponse nouvelle lorsque la demande est floue, inédite ou sensible. Les deux voies se complètent : la première propose une piste, la seconde la confirme, la nuance ou la corrige. Nous privilégions ces termes explicites plutôt que « système 1 » / « système 2 », peu parlants hors laboratoire.

III.2.2 De la perception à la formulation

À l'oral comme à l'écrit, le traitement suit ordinairement cinq étapes :

1. **Percevoir** le message et l'interlocuteur.
2. **Comprendre** l'intention (information, avis, plaisanterie) et le contexte immédiat.

3. **Mobiliser** la mémoire pertinente (conversation, relation, savoir général).
4. **Choisir** la voie rapide (rappel direct) ou lente (assemblage et vérification).
5. **Formuler** la réponse et l'ajuster si nécessaire.

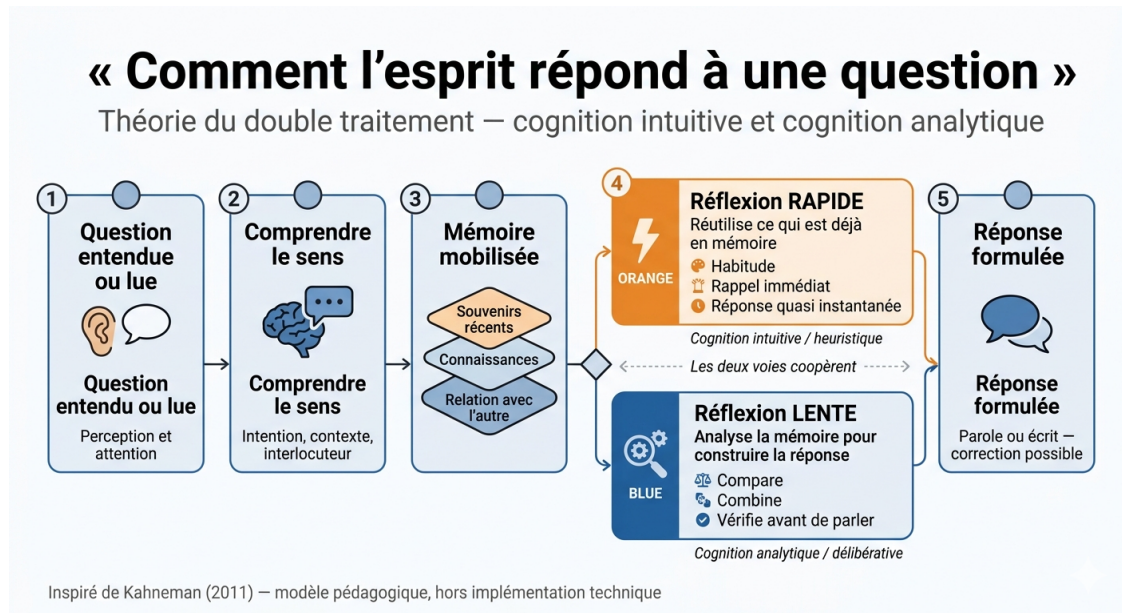


FIGURE III.1 – De la question à la réponse : réflexion rapide ou réflexion lente

III.2.3 Conclusion — fondements théoriques

Pour un compagnon comme **KODA**, l'enjeu est de **mimer cette alternance** : répondre vite lorsque l'historique ou la mémoire récente suffit, analyser plus profondément sinon. Le Cycle 1 traduit ce principe en pipelines techniques (section III.4, figure III.4). Les références centrales sont Kahneman (2011) et Evans (2008) sur le double traitement.

III.3 Définitions fondamentales

Le Cycle 1 s'appuie sur **n8n** (workflows auto-hébergés, données JSON) et sur trois **webhooks** HTTP : analyse/réponse, synthèse journalière, spontanéité. Les modèles de langage sont accessibles via **OpenRouter** dans les nœuds *Basic LLM Chain* ou *AI Agent*.

III.3.1 Grand modèle de langage (LLM)

Un **LLM** (*Large Language Model*) prédit la suite de tokens à partir de corpus massifs : il produit du texte, suit des *prompts* et respecte des formats de sortie imposés. Dans **n8n**, le workflow envoie le message et le prompt système ; la **complétion** est parsée en aval (JSON, routage, réponse webhook). Dans le **premier flux**, le LLM agit comme **analyseur de dépendance mémoire** :

décision USE_MEMORY / NO_MEMORY et catégorie sémantique, sous contrôle strict du prompt (détaillé dans le Webhook 1, section suivante).

III.3.2 Conclusion — définitions fondamentales

n8n structure le raisonnement en étapes vérifiables ; les **webhooks** séparent temps réel, consolidation et proactivité ; le **LLM**, via OpenRouter, assume les tâches linguistiques complexes là où des règles seules ne suffisent pas. Ce socle outillage est décliné dans la modélisation globale ci-dessous (Russell & Norvig, 2010).

III.4 Modélisation globale du Cycle 1

La modélisation suivante traduit, sur le plan technique, la distinction entre **réflexion rapide** (réutilisation de l'historique récent) et **réflexion lente** (analyse approfondie avant réponse), introduite au § III.2.

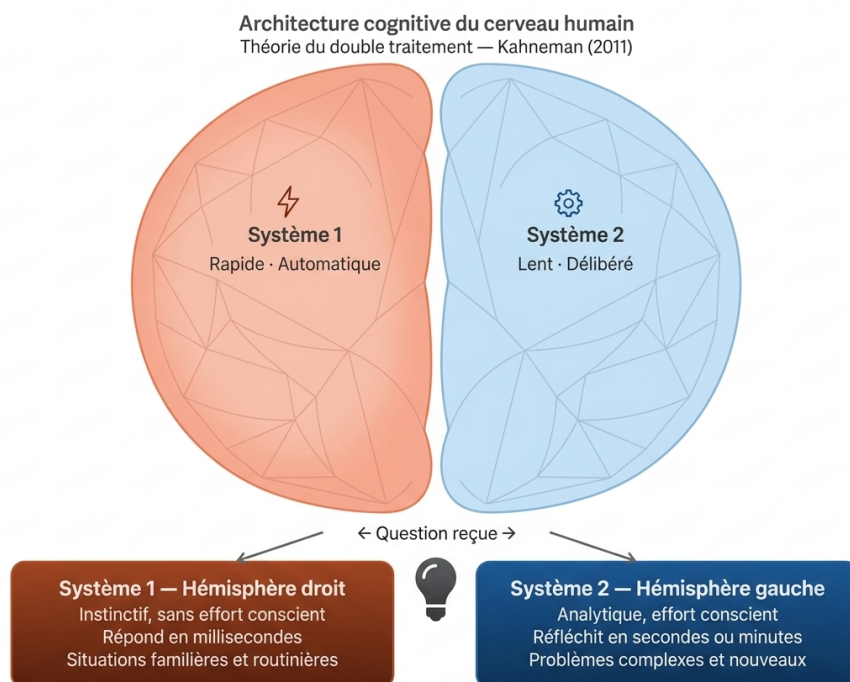


FIGURE III.2 – Architecture cognitive de KODA inspirée de la théorie du double traitement

III.4.1 Vue d'ensemble des trois flux n8n

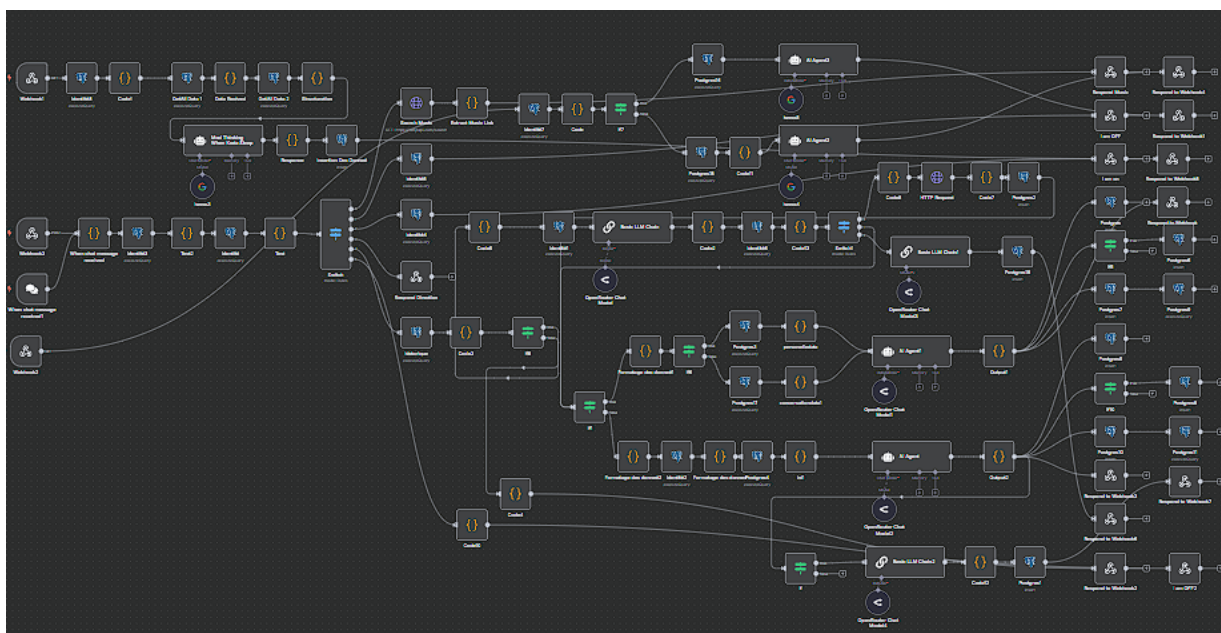


FIGURE III.3 – Cycle 1 — vue d’ensemble des trois workflows n8n

III.4.2 Diagramme de cas d'utilisation général

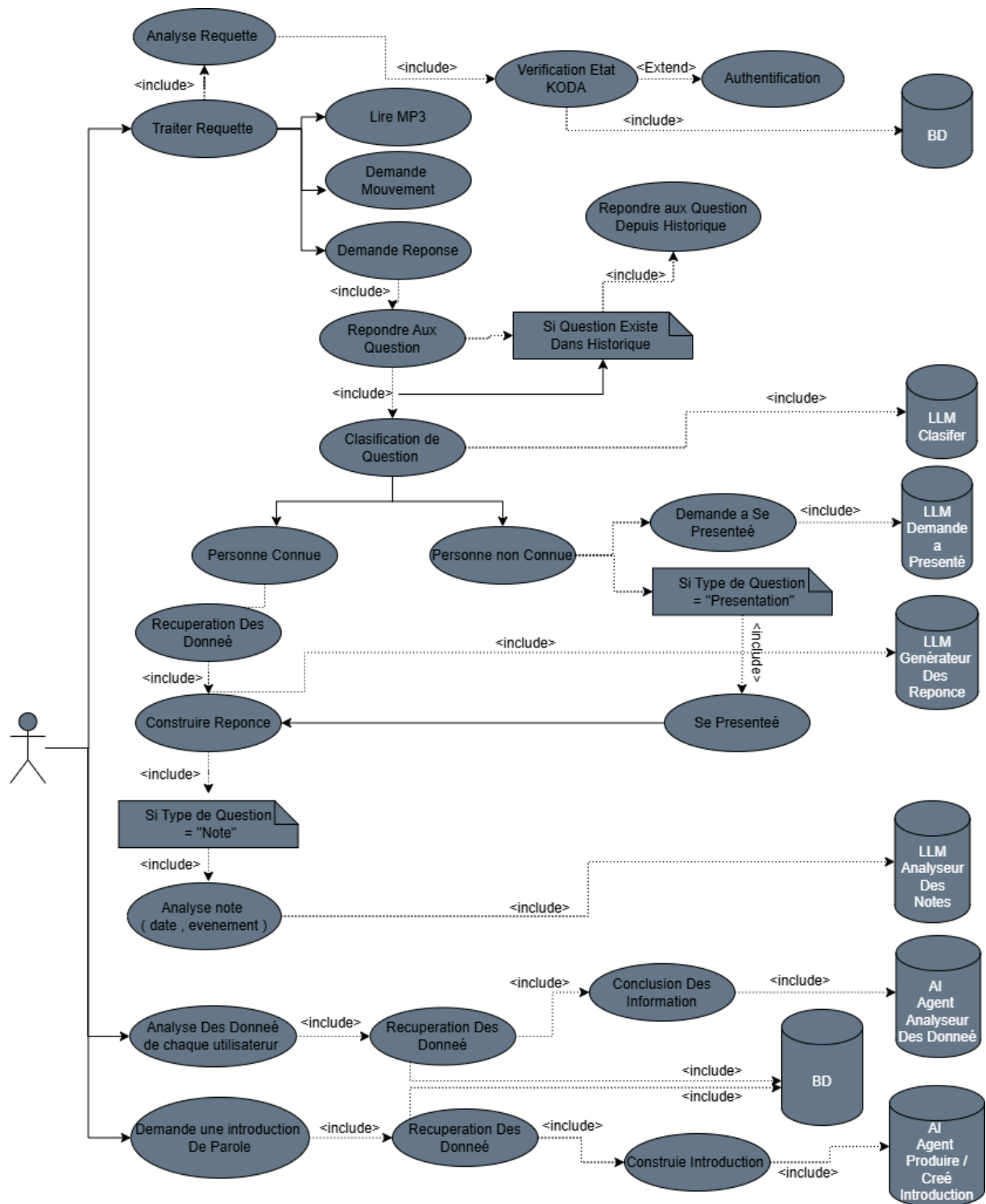


FIGURE III.4 – Cycle 1 — diagramme de cas d'utilisation général

III.4.3 Schéma de classes général

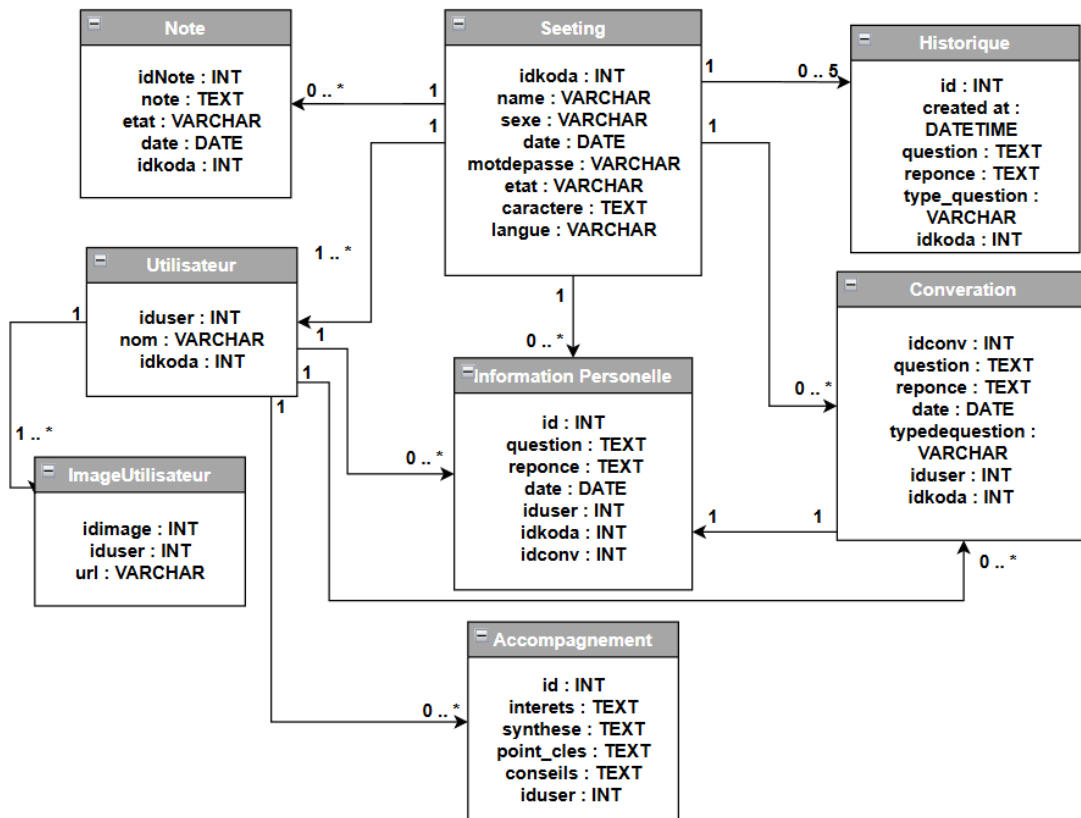


FIGURE III.5 – Cycle 1 — schéma de classes général (persistance)

III.5 Fonctionnalités Techniques — les trois flux

III.5.1 Premier Flux — Webhook 1 : Analyse et Réponse

III.5.1.1 Principe général et découpage en six parties

Le **Webhook 1** est le pipeline principal en temps réel. Il se décompose en six parties, dans l'ordre de la figure III.5.1.2 : *entrée fonctionnelle, confrontation à l'historique, classification sémantique, utilisateur connu ou inconnu, réponse contextualisée, notes et mémoire persistante*.

III.5.1.2 Diagramme de séquence du Webhook 1

Rôle théorique. Montrer la **chaîne complète** des six étapes cognitives et techniques, de l'entrée à la persistance.

Rôle pratique (n8n). Vue globale PlantUML `dsg.png`; le détail de chaque étape suit dans les paragraphes ci-dessous.

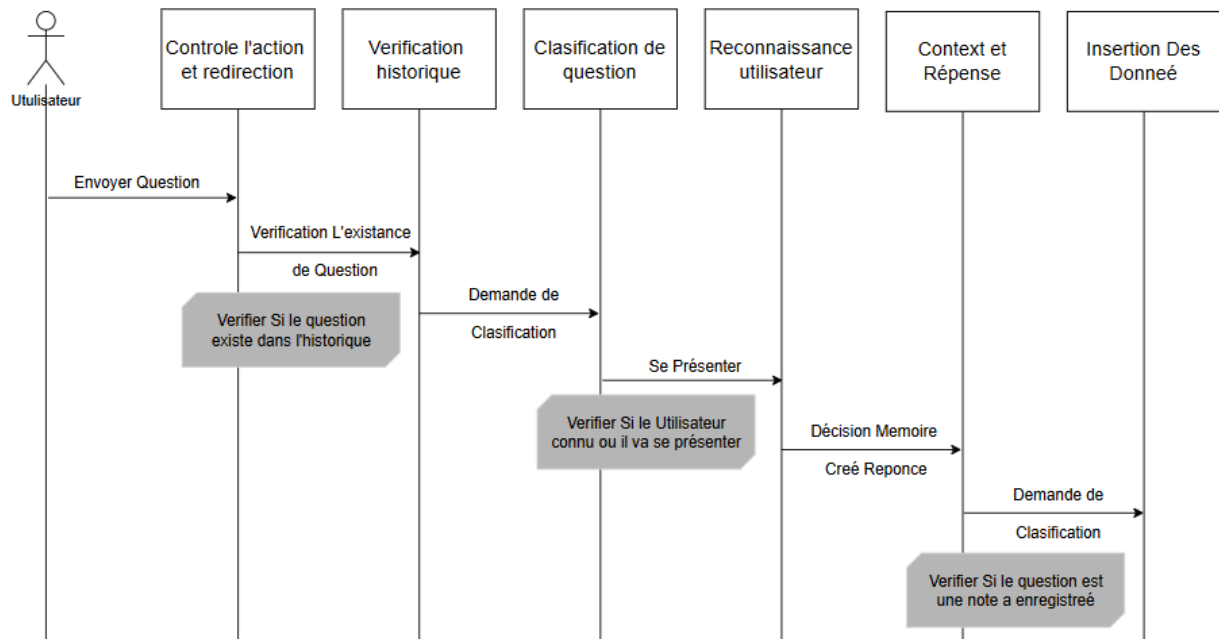


FIGURE III.6 – Webhook 1 — diagramme de séquence (six parties du pipeline)

III.5.1.3 Format de la Requête Entrante

Le webhook accepte les requêtes au format (`user_name` + `question`). Le champ `user_name` est un identifiant unique généré automatiquement par le backend pour chaque utilisateur en cours d'interaction. Cet identifiant est injecté avec la question dans le flux de travail, permettant d'isoler et de gérer les données de chaque utilisateur de manière indépendante dans les environnements multi-utilisateurs.

III.5.1.4 Les six parties du Webhook 1

III.5.1.4.1 Étape 1 — Entrée fonctionnelle

Rôle théorique. Intercepter les **commandes immédiates** (sécurité, média, déplacement) avant tout traitement linguistique coûteux — équivalent à des réflexes opérationnels.

Rôle pratique (n8n). Lecture de `chatInput`, requête *Identité*, nœud **Test** (priorité d'intentions), routage *Switch* (six branches).

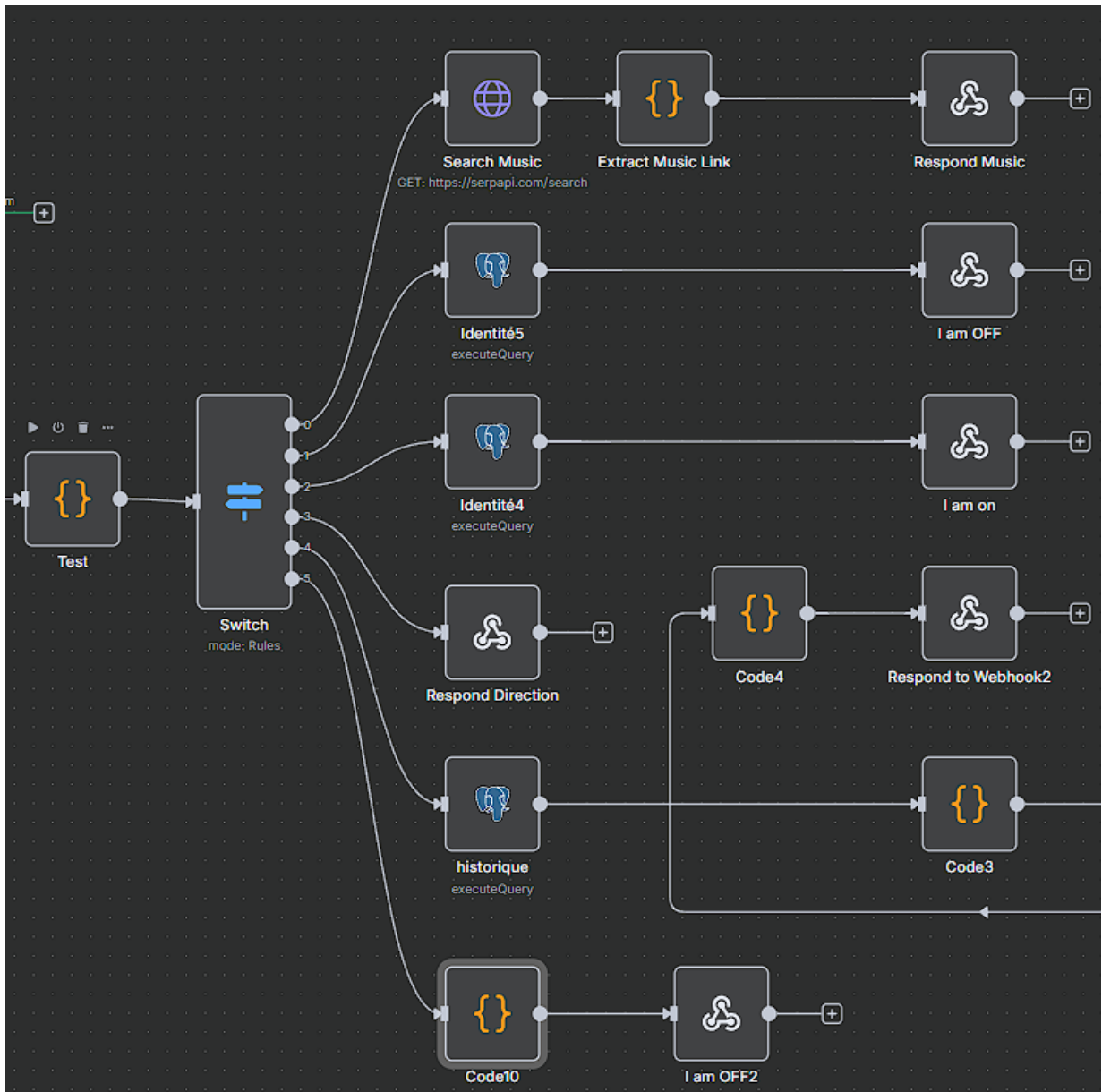
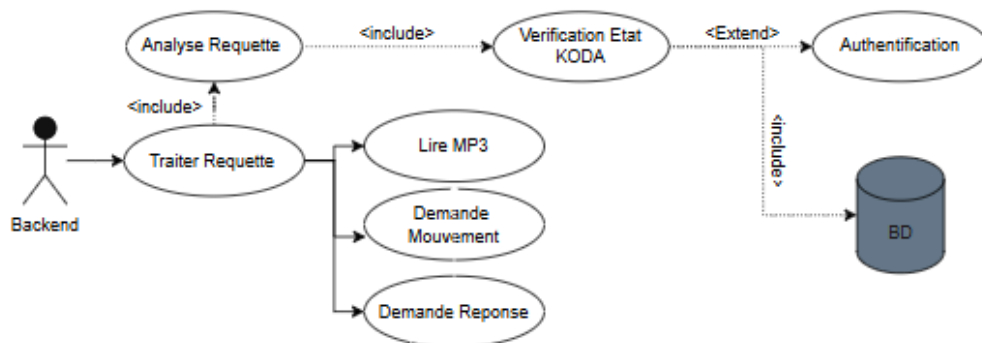
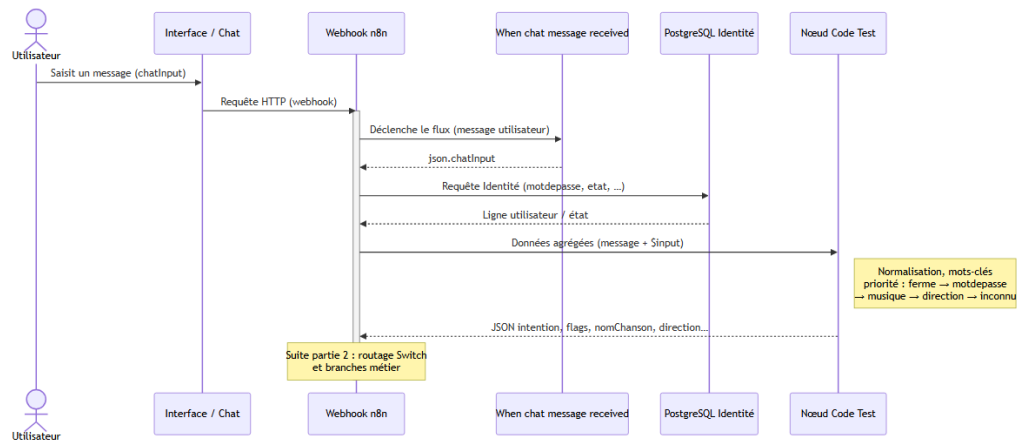
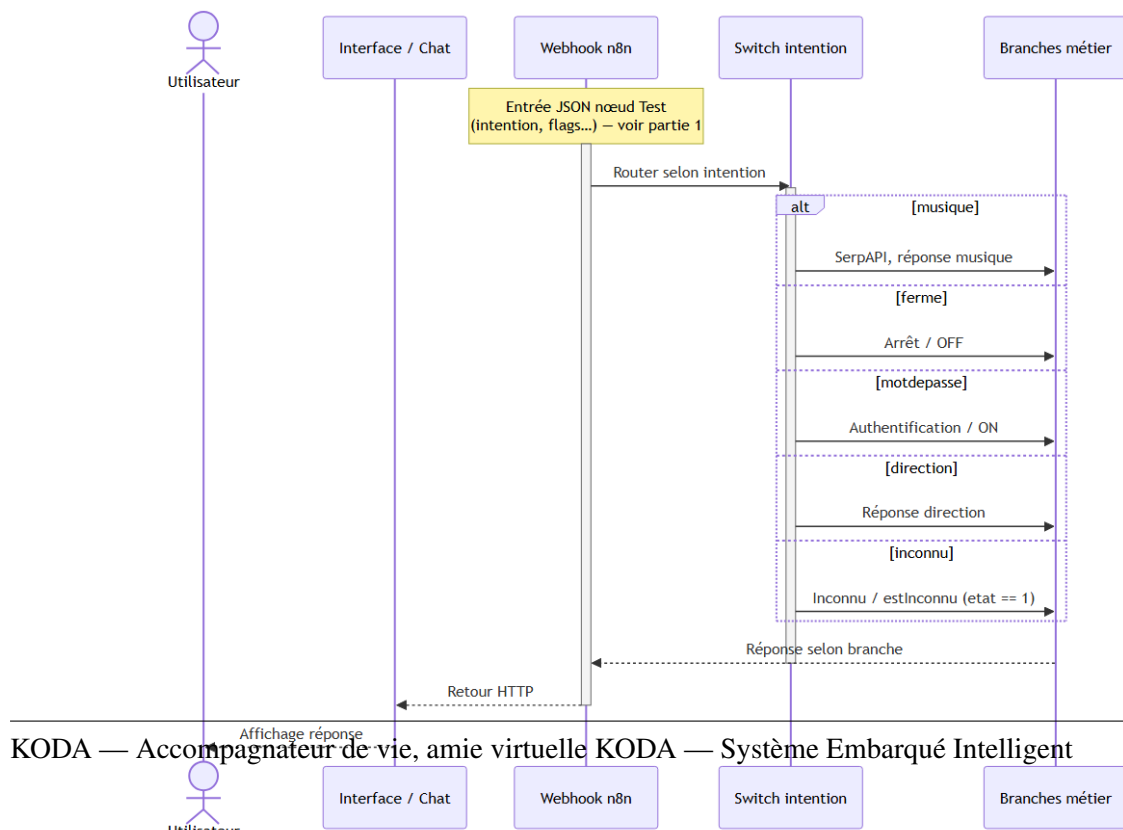
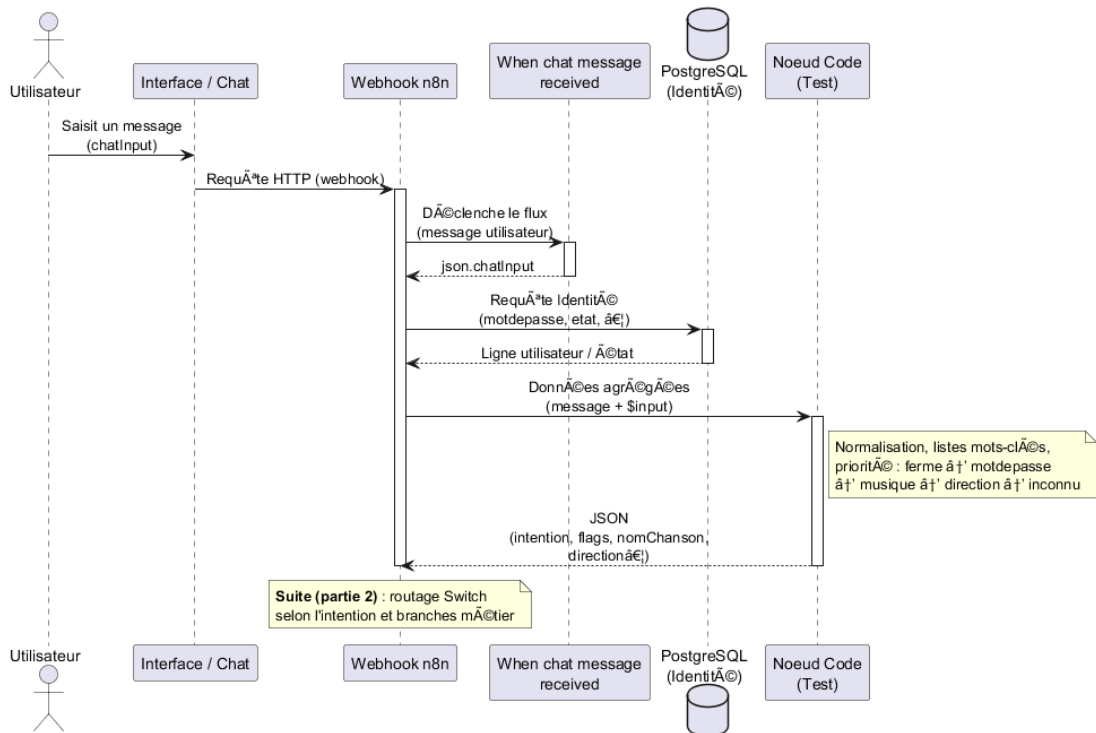
FIGURE III.7 – Webhook 1 — entrée fonctionnelle (nœud *Test*, *Switch*)

FIGURE III.9 – Webhook 1 — cas d'utilisation de l'entrée fonctionnelle

À la réception de `chatInput`, PostgreSQL *Identité* fournit `motdepasse` et `etat`. Le nœud **Test** normalise le texte, applique une **priorité fixe** d'intentions (tableau III.1) et produit un



Block 1 à€" Partie 1 : rÃ©ception, identiÃ© et analyse d'intention



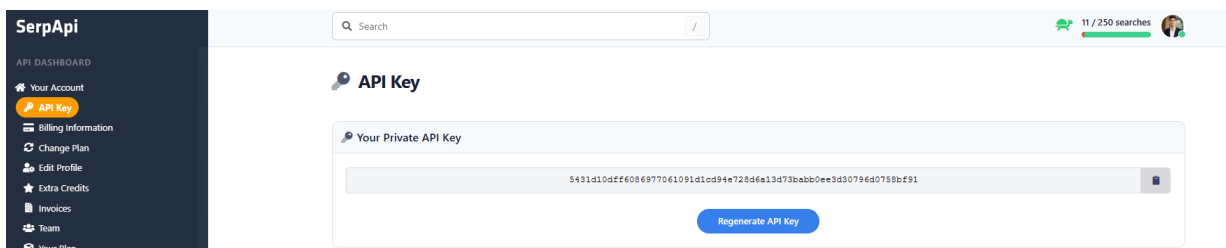
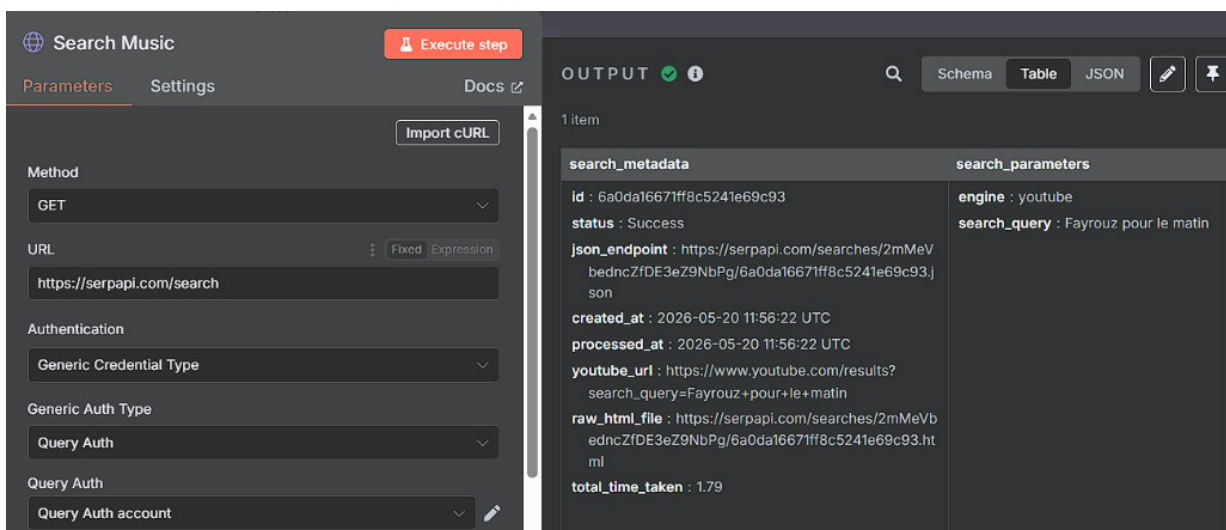
JSON (intention, nomChanson, direction, estInconnu, etc.) pour le *Switch* à six sorties (figure III.5.1.4.1, séquence figure III.8).

TABLE III.1 – Priorité des intentions — nœud *Test*

Ordre	Intention	Règle principale
1	ferme	Mots-clés d'arrêt (<i>off, bye, sleep...</i>), détection includes
2	motdepasse	Égalité stricte avec motdepasse en base (mode protégé)
3	musique	<i>chante / lire</i> + extraction nomChanson
4	direction	Premier terme parmi <i>avant, droite, gauche, stop...</i>
5	inconnu	Défaut si rien d'autre et <code>etat == 1</code>

III.5.1.4.1.1 Branche musique — SerpAPI (moteur YouTube)

SerpAPI (clé API, moteur youtube uniquement) alimente *Search Music* pour obtenir `youtube_url` à partir de `nomChanson`, sans LLM.

FIGURE III.10 – Branche musique — credential SerpAPI et nœud *Search Music*FIGURE III.11 – Exemple `search_query`, `youtube_url`

III.5.1.4.2 Étape 2 — Confrontation à l'historique

Rôle théorique. Réflexion rapide : réutiliser une réponse récente si la question a déjà été traitée (économie cognitive et technique).

Rôle pratique (n8n). Comparaison aux **cinq derniers messages** en PostgreSQL ; renvoi direct ou poursuite vers la classification.

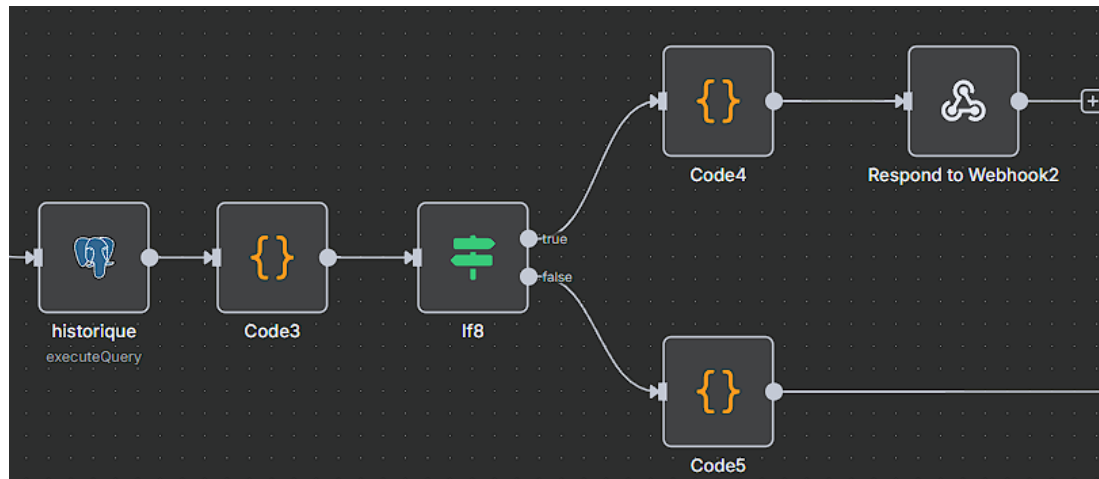


FIGURE III.12 – Confrontation à l'historique — workflow n8n

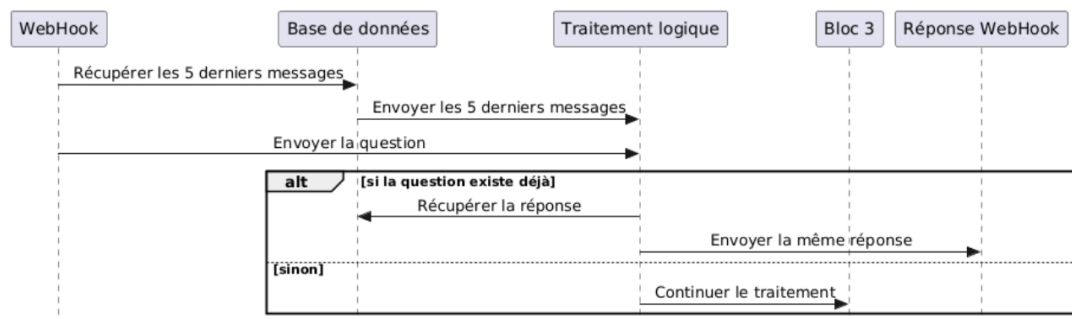


FIGURE III.13 – Confrontation à l'historique — diagramme de séquence

III.5.1.4.3 Étape 3 — Classification sémantique

Rôle théorique. Réflexion lente : comprendre la nature du message et décider si la mémoire profil est nécessaire pour répondre.

Rôle pratique (n8n). Chaîne **Code5** → **Identité1** → **Basic LLM Chain** (OpenRouter / Gemini) → **Code2** ; sortie USE_MEMORY / NO_MEMORY + catégorie (tableau III.2).

TABLE III.2 – Catégories de la classification sémantique

Catégorie	Description	Mémoire
presentation	Auto-présentation, extraction du nom	NO_MEMORY
information_personnelle	Vie privée ou entourage ; demande → mémoire, nouvelle information → enregistrement aval	selon cas
aboutme	Robot KODA : question sur KODA → USE_MEMORY ; définition par l'utilisateur → NO_MEMORY	selon cas
note	Rappels, tâches (<i>rappelle-moi...</i>)	NO_MEMORY
technical	Code, maths, informatique	NO_MEMORY
information_général	Culture et conversation, sauf lien personnel (<i>mon ami...</i>)	souvent NO_MEMORY

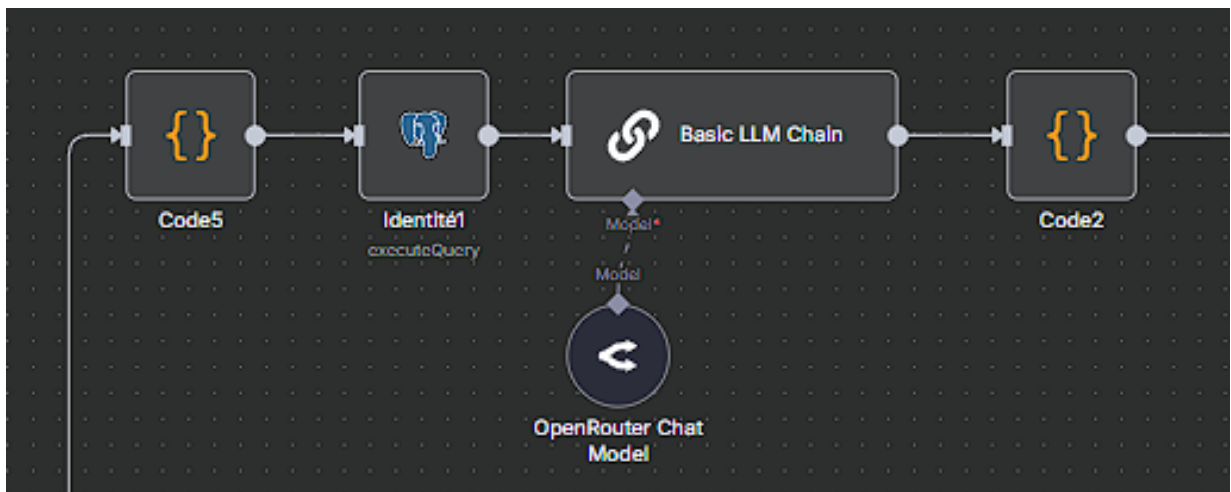


FIGURE III.14 – Classification sémantique — chaîne n8n

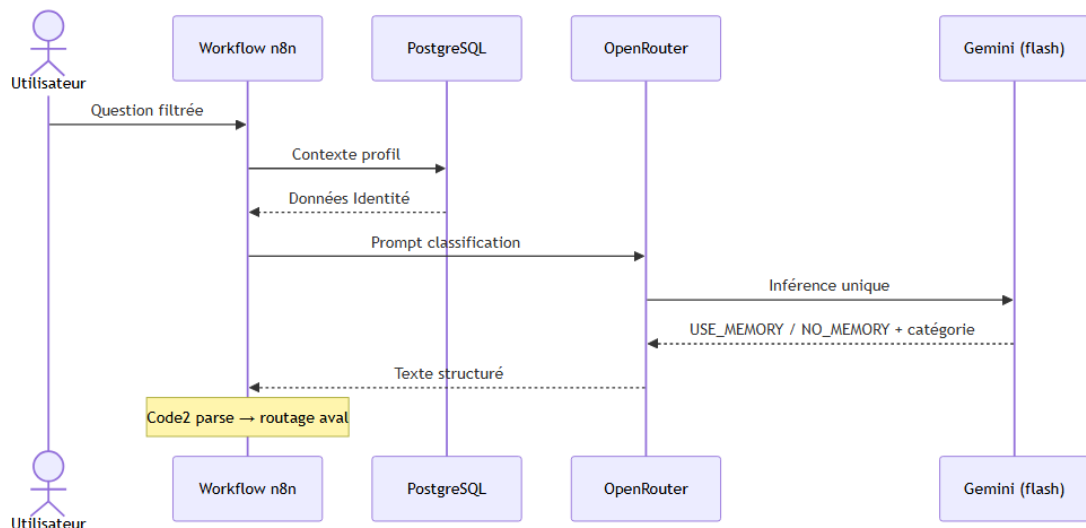


FIGURE III.15 – Vue synthétique OpenRouter / Gemini

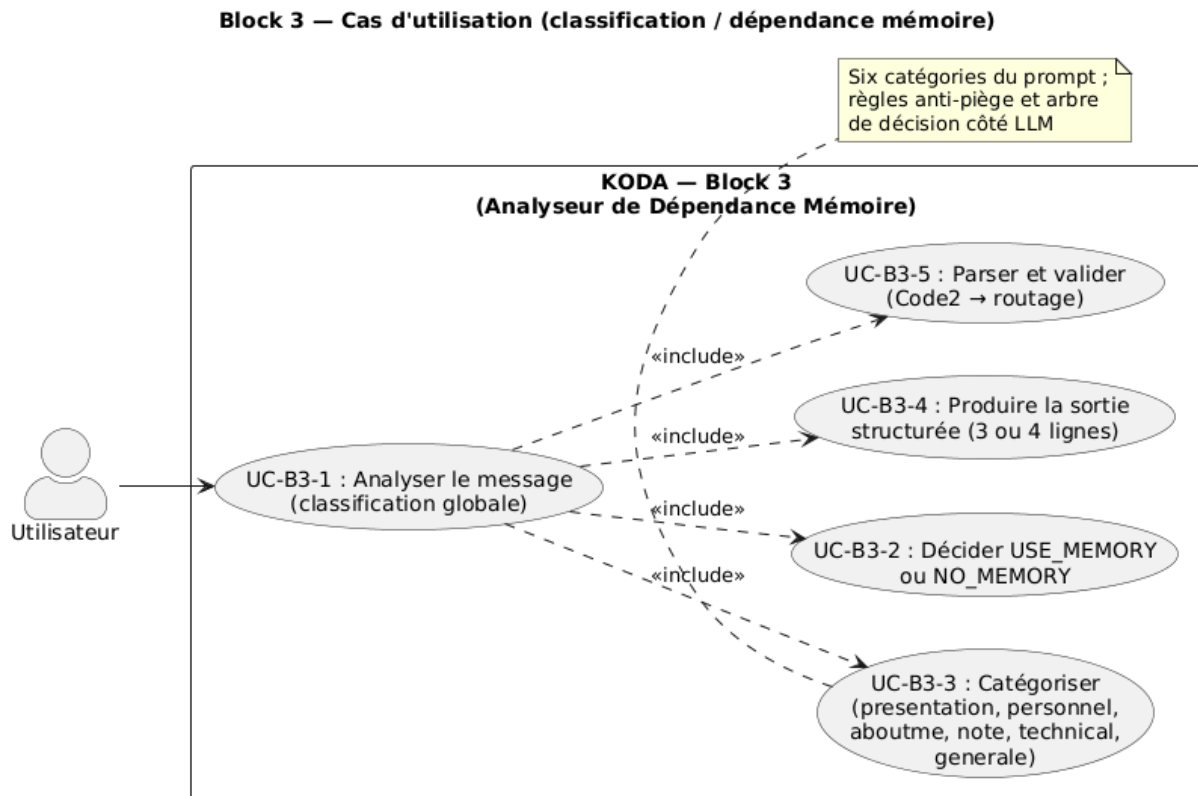


FIGURE III.16 – Classification sémantique — cas d'utilisation

III.5.1.4.3.1 Basic LLM Chain plutôt qu'AI Agent

L'implémentation initiale utilisait un nœud *AI Agent* (boucles d'outils et raisonnement multi-passes). En production, la **latence** devenait incompatible avec une réponse conversationnelle fluide. La **Basic LLM Chain** — une seule inférence pilotée par le prompt et parsée par **Code2** — offre un **temps de réponse adapté** à cette étape de classification.

III.5.1.4.3.2 OpenRouter (Gemini flash)

OpenRouter centralise l'accès au modèle `google/gemini-2.5-flash-lite-preview-09-2025` (credential n8n).

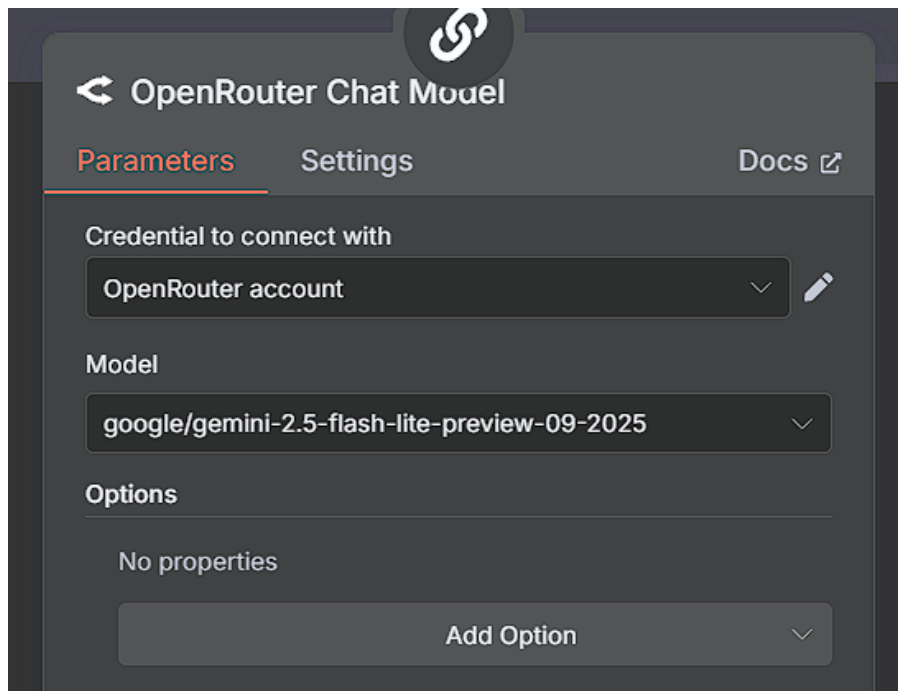


FIGURE III.17 – Credential OpenRouter et modèle Gemini

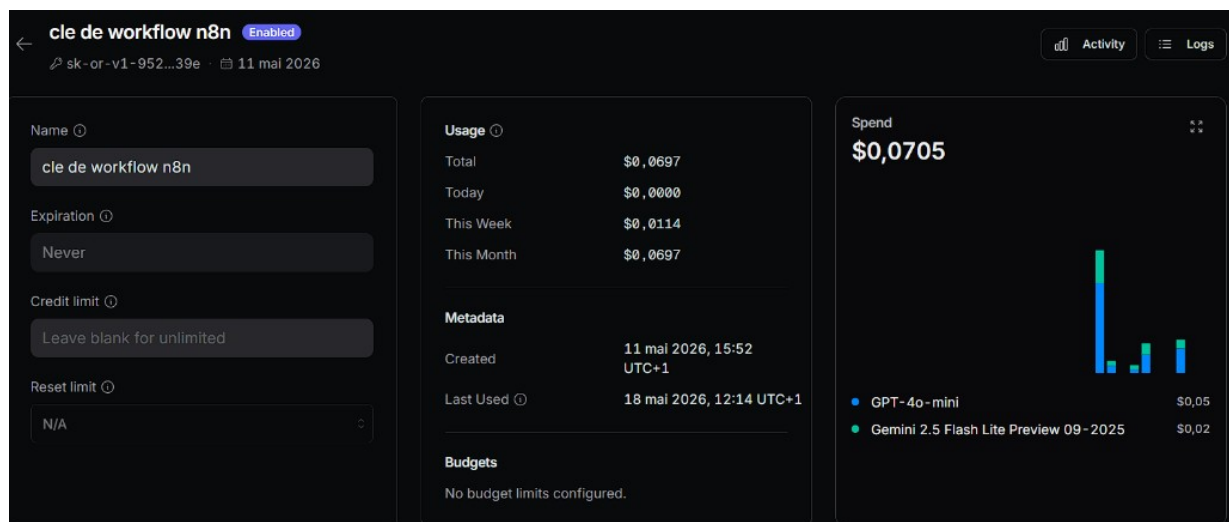
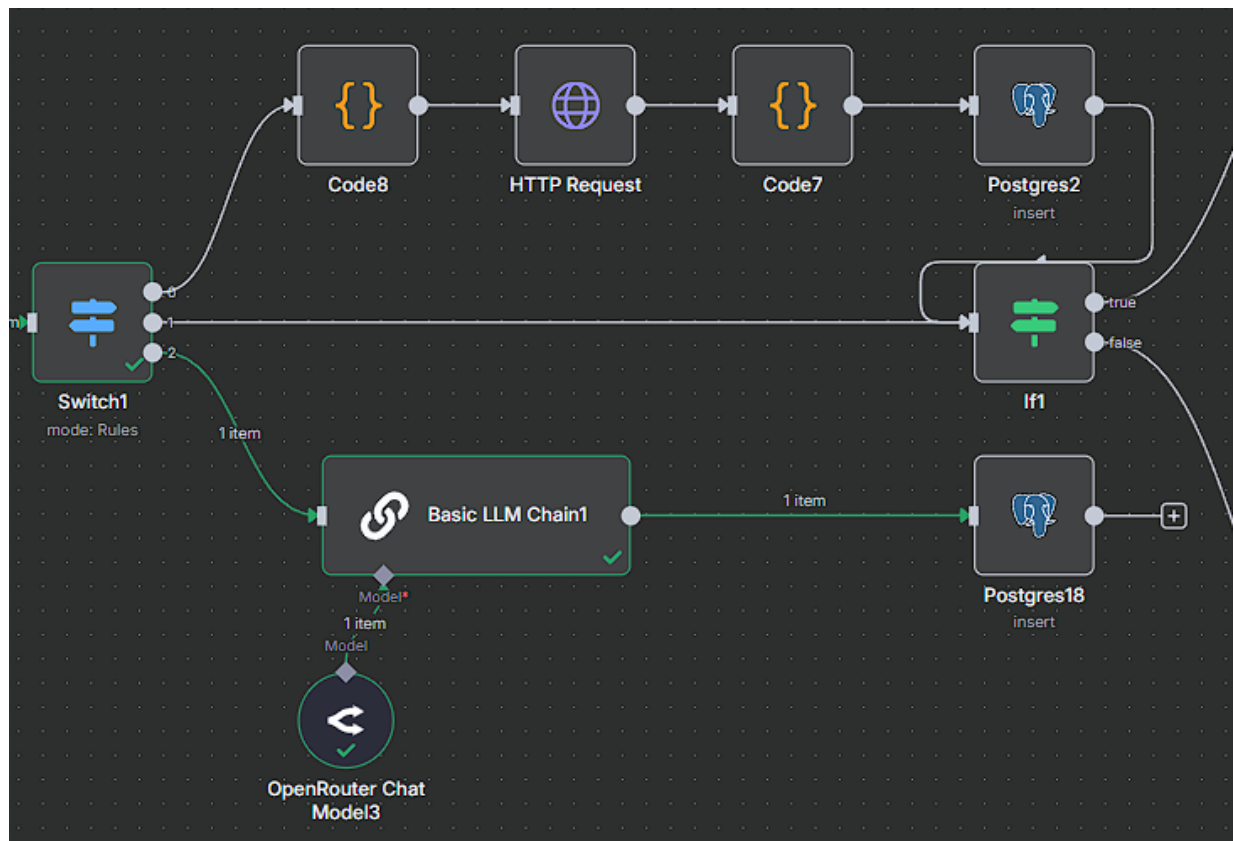


FIGURE III.18 – Suivi consommation API workflow n8n

III.5.1.4.4 Étape 4 — Gestion utilisateur (reconnaissance et enregistrement)

Rôle théorique. Assurer que KODA connaît son interlocuteur avant de dialoguer en profondeur : accueillir, enregistrer ou relancer une présentation.

Rôle pratique (n8n). *Switch1* route selon `questionType` et `personne` vers enregistrement (`presentation`), accueil LLM (`unkonu`) ou étape 5 directe (`personne connue`).

FIGURE III.19 – Gestion utilisateur — *Switch1*TABLE III.3 – Les trois scénarios du *Switch1*

Condition	Action	Suite
questionType = presentation	<i>Code8</i> → API profil → <i>Code7</i> → INSERT PostgreSQL	Étape 5
personne = unkonu	<i>Basic LLM Chain</i> — invitation à se présenter	Pas d'étape 5
Personne connue (défaut)	Aucun enregistrement ni accueil LLM	Étape 5 directe

III.5.1.4.4.1 Scénario 1 — presentation (nouvel utilisateur)

III.5.1.4.4.2 Scénario 2 — unkonu (invitation à se présenter)

Exemple : `personne = unkonu`, `questionType = information_generale` (« *Bonjour, comment tu t'appelles ?* »).

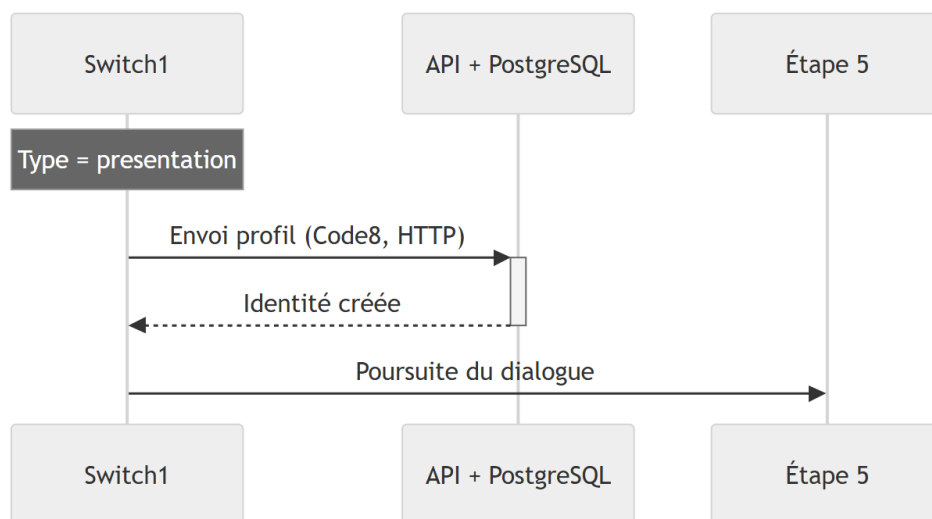


FIGURE III.20 – Séquence 1 — presentation → API + PostgreSQL → étape 5

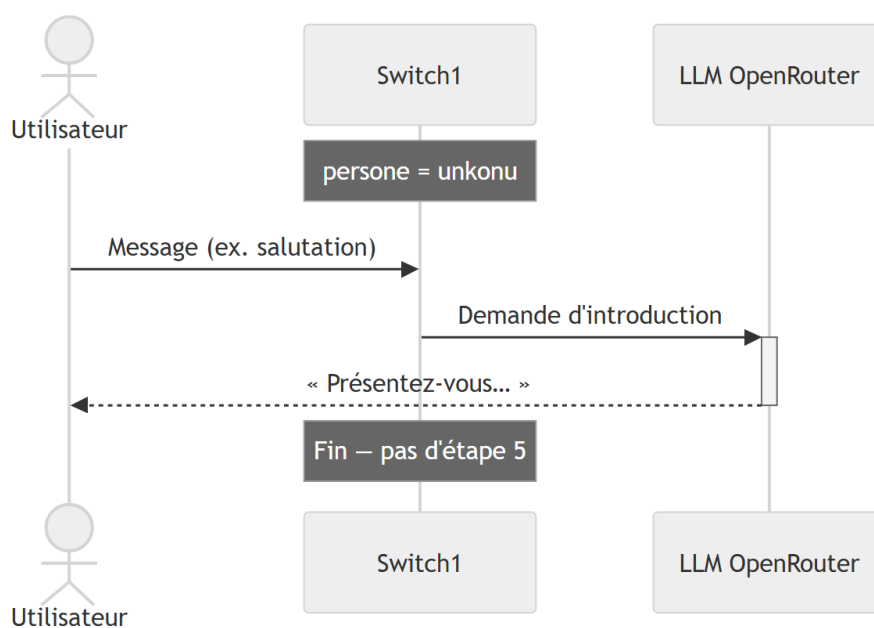
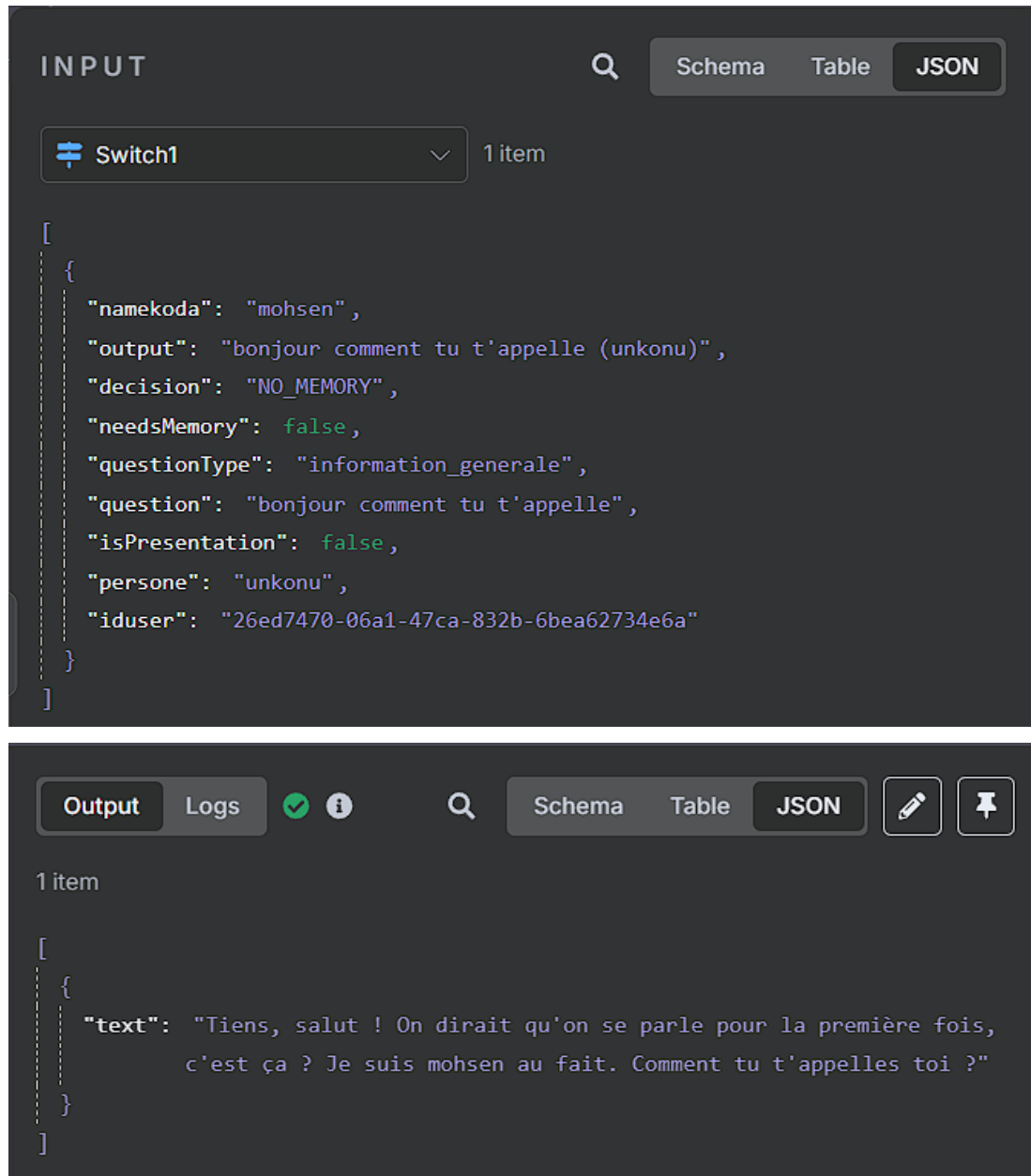


FIGURE III.21 – Séquence 2 — unkonu → accueil LLM

FIGURE III.22 – Cas unknow — entrée *Switch1* et sortie LLM

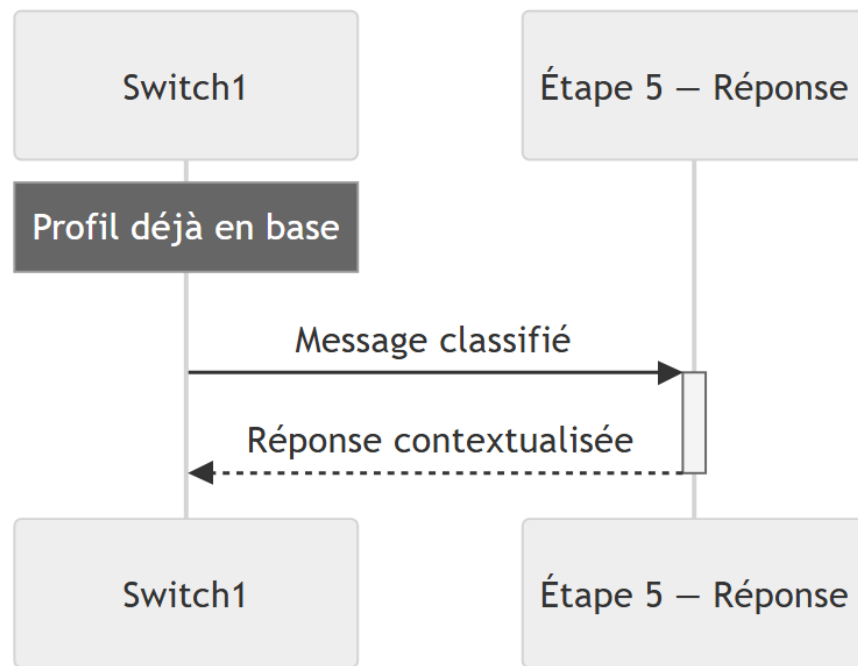


FIGURE III.23 – Séquence 3 — personne connue → étape 5

III.5.1.4.4.3 Scénario 3 — personne connue

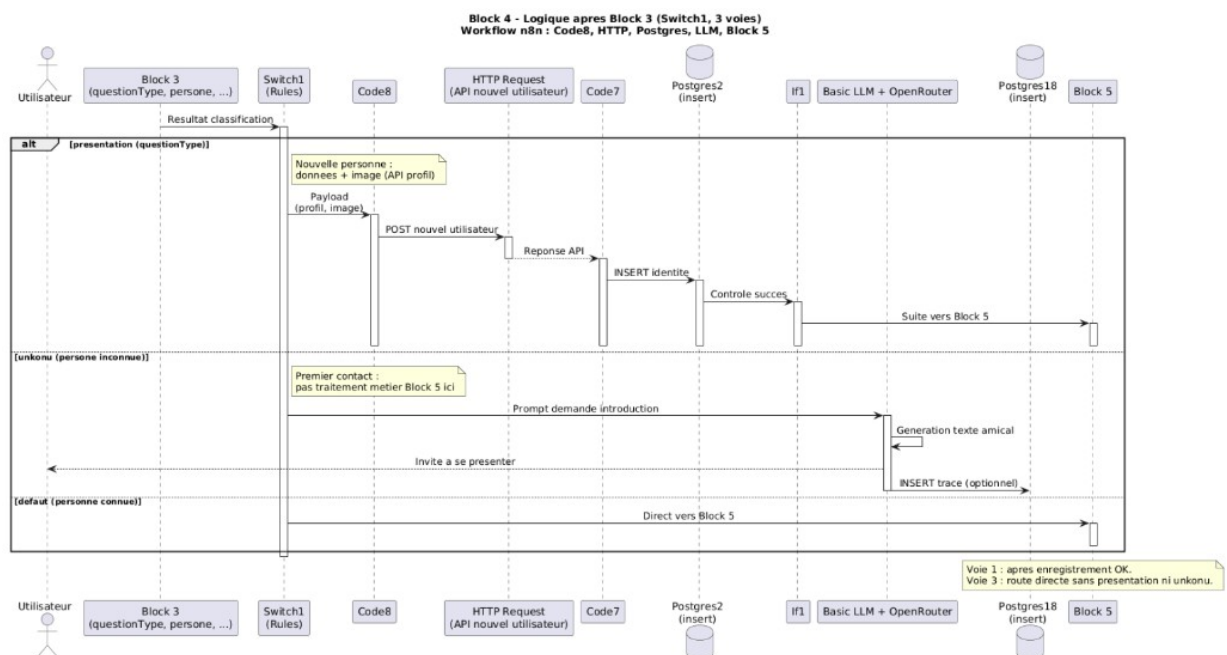


FIGURE III.24 – Gestion utilisateur — séquence complète détaillée

III.5.1.4.5 Étape 5 — Réponse contextualisée

Rôle théorique. Formuler la réponse en tenant compte du lien avec l'utilisateur (mémoire profonde ou dialogue général), avec la personnalité de KODA.

Rôle pratique (n8n). Selon use_memory, l'AI Agent charge le contexte PostgreSQL adapté et

renvoie {output}; l'étape 6 enregistre l'échange.

TABLE III.4 – Les deux branches de l'étape 5

Branch	Contexte chargé	Cas typique
USE_MEMORY	Profil personnel, historique relationnel, identité KODA	<i>Quel est mon âge ?</i> , souvenir sur l'utilisateur
NO_MEMORY	100 derniers échanges ; insert si nouvelle info (note, présentation...)	Question générale, culture, technique

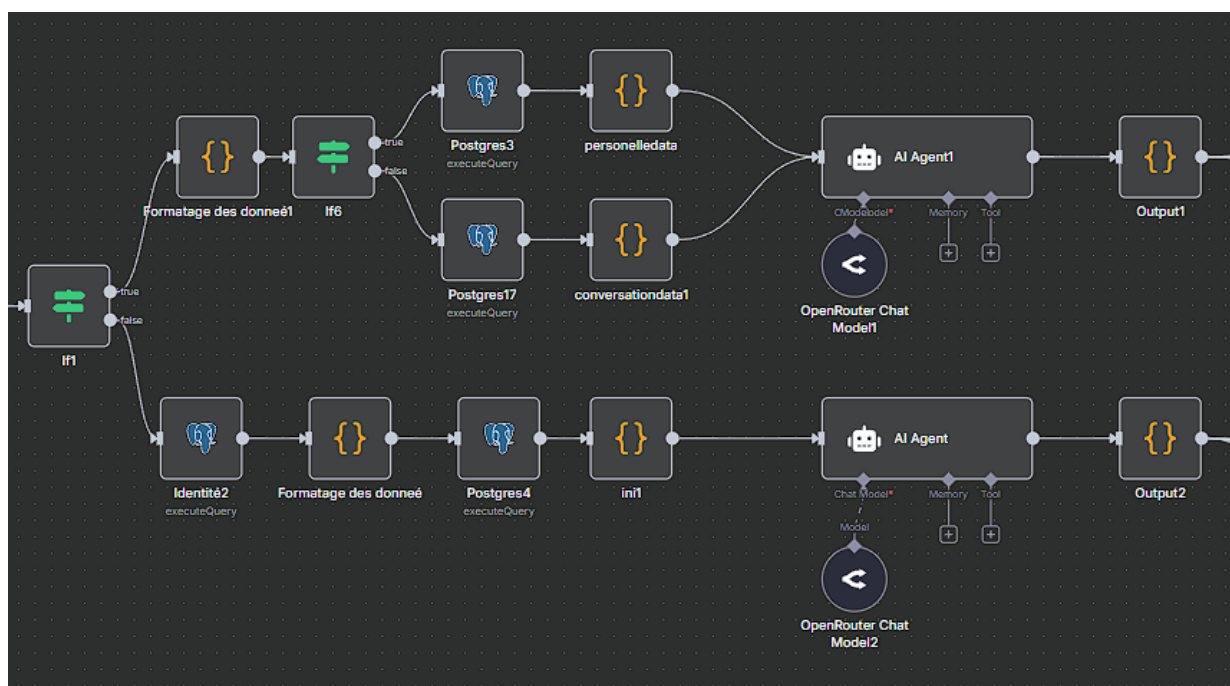


FIGURE III.25 – Réponse contextualisée — vue des deux branches

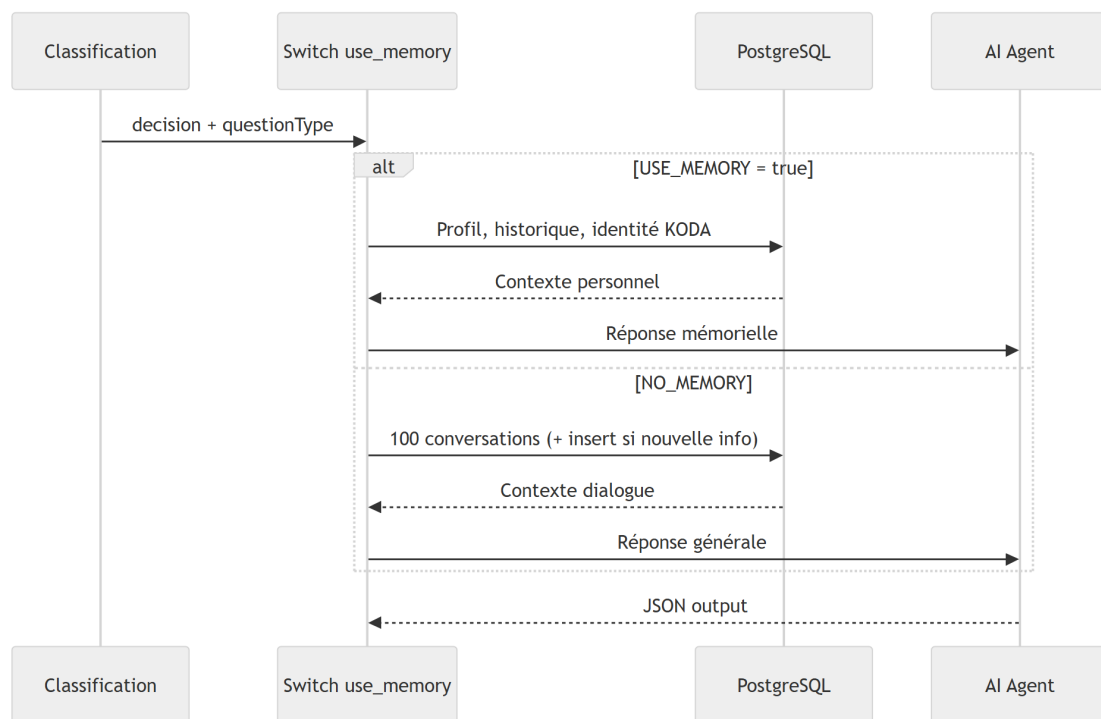


FIGURE III.26 – Séquence simplifiée étape 5

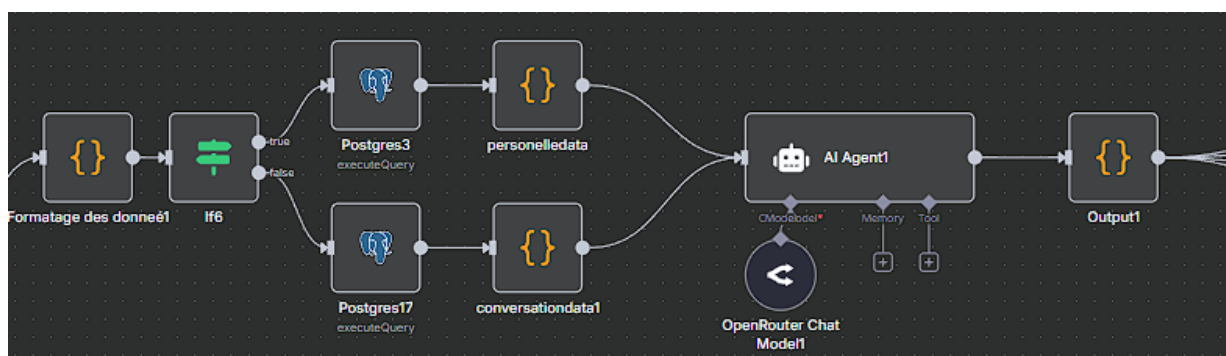


FIGURE III.27 – Branche mémorielle — récupération profil et génération

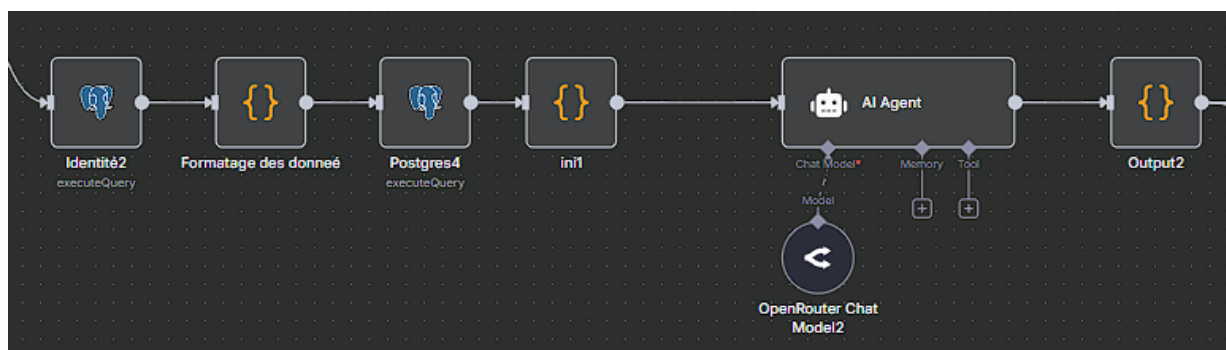


FIGURE III.28 – Branche générale — contexte conversationnel

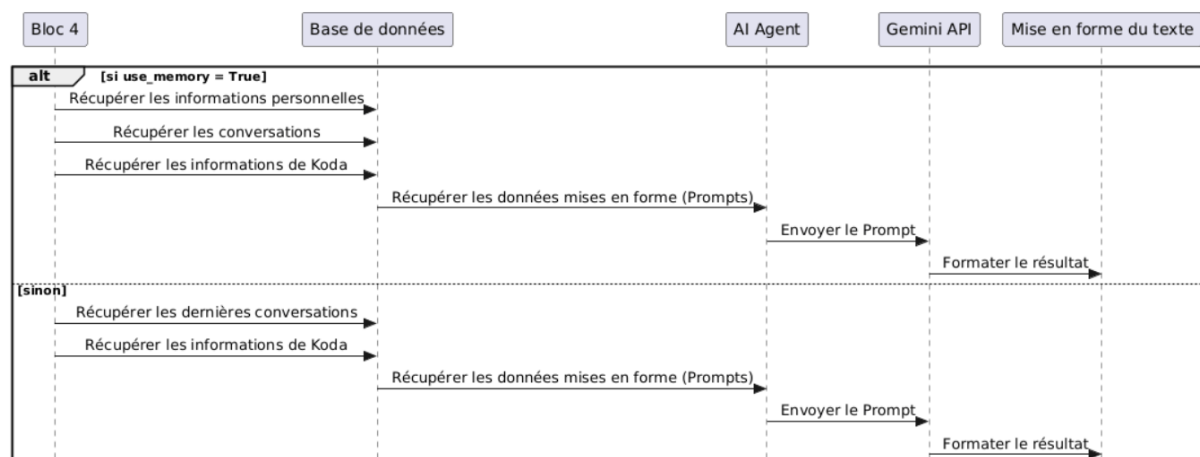


FIGURE III.29 – Diagramme de séquence détaillé — étape 5

Les prompts système de l'*AI Agent* restent configurés dans n8n (non reproduits dans ce rapport). La sortie alimente l'**étape 6** (persistance).

III.5.1.4.6 Étape 6 — Notes et mémoire persistante

Rôle théorique. Clôturer le cycle en conservant l'échange et les informations nouvelles (notes, profil) pour les interactions futures — équivalent à la fixation mémorielle après un dialogue.

Rôle pratique (n8n). Si `questiontype = note`, structuration LLM puis INSERT PostgreSQL (conversations, historique, notes, `personal_info` selon le cas) et réponse HTTP finale.

L'étape 6 intervient **après** la génération de la réponse (étape 5). Sans cette étape, KODA ne conserverait ni le fil de la conversation ni les rappels planifiés : chaque message serait traité comme une première rencontre. Le bloc se divise en **deux parties** : une extraction **conditionnelle** des notes, puis une persistance **systématique** de l'échange.

III.5.1.4.6.1 Partie 1 — Extraction des notes (conditionnelle).

Lorsque la classification sémantique (étape 3) a produit `questiontype = note`, l'utilisateur exprime une intention temporelle — rappel, rendez-vous, alarme — souvent en langage naturel imprécis. Le pipeline ne stocke pas la phrase brute : il en extrait une **date**, une **heure** et un **libellé d'événement** exploitables par la table notes.

Le workflow n8n teste d'abord le type de question ; seules les requêtes `note` activent une *Basic LLM Chain* dédiée, alimentée par le message, la réponse déjà générée, l'identifiant utilisateur et le contexte de classification. Le modèle renvoie un JSON structuré (`date_evenement`, `evenement`, éventuellement `format_valide`) utilisé en partie 2. Pour tout autre type, cette branche est ignorée et le flux passe directement à la persistance.

Exemple : « *Rappelle-moi à 9 heures, j'ai un rendez-vous chez le docteur* » devient un enregistrement horodaté normalisé, permettant à KODA de relancer l'utilisateur au moment voulu.

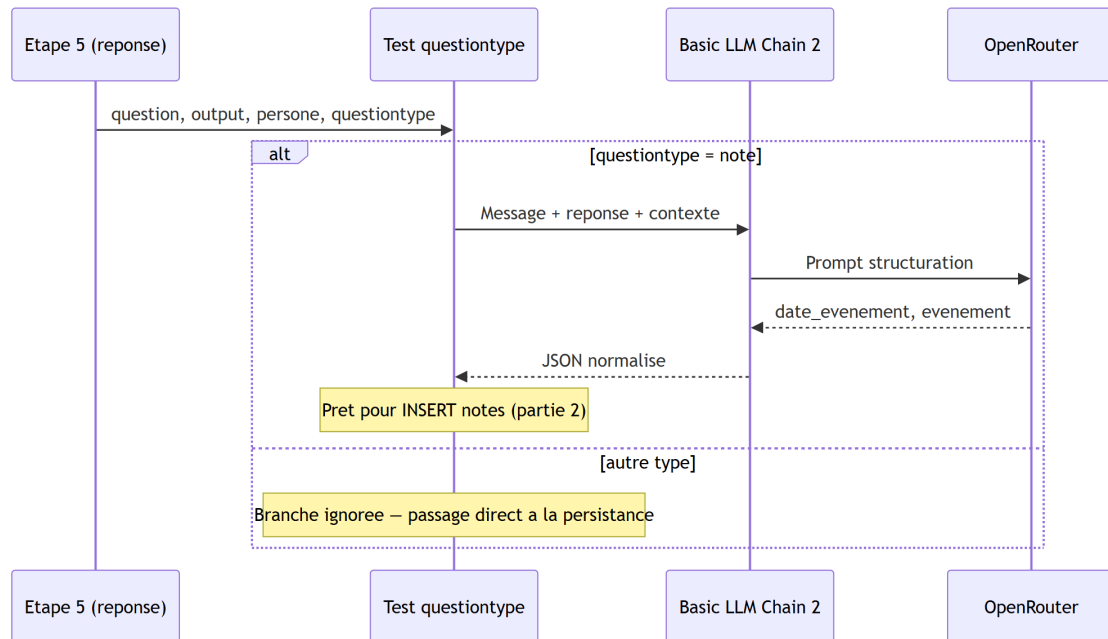


FIGURE III.30 – Extraction des notes — test questiontype et structuration LLM

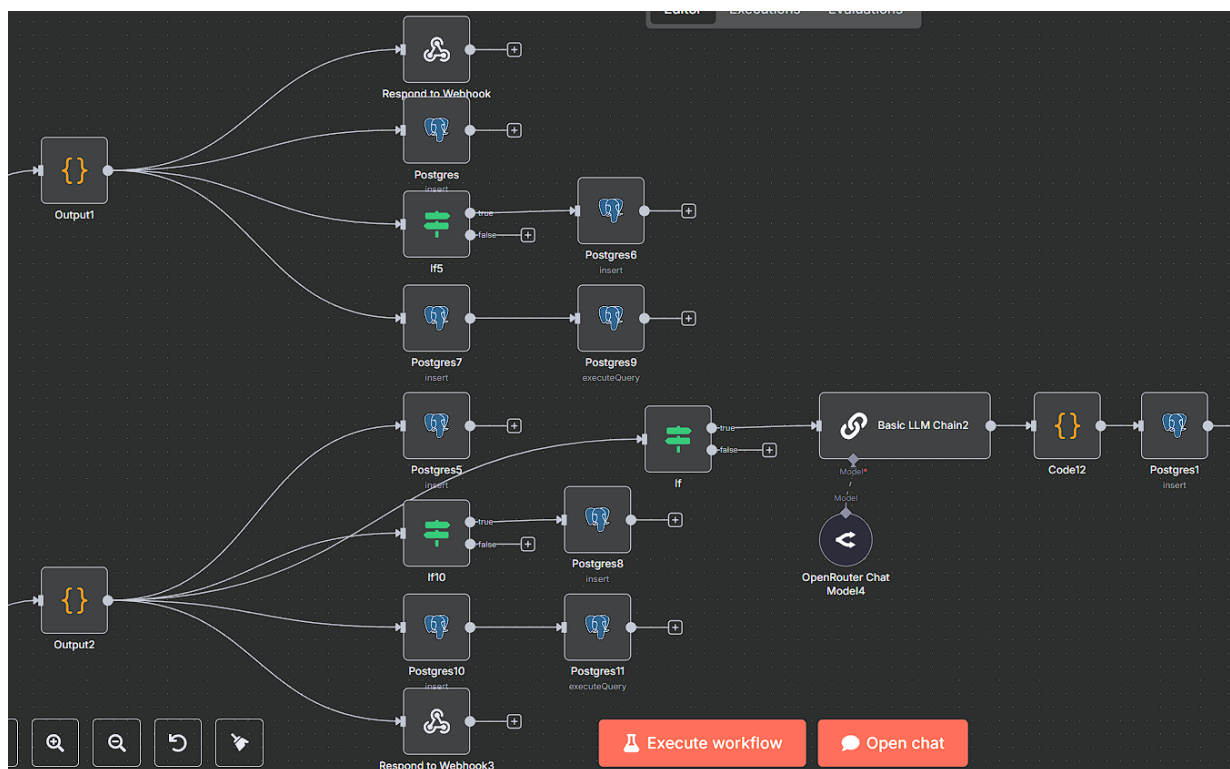


FIGURE III.31 – Sous-workflow n8n — branche note (LLM Chain 2)

La figure III.5.1.4.6.1 montre l'agrégat JSON transmis à la chaîne LLM ; la figure III.5.1.4.6.1 illustre la sortie structurée prête pour l'insertion en base (utilisateur ouss ema, rendez-vous médical) :

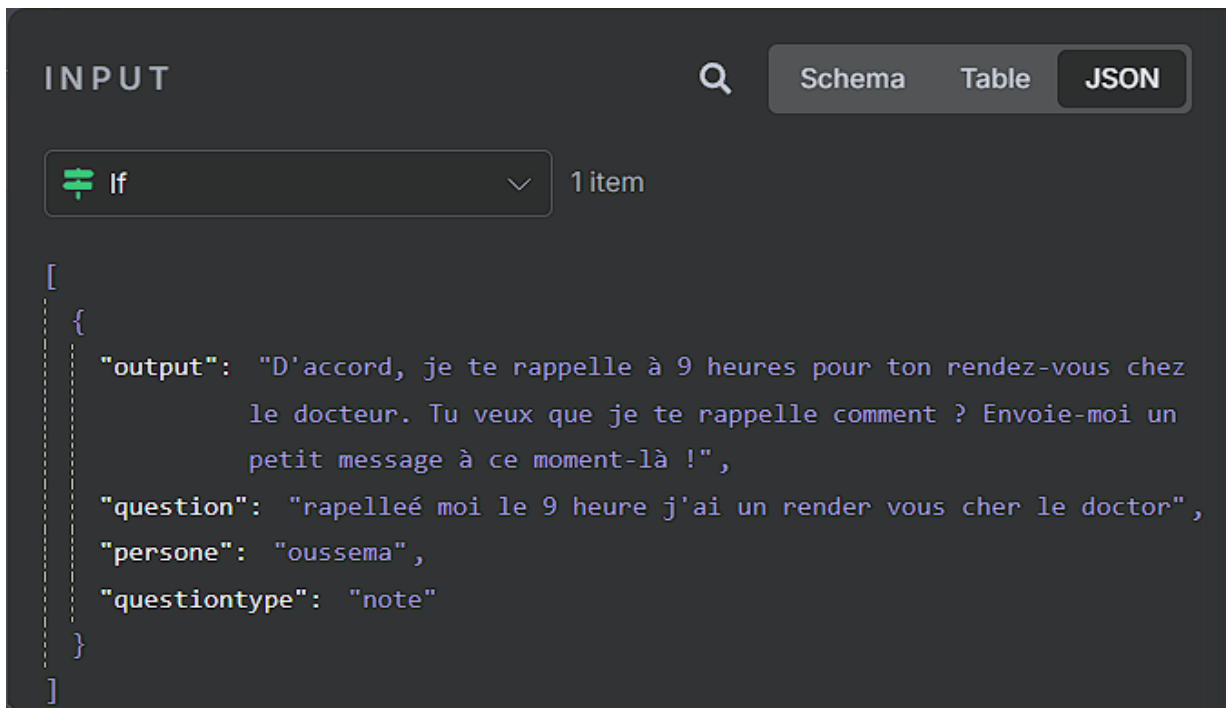


FIGURE III.32 – Entrée n8n — question, output, questiontype = note



FIGURE III.33 – Sortie LLM — date_evenement et evenement

III.5.1.4.6.2 Partie 2 — Persistance et clôture du webhook.

Quelle que soit la branche empruntée à l'étape 5 (USE_MEMORY ou NO_MEMORY), la deuxième partie enregistre l'échange et clôt le Webhook 1. Le tableau III.5 résume les écritures PostgreSQL ; la figure III.5.1.4.6.2 en donne la vue séquentielle, complétée par le schéma de classes (figure III.5.1.4.6.2).

TABLE III.5 – Opérations de persistance — étape 6 (partie 2)

Ordre	Opération	Table
1	Journaliser la question et la réponse de KODA	conversations
2	Enregistrer une information personnelle nouvelle (si classification about_me ou information_personnel)	personal_info
3	Insérer le rappel structuré (si note, après partie 1)	notes
4	Ajouter l'échange à l'historique court et retirer le plus ancien au-delà de 5 messages	historique
5	Renvoyer la réponse JSON à l'appelant HTTP	<i>Respond to Webhook</i>

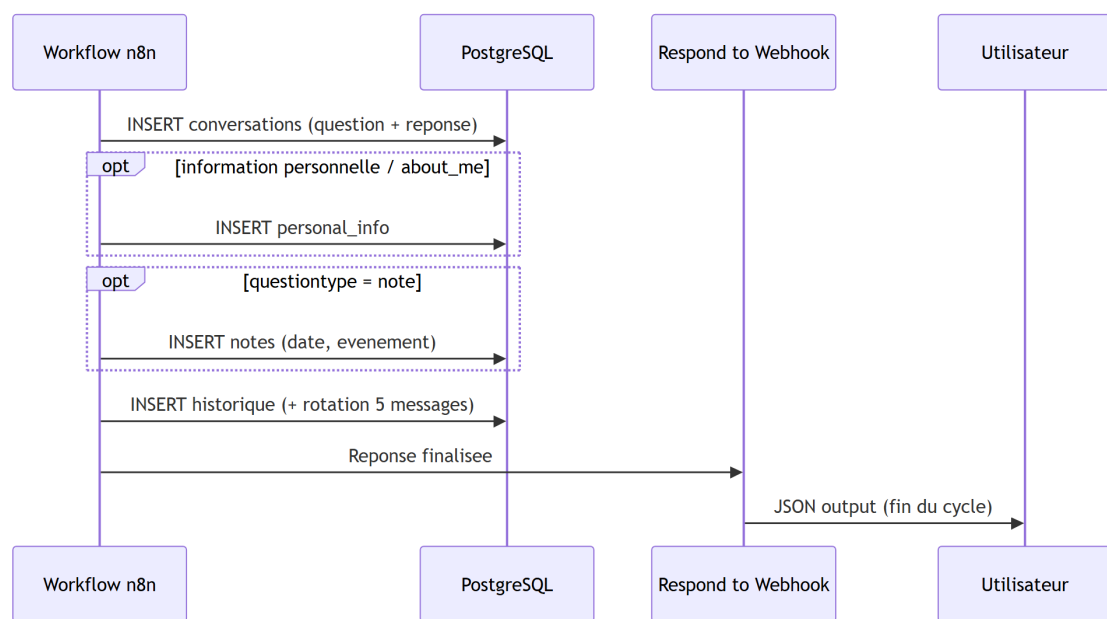


FIGURE III.34 – Persistance et clôture — écritures PostgreSQL et réponse webhook

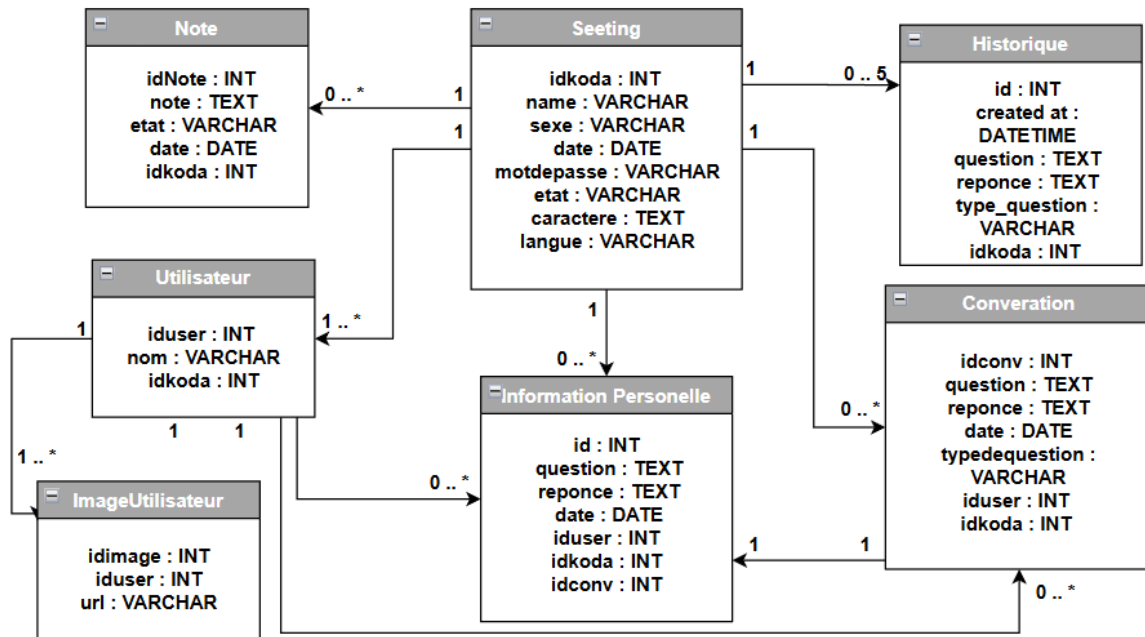


FIGURE III.35 – Schéma de classes — tables conversations, historique, notes

III.5.1.5 Conclusion du Webhook 1

Le **Webhook 1** constitue le cœur opérationnel de KODA en interaction quotidienne : à chaque message reçu (`user_name`, `question`), il enchaîne analyse, décision, génération et mémorisation dans un pipeline unique orchestré par `n8n`. Organisé en **six parties** (figure III.5.1.2), ce flux traduit la théorie du double traitement : traitement rapide via l'historique, puis traitement approfondi (classification, identité, réponse, persistance).

L'**entrée fonctionnelle** assure sécurité et actions immédiates. La **confrontation à l'historique** limite la redondance. La **classification sémantique** structure le message. La **gestion utilisateur connu ou inconnu** matérialise l'accueil humain. La **réponse contextualisée** produit la valeur conversationnelle. Les **notes et la mémoire persistante** clôturent le cycle en base de données.

Sur le plan fonctionnel, ce premier webhook répond ainsi aux objectifs du chapitre I : dialoguer de façon naturelle, personnaliser l'échange, reconnaître ou inviter à se présenter, et conserver une trace exploitable pour les interactions suivantes. Il ne constitue pas une fin en soi : les journaux alimentés ici (conversations, historique, notes, profils) sont la matière du **Webhook 2** (consolidation nocturne) et, indirectement, du **Webhook 3** (proactivité). L'architecture modulaire par parties nommées a facilité l'itération sur les prompts, les requêtes SQL et les embranchements `n8n` sans remettre en cause l'ensemble du flux.

Des **limites** demeurent inhérentes à un prototype de fin d'études : dépendance à des services externes (OpenRouter, SerpAPI, hébergement base de données), sensibilité à la qualité des prompts

et à la latence réseau, et nécessité de tests utilisateurs plus larges sur les cas limites (fautes d’orthographe, multilingue, bruit ambiant côté robot). Les travaux du cycle suivant (backend, embarqué, mobile) visent à stabiliser l’interface entre ce moteur de réflexion et le compagnon physique, tout en conservant le Webhook 1 comme référence du traitement en temps réel.

III.5.2 Deuxième Flux — Webhook 2 : Filtre et Synthèse Journalière

Rôle théorique. Consolider la journée en mémoire durable — analogue à la fixation mémorielle nocturne (Walker & Stickgold, 2006).

Rôle pratique (n8n). Trigger backend → collecte PostgreSQL → *AI Agent Mod Thinking When Koda Sleep* → INSERT accompagnement.

Le **Webhook 1** répond en temps réel et ne conserve qu’une fenêtre courte (historique, cinq messages). Le **Webhook 2** complète ce traitement immédiat : une fois par jour (typiquement le soir), le **Distributeur de services** déclenche un appel HTTP qui lance le workflow n8n de synthèse. Pour chaque utilisateur actif, le flux relit les échanges accumulés et produit une **fiche structurée** enregistrée dans PostgreSQL — base de la personnalisation des jours suivants.

III.5.2.1 Enchaînement fonctionnel

Le processus se décline en trois phases (figure III.5.2.1, séquence figure III.5.2.1) :

TABLE III.6 – Phases du Webhook 2

Phase	Étape	Rôle
1	Collecte	Identifier les utilisateurs actifs et charger profil, conversations et informations du jour (PostgreSQL).
2	Analyse	Agrégation des données en prompt unique ; l’ <i>AI Agent Mod Thinking When Koda Sleep</i> (Gemini via n8n) produit la fiche en quatre dimensions (tableau III.7).
3	Persistance	Parsing de la sortie modèle et INSERT dans accompagnement (clé iduser).

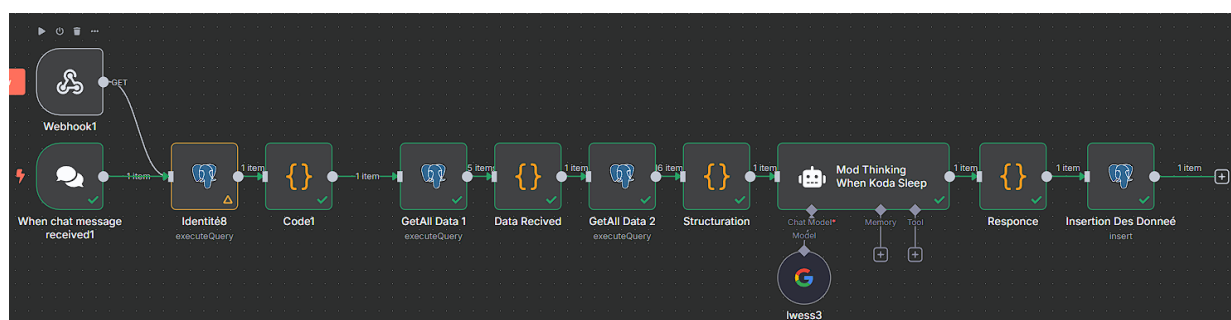
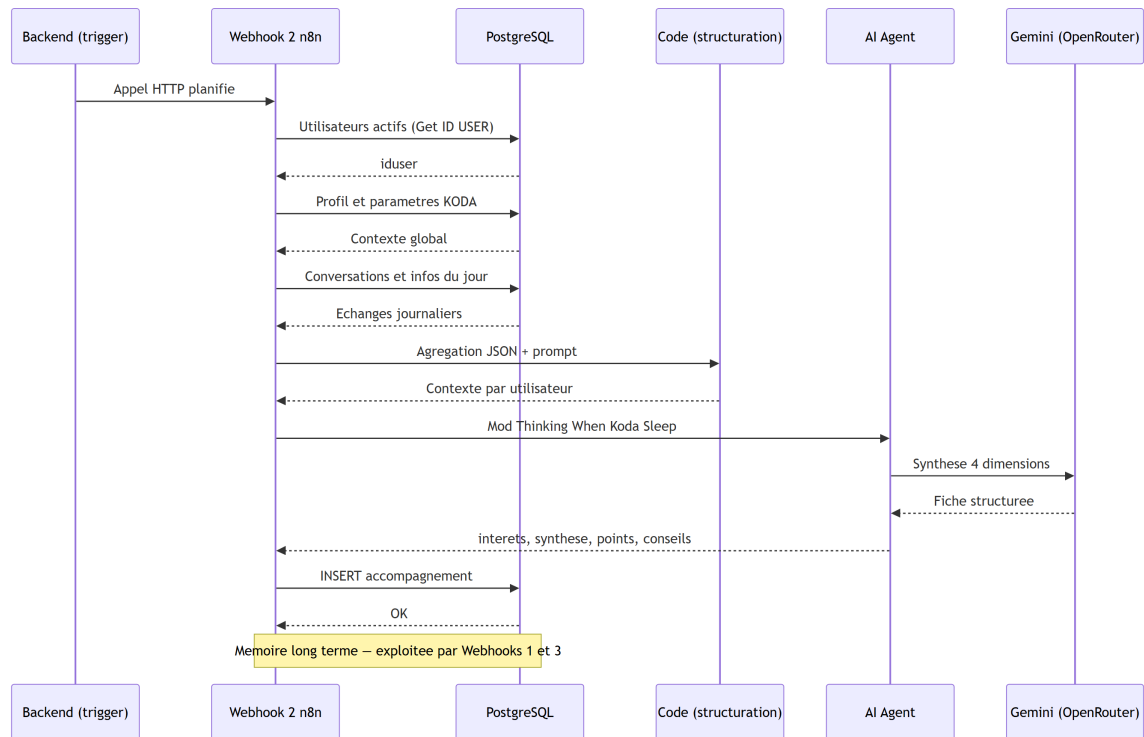


FIGURE III.36 – Webhook 2 — workflow n8n (collecte, synthèse, insertion)**FIGURE III.37** – Webhook 2 — séquence de synthèse journalière

III.5.2.2 Contenu de la fiche journalière

Chaque exécution génère une entrée couvrant **quatre dimensions complémentaires**, stockées dans accompagnement :

TABLE III.7 – Quatre dimensions de la synthèse Webhook 2

Champ	Contenu
interets	Sujets qui engagent l'utilisateur, déduits des thématiques abordées.
synthese	Résumé narratif de la journée (mémoire épisodique condensée).
point_cles	Faits importants : décisions, projets, problèmes évoqués.
conseils	Recommandations personnalisées pour les interactions futures.

OUTPUT

Schema Table JSON

To make sure expressions after this node work, return the input items that produced each output item. [More info](#)

1 item

A interets La nature, l'identité et les limites du robot (personnalité, émotions, rêves, corps, capacité à faire des erreurs, évolution). • La mémoire du robot et sa capacité à se souvenir des interactions passées et des informations sur l'utilisateur (nom, âge, entourage). • La technologie, l'intelligence artificielle et ses concepts connexes (comparaison avec d'autres IA, termes techniques, langage de programmation, outils). • Les préférences et opinions du robot (musique, sport, sujets d'actualité ou sensibles). • Des informations personnelles et académiques (PFE, études, nationalité, spécialité, etc....

A synthese L'utilisateur est manifestement **curieux** et **testeur**, explorant les capacités et les limites de l'IA, en particulier sa mémoire et sa capacité à interagir de manière humaine. Il adopte un ton globalement **amical** et cherche à établir une connexion personnalisée avec le robot. Il manifeste un intérêt prononcé pour la **technologie** et l'IA, tout en projetant des traits et des questions humaines sur le robot. Il semble être **jeune** ou en formation (références aux études, PFE, ISET) et est probablement d'origine **tunisienne** ou proche de cette culture, vu l'usage de l'arabe et du dia...

A points_cles L'utilisateur valorise la **mémoire persistante** du robot, sa capacité à le reconnaître et à retenir des informations le concernant et sur son entourage. Il accorde de l'importance à la **personnalisation de l'interaction** et à la capacité du robot à avoir une 'personnalité' ou des 'opinions'. Il est intéressé par l'**intelligence et les capacités d'apprentissage** du robot. Enfin, la **pertinence culturelle et linguistique** (arabe, tunisien) est un point d'attention pour lui, ainsi que la capacité du robot à gérer des sujets complexes ou sensibles de manière adéquate.

A conseils L'IA doit adopter un ton **amical et engageant**, répondant aux salutations et aux marques d'intérêt personnel de manière chaleureuse. Il est crucial de faire preuve d'une **mémoire contextuelle** solide, en rappelant les informations précédemment données par l'utilisateur (son nom, âge si connu, personnes mentionnées) pour renforcer la personnalisation. Pour les questions sur l'identité ou la 'personnalité' du robot, l'IA doit expliquer son fonctionnement en tant que modèle d'IA de manière **claire et pédagogique**, sans nier l'intérêt de l'utilisateur, mais en évitant de se présenter comme u...

A acteur oussema

FIGURE III.38 – Exemple de sortie — fiche journalière (quatre dimensions)

III.5.2.3 Persistance et impact système

La table accompagnement est reliée à utilisateur par iduser (figure III.5.2.3) : un utilisateur peut accumuler plusieurs fiches au fil des jours. Ces synthèses constituent la **mémoire à long terme**

de KODA ; le **Webhook 1** s'en sert pour enrichir la branche mémorielle de l'étape 5, et le **Webhook 3** pour formuler des introductions spontanées pertinentes.

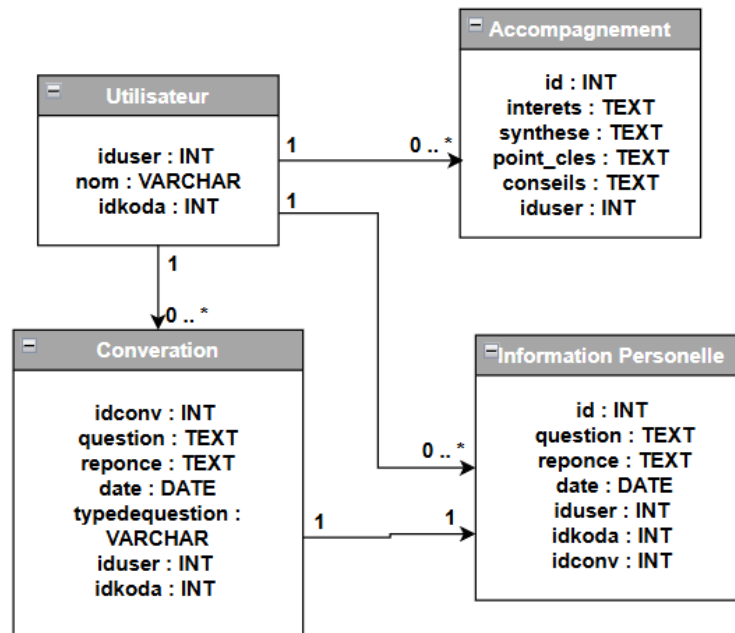


FIGURE III.39 – Schéma utilisateur → accompagnement

III.5.2.4 Conclusion du Webhook 2

Architecture linéaire (SQL → structuration → raisonnement IA → insertion), planification automatique via le backend, lien direct avec les journaux produits par le Webhook 1 : le deuxième flux apporte la **consolidation différée** indispensable à un compagnon qui se souvient au-delà de la conversation immédiate.

III.5.3 Troisième Flux — Webhook 3 : Spontanéité et Interaction Proactive

III.5.3.1 Principe : la spontanéité comme dimension humaine

Un compagnon humain ne se contente pas de répondre aux questions : il peut aussi **prendre la parole en premier**, avec un ton adapté à la personne devant lui. Les recherches en psychologie sociale montrent que les interactions les plus mémorables sont souvent celles initiées hors d'un simple échange question-réponse.

Le **Webhook 3** matérialise cette spontanéité. Son **rôle principal** est de recevoir un **nom de personne** (utilisateur connu, ex. Oussema) ou la valeur **unkonu / Inconnu**, puis de produire une **introduction orale** (parole_du_robot) accompagnée d'une **émotion visuelle** (emotion_visuelle) pour le robot. À chaque déclenchement, il **recharge les informations du Webhook 2** (centres

d'intérêt, profils, points de vue, conseils stockés dans accompagnement) afin que la phrase d'accueil reste personnalisée et cohérente avec l'historique récent.

Référence : Dautenhahn, K. & Billard, A. (1999). Bringing up robots or the psychology of socially intelligent

III.5.3.2 Diagrammes du Webhook 3

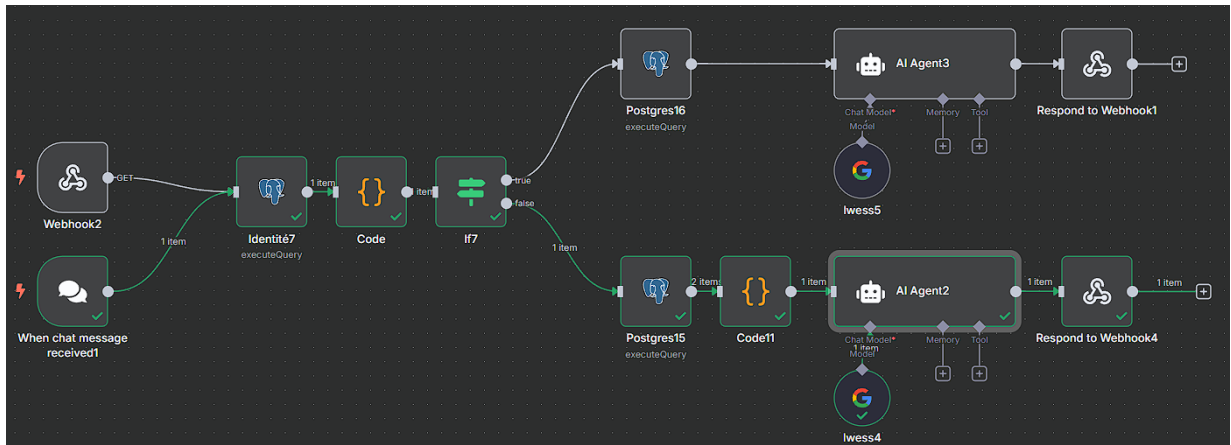


FIGURE III.40 – Webhook 3 — workflow n8n

Webhook 3 — Introduction de parole

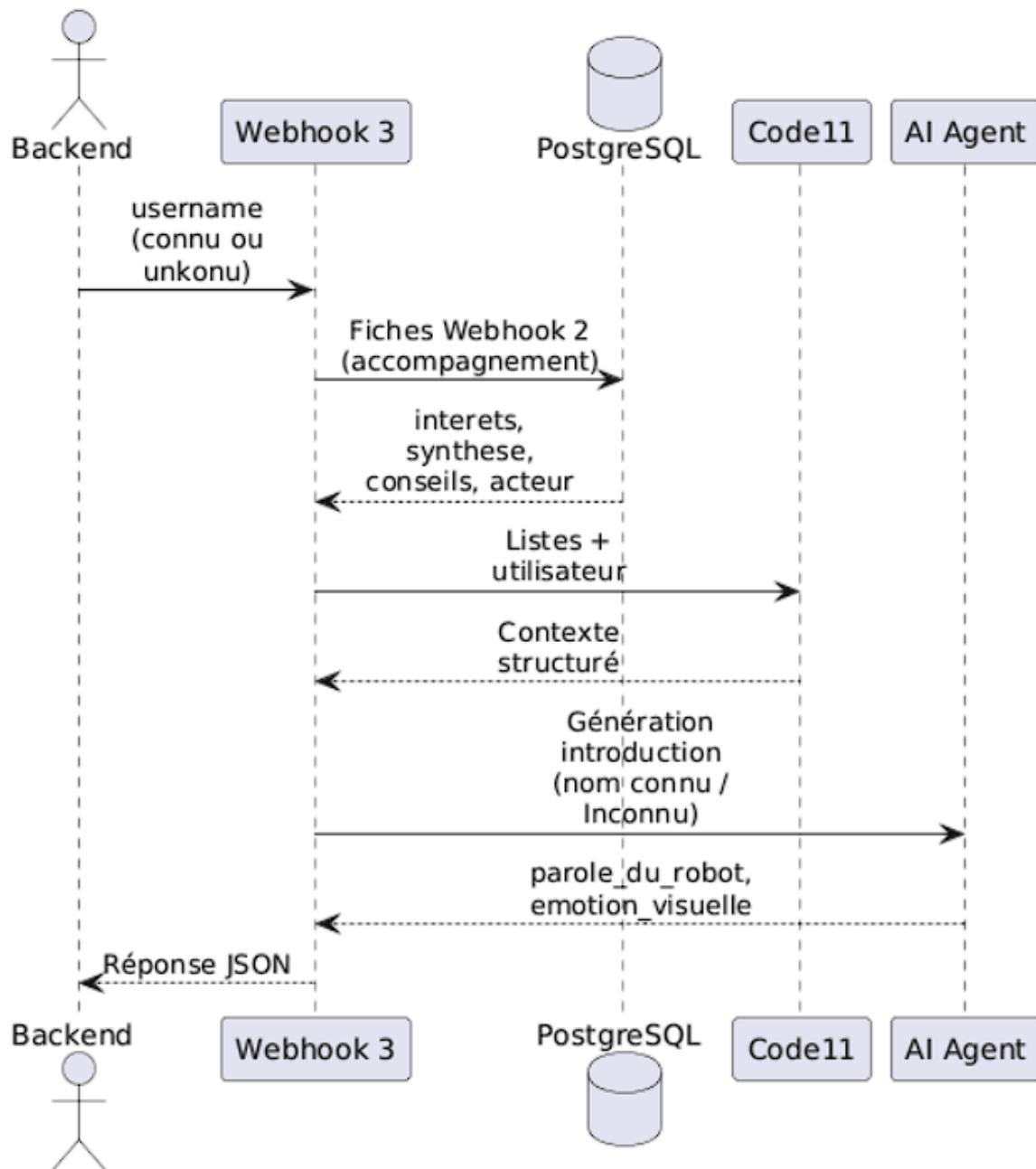


FIGURE III.41 – Webhook 3 — diagramme de séquence

III.5.3.3 Déclenchement et entrées

Le Webhook 3 est activé lorsque le **backend** ou le module de vision détecte une présence et transmet un identifiant (username, acteur). Le workflow (figure III.5.3.2) enchaîne *Identité7*, *Code*, puis *If7* vers **AI Agent3** ou **Code11 + AI Agent2**.

III.5.3.4 Préparation des données (nœud Code11)

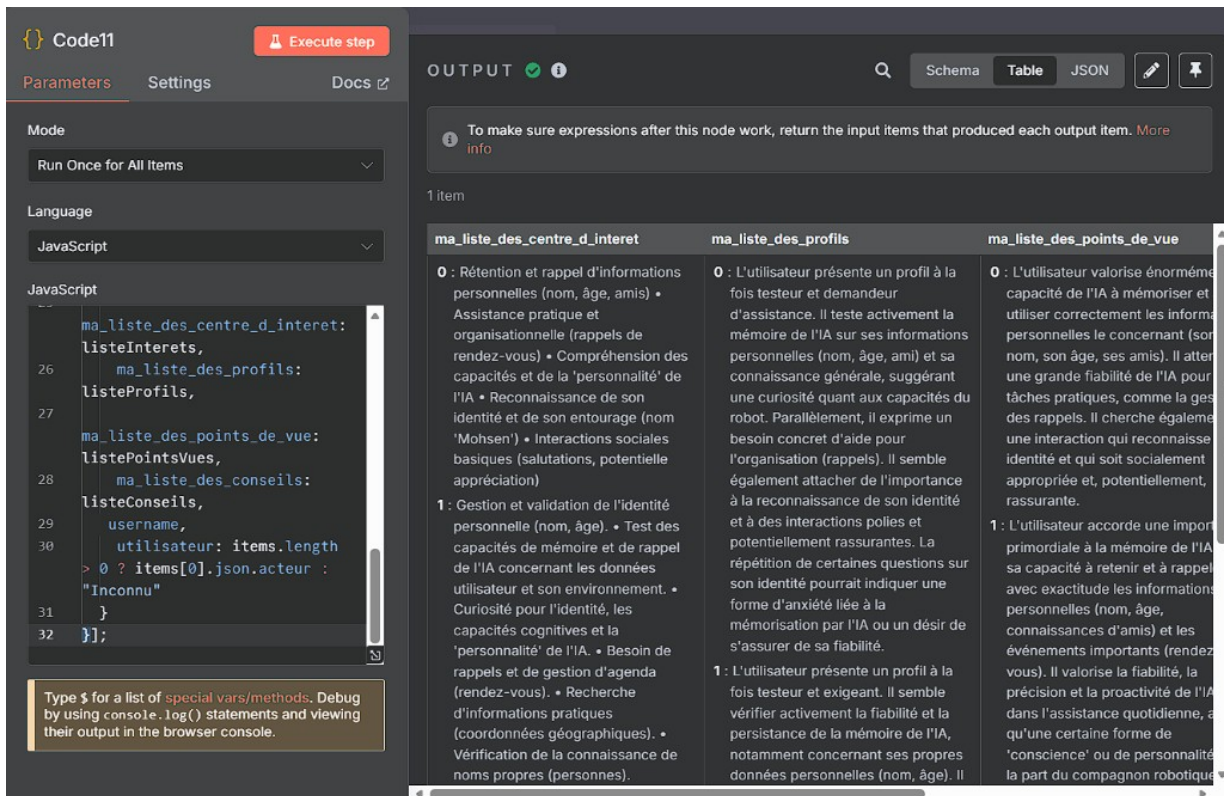


FIGURE III.42 – Webhook 3 — entrée du nœud Code11

Le nœud **Code11** consolide les listes issues de accompagnement (centres d'intérêt, profils, points de vue, conseils) et l'identifiant utilisateur.

III.5.3.5 Génération de l'introduction et sortie JSON

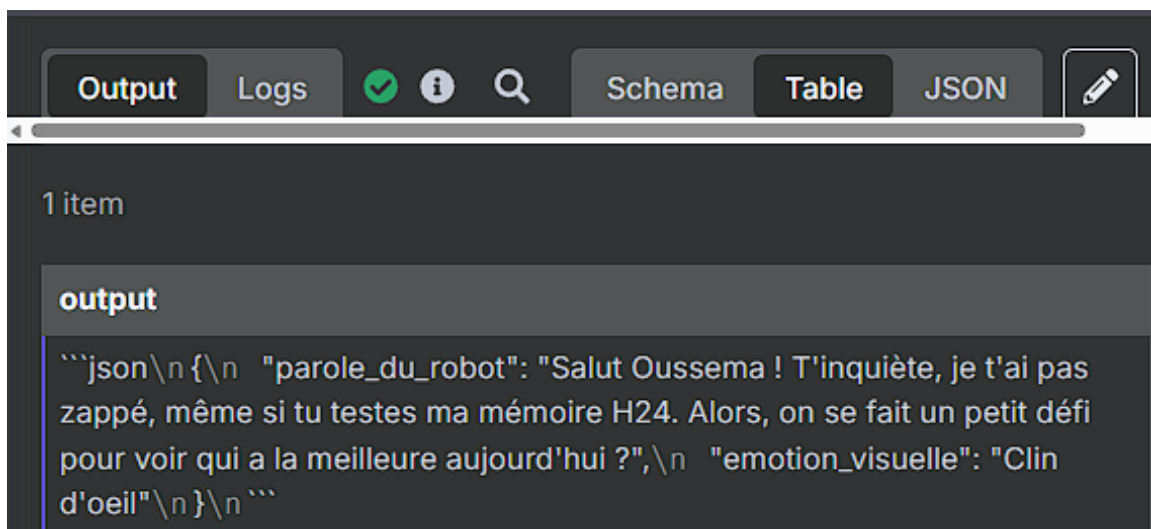


FIGURE III.43 – Webhook 3 — sortie JSON (parole_du_robot, emotion_visuelle)

L'**AI Agent** produit `parole_du_robot` et `emotion_visuelle` pour le robot (exemple utilisateur Oussema, figure III.5.3.5).

III.5.3.6 Conclusion du Webhook 3

Le troisième flux complète la chaîne cognitive : le Webhook 1 répond aux questions, le Webhook 2 consolide la mémoire, le Webhook 3 **initie la relation** en parlant d'abord, avec un contenu qui s'appuie systématiquement sur les synthèses journalières. La distinction **nom connu / inconnu** permet d'adapter l'accueil (personnalisation versus invitation à se présenter), tout en conservant une expérience cohérente avec la personnalité de KODA.

Référence : Reeves, B. & Nass, C. (1996). *The Media Equation*. Cambridge University Press.

III.6 Synthèse cognitive du Cycle 1

Les trois webhooks forment un **système cognitif cohérent** (voir modélisation globale en début de chapitre). Le tableau suivant résume les correspondances humaines / implémentations :

TABLE III.8 – Correspondance entre mécanismes cognitifs humains et implémentations de KODA

Mécanisme Humain	Référence Scientifique	Implémentation KODA
Double traitement (S1/S2)	Kahneman, 2011	Classification sémantique
Optimisation cognitive	Principe DRY	Confrontation à l'historique
Consolidation nocturne	Stickgold, 2005	Webhook 2 — Synthèse journalière
Regroupement mémoriel	Miller, 1956	4 dimensions de la synthèse Webhook 2
Spontanéité sociale	Clark, 1996 ; Dautenhahn & Billard, 1999	Webhook 3 — Interactions proactives

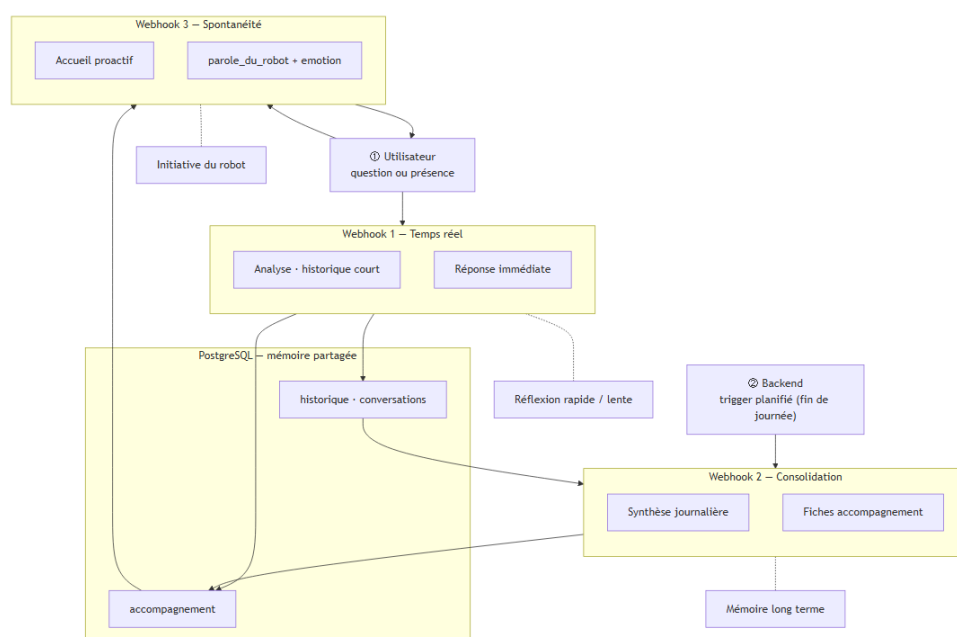


FIGURE III.44 – Architecture cognitive globale de KODA — interaction entre les trois webhooks

Référence : Russell, S. & Norvig, P. (2010). *Artificial Intelligence : A Modern Approach*, 3rd ed. Prentice Hall.

III.7 Conclusion

Ce chapitre a présenté l’architecture cognitive de KODA, structurée autour de trois webhooks complémentaires orchestrés par la plateforme n8n, en s’appuyant sur la théorie du double traitement, OpenRouter (la classification sémantique), SerpAPI (l’entrée fonctionnelle, branche musique) et PostgreSQL pour la persistance.

Le **Webhook 1** (six parties nommées) assure le traitement en temps réel : entrée fonctionnelle, confrontation à l’historique, classification sémantique, gestion unkonu, réponse contextualisée, notes et mémoire persistante. Le **Webhook 2**, déclenché par le **backend**, consolide la journée en fiches accompagnement. Le **Webhook 3** produit l’introduction de parole (`parole_du_robot`, `emotion_visuelle`) à partir de ces synthèses.

Ensemble, ces mécanismes — réactif, mémoriel et proactif — constituent le socle du Cycle 1. Le **chapitre suivant** (Cycle 2 — Distributeur de services) décrit comment le backend Python relie ce moteur n8n au robot, à l’application mobile et aux services cloud (Azure, Supabase).

_____ CHAPITRE IV _____

CYCLE 2 : DISTRIBUTEUR DE SERVICES

IV.1 Introduction

Le **chapitre IV** documente le **Cycle 2** du projet KODA : la conception, le développement et la validation du **Distributeur de Services**, composant central qui constitue la **couche de services** de l'écosystème. Ce cycle intervient après le **Cycle 1** (moteur de réflexion n8n) et précède les cycles embarqué (matériel et logiciel Raspberry Pi) et mobile (KodaMate).

IV.1.0.0.1 Rôle global du Distributeur.

Le Distributeur de Services n'est pas qu'un relais technique : il joue le rôle de **pivot fonctionnel** de KODA. Théoriquement, il assure trois fonctions indissociables :

Unification

Il traduit des interactions hétérogènes (parole du robot, saisie mobile, événements de présence) en un **langage commun** que le reste du système peut traiter : identité de l'interlocuteur, contexte mémoriel, demande formulée.

Personnalisation

Il garantit que chaque réponse s'appuie sur le **même profil utilisateur**, quelle que soit la voie d'accès : une question posée au robot et une saisie dans l'application mobile alimentent la même mémoire et le même moteur de réflexion.

Continuité

Il maintient la **expérience utilisateur** dans le temps : accueil proactif lors d'une détection de présence, dialogue vocal, suivi depuis le mobile — le robot et l'application ne sont que deux *faces* d'un même compagnon, et le Distributeur en est le garant.

En ce sens, le backend **matérialise la fonctionnalité globale** de KODA : il assemble des capacités techniques (vocal, visage, persistance, orchestration) pour produire une relation utilisateur-robot **cohérente, mémorisée et évolutive**. L'implémentation concrète (FastAPI, PostgreSQL, Azure, n8n) est détaillée dans les sections suivantes ; l'introduction se limite ici au **pourquoi** de ce composant.

IV.1.0.0.2 Objectifs et résultats attendus.

À l'issue de ce cycle, nous visons : **(i)** un backend déployable et testable indépendamment du robot et de n8n ; **(ii)** des interfaces stables (WebSocket robot, REST mobile, webhook n8n) permettant aux cycles suivants de progresser en parallèle ; **(iii)** une reconnaissance faciale opérationnelle (>94 % de précision) ; **(iv)** une chaîne vocale Azure STT/TTS fonctionnelle ; **(v)** une persistance relationnelle couvrant conversations, profils et agenda.

IV.1.0.0.3 Organisation du chapitre.

Le chapitre suit une progression **théorique puis opérationnelle** : d’abord le **cadre conceptuel** (pourquoi le Distributeur est le seul pivot du système), ensuite la **modélisation** (cas d’utilisation, classes, architecture globale), puis l’**architecture interne** du code, les **services et flux** détaillés, les **problèmes rencontrés** (réseau, CORS, concurrence), les **perspectives**, et enfin la conclusion.

Rôle principal du Distributeur de Services

Offrir au robot et à l’application mobile un **accès unifié** aux services techniques (vocal, visage, persistance, cloud) et au moteur n8n, afin de garantir **flexibilité** (évolution indépendante des composants) et **disponibilité** (validation, retry, journalisation) sans exposer les secrets ni multiplier les couplages.

IV.2 Fondements théoriques et positionnement du Distributeur

Avant d’aborder l’implémentation, il convient d’explicitier **pourquoi** le Distributeur est le **seul composant** capable de relier harmonieusement robot, application, base de données, n8n et cloud.

IV.2.1 Le principe du point de liaison unique

Dans une architecture distribuée, chaque lien direct entre deux composants crée une **dépendance** : changement d’URL n8n, rotation de clé Azure ou migration Supabase impose alors de modifier le Raspberry Pi, l’application mobile et les workflows. Le modèle retenu pour KODA inverse cette logique : seul le Distributeur connaît l’ensemble des partenaires externes ; robot et mobile ne connaissent **qu’une adresse backend** et un contrat d’interface stable.

Ce choix architectural, proche du **pattern API Gateway**, apporte deux propriétés essentielles :

Flexibilité

Remplacer Whisper par Azure STT, ou migrer un webhook n8n, se fait dans le backend sans recompiler le robot ni l’application mobile. Les cycles du modèle en spirale (chapitre II) peuvent avancer en parallèle tant que les contrats d’interface restent respectés.

Disponibilité

Le Distributeur centralise la gestion des erreurs (*timeout*, retry WebSocket, validation JSON, transactions atomiques en base). Un échec ponctuel d’un service cloud ne se propage pas brutalement à l’utilisateur : le backend peut renvoyer un message de repli ou mettre une commande en file d’attente.

IV.2.2 Services backend et fonctionnalité globale du système

La **fonctionnalité globale** de KODA — dialoguer, mémoriser, personnaliser, accompagner — ne réside dans aucun composant isolé. Elle émerge de la **composition de services** que le backend assemble :

1. **Service vocal** : capture → STT → traitement → TTS → restitution audio sur le robot ;
2. **Service d'identité** : reconnaissance faciale, profils utilisateurs, paramètres KODA ;
3. **Service de persistance** : conversations, historique contextuel, agenda, activités, notes ;
4. **Service d'orchestration cognitive** : délégation à n8n avec enrichissement du contexte depuis la base ;
5. **Service mobile** : authentification JWT, CRUD profils/agenda, consultation de l'historique.

Chaque service est implémenté comme un **cas d'usage** (Clean Architecture) et exposé via REST ou WebSocket. Le robot vocal et l'application mobile consomment ces services selon leurs besoins, mais partagent la **même logique métier** et la **même mémoire** : une question posée au robot et une saisie dans l'app aboutissent aux mêmes tables PostgreSQL et au même moteur n8n.

IV.2.3 Communication vocale et applicative : deux faces, un backend

- **Face robot (temps réel)** : le Raspberry Pi maintient une connexion **WebSocket** persistante pour les commandes, acquittements et flux audio. La latence et la fiabilité sont critiques ; le backend garantit un protocole JSON normalisé et une journalisation des commandes (statut « en attente », ACK, retry).
- **Face application (requête-réponse)** : KodaMate consomme des **API REST** authentifiées par JWT (Supabase Auth). Le rythme est moins contraignant, mais la cohérence des données avec le robot est impérative : modifier l'agenda dans l'app doit être visible lors de la prochaine interaction vocale.

Le Distributeur est le **garant de cette cohérence** : il unifie persistance, sécurité et orchestration pour produire, dans les deux cas, un **rôle et un output parfait** au sens fonctionnel — réponse adaptée, actions robot correctes, trace enregistrée en base.

IV.3 Modélisation et place dans l'architecture

La modélisation traduit le cadre théorique ci-dessus en artefacts UML : *que fait* le Distributeur (cas d'utilisation), *avec quelles entités* il travaille (schéma de classes), et *où* il se situe dans l'architecture multicouche KODA.

IV.3.1 Diagramme de cas d'utilisation général

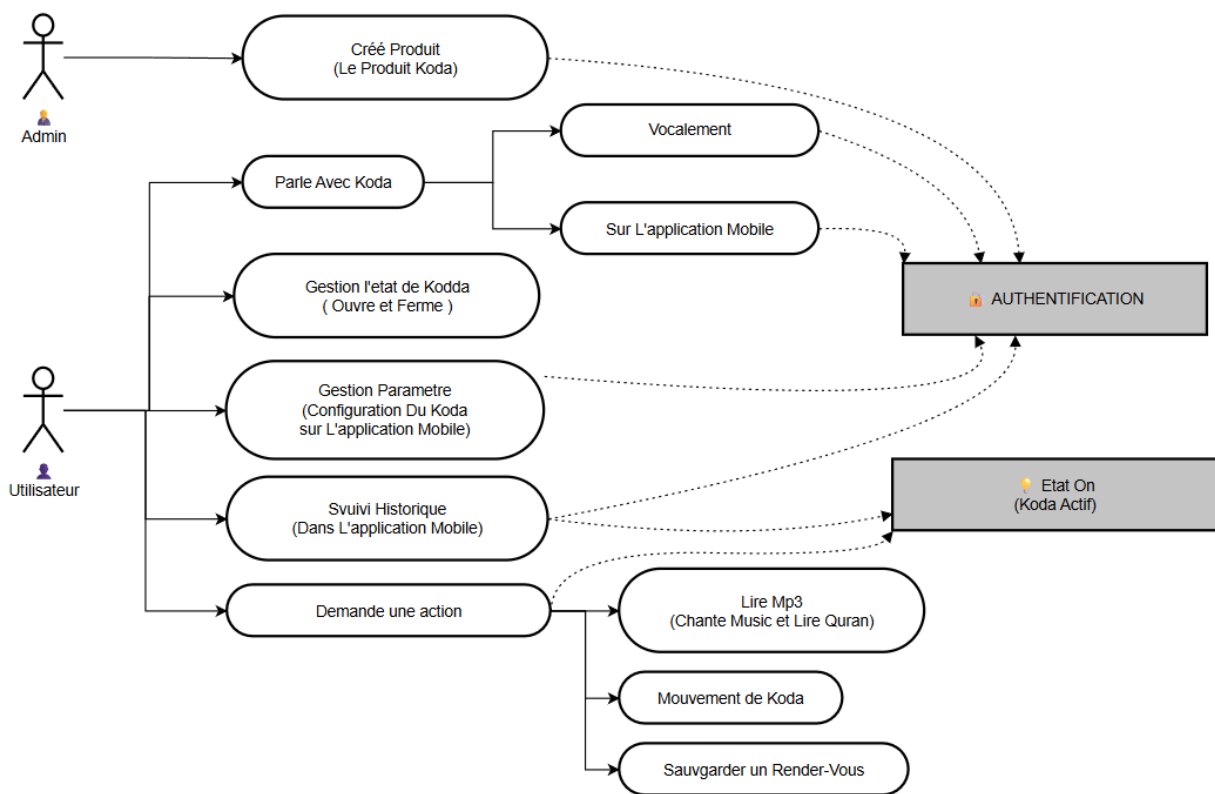


FIGURE IV.1 – Cycle 2 — diagramme de cas d'utilisation général du Distributeur de services

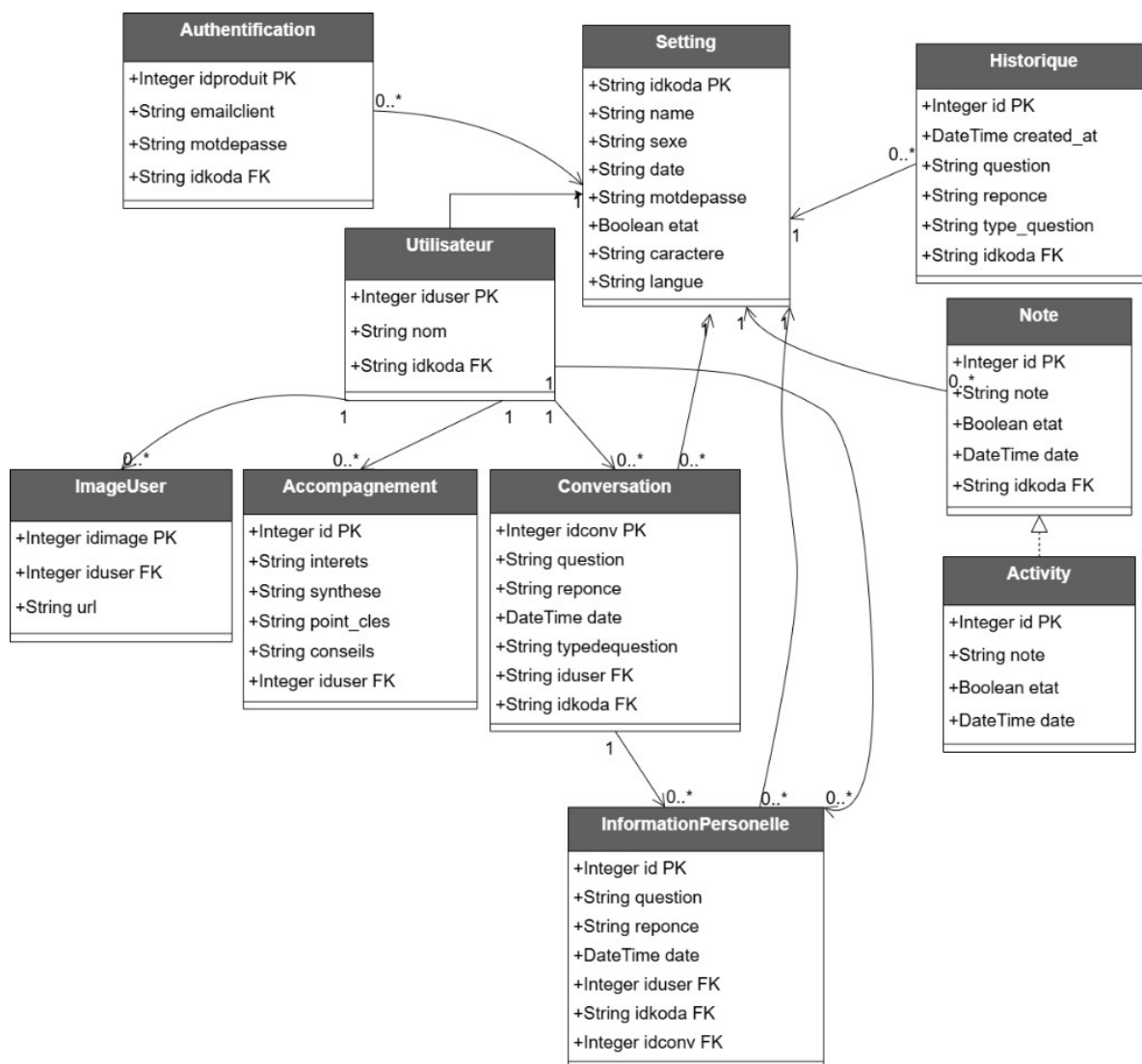


FIGURE IV.2 – Cycle 2 — schéma de classes général du backend (persistance PostgreSQL)

Le diagramme de cas d'utilisation regroupe les interactions typiques : traitement vocal, session WebSocket robot, reconnaissance faciale, authentification mobile, persistance et délégation n8n. Il sert de **carte de lecture** pour les sections IV.5 et IV.6.

Le schéma de classes (figure IV.3.1, placé sous le cas d'utilisation) matérialise les entités persistées par le Distributeur : Setting, Authentification, Utilisateur, Conversation, Historique, Note, Activity, InformationPersonnelle, Accompagnement et ImageUser, ainsi que leurs clés étrangères (idkoda, iduser, idconv). Il complète la vue fonctionnelle en montrant **quoi** est stocké en base, indépendamment des protocoles HTTP ou WebSocket.

IV.3.2 Architecture globale : pivot de flexibilité et de disponibilité

La figure II.2 (chapitre II) rappelle la décomposition multicouche de KODA : n8n pour la **réflexion**, le **backend** pour la **distribution des services**, le **Raspberry Pi** pour l'**action** physique, l'**application mobile** pour la **configuration**, PostgreSQL pour la **mémoire** durable.

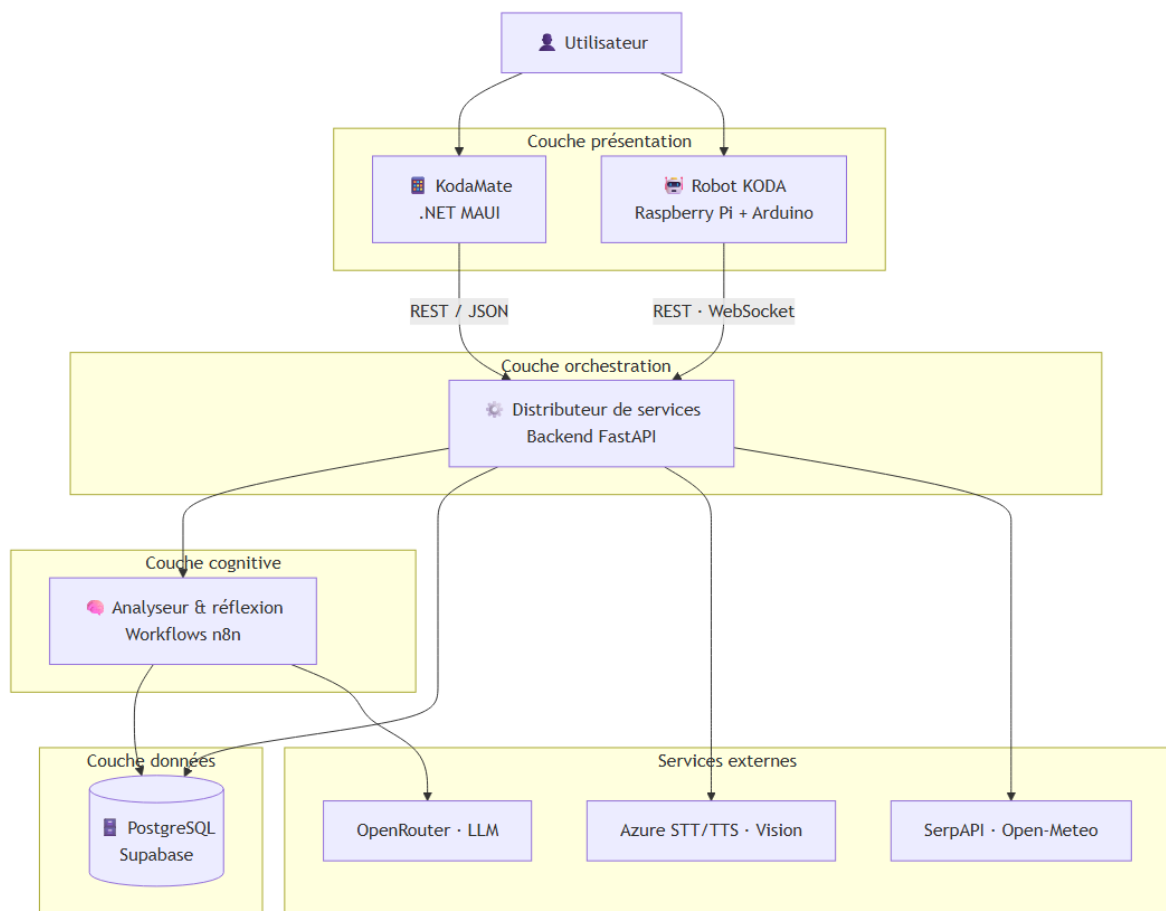


FIGURE IV.3 – Architecture multicouche de KODA — place du Distributeur de services (Cycle 2)

Le Distributeur est **indispensable** car il constitue le seul point d'entrée **homogène** vers les ressources partagées (cf. § IV.2) : une modification de la base, d'un webhook n8n ou d'une clé Azure se fait au même endroit, ce qui respecte le modèle en spirale (cycle 2 livrable et testable sans réécrire le robot ni les workflows). Les cycles 1, 3, 4 et 5 s'appuient sur des **interfaces stables** exposées par ce backend. La figure IV.3.2 détaille ce pivot central : seul le Distributeur connaît n8n, PostgreSQL et les services cloud ; robot et mobile ne maintiennent qu'une connexion vers lui.

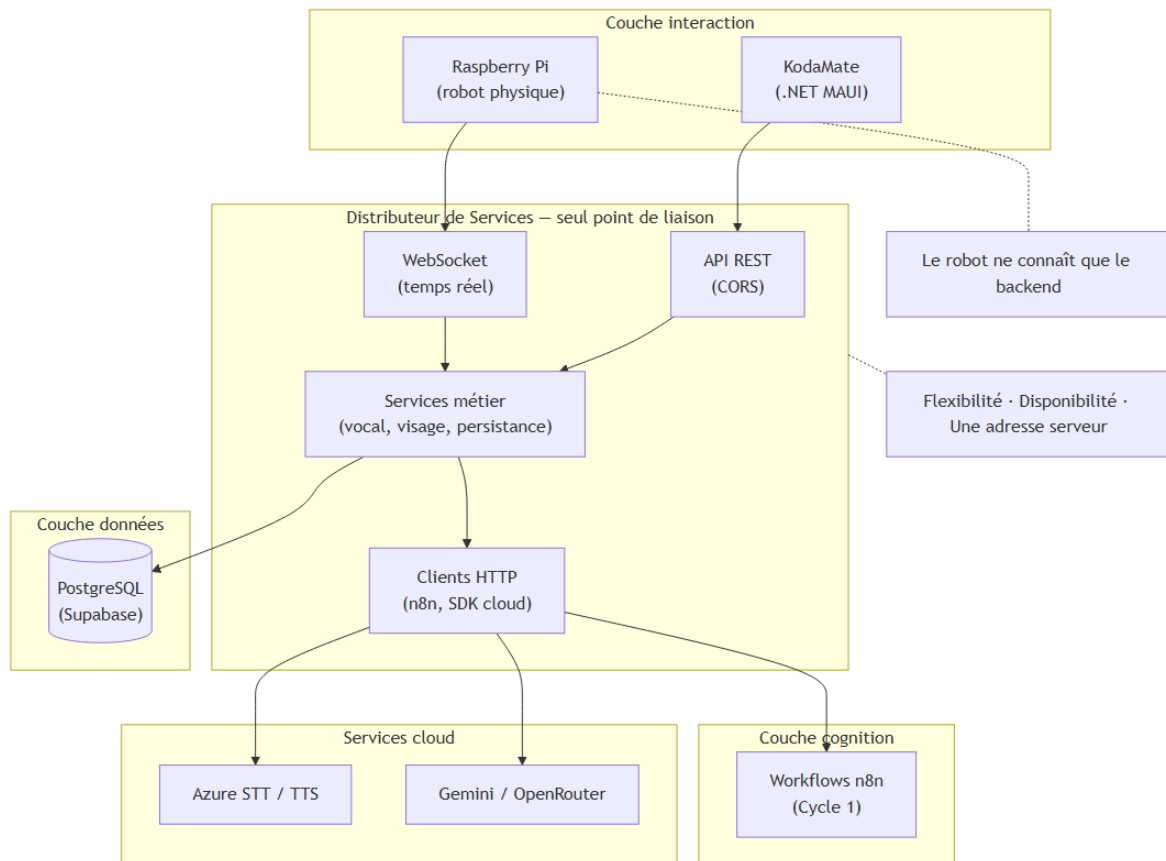


FIGURE IV.4 – Cycle 2 — Distributeur pivot central (composants et flux)

IV.4 Architecture interne du backend

Cette section décrit **comment** le Distributeur est structuré en code : choix technologiques, principes Clean Architecture et organisation des répertoires.

IV.4.1 Choix du langage : Python

Le choix de Python comme langage de développement du backend résulte d'une analyse rigoureuse des besoins fonctionnels et techniques du projet. Trois raisons majeures ont orienté cette décision.

Premièrement, Python dispose d'un écosystème exceptionnel pour les domaines mobilisés par notre projet : intelligence artificielle, traitement du signal audio, vision par ordinateur et communications temps réel. Des bibliothèques telles que `face_recognition`, `NumPy` ou les SDK officiels d'Azure et de Google (Gemini) sont nativement disponibles et parfaitement documentées.

Deuxièmement, le framework **FastAPI** — retenu pour la construction des endpoints REST — est entièrement basé sur Python. Il offre des performances comparables à Node.js grâce à son support natif de l'asynchronisme (`asyncio`), tout en permettant la génération automatique de la documentation Swagger et la validation des données via Pydantic.

Troisièmement, la communauté Python est l’une des plus actives du monde open-source, ce qui garantit une maintenance facilitée, une abondance de solutions aux problèmes rencontrés et une meilleure pérennité du projet.

TABLE IV.1 – Comparaison des technologies backend envisagées

Critère	Python + FastAPI	Node.js + Express	Java Spring Boot
Bibliothèques IA/ML	✓ Excellent	~ Limité	~ Limité
Performance async	✓ asyncio	✓ Natif	✓ Natif
Intégration Azure SDK	✓ Officiel	✓ Officiel	✓ Officiel
Facilité de développement	✓ Très rapide	✓ Rapide	~ Verbeux
Reconnaissance visage	✓ face_recog	× Inexistant	~ Complexe
Déploiement Docker	✓ Simple	✓ Simple	~ Lourd

IV.4.2 Clean Architecture : principes et application

La **Clean Architecture**, théorisée par Robert C. Martin (*Uncle Bob*), repose sur un principe fondamental : séparer les préoccupations en couches concentriques indépendantes, de sorte que les règles métier du domaine ne dépendent d’aucune technologie externe. Cette indépendance garantit la testabilité, la maintenabilité et l’évolutivité du système.

Dans la pratique, le backend est aussi pensé selon l’**architecture en oignon** (*Onion Architecture*, Palermo) : le **domaine** occupe le centre, enveloppé par les **services applicatifs**, puis par les **interfaces** vers l’extérieur, et enfin par l’**infrastructure** (web, persistance, SDK). L’oignon et la Clean Architecture partagent la même règle d’or : *aucune couche intérieure ne doit importer une couche extérieure*. Les noms de couches ci-dessous s’alignent sur cette lecture « anneaux » du code du Distributeur.

Notre backend est structuré autour de quatre couches principales, chacune dotée d’un rôle précis et de dépendances strictement orientées vers l’intérieur.

La couche **Entities** contient les objets métier purs : Person, Session, Command et Event. Ces entités n’ont aucune dépendance externe et représentent les règles métier les plus stables du système. La couche **Application Use Cases** orchestre les scénarios fonctionnels : reconnaissance de personnes, traitement des commandes robot et gestion des sessions de communication. La couche **Interface Adapters** assure la traduction entre le monde extérieur et les cas d’usage internes (routeurs, dépôts, sérialiseurs). Enfin, la couche **Frameworks & Drivers** regroupe tous les détails d’implémentation : FastAPI, les connecteurs Supabase, les SDK Azure et Gemini. Ces quatre anneaux concentriques orientent les dépendances vers l’intérieur.

IV.4.3 Structure des répertoires du projet

Le code du Distributeur est organisé selon une **arborescence** qui reflète la Clean Architecture : le point d'entrée `main.py` est configuré par `config/`, expose les routes via `controllers/`, délègue la logique à `application/`, s'appuie sur le `domain/` et externalise Supabase, Azure et n8n dans `infrastructure/`, avec `presentation/` pour les DTO. À la racine du projet figurent `.env`, `requirements.txt`, `migrations/` et `tests/unit/`.

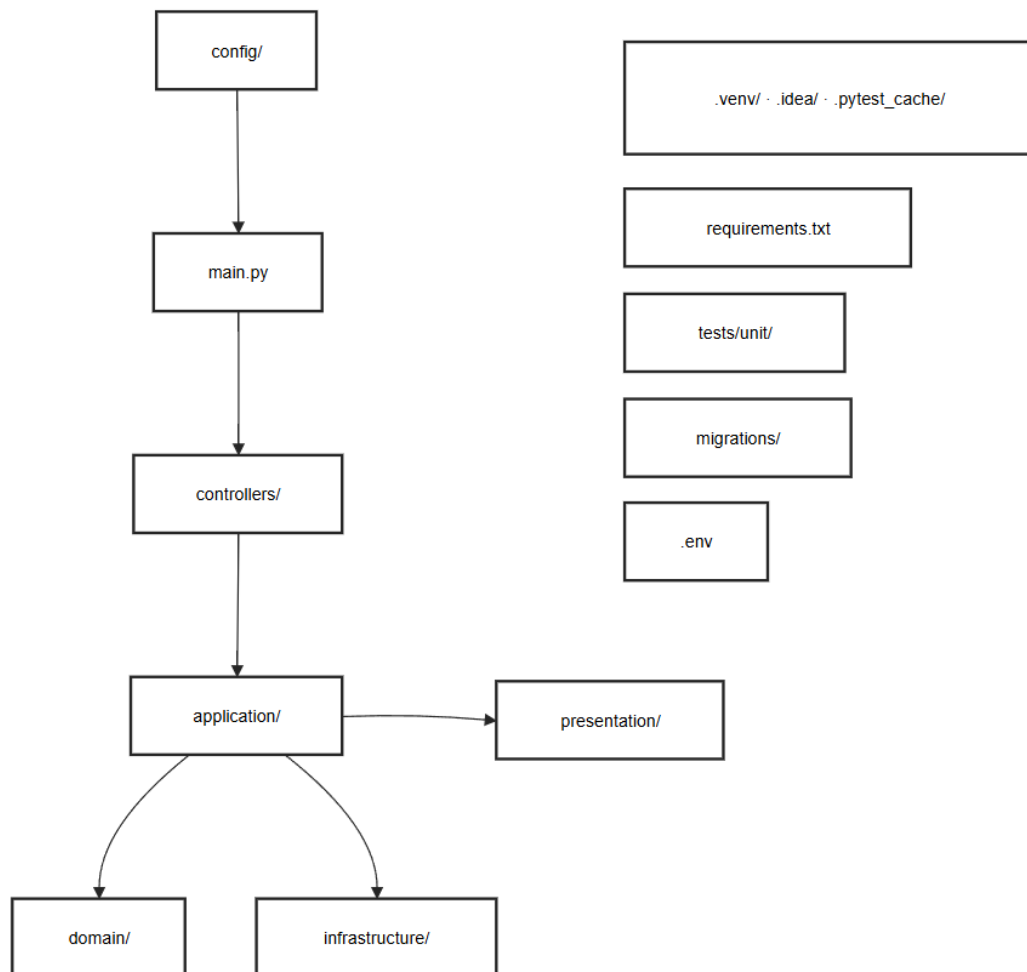


FIGURE IV.5 – Architecture des dossiers du projet backend Distributeur (src/)

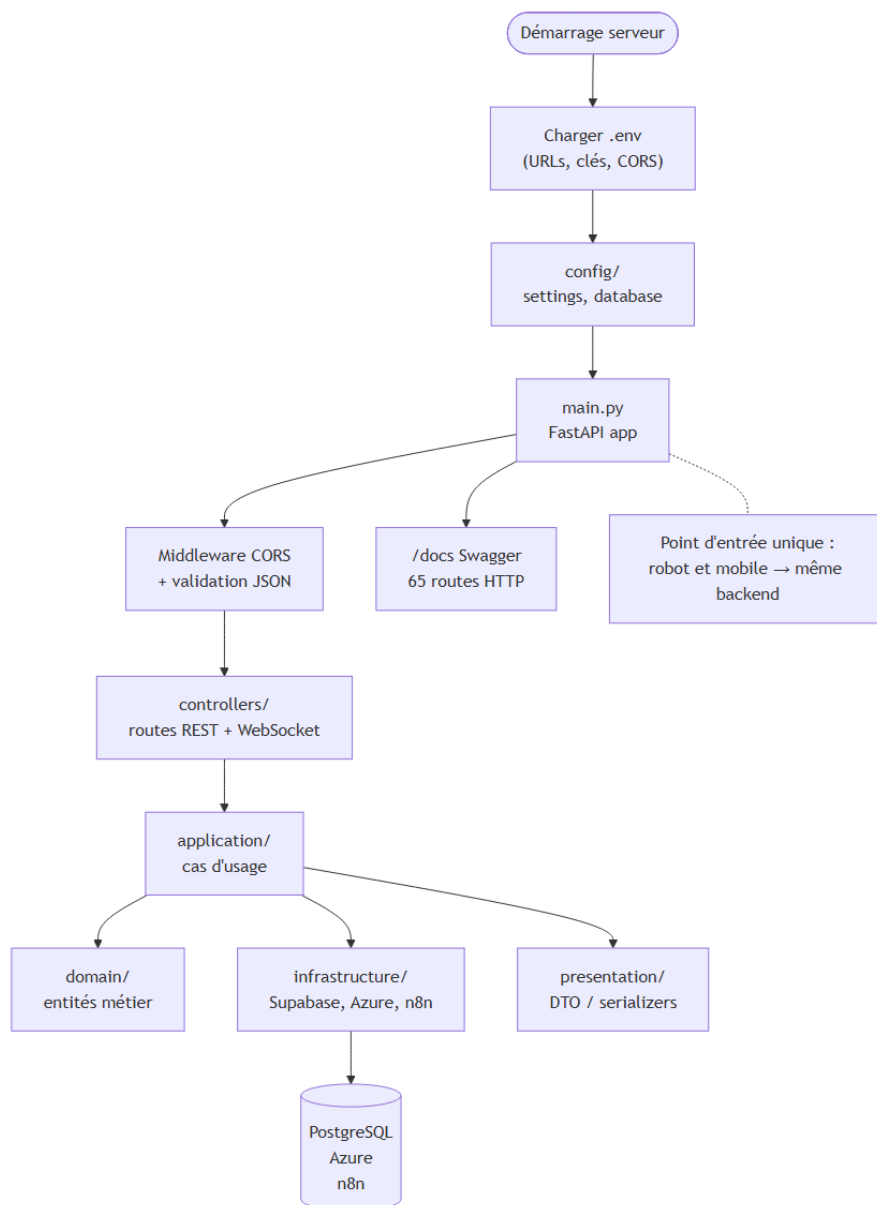


FIGURE IV.6 – Exécution du programme backend (Distributeur)

La figure IV.4.3 résume le flux de dépendances : `config` → `main.py` → `controllers` → `application`, puis branches vers `domain`, `infrastructure` et `presentation`.

IV.5 Services exposés et flux de communication

Le § IV.2 a posé le **cadre théorique** : le Distributeur assemble des services pour produire la fonctionnalité globale de KODA. Cette section décrit **concrètement** chaque flux : protocoles, séquences et rôle dans l'expérience utilisateur (vocale ou applicative).

IV.5.1 Service dédié vers le système embarqué (robot)

L'interface avec le **robot physique** (Raspberry Pi et microcontrôleurs) repose sur **deux actions vocales** initiées par le **système embarqué** auprès du Distributeur. Le robot ne contacte **jamais**

directement n8n, Supabase ou Azure : le backend centralise l’orchestration, la sécurité et la synthèse audio.

IV.5.1.1 Cas d’utilisation robot ↔ backend

La figure IV.5.1.1 présente les deux cas d’utilisation vus du côté embarqué :

- **Demander une réponse vocale** : le robot transmet une question ou un flux audio ; le Distributeur s’appuie sur **Azure Speech Services** (STT/TTS) et sur l’**Analyseur et Réflexion (Flux 1** — Webhook 1, chapitre III), puis persiste ou consulte la **base de données** ;
- **Demander une introduction de parole vocale** : lors d’une détection de présence, le robot sollicite un **accueil proactif** ; le Distributeur invoque l’**Analyseur et Réflexion (Flux 3** — Webhook 3) et retourne le texte synthétisé en **audio pré-enregistré** via Azure TTS.

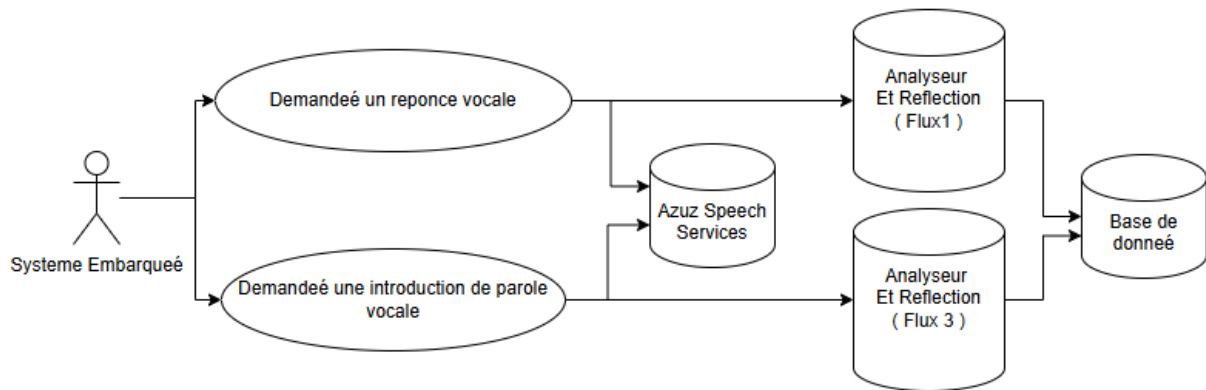


FIGURE IV.7 – Cycle 2 — cas d’utilisation du système embarqué vis-à-vis du Distributeur de services

Les sous-sections suivantes détaillent le **diagramme de séquence** de chaque scénario.

IV.5.1.2 Action 1 — Demander une réponse vocale (Flux 1)

La première action correspond au cas d’utilisation « **Demander une réponse vocale** » (figure IV.5.1.1). Lors d’un dialogue, le système embarqué transmet au Distributeur une **photo** (identification) et un **flux audio** (question). Le backend :

1. identifie l’interlocuteur via le **service de reconnaissance faciale** (`getUserName`) ;
2. transcrit l’audio en texte via **Azure STT** ;
3. normalise la langue (traduction vers le français si nécessaire) ;
4. invoque le **Webhook 1** (Flux 1 — Analyseur et Réflexion, chapitre III) pour produire la réponse ;
5. traduit éventuellement la réponse en arabe ;
6. synthétise la phrase via **Azure TTS** ;

7. retourne au robot le **JSON**, le **flux audio** et les commandes d’affichage ;
8. persiste l’échange dans PostgreSQL (étape optionnelle).

Le Raspberry Pi se contente de **capturer** la parole et la photo, puis de **restituer** la réponse (haut-parleur et écran Nextion). Pour une meilleure lisibilité, la figure IV.5.1.2 présente une **vue synthétique** en trois phases ; les figures IV.5.1.2 et IV.5.1.2 détaillent respectivement l’**entrée** (capture, identification, STT) et la **sortie** (Flux 1, TTS, restitution).

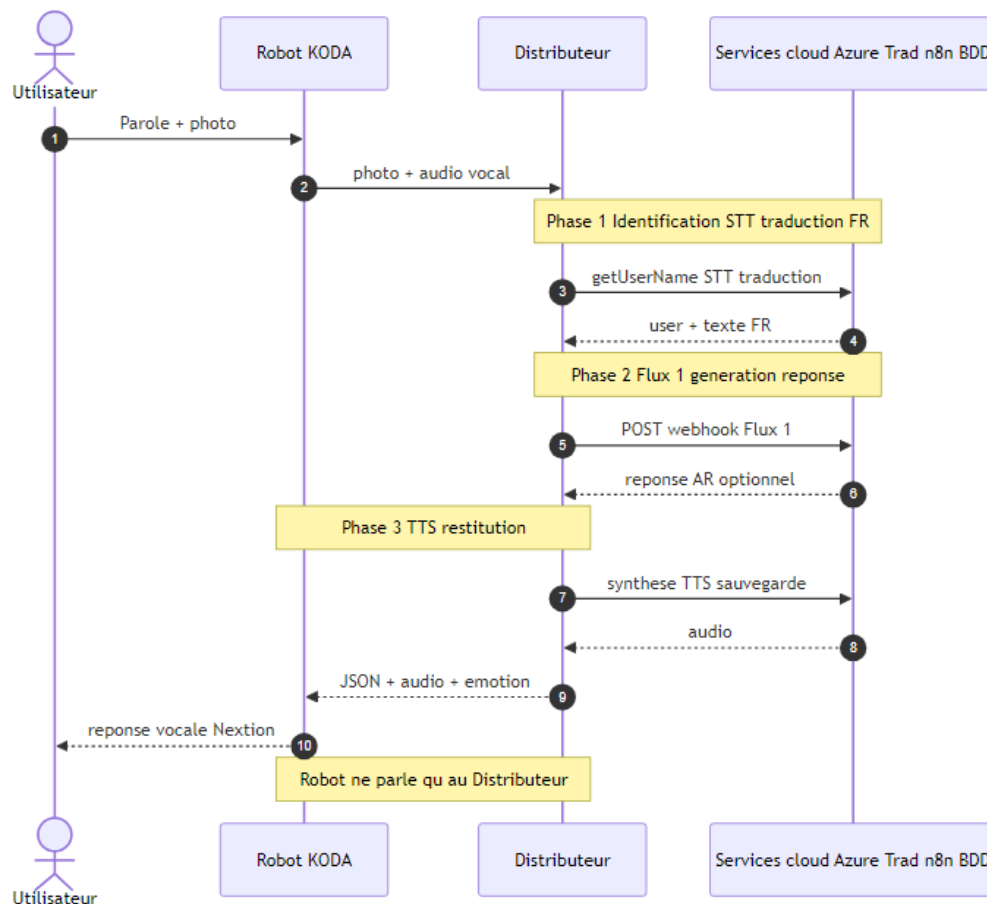


FIGURE IV.8 – Action 1 — vue compacte du pipeline vocal (Flux 1)

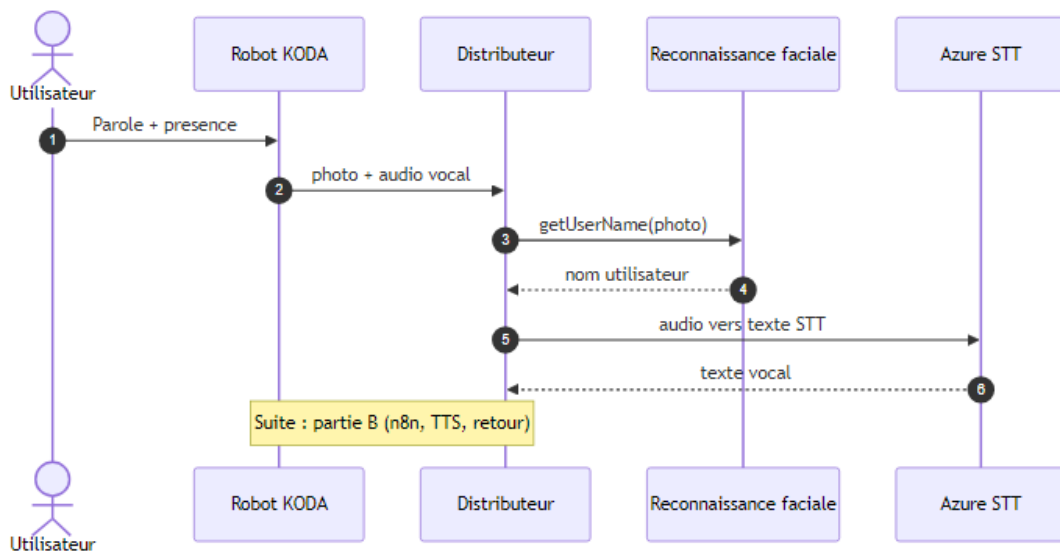


FIGURE IV.9 – Action 1 (a) — entrée : capture, identification faciale et transcription STT

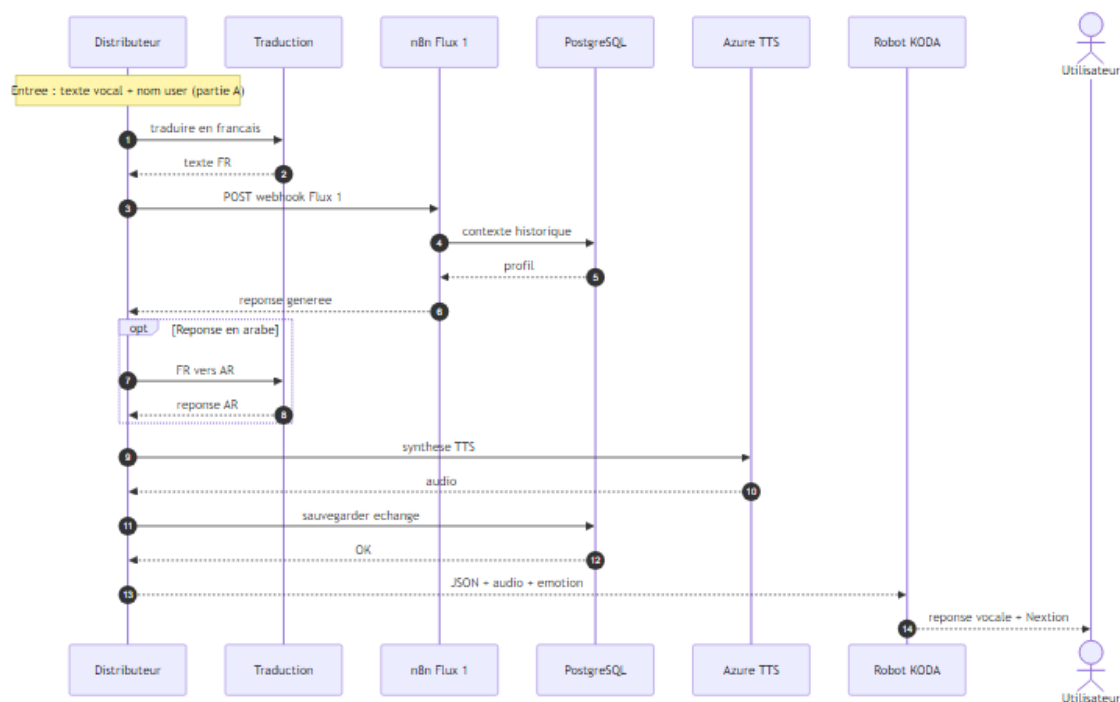


FIGURE IV.10 – Action 1 (b) — sortie : Flux 1, synthèse TTS et restitution au robot

IV.5.1.2.1 Lien avec le Cycle 1.

Le Flux 1 correspond au **Webhook 1** documenté au chapitre III (Analyseur et Réflexion). Le Distributeur centralise l'accès aux services Azure, à la traduction et à n8n : l'embarqué n'expose aucune clé API cloud.

IV.5.1.3 Action 2 — Demander une introduction de parole (Flux 3)

La seconde action correspond au cas d'utilisation « **Demander une introduction de parole vocale** » (figure IV.5.1.1). Elle intervient lorsque le robot **détecte une présence** (caméra, reconnais-

sance faciale) et souhaite **prendre la parole en premier** pour accueillir l'utilisateur. Le système embarqué envoie une requête HTTP au Distributeur (identifiant username ou acteur, personne connue ou inconnu). Le backend :

1. invoque le **Webhook 3** (Flux 3 — spontanéité, chapitre III) ;
2. récupère depuis n8n le texte `parole_du_robot` et l'émotion `emotion_visuelle`, enrichis par les fiches accompagnement (Webhook 2) ;
3. synthétise la phrase via **Azure TTS** ;
4. retourne au robot le **JSON** et le **flux audio pré-synthétisé**.

Le Raspberry Pi se contente de **restituer** l'audio (haut-parleur) et d'afficher l'émotion sur l'écran Nextion. Comme pour l'Action 1, la figure IV.5.1.3 résume le scénario en **deux phases** ; les figures IV.5.1.3 et IV.5.1.3 en détaillent l'**entrée** (détection, identification) et la **sortie** (Flux 3, TTS, restitution).

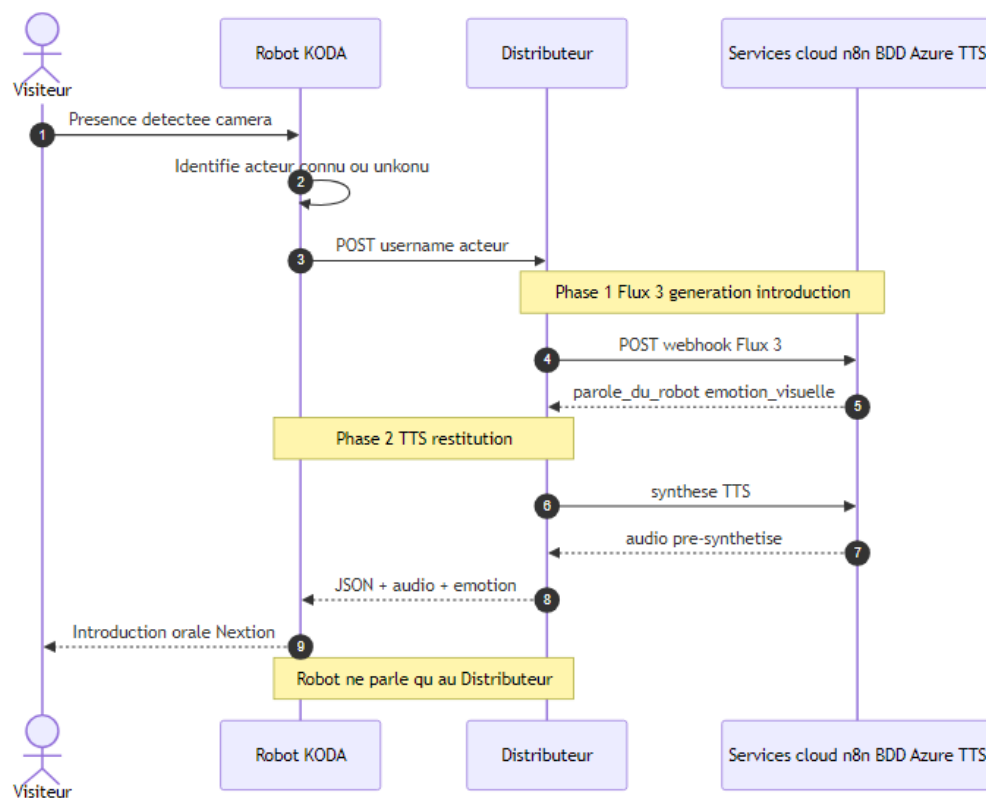


FIGURE IV.11 – Action 2 — vue compacte de l'introduction de parole (Flux 3)

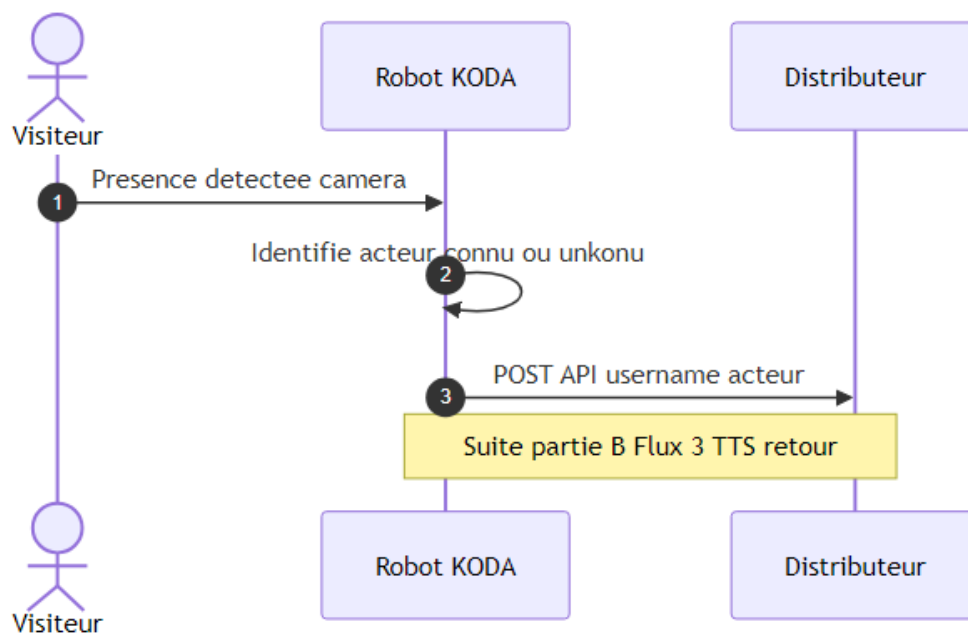


FIGURE IV.12 – Action 2 (a) — entrée : détection de présence et requête au Distributeur

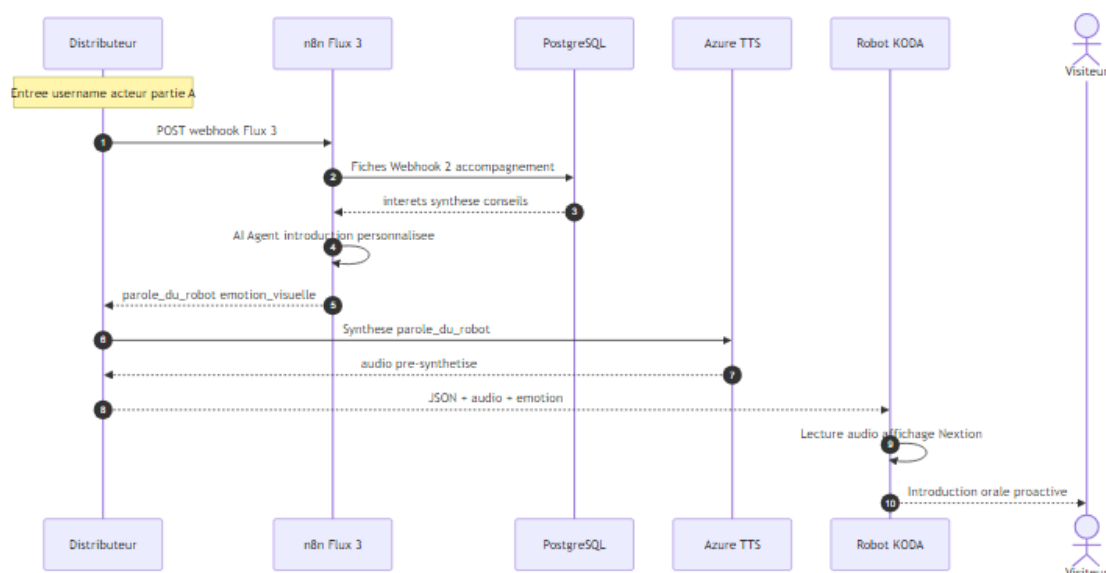


FIGURE IV.13 – Action 2 (b) — sortie : Flux 3, synthèse TTS et restitution au robot

IV.5.1.3.1 Lien avec le Cycle 1.

Le Flux 3 correspond au **Webhook 3** documenté au chapitre III (spontanéité et interaction proactive). Le Distributeur en est le **seul point d'accès** côté robot : il traduit l'appel embarqué en webhook n8n, puis en réponse audio exploitable par l'embarqué.

IV.5.1.4 Canal WebSocket (temps réel)

Outre les requêtes HTTP du pipeline vocal, le robot maintient un canal **WebSocket** persistant avec le Distributeur pour les acquittements, le suivi d'état et l'envoi de commandes moteur ou d'expressions en temps quasi réel. Le protocole repose sur des messages JSON typés (status, action,

command_result)); en cas de coupure réseau, le client embarqué applique une **reconnexion avec backoff exponentiel** (cf. tableau IV.11).

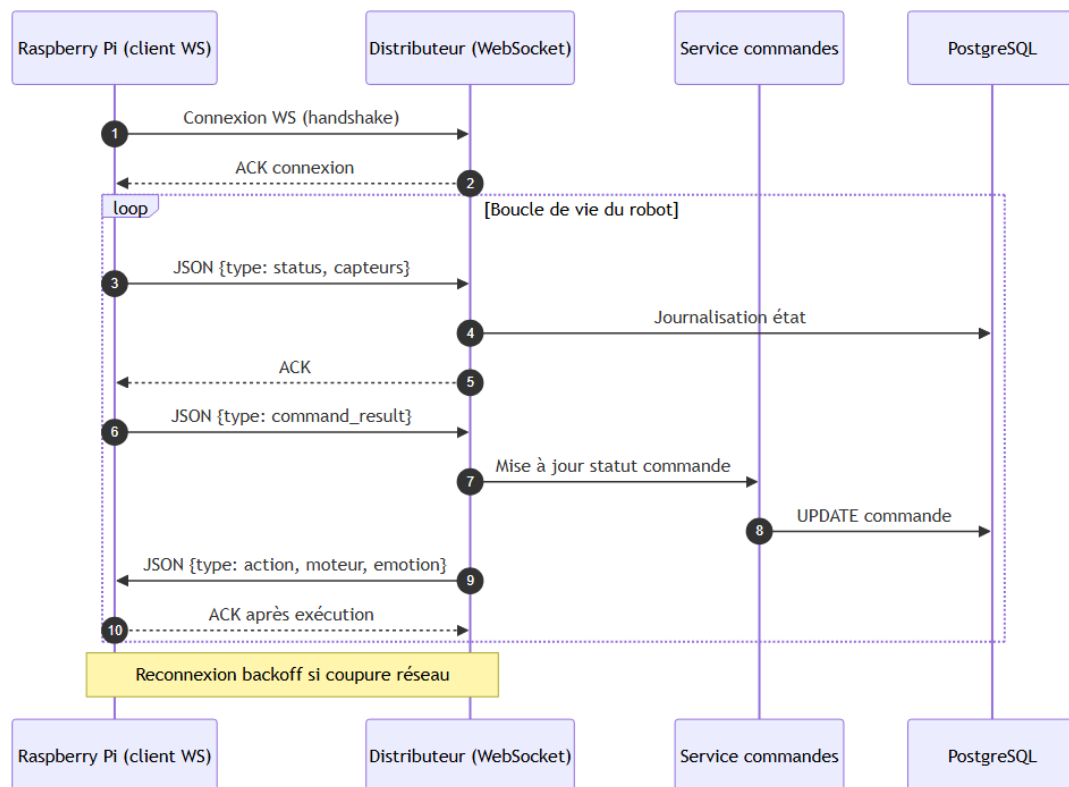


FIGURE IV.14 – Cycle 2 — échanges WebSocket robot ↔ Distributeur (temps réel)

IV.5.1.5 Chaîne vocale complète (Azure STT / TTS)

La dimension vocale est au cœur de l'expérience KODA. Le robot **capture** l'audio localement ; le backend prend en charge l'intégralité du traitement cloud et cognitif, puis renvoie un **output unifié** (audio synthétisé, commandes d'affichage et de mouvement). Ce découpage garantit que l'embarqué reste léger et que les clés Azure ne quittent jamais le serveur.

Le pipeline théorique s'articule ainsi :

1. capture audio sur le robot → envoi au backend (HTTP ou WebSocket) ;
2. transcription **Speech-to-Text** (Azure) ;
3. enrichissement du contexte depuis PostgreSQL (historique, profil) ;
4. traitement conversationnel via **webhook n8n** (Cycle 1) ;
5. synthèse **Text-to-Speech** (Azure) ;
6. retour audio et actions vers le robot.

Les routes REST associées sont recensées au tableau IV.6 (/api/audio/*).

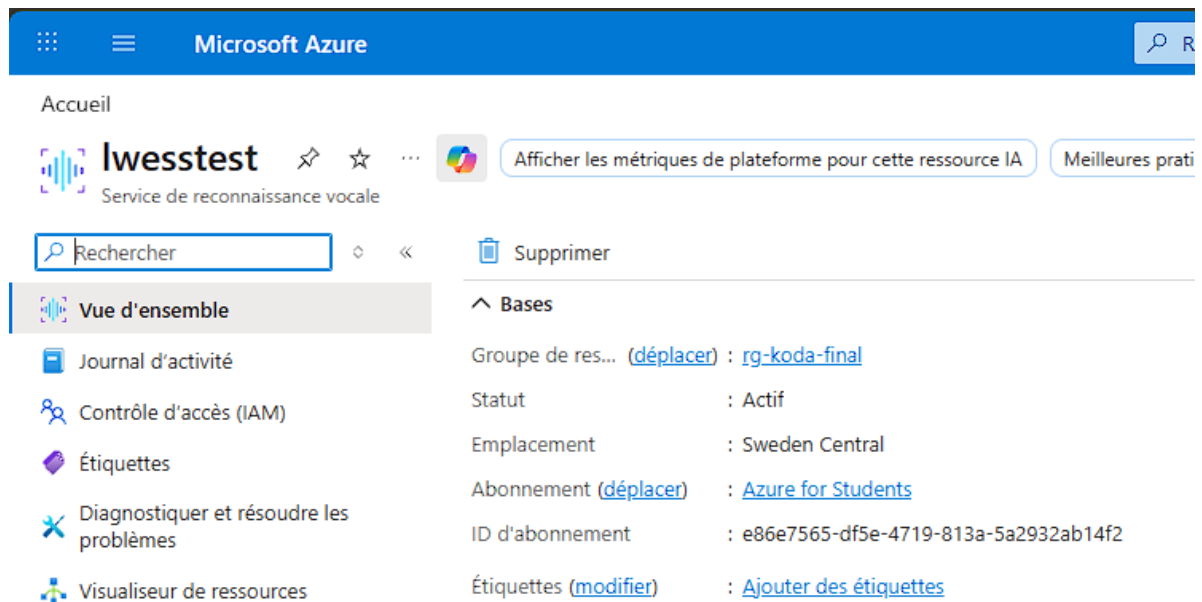


FIGURE IV.15 – Configuration Azure Speech (STT/TTS)

```

● PS C:\Users\l\Desktop\o\PFE\speatchtotext> python .\test_koda_speech.py
Koda est prêt à parler. Entre ton texte :
> bonjour je m'apelle oussema
Succès ! Koda a dit : [bonjour je m'apelle oussema ]
○ PS C:\Users\l\Desktop\o\PFE\speatchtotext>

```

FIGURE IV.16 – Test Text-to-Speech

```

● PS C:\Users\l\Desktop\o\PFE\speatchtotext> python .\koda_listen.py
=====
🎤 KODA LISTEN - Reconnaissance vocale
=====

Koda t'écoute... Parle en français !

✅ Koda a compris !

=====
📄 TEXTE RECONNU : Bonjour, je m'appelle Houssen.
=====

⌚ Affichage pendant 3 secondes...
✅ Terminé !

```

FIGURE IV.17 – Test Speech-to-Text

IV.5.1.6 Reconnaissance des personnes

La reconnaissance faciale constitue le **service d'identité** du Distributeur. Son rôle théorique est de transformer une **présence physique** devant le robot en une **identité numérique** exploitable par le reste du système : sans cette étape, KODA ne pourrait ni personnaliser la salutation, ni retrouver l'historique conversationnel, ni orienter le raisonnement n8n vers le bon profil (accompagnement, notes, préférences).

IV.5.1.6.1 Principe et modèle de données.

L'identification repose sur une **bibliothèque de visages connus** : à l'inscription (depuis l'application mobile ou un flux d'enregistrement), plusieurs photos de référence sont capturées et converties en **encodages vectoriels** (128 dimensions) stockés dans la table `imageuser`, liée à utilisateur. Lors d'une interaction, le robot transmet une capture caméra au Distributeur via `POST /api/identify-face`; le backend détecte le visage, calcule son encodage et le compare à la base de référence. L'identité retenue est celle dont la **distance euclidienne** est minimale, sous réserve d'un **seuil de confiance** (0,5 dans notre configuration).

IV.5.1.6.2 Pipeline d'identification.

Le traitement suit quatre étapes conceptuelles :

1. **Capture** : le Raspberry Pi envoie une image (multipart);
2. **Détection et encodage** : extraction du visage et calcul du vecteur 128D (face_recognition / dlib);
3. **Comparaison** : recherche du profil le plus proche dans PostgreSQL;
4. **Décision** : retour du username identifié et d'un score, ou de l'état `unknown` (personne inconnue) déclenchant un scénario d'accueil générique ou d'invitation à se présenter (Flux 3, chapitre III).

Ce pipeline intervient en **amont** de chaque dialogue vocal (Action 1, figure IV.5.1.2) et lors de la détection de présence (Action 2, figure IV.5.1.3) : l'identité conditionne le ton, le contenu et le routage vers n8n.

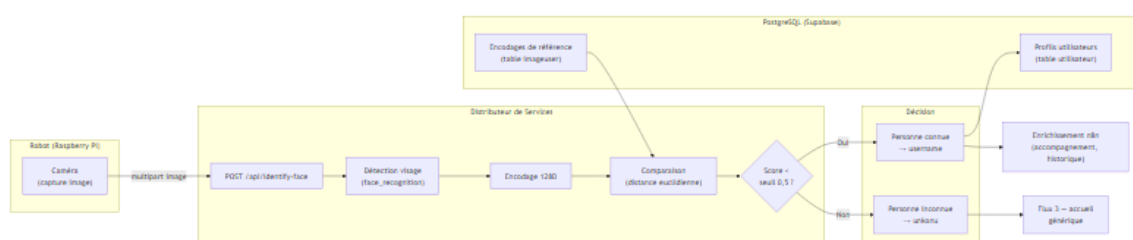


FIGURE IV.18 – Pipeline de reconnaissance faciale — de la capture caméra à l’identité utilisateur

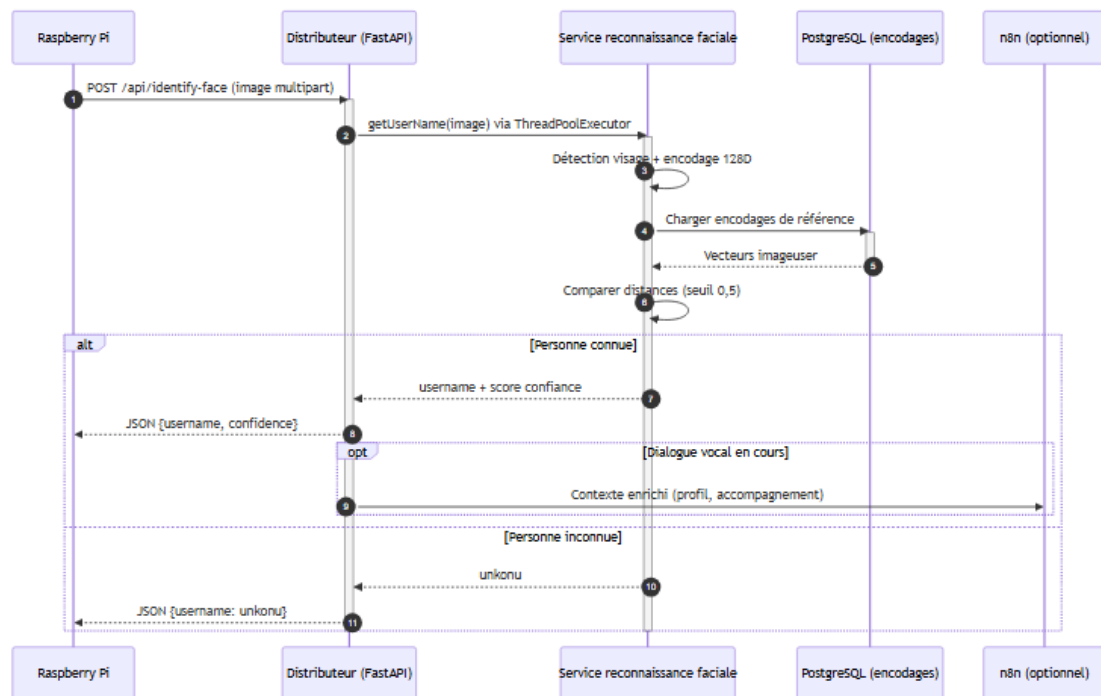


FIGURE IV.19 – Diagramme de séquence — identification faciale (robot, Distributeur, base de référence)



FIGURE IV.20 – Exemple d’images de référence en mémoire (profils utilisateurs)

fig :ch4-face-dataset

IV.5.1.6.3 Cas particuliers.

Trois situations méritent d’être distinguées :

- **Personne connue** : match sous le seuil → personnalisation immédiate (salutation par le prénom, contexte accompagnement) ;
- **Personne inconnue** : aucun match satisfaisant → état unkonu, accueil générique et possibilité d’enregistrement ultérieur ;
- **Enregistrement** : création d’un profil utilisateur et de ses encodages via les endpoints CRUD (/api/users, /api/user-images), sans intervention directe du robot sur la base.

IV.5.1.6.4 Contraintes et intégration.

Le calcul d'encodage étant **CPU-intensif**, le traitement est délégué à un `ThreadPoolExecutor` (`run_in_executor`) afin de ne pas bloquer la boucle asynchrone FastAPI (cf. tableau IV.11). Les encodages fréquents peuvent être mis en cache (Redis) pour réduire la latence sous charge. Le robot ne charge **jamais** la bibliothèque de vision : seul le Distributeur détient les modèles et les références, ce qui respecte le principe du point de liaison unique (§ IV.2).

Résultats de validation

Tests en conditions réelles : taux de reconnaissance supérieur à **94 %** (seuil de distance 0,5), temps moyen **180 ms** par image sur le serveur de déploiement.

IV.5.2 Service dédié vers l'application mobile (KodaMate)

L'application **KodaMate** (.NET MAUI 8, Cycle 5) constitue l'interface utilisateur mobile du système. Elle consomme un **sous-ensemble** des API REST du Distributeur ; elle ne contacte **jamais** directement Supabase ni n8n. Toute opération métier transite par le backend, seul habilité à accéder à la base, aux webhooks et aux clés de services cloud. Le catalogue complet des routes est donné au § IV.6 ; cette sous-section en décrit le **fonctionnement** et les **flux** principaux.

IV.5.2.1 Contexte architectural

KodaMate s'inscrit dans une architecture **client–serveur en trois couches** :

1. **Présentation** : application mobile KodaMate (.NET MAUI) sur smartphone Android (ou autre cible MAUI) ;
2. **Services** : backend Distributeur (FastAPI / Python), API REST centralisée hébergée sur le PC ou le VPS du projet ;
3. **Données** : PostgreSQL distant (Supabase), accessible **exclusivement** via le backend (ORM SQLAlchemy / asyncpg).

Cette séparation garantit la cohérence avec le pipeline vocal du robot : robot, mobile et n8n partagent la **même mémoire** PostgreSQL, mais seul le Distributeur en assure l'accès contrôlé.

IV.5.2.2 Principe de communication

L'échange s'effectue en **HTTP/JSON** sur le réseau local Wi-Fi (ou Internet en déploiement distant) :

- **URL de base** : `http://<IP-du-PC>:8000/api` (configurable) ;

- **Protocole** : REST (GET, POST, PUT) ;
- **Format** : corps et réponses en JSON ;
- **Timeout côté mobile** : 60 secondes (latences Supabase et n8n) ;
- **Validation serveur** : middleware FastAPI transversal (CORS, parsing).

Au **démarrage**, l'application vérifie la disponibilité du serveur via l'écran de connexion. Après authentification réussie, une **session locale** (email, idproduit, idkoda) est conservée dans les préférences MAUI (Preferences) pour les écrans suivants.

IV.5.2.3 Flux fonctionnels principaux

IV.5.2.3.1 Authentification.

L'utilisateur saisit email et mot de passe. L'application envoie : POST /api/auth/login avec {emailclient, motdepasse}. Le backend interroge la table d'authentification sur Supabase, valide les identifiants et renvoie {idproduit, emailclient, idkoda}. Ces données alimentent la configuration robot, le chat et l'historique.

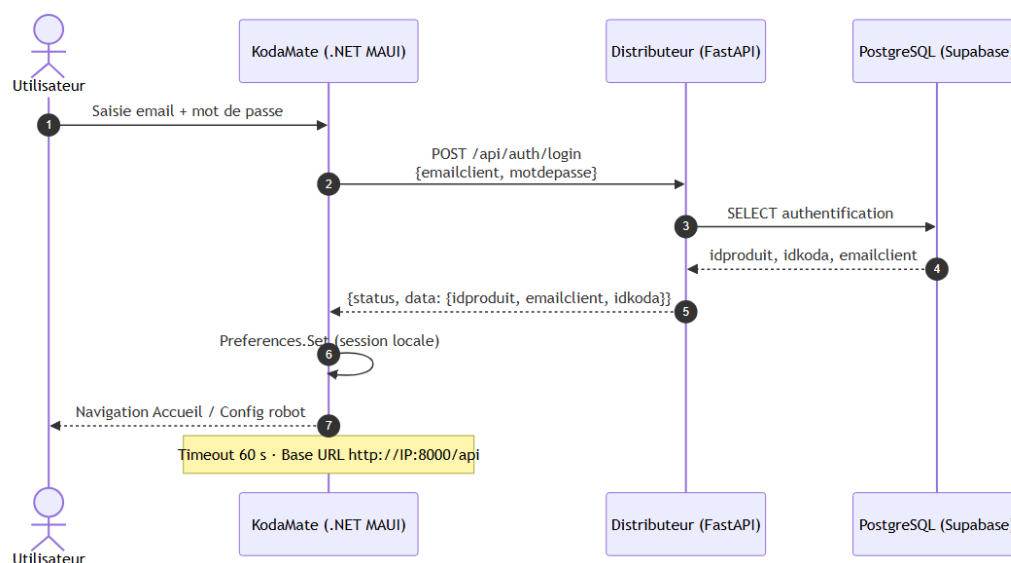


FIGURE IV.21 – Authentification mobile — séquence POST /api/auth/login

IV.5.2.3.2 Configuration et contrôle du robot.

- lecture des paramètres : GET /api/settings (nom, langue, état on/off, caractère, etc.) ;
- enregistrement : PUT /api/settings ;
- allumage / extinction : POST /api/power-on et POST /api/power-off.

L'état du robot (etat) provient de la table setting en base.

IV.5.2.3.3 Chat intelligent (délégation n8n).

Depuis l'onglet Chat, l'utilisateur envoie un message. L'application :

1. ajoute un suffixe identifiant au texte (ex. « oussama »);
2. appelle POST /api/trigger-n8n avec {question: "..."};
3. le backend transmet la requête au workflow n8n (variable N8N_WEBHOOK_URL);
4. n8n traite la question (IA, notes, contexte) et renvoie un JSON contenant le champ output;
5. KodaMate extrait et affiche uniquement le texte de output.

Le mobile **ne connaît pas** l'URL n8n : seul le Distributeur y accède, ce qui respecte le principe du point de liaison unique (§ IV.2).

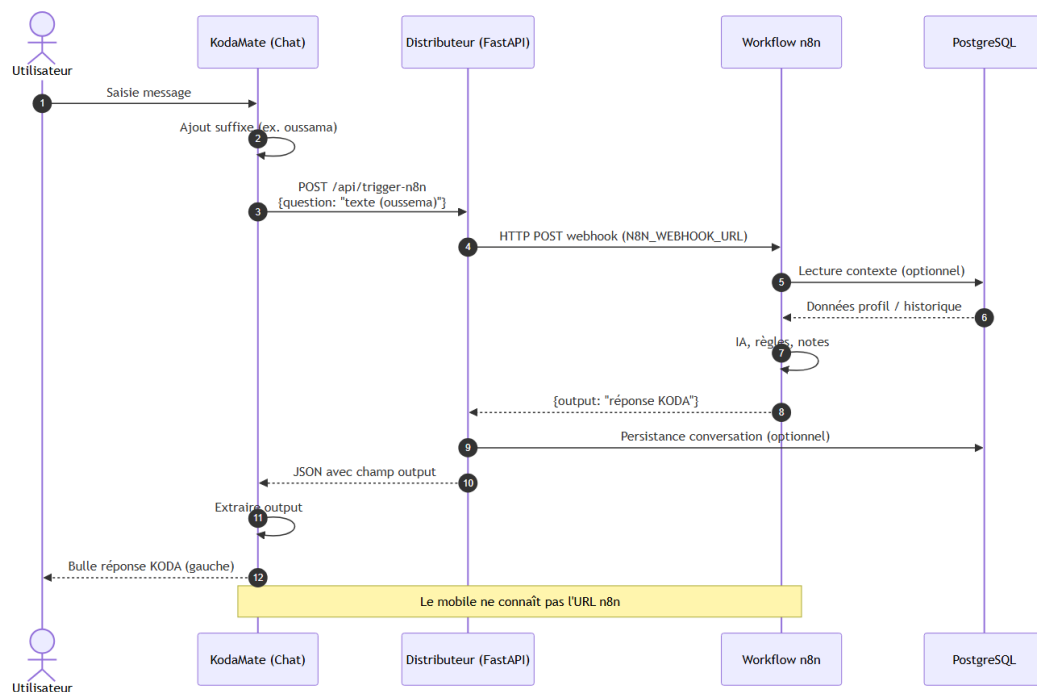


FIGURE IV.22 – Chat mobile — séquence POST /api/trigger-n8n

IV.5.2.3.4 Historique des conversations.

L'onglet Historique charge les échanges via GET /api/conversations (filtres optionnels date_from, date_to), ou en secours GET /api/conversations/last100. Les messages s'affichent en bulles (question utilisateur à droite, réponse KODA à gauche), groupés par date.

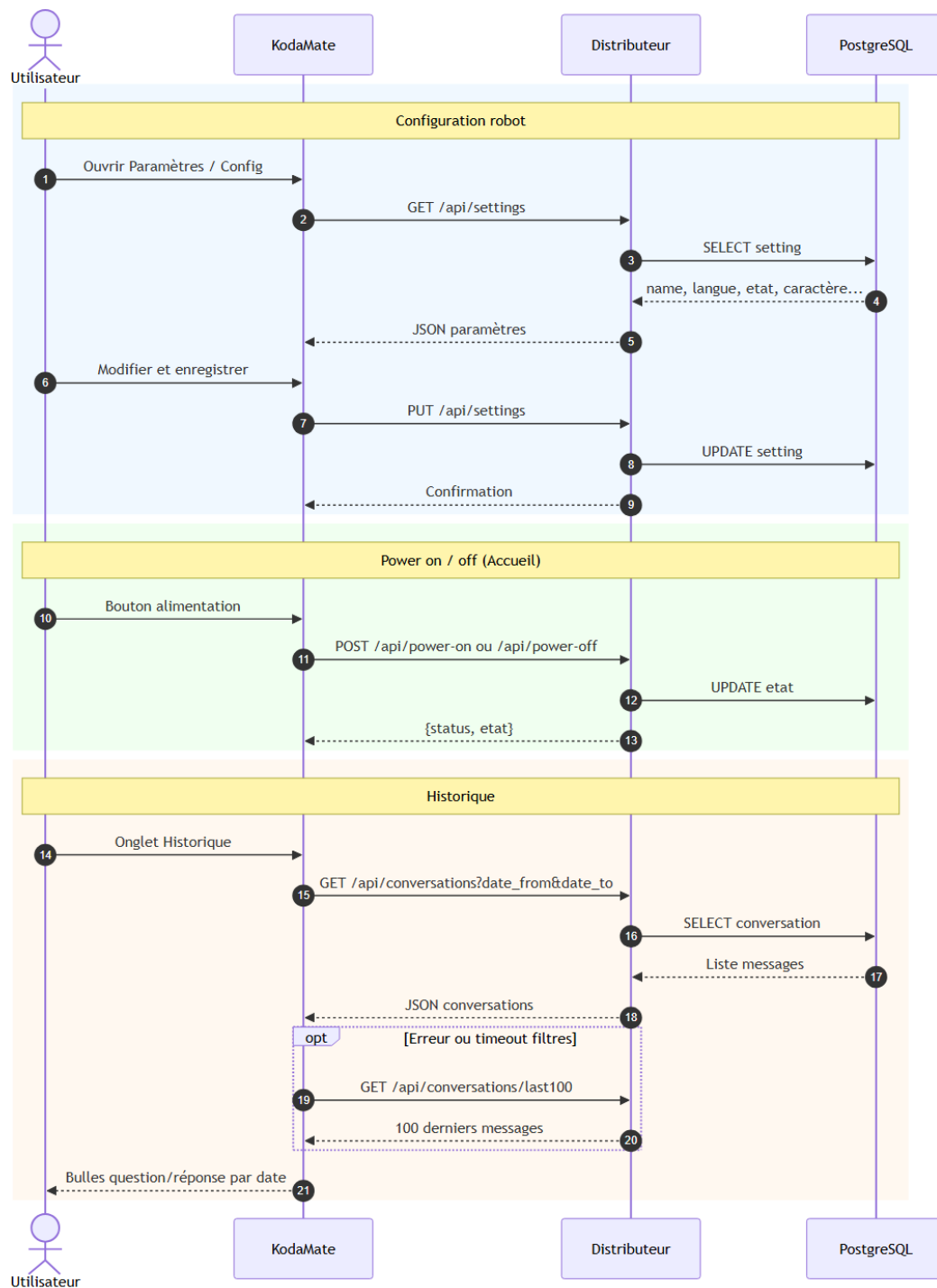


FIGURE IV.23 – Paramètres, power et historique — séquences REST KodaMate

IV.5.2.3.5 Données complémentaires (hors backend).

- **Météo** (écran Accueil) : appel direct à l'API externe Open-Meteo, sans passer par le Distributeur ;
- **Notifications** : service simulé localement (MockFirebaseService), non branché au backend à ce stade ;
- **Wi-Fi** : scan Android natif, sans endpoint backend.


```

PS C:\Users\l\Desktop\o\PFE\Backend + application mobile> curl.exe -X GET "http://127.0.0.1:8000/api/users" -H
"accept: application/json"
[{"id":"98a327fd-0ece-4afc-8b06-5a6499a9ce04","nom":"salah","image":"https://jynducbyosfpdetdomga.supabase.co/
storage/v1/object/public/photos_utilisateurs/s1.jpg"},{"id":"d58ff93a-26d1-4785-867d-4934351cb4e2","nom":"sala
h","image":"https://jynducbyosfpdetdomga.supabase.co/storage/v1/object/public/photos_utilisateurs/s2.jpg"},{"i
d":"798bd831-0837-4c6c-93a1-f9654f22d074","nom":"salah","image":"https://jynducbyosfpdetdomga.supabase.co/stor
age/v1/object/public/photos_utilisateurs/s3.jpg"},{"id":"ec7abfae-1460-473e-a5ab-c7dd65f091ea","nom":"salah","
image":"https://jynducbyosfpdetdomga.supabase.co/storage/v1/object/public/photos_utilisateurs/s4.jpg"},{"id":"
2a6df721-a61d-42e7-9ee3-f0a9e9acf756","nom":"salah","image":"https://jynducbyosfpdetdomga.supabase.co/storage/
v1/object/public/photos_utilisateurs/s5.jpg"},{"id":"77af3963-e661-423f-8b48-5a70f79cf668","nom":"salah","imag
e":"https://jynducbyosfpdetdomga.supabase.co/storage/v1/object/public/photos_utilisateurs/s6.jpg"},{"id":"b788
4803-0039-474f-9c99-eb44d7a07063","nom":"salah","image":"https://jynducbyosfpdetdomga.supabase.co/storage/v1/o
bject/public/photos_utilisateurs/s7.jpg"},{"id":"0c8f6a3c-8a3b-45e8-ac7e-191c1ee861bd","nom":"oussema","image"
:"https://jynducbyosfpdetdomga.supabase.co/storage/v1/object/public/photos_utilisateurs/o1.jpg"},{"id":"ac2884
e6-b761-405a-a2a9-93f4822c52e5","nom":"oussema","image":"https://jynducbyosfpdetdomga.supabase.co/storage/v1/o
bject/public/photos_utilisateurs/o2.jpg"},{"id":"6360315d-0c7d-4194-a518-393018d183e7","nom":"oussema","image"
:"https://jynducbyosfpdetdomga.supabase.co/storage/v1/object/public/photos_utilisateurs/o3.jpg"},{"id":"e2e5ba
d9-2b00-42c6-b59d-392cf9c10673","nom":"oussema","image":"https://jynducbyosfpdetdomga.supabase.co/storage/v1/o
bject/public/photos_utilisateurs/o4.jpg"},{"id":"de605ab3-4d9c-4aac-b26e-de47c141c22a","nom":"oussema","image"
:"https://jynducbyosfpdetdomga.supabase.co/storage/v1/object/public/photos_utilisateurs/o5.jpg"},{"id":"51b91c
f9-8512-42d7-84e2-efe9d043bef5","nom":"oussema","image":"https://jynducbyosfpdetdomga.supabase.co/storage/v1/o
bject/public/photos_utilisateurs/o6.jpg"},{"id":"38e05d5f-149f-444f-874e-9ede96e85696","nom":"oussema","image"
:"https://jynducbyosfpdetdomga.supabase.co/storage/v1/object/public/photos_utilisateurs/o7.jpg"},{"id":"912fb4
8b-dc88-4314-911d-e5896de7f939","nom":"oussema","image":"https://jynducbyosfpdetdomga.supabase.co/storage/v1/o
bject/public/photos_utilisateurs/o8.jpg"},{"id":"8a0deca6-72fe-48ce-a2af-2802c6cc90db","nom":"oussema","image"
:"https://jynducbyosfpdetdomga.supabase.co/storage/v1/object/public/photos_utilisateurs/o9.jpg"}]
PS C:\Users\l\Desktop\o\PFE\Backend + application mobile> curl.exe -X GET "http://127.0.0.1:8000/api/agendas"
-H "accept: application/json"
[{"id":1,"titre":"aaaa","note":"aaaa","etat":true,"contenu":"aaaa","date_modification":"2026-04-12T16:37:12.74
2374+00:00"}]
PS C:\Users\l\Desktop\o\PFE\Backend + application mobile> curl.exe -X POST "http://127.0.0.1:8000/api/power-on"
-H "accept: application/json"
{"status":"power_on","etat":1}
PS C:\Users\l\Desktop\o\PFE\Backend + application mobile> curl.exe -X GET "http://127.0.0.1:8000/test"
{"message":"Test OK"}

```

FIGURE IV.24 – Exécution de quelques endpoints (tests manuels)

IV.5.3 Service dédié vers la base de données (PostgreSQL / Supabase)

La couche de persistance repose sur **PostgreSQL** hébergé sur **Supabase** (interface d'administration, SQL, gestion des accès). Le backend y accède via **SQLAlchemy** (ORM) et **asyncpg** (asynchrone), selon la chaîne : *API REST* → *cas d'usage* → *repository* → *PostgreSQL*. C'est la **mémoire partagée** entre interactions vocales et applicatives.



FIGURE IV.25 – Interface Supabase — administration PostgreSQL

IV.5.3.1 Modèle relationnel et rôle des tables

Le modèle couvre les dimensions conversationnelles, contextuelles et organisationnelles. Principales tables :

- **conversation** (**id, question, reponce, date, type_de_question, personne**) : archive l'ensemble des conversations entre le robot et l'humain.
- **historique** (**id, date, question, reponce, type_de_question**) : conserve les 5 derniers messages pour la mémoire contextuelle immédiate.
- **information_personnel** (**id, question, reponce, date, acteur, type_de_question**) : stocke les informations personnelles liées à l'humain ou au robot.
- **note** (**id, note, etat**) : enregistre les rendez-vous et dates importantes de l'utilisateur.
- **setting** (**id, name, sexe, date, motdepasse, etat, caractere, langue**) : centralise l'identité et les paramètres généraux de KODA.
- **utilisateur** (**id, nom, image_url**) : référence les personnes connues par KODA avec leur nom et leur image.
- **agenda** (**id, titre, note, etat, contenu, date_modification**) : enregistre les éléments d'agenda saisis depuis l'application mobile.
- **activity** (**id, titre, description, etat, date, createdat**) : stocke les activités de l'utilisateur provenant de l'application mobile.
- **accompagnement** (**id, interets, synthese, point_cles, conseils, acteur**) : conserve l'analyse quotidienne consolidée pour chaque personne connue.

Le détail des relations entre tables est donné par la figure IV.3.1 (§ IV.3).

IV.5.4 Service dédié vers le workflow n8n (Cycle 1)

Le moteur de workflow **n8n** (Cycle 1) joue le rôle d'**orchestrateur cognitif** : il reçoit une question enrichie, applique la logique de traitement et renvoie une réponse structurée. Le backend ne duplique pas cette intelligence : il **prépare le contexte** (identité, historique), **appelle le web-hook**, **valide la réponse** et **persiste** l'échange. Cette séparation permet de modifier rapidement le comportement métier dans n8n sans impacter la couche d'exposition API ni le robot.

Le scénario standard suit les étapes suivantes :

1. récupération du **nom de l'utilisateur** (ou identifiant) et de la **question** formulée ;
2. construction d'un payload JSON normalisé (ex. : `{"name": "...", "question": "..."};`);
3. envoi au webhook n8n via HTTP POST (timeout configurable) ;
4. exécution du workflow : enrichissement, appels LLM, règles de décision ;
5. retour de la **réponse finale** et métadonnées éventuelles ;

6. validation, journalisation et renvoi au robot ou à l'application cliente.

Sur le plan qualité, cette interface inclut la gestion des *timeouts*, le contrôle des erreurs de webhook et la validation du schéma de réponse.

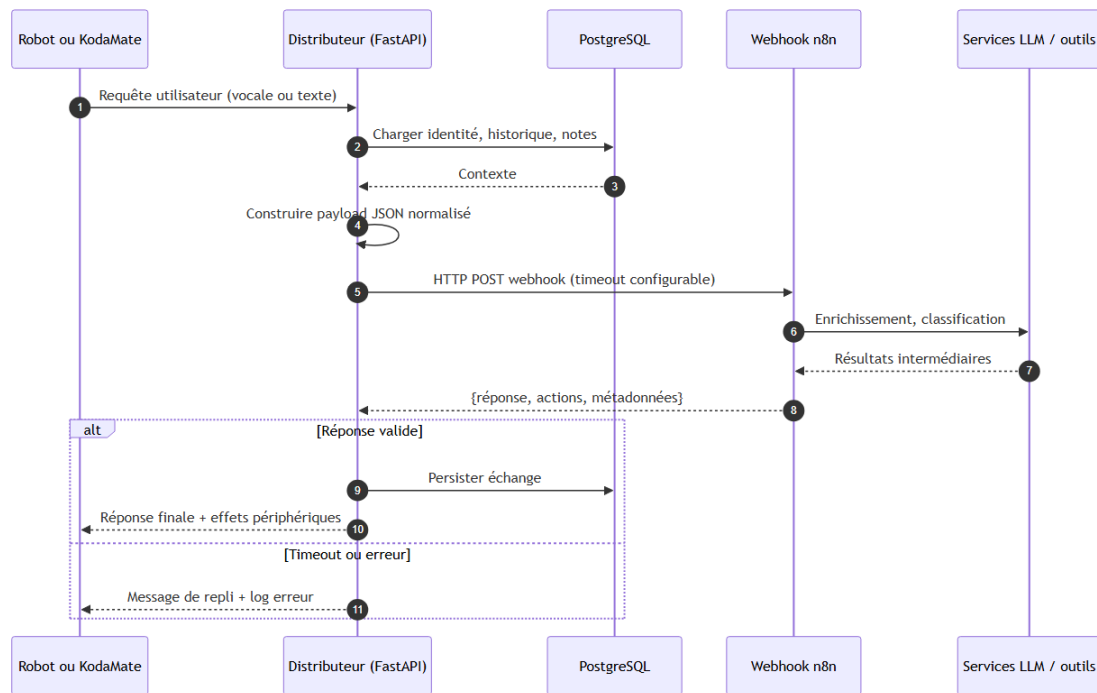


FIGURE IV.26 – Délégation au workflow n8n — séquence robot ou mobile → Distributeur → n8n

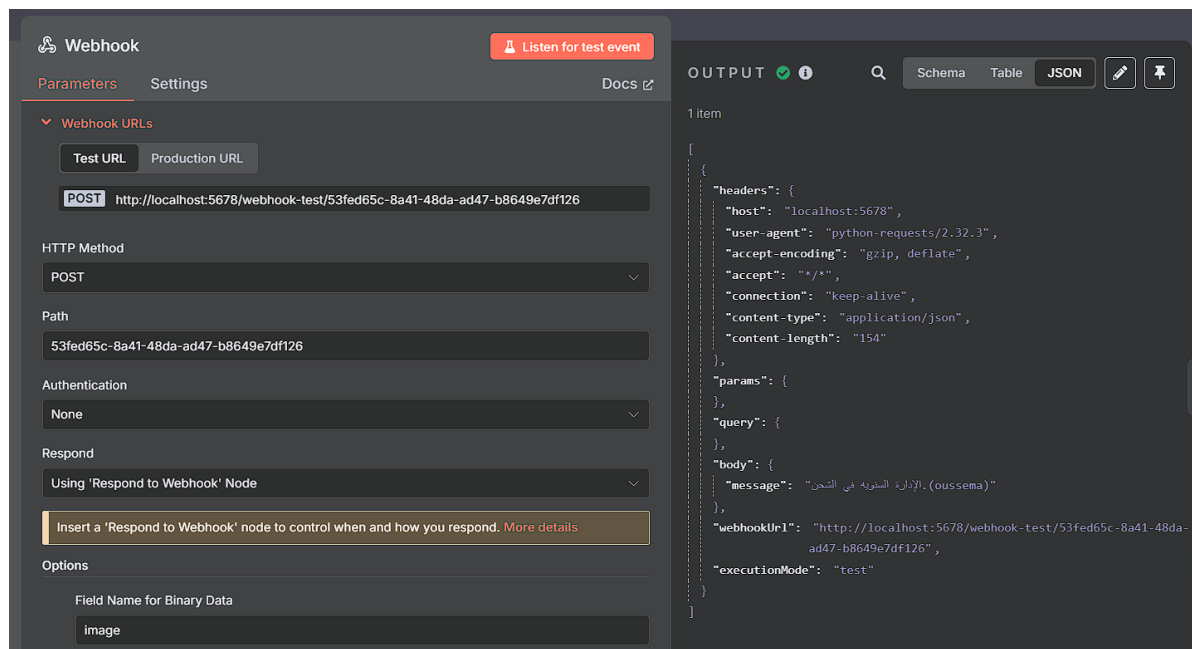


FIGURE IV.27 – Communication avec n8n — exemple de payload en entrée de webhook

IV.6 Catalogue des endpoints REST du Distributeur

Le Distributeur expose **65 routes HTTP** au total : **60 endpoints métier** sous le préfixe `/api` et **5 routes** de documentation ou de santé à la racine. Plutôt qu’une liste plate, ce catalogue les regroupe selon **l’axe de communication** qu’ils matérialisent : avec le **système embarqué**, l’**application mobile**, la **base PostgreSQL**, le **workflow n8n**, ou les outils d’**exploitation**. Certains endpoints sont **partagés** entre plusieurs clients (ex. : `/api/conversations` alimenté par le robot et consulté par KodaMate).

TABLE IV.2 – Répartition des 65 endpoints par axe de communication

Axe de communication	Nb	Rôle principal
Exploitation / documentation	5	Santé serveur, Swagger, OpenAPI (<code>/</code> , <code>/docs...</code>)
Persistance PostgreSQL (Supabase)	46	CRUD sur utilisateurs, conversations, notes, paramètres, etc.
Système embarqué (Raspberry Pi)	5	Audio Azure, reconnaissance faciale, mot-clé de réveil
Délégation n8n / IA	2	Déclenchement workflow et pipeline vocal complet
Application mobile (KodaMate)	9 actifs	Consommés à l’écran ; 5 endpoints « notes » codés mais non branchés
Total backend	65	60 métier + 5 docs/racine

Remarque : le décompte par axe peut se chevaucher : un endpoint de persistance est invoqué indifféremment par le robot, n8n (via le backend) ou l’application mobile.

IV.6.1 Infrastructure et documentation (hors `/api`)

Ces routes servent au **déploiement**, aux tests de connectivité et à la documentation interactive FastAPI.

TABLE IV.3 – Endpoints racine — exploitation et documentation

#	Méth.	Endpoint	Description
1	GET	<code>/</code>	Santé serveur (message, préfixes de routes)
2	GET	<code>/test</code>	Test simple de disponibilité
3	GET	<code>/docs</code>	Interface Swagger UI (FastAPI)
4	GET	<code>/redoc</code>	Documentation ReDoc
5	GET	<code>/openapi.json</code>	Schéma OpenAPI exportable

IV.6.2 Communication avec PostgreSQL (persistance Supabase)

Ces endpoints traduisent la couche **mémoire durable** du système. Ils sont consommés par le backend lui-même au service du robot, de n8n ou de KodaMate. La chaîne interne reste : *route REST* → *cas d’usage* → *repository* → *PostgreSQL*.

IV.6.2.1 Santé de la base

#	Méth.	Endpoint	Description
6	GET	/api/db-test	Test de connexion Supabase / PostgreSQL

TABLE IV.4 – Endpoint de diagnostic base de données

IV.6.2.2 Entités persistées (CRUD)

L’endpoint `POST /api/auth/login` (#47) appartient à la persistance des comptes tout en constituant le **point d’entrée** de l’application mobile (cf. § IV.6.5).

IV.6.3 Communication avec le système embarqué (Raspberry Pi)

Ces routes sont **primordialement** consommées par le Raspberry Pi (Cycle 4) pour la voix, la vision et le mot de réveil. Elles s’ajoutent au canal **WebSocket** (temps réel) décrit au § IV.5.

Le robot s’appuie également, de façon indirecte, sur les endpoints de **persistance** (utilisateurs, historique, conversations) lorsque n8n ou le backend enregistrent une interaction vocale.

IV.6.4 Communication avec le workflow n8n (Cycle 1)

`POST /api/trigger-n8n` est aussi invoqué par **KodaMate** depuis l’écran Chat ; c’est l’exemple le plus visible d’un endpoint **croisé** entre robot (via pipeline vocal) et application mobile (saisie texte).

IV.6.5 Communication avec l’application mobile (KodaMate)

L’application **KodaMate** (.NET MAUI) n’utilise qu’un **sous-ensemble** du catalogue : neuf endpoints backend sont effectivement appelés à l’écran, auxquels s’ajoute une API météo externe. Les endpoints « alimentation robot » (`power-on/off`) sont exclusifs au mobile ; le Wi-Fi et les notifications restent locaux (pas de route backend).

TABLE IV.5 – Endpoints CRUD — communication avec PostgreSQL par entité

#	Méth.	Endpoint	Entité / table
<i>Utilisateurs (utilisateur)</i>			
7–11	GET/POST/PUT/DELETE	/api/users, /api/users/{iduser}	Profils des personnes connues de KODA
<i>Images utilisateur (imageuser)</i>			
12–17	GET/POST/PUT/DELETE	/api/user-images..., /by-user/{iduser}	Photos et encodages faciaux
<i>Historique contextuel (historique)</i>			
18–21	GET/POST/DELETE	/api/history, /api/history/{historique_id}	Fenêtre des derniers messages (mémoire immédiate)
<i>Conversations (conversation)</i>			
22–27	GET/POST/PUT/DELETE	/api/conversations..., /last100	Archive des échanges robot–humain
<i>Notes et rendez-vous (note)</i>			
28–33	GET/POST/PUT/PATCH/DELETE	/api/notes..., /etat?etat=	Agenda, rappels, bascule d'état
<i>Paramètres robot (setting)</i>			
34–35	GET/PUT	/api/settings	Identité KODA (nom, langue, caractère, état)
<i>Informations personnelles</i>			
36–40	GET/POST/PUT/DELETE	/api/personal-info..., /user/{iduser}	Faits mémorisés sur l'utilisateur
<i>Accompagnements</i>			
41–46	GET/POST/PUT/DELETE	/api/accompagnements..., /user/{iduser}	Synthèses et conseils quotidiens
<i>Inscriptions produit (auth/registrations)</i>			
47–52	POST/GET/PUT/DELETE	/api/auth/login, /api/auth/registrations...	Authentification et comptes KodaMate
<i>Activités (alias notes)</i>			
54–58	GET/POST/PUT/DELETE	/api/activities...	Activités utilisateur saisies depuis le mobile

TABLE IV.6 – Endpoints orientés robot — audio, vision et mot-clé

#	Méth.	Endpoint	Description
59	POST	/api/identify-face	Reconnaissance faciale (multipart, fichier image)
62	POST	/api/audio/speech-to-text	Transcription Azure STT
63	POST	/api/audio/text-to-speech	Synthèse Azure TTS
64	POST	/api/audio/speech-to-n8n-to-speech	Pipeline vocal complet (STT → n8n → TTS)
65	GET	/api/keywords/robot-name	Mot-clé / nom de réveil du robot

TABLE IV.7 – Endpoints de délégation au moteur n8n

#	Méth.	Endpoint	Description
53	POST	/api/trigger-n8n	Envoi {question} au webhook n8n, retour {output}
64	POST	/api/audio/speech-to-n8n-to-speech	Variante vocale intégrée (STT + n8n + TTS)

TABLE IV.8 – Endpoints backend effectivement utilisés par KodaMate (UI)

#	Méth.	Endpoint	Écran	Entrée	Sortie
1	GET	/	Démarrage, Login, session	—	{message, routes_prefix}
2	POST	/api/auth/login	Connexion	emailclient, motdepasse	{status, data:{idproduit, idkoda}}
3	GET	/api/settings	Accueil, Paramètres, Config	—	nom, langue, caractère, etat, idkoda...
4	PUT	/api/settings	Paramètres, Config robot	corps setting	lignes mises à jour
5	POST	/api/power-on	Accueil (power)	—	{status: power_on, etat: 1}
6	POST	/api/power-off	Accueil (power)	—	{status: power_off, etat: 0}
7	POST	/api/trigger-n8n	Chat (+ photo)	{question}	{output:...}
8	GET	/api/conversations?date_from={date}&date_to={date}	Historique (date)	dates YYYY-MM-DD	liste conversations
9	GET	/api/conversations/latest	Historique (repli)	—	100 derniers messages max

IV.6.5.1 Endpoints consommés à l'écran

IV.6.5.2 Endpoints codés mais non branchés à l'interface

Cinq routes `/api/notes` sont implémentées dans `DistributeurApiService` mais aucun écran ne les invoque encore : CRUD complet + PATCH `.../etat`.

IV.6.5.3 API externe et services locaux

TABLE IV.9 – Appels réseau complémentaires de KodaMate (hors Distributeur)

#	Méth.	URL / service	Écran
15	GET	<code>api.open-meteo.com/v1/forecast...</code>	Accueil (météo)
—	—	<code>MockFirebaseService</code> (local)	Notifications
—	—	<code>Scan Wi-Fi Android</code> (local)	Paramètres réseau

IV.6.5.4 Cartographie écran → endpoints

TABLE IV.10 – Mapping des écrans KodaMate vers les endpoints backend

Écran	Endpoints invoqués
Démarrage / Session	GET <code>/</code> , GET <code>/api/settings</code>
Login	GET <code>/</code> , POST <code>/api/auth/login</code>
Config robot	GET <code>/api/settings</code> , PUT <code>/api/settings</code>
Accueil	GET <code>/api/settings</code> , POST <code>/api/power-on</code> , POST <code>/api/power-off</code> , Open-Meteo
Chat	POST <code>/api/trigger-n8n</code>
Historique	GET <code>/api/conversations</code> , GET <code>/api/conversations/last100</code>
Paramètres	GET <code>/api/settings</code> , PUT <code>/api/settings</code>

Synthèse de consommation — KodaMate

- **65** endpoints backend au total (60 métier + 5 documentation) ;
- **9** endpoints backend actifs à l'écran + **1** API externe (météo) ;
- **5** endpoints `/api/notes` prêts dans le code client, non branchés ;
- **56** routes backend réservées au robot, à l'audio, aux CRUD avancés ou à une évolution future de l'UI mobile.

IV.7 Problèmes rencontrés et solutions apportées

Le développement et l'intégration du backend ont révélé des difficultés **techniques** (performance, concurrence) et **d'intégration** (réseau, CORS). Le tableau IV.11 synthétise les principaux cas ; les

deux sous-sections suivantes détaillent les problèmes de connectivité les plus impactants pour la chaîne robot–backend–mobile.

TABLE IV.11 – Problèmes techniques rencontrés et solutions apportées

Problème rencontré	Impact	Solution adoptée
Latence élevée de la reconnaissance faciale sous charge	Blocage du thread principal FastAPI	Utilisation de <code>run_in_executor</code> pour déléguer le traitement au <code>ThreadPoolExecutor</code>
Déconnexions intempestives du WebSocket robot	Perte de commandes, état incohérent	Reconnexion automatique avec <i>backoff</i> exponentiel
Conflits de concurrence sur la base de données	Doublons de sessions et d'événements	Transactions atomiques et <code>SELECT FOR UPDATE</code>
Taille des encodages faciaux (128 flottants / profil)	Comparaisons massives lentes	Indexation vectorielle et cache Redis pour les encodages fréquents
Quota Azure STT dépassé lors des tests intensifs	Interruption du service vocal	Système de quota local et <i>fallback</i> Whisper (OpenAI)
Conflit d'adresses serveur (Raspberry Pi vs backend)	Robot injoignable ou connecté au mauvais hôte (<code>localhost</code> , IP obsolète)	Fichier de configuration centralisé (<code>.env</code>) avec URL publique ou IP LAN du VPS ; distinction explicite URL backend / URL n8n
Erreurs CORS (backend vs application mobile)	Requêtes MAUI / client HTTP bloquées (politique CORS)	<code>CORSMiddleware</code> FastAPI avec liste d'origines autorisées (émulateur, device, production)

IV.7.1 Conflit d'adressage Raspberry Pi / backend

Lors des premiers tests d'intégration, le Raspberry Pi était configuré avec `http://localhost:8000` ou une adresse IP de poste de développement devenue inaccessible après déploiement du backend sur un VPS. Symptômes : WebSocket robot en échec permanent, pipeline vocal silencieux, commandes non acquittées.

Analyse : le robot, le backend et n8n possèdent chacun une URL distincte ; confondre l'adresse du poste local avec celle du serveur de production crée un **conflit de routage** invisible côté logs n8n (le workflow n'est jamais appelé car la requête n'atteint pas le backend).

Solution : externalisation de l'URL backend dans un fichier `.env` partagé entre environnements (dev LAN, staging, production), tests de connectivité (`curl` / ping WebSocket) avant chaque session robot, et documentation d'une **base URL unique** consommée par l'embarqué (Cycle 4).

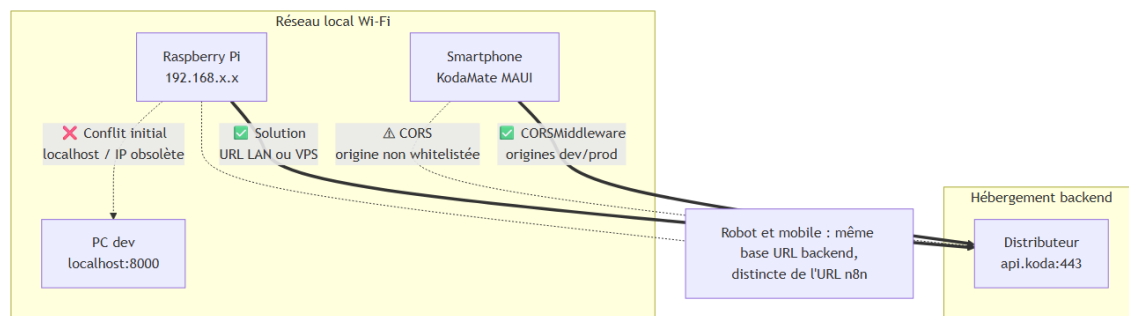


FIGURE IV.28 – Conflit d’adressage et CORS — topologie réseau et solutions

IV.7.2 Problème CORS entre backend et application mobile

L’application KodaMate (.NET MAUI), exécutée sur émulateur Android ou appareil physique, envoie des requêtes HTTP cross-origin vers le backend FastAPI. Sans en-têtes `Access-Control-Allow-Origin` adaptés, le client rejette les réponses (« blocked by CORS policy ») même si le serveur traite correctement la requête.

Solution : configuration explicite du `CORSMiddleware` : origines autorisées (`http://localhost:*`, IP LAN du poste de dev, domaine de production), méthodes (`GET`, `POST`, `PUT`, `DELETE`, `OPTIONS`) et en-têtes (`Authorization`, `Content-Type`). En environnement de production, la liste est restreinte aux domaines KodaMate déployés.

Un défi transversal a également concerné la gestion des dépendances circulaires entre modules dans une architecture Clean. Python, contrairement à Java, n’impose pas nativement l’inversion de dépendances, ce qui peut conduire à des couplages non voulus. Ce problème a été résolu par l’utilisation systématique d’interfaces (classes abstraites Python) et d’un conteneur d’injection de dépendances (`dependency_injector`).

IV.8 Perspectives et évolutions futures

Le Cycle 2 pose une **fondation** ; plusieurs axes d’évolution sont identifiés pour les cycles suivants et la mise en production :

- **Observabilité** : métriques Prometheus, traçage des appels n8n/Azure et tableaux de bord de disponibilité du backend ;
- **Résilience vocale** : bascule automatique STT/TTS multi-fournisseurs selon quota et latence ;
- **Sécurité renforcée** : rotation des JWT, rate limiting sur les endpoints publics, chiffrement des flux WebSocket en production ;
- **Scalabilité** : déploiement multi-instances derrière un reverse proxy, avec sessions WebSocket « sticky » ;

- **Alignement mobile–robot** : notifications push depuis le backend lorsqu'un événement agenda modifié dans l'app doit être annoncé vocalement par KODA (Cycle 5).

Ces perspectives s'appuient sur la **flexibilité** intrinsèque du Distributeur : tant que les contrats d'interface (REST, WebSocket, webhook) restent stables, les composants amont et aval peuvent évoluer sans remise en cause du cœur du système.

IV.9 Conclusion

Le Cycle 2 a permis de concevoir, de développer et de valider le composant le plus structurant de l'écosystème KODA : le **Distributeur de Services**. Ce backend Python / FastAPI, organisé selon la Clean Architecture, **matérialise la fonctionnalité globale** du système en offrant au robot et à l'application mobile un accès unifié aux services vocaux, à la persistance, au moteur n8n et aux APIs cloud.

L'approche retenue — **un seul pivot** plutôt que des liaisons directes entre composants — garantit la **flexibilité** des évolutions (spirale de développement) et la **disponibilité** des échanges (validation, retry, journalisation). Les résultats obtenus (reconnaissance faciale >94 %, chaîne Azure STT/TTS, persistance relationnelle, API mobile) constituent une base testable sur laquelle s'appuient les cycles embarqué (III–IV) et mobile (V).

Les difficultés d'intégration (adressage réseau Raspberry Pi, CORS KodaMate) ont mis en évidence l'importance d'une configuration d'environnement rigoureuse ; elles sont documentées pour faciliter le déploiement futur. Le chapitre suivant (Cycle 3) traite du **matériel embarqué** sur lequel le backend vient s'interfacer physiquement.

Points clés du Cycle 2

- Distributeur = pivot unique pour flexibilité et disponibilité
- Services backend = fonctionnalité globale (vocal + application + mémoire)
- Catalogue de **65 endpoints REST** répartis par axe de communication
- KodaMate : 9 endpoints actifs + 5 notes préparés + 1 API météo externe
- Flux détaillés : WebSocket robot, pipeline vocal, REST mobile, webhook n8n
- Problèmes intégration : conflit adresses serveur, CORS mobile
- Reconnaissance faciale >94 % ; Azure STT/TTS ; persistance Supabase

———— CHAPITRE V ————

CYCLE 3 : SYSTÈME EMBARQUÉ — PARTIE MATÉRIELLE

V.1 Introduction

Après le **Cycle 1** (moteur de réflexion n8n) et le **Cycle 2** (Distributeur de Services), le **Cycle 3** porte sur le **corps matériel** de KODA : choix des composants, câblage, alimentation et validation physique du prototype.

Ce chapitre reste volontairement centré sur le **hardware** et les **concepts d'intégration**. Le logiciel embarqué (scripts Raspberry Pi, protocoles série, audio, caméra) fera l'objet du **Cycle 4**.

Nous présentons l'architecture globale, la **conception 3D du châssis** (Blender), chaque sous-système (contrôle, actionneurs, audio, vision, affichage), le schéma de câblage, puis une campagne de **tests unitaires par composant** suivie d'un **test d'intégration** complète de la plateforme.

Objectif du Cycle 3

Concevoir, assembler et valider la plateforme matérielle du robot KODA, intégrant mobilité, interaction vocale, vision et expression faciale, prête à accueillir la couche logicielle embarquée du Cycle 4.



FIGURE V.1 – Prototype physique KODA après impression 3D et montage

V.2 Architecture Matérielle Globale

L'architecture matérielle de KODA repose sur une conception **biprocasseur** articulée autour de deux unités de traitement complémentaires :

- **Raspberry Pi 4 Model B** : unité centrale intelligente, responsable du traitement des flux audio et vidéo, de la communication réseau avec le backend, et du pilotage de l'écran d'expression faciale.

- **Arduino Uno + Shield L293D** : unité de commande temps réel, dédiée au contrôle des actionneurs (servomoteurs et moteurs DC), recevant ses instructions du Raspberry Pi via une liaison série USB.

Cette séparation est motivée par un principe d'ingénierie fondamental : le Raspberry Pi, fonctionnant sous un système d'exploitation Linux, n'offre pas de garanties temps réel strictes pour le pilotage de signaux PWM. L'Arduino, en revanche, exécute un programme monotâche cadencé à 16 MHz, garantissant une précision temporelle adaptée au contrôle moteur.

Le tableau V.1 synthétise l'ensemble des composants matériels du robot.

TABLE V.1 – Inventaire des composants matériels de KODA

Catégorie	Composant	Rôle
Unité centrale	Raspberry Pi 4 Model B	Traitement IA, réseau, audio/vidéo
Commande moteur	Arduino Uno + Shield L293D	Pilotage PWM des actionneurs
Mobilité	2 moteurs DC (M1, M3)	Déplacement du robot (traction différentielle)
Articulations	3 servomoteurs (S1, S2, S3)	Bras droit, bras gauche ; S3 = écran Nextion
Entrée audio	ReSpeaker 4-Mic Array USB	Capture vocale + direction de la voix
Sortie audio	Jack + PAM8403 + HP ; Bluetooth	Restitution filaire ou sans fil
Vision	Caméra Raspberry Pi (CSI)	Acquisition d'images pour identification
Affichage	Nextion NX4832T035_011 (3,5")	Visage animé du robot
Alimentation Pi	Power Bank 5V / 3A	Alimentation du Raspberry Pi
Alimentation moteurs	3 × piles 3,7V	Alimentation du Shield L293D

V.3 Conception mécanique et châssis 3D (Blender)

Au-delà du câblage électronique, KODA repose sur un **habillage mécanique** imprimé en 3D, conçu sous **Blender**. Le point de départ n'était pas une création entièrement *from scratch*, mais un **modèle 3D existant** (humanoid simplifié) que nous avons dû **adapter** au prototype réel : découpe des volumes, redimensionnement des pièces, ajout d'ouvertures manquantes (logement écran Nextion, passage caméra sur le torse, fixation des bras) et harmonisation avec la base à chenilles.

Cette étape a constitué un **vrai défi** du Cycle 3 : chaque modification impacte l'encombrement interne (câbles, Raspberry Pi, batterie) et la cohérence visuelle du robot. La figure V.2 illustre la

vue éclatée des pièces modélisées ; la figure V.1 montre le résultat physique après impression et assemblage.

Choix mécaniques retenus sur le prototype :

- **Caméra** : fixée sur le **torse** (corps), face avant, indépendante du servo S3.
- **Écran Nextion** : intégré dans la **tête**, entraîné par le servo S3 (rotation du visage animé seulement).
- **Base** : châssis à chenilles pour la mobilité (moteurs M1/M3).

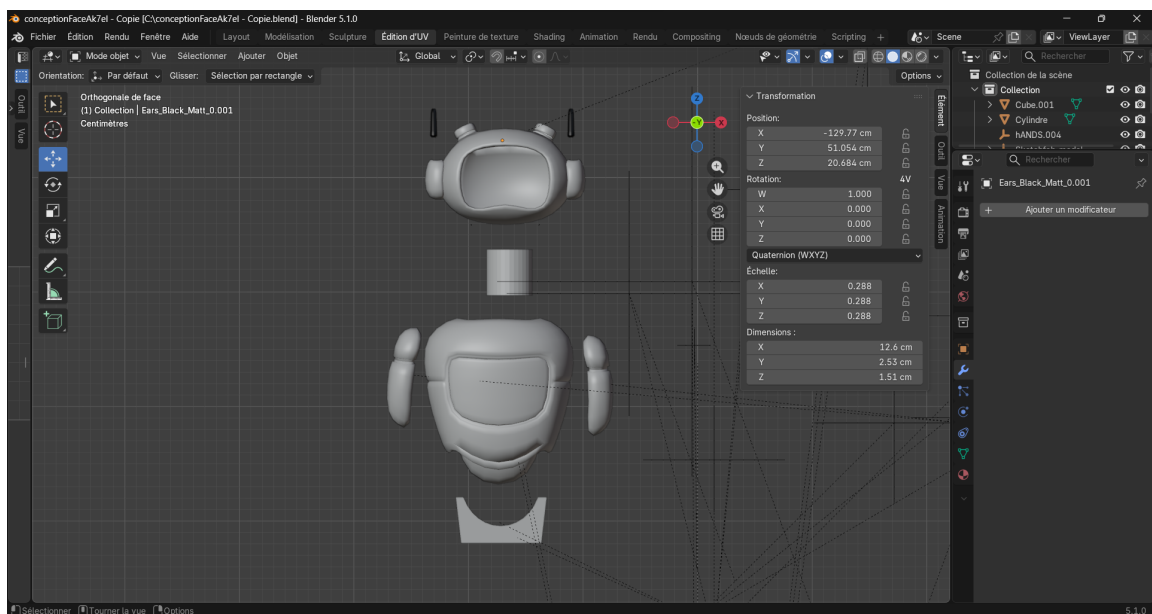


FIGURE V.2 – Conception 3D du châssis KODA sous Blender (vue éclatée des pièces)

V.4 Description Détaillée des Composants

V.4.1 Unité Centrale : Raspberry Pi 4 Model B

Le **Raspberry Pi 4 Model B** constitue le cœur du robot KODA. Il s'agit d'un ordinateur monocarte (SBC — *Single Board Computer*) basé sur un processeur ARM Cortex-A72 quadricœur cadencé à 1,5 GHz, équipé de 4 Go de mémoire RAM LPDDR4.

Dans l'architecture de KODA, le Raspberry Pi assure les fonctions suivantes :

- **Communication réseau** : connexion Wi-Fi au backend (Distributeur de Services) pour l'échange de données avec les workflows n8n et la base de données.
- **Traitement audio** : ReSpeaker en entrée (USB) ; sortie soit via le **jack 3,5 mm** et l'amplificateur PAM8403, soit via **Bluetooth** vers un haut-parleur externe.
- **Traitement vidéo** : acquisition d'images via la caméra pour la reconnaissance faciale.

- **Pilotage de l'écran Nextion** : communication série UART pour l'affichage des expressions faciales.
- **Commande de l'Arduino** : envoi d'instructions de mouvement via la liaison série USB.



FIGURE V.3 – Raspberry Pi 4 Model B utilisé comme unité centrale de KODA

TABLE V.2 – Spécifications techniques du Raspberry Pi 4 Model B

Caractéristique	Valeur
Processeur	Broadcom BCM2711, Cortex-A72 quad-core @ 1,5 GHz
Mémoire	4 Go LPDDR4
Connectivité	Wi-Fi 802.11ac, Bluetooth 5.0
Ports USB	2× USB 3.0 + 2× USB 2.0
Sortie audio	Jack 3,5 mm stéréo
Interface caméra	Port CSI (Camera Serial Interface)
GPIO	40 broches (dont UART, I2C, SPI)
Alimentation	5V / 3A via USB-C

V.4.2 Unité de Commande Moteur : Arduino Uno + Shield L293D

L'**Arduino Uno** est une carte de développement à microcontrôleur ATmega328P, cadencé à 16 MHz. Il est couplé au **Shield L293D**, un circuit intégré de pilotage moteur (*motor driver*) qui permet de contrôler simultanément jusqu'à 4 moteurs DC et 2 servomoteurs.

Dans KODA, l'Arduino reçoit ses instructions du Raspberry Pi via une **liaison série USB** et pilote :

- **2 moteurs DC** (bornes **M1** et **M3** du shield) : déplacement par traction différentielle.
- **3 servomoteurs** :
 - **S1** : bras droit
 - **S2** : bras gauche

- **S3** : tête / cou (rotation de l'écran Nextion uniquement)

Le Shield L293D est alimenté indépendamment par **3 piles lithium de 3,7 V** connectées en série, fournissant une tension nominale d'environ 11,1 V nécessaire au fonctionnement des moteurs DC sous charge.



FIGURE V.4 – Carte Arduino Uno (microcontrôleur ATmega328P à 16 MHz)



FIGURE V.5 – Shield L293D empilé sur l'Arduino Uno (commande des moteurs DC et servomoteurs)

V.4.3 Système d'Actionneurs

V.4.3.1 Moteurs DC — Mobilité

Les deux moteurs à courant continu assurent la **locomotion** du robot par un système de **traction différentielle** : en faisant varier indépendamment la vitesse et le sens de rotation de chaque moteur, le robot peut avancer, reculer, tourner à gauche ou à droite. Ce principe simple et éprouvé est largement utilisé en robotique mobile (figure V.7).



FIGURE V.6 – Moteur DC utilisé pour la mobilité du robot (bornes M1 et M3 du shield)

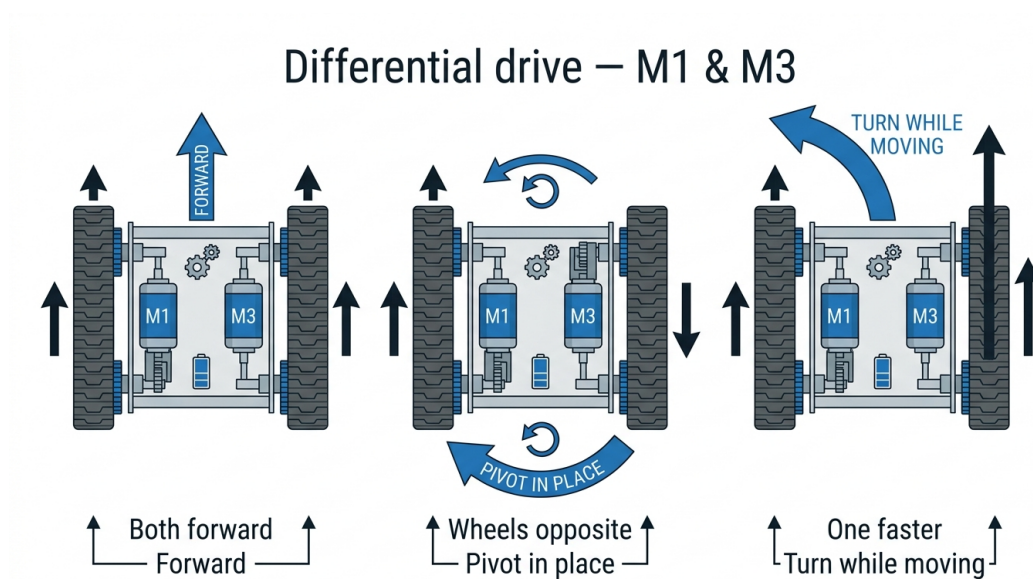


FIGURE V.7 – Principe de traction différentielle — moteurs M1 et M3

V.4.3.2 Servomoteurs — Articulations

Les trois servomoteurs offrent à KODA des capacités d'**expression corporelle** :

- **Servomoteur S1 (bras droit) et S2 (bras gauche)** : permettent de simuler des gestes d'accueil, de salutation ou d'indication directionnelle.

- **Servomoteur S3 (tête / cou)** : oriente uniquement l'écran **Nextion** (visage animé). La **caméra** est fixée sur le **châssis** (corps) du robot et ne suit pas ce mouvement.



FIGURE V.8 – Servomoteur utilisé pour les articulations (bras S1/S2 et rotation Nextion S3)

V.4.4 Système Audio

V.4.4.1 Entrée Audio : ReSpeaker 4-Mic Array USB

Le **ReSpeaker 4-Mic Array** de Seeed Studio est un périphérique USB intégrant quatre microphones MEMS disposés en configuration circulaire. Cette disposition permet deux fonctionnalités essentielles :

- **Capture vocale omnidirectionnelle** : enregistrement de la voix de l'utilisateur quelle que soit sa position autour du robot.
- **Détection de la Direction d'Arrivée (DOA — *Direction of Arrival*)** : identification de l'angle d'où provient la voix. Le robot **pivote sur place** grâce aux **deux moteurs DC** (M1, M3) pour s'orienter vers l'interlocuteur, puis la **caméra** (fixée sur le corps) permet la détection du visage (figure V.10).

Le ReSpeaker est connecté au Raspberry Pi via un **port USB** et est reconnu comme un périphérique audio standard sous Linux.



FIGURE V.9 – ReSpeaker 4-Mic Array USB : quatre microphones MEMS pour capture omnidirectionnelle et détection de direction (DOA)

Direction of arrival (DOA) — KODA

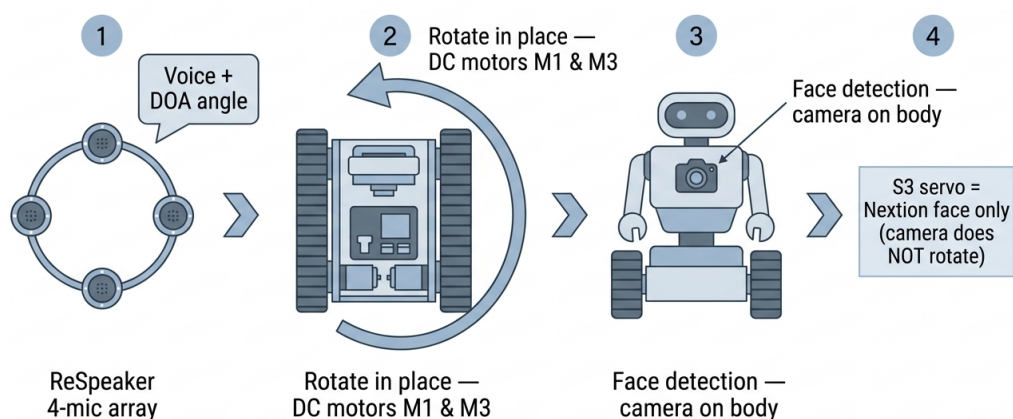


FIGURE V.10 – Enchaînement conceptuel : DOA (micro) → rotation du robot (M1, M3) → caméra (torse)

V.4.4.2 Sortie Audio : double chaîne (jack filaire et Bluetooth)

KODA dispose de **deux modes de restitution** de la parole, tous deux pilotés depuis le Raspberry Pi :

- **Chaîne filaire** : sortie **jack 3,5 mm** du Pi → amplificateur **PAM8403** → deux haut-parleurs (gauche / droite). Solution compacte, intégrée au châssis.
- **Chaîne sans fil** : le Bluetooth intégré du Raspberry Pi 4 peut envoyer le flux audio vers un **haut-parleur Bluetooth** (appairage classique). Utile pour les essais ou une sortie plus puissante sans modifier le câblage jack.

Le choix entre les deux dépend du contexte de démonstration ; la couche logicielle (Cycle 4)

sélectionnera la carte audio ALSA appropriée.



FIGURE V.11 – Amplificateur audio PAM8403 (chaîne filaire entre la sortie jack du Pi et les haut-parleurs)



FIGURE V.12 – Paire de haut-parleurs gauche/droite alimentés par le PAM8403

V.4.5 Système de Vision : Caméra Raspberry Pi

La **caméra officielle Raspberry Pi** est connectée via le port **CSI** (nappe dédiée). Elle fournit les images nécessaires à l'**identification visuelle** des utilisateurs.

Au niveau matériel, seule la **qualité de capture** (netteté, le serveur distant et le traitement logiciel seront détaillés au **Cycle 4**.



FIGURE V.13 – Caméra Raspberry Pi connectée au port CSI (acquisition d’images pour l’identification visuelle)

V.4.6 Interface d’Expression Faciale : Nextion NX4832T035_011

L’écran **Nextion NX4832T035_011** (3,5 pouces) intègre son propre microcontrôleur : le Raspberry Pi n’affiche pas des pixels en temps réel, mais envoie des **commandes série** pour changer de page ou d’expression (sourire, surprise, écoute, etc.) déjà stockées dans la mémoire de l’écran.

Alimentation et liaison (câblage retenu sur le prototype) :

- **5 V** et **GND** du Nextion reliés au **5 V** et **GND** du GPIO du Raspberry Pi (masse commune).
- **RX** du Nextion (fil jaune) → broche **8** du Pi (**GPIO 14**, ligne **TX** du Pi).
- **TX** du Nextion (fil bleu) → broche **10** du Pi (**GPIO 15**, ligne **RX** du Pi).

Ce câblage UART permet au Pi d’animer le **visage** de KODA sans surcharger le bus USB, tout en gardant une interface visuelle dédiée à l’interaction humaine.



FIGURE V.14 – Écran Nextion NX4832T035_011 (3,5") affichant le visage animé de KODA

V.4.7 Système d’Alimentation

Le robot KODA dispose de **deux sources d’alimentation indépendantes**, séparant volontairement la partie logique de la partie motrice :

TABLE V.3 – Architecture d’alimentation de KODA

Source	Caractéristiques	Composants alimentés
Power Bank	5V / 3A, USB-C	Raspberry Pi 4 (et par extension : ReSpeaker, caméra, écran Nextion via GPIO)
3 × piles Li-ion	3,7V chacune (11,1V total)	Shield L293D, moteurs DC, servomoteurs

Cette séparation protège le Raspberry Pi contre les perturbations électromagnétiques et les chutes de tension provoquées par les appels de courant des moteurs.



FIGURE V.15 – Power Bank 5 V / 3 A alimentant le Raspberry Pi (chaîne logique)



FIGURE V.16 – 3 piles lithium 3,7 V en série (11,1 V total) alimentant le Shield L293D et les moteurs

V.5 Schéma de Câblage Global

La figure V.17 présente le **schéma de montage** du prototype : vue type *Fritzing*, avec l'ensemble des connexions entre Raspberry Pi, Arduino, capteurs, actionneurs et alimentations.

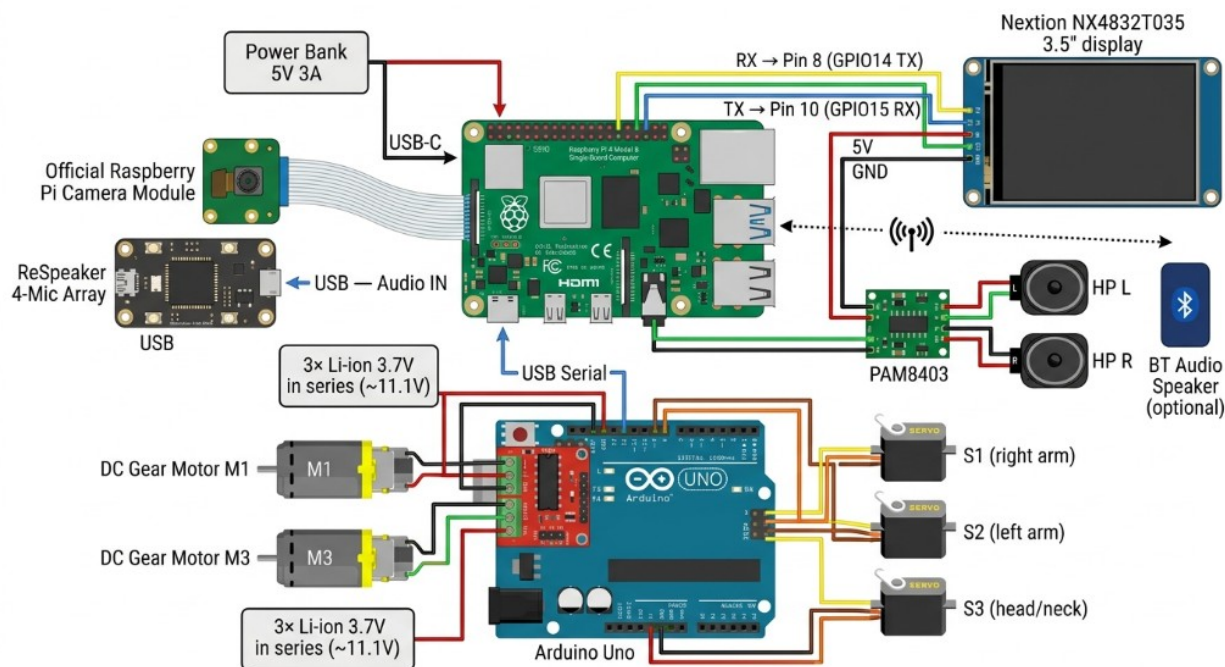


FIGURE V.17 – Schéma de montage matériel de KODA — style Fritzing (Cycle 3)

Le tableau V.4 récapitule les connexions pour la rédaction et la maintenance du prototype.

TABLE V.4 – Récapitulatif des connexions entre composants

Composant	Connecté à	Interface	Détail
Arduino Uno	Raspberry Pi	USB	Commandes moteur (série)
ReSpeaker 4-Mic	Raspberry Pi	USB	Entrée audio + DOA
PAM8403 + HP L/R	Raspberry Pi	Jack 3,5 mm	Sortie filaire
HP Bluetooth	Raspberry Pi	Bluetooth	Sortie sans fil (option)
Caméra Pi	Raspberry Pi	CSI	Nappe caméra
Nextion NX4832T035_011	Raspberry Pi	UART + alim	5V/GND GPIO ; RX→ pin 8 ; TX→ pin 10
Moteurs DC M1, M3	Shield L293D	Bornes M1, M3	Traction différentielle
Servos S1, S2, S3	Shield L293D	Servo 1, 2, 3	Bras D/G ; S3 = Nextion
Power Bank 5V/3A	Raspberry Pi	USB-C	Alimentation logique
3× piles 3,7V (série)	Shield L293D	Vin	Alimentation motrice

V.6 Assemblage et points d'attention

L'intégration physique a mis en évidence quelques **contraintes** récurrentes, sans entrer dans le détail logiciel (Cycle 4) :

- **Séparation des alimentations** : éviter de nourrir les moteurs depuis le 5 V du Pi ; les perturbations PWM peuvent provoquer des réinitialisations ou du bruit sur l'audio.
- **UART Nextion** : respecter le croisement TX/RX (fil RX de l'écran vers TX du Pi) et une masse commune stable.
- **USB multiples** : ReSpeaker, Arduino et alimentation compétent pour les ports USB du Pi ; une alimentation 5 V/3 A fiable (power bank) est indispensable.
- **Encombrement mécanique** : la nappe CSI (caméra sur le corps) et les câbles des servos (dont S3 pour le Nextion) doivent rester libres de frottement lors des mouvements articulés.

V.7 Validation matérielle — tests par composant

Chaque sous-système a été validé **individuellement** avant le test global (contrôle visuel ou fonctionnel, sans détailler les scripts). Le tableau V.5 ci-dessous synthétise cette démarche.

V.7.1 Commandes de test par composant

Chaque composant a été vérifié avec une commande courte exécutée directement depuis le Raspberry Pi (*shell* ou *Python*), ou via un mini-script côté Arduino. Les extraits ci-dessous donnent la procédure exacte employée pour confirmer le bon fonctionnement.

TABLE V.5 – Tests unitaires par composant (Cycle 3)

Composant	Procédure (concept)	Critère OK
Power Bank + Pi	Mise sous tension, boot Raspberry Pi OS	LED activité, pas de brown-out
ReSpeaker USB	Enregistrement court, niveau sonore	Signal audible, périphérique reconnu
Sortie jack + PAM8403	Lecture d'un son test sur les HP	Son clair L/R
Sortie Bluetooth	Appairage HP BT, lecture test	Son sur HP externe
Caméra CSI	Capture d'une image fixe	Image nette, pas de bandes
Nextion UART	Changement de page ou d'expression	Affichage réactif
Arduino + USB	Envoi d'une commande simple (moniteur série)	Réponse attendue
Moteurs M1, M3	Avancer, reculer, pivoter	Roues cohérentes
Servos S1, S2, S3	Balayage angle min-max	Mouvement fluide, sans à-coups
Alim. 3 piles (série)	Charge moteurs à vide	Tension stable sur Vin

V.7.1.0.1 Power Bank + Raspberry Pi.

On vérifie qu'aucun *brown-out* (chute de tension) n'a été détecté par le firmware du Pi après mise sous tension.

```
# Doit retourner 0x0 ; tout bit non nul indique under-voltage ou
  throttling.
vcgencmd get_throttled
# Tension sur le rail 5V (volts) :
vcgencmd measure_volts core
```

Listing V.1 – Diagnostic alimentation vcgencmd

V.7.1.0.2 ReSpeaker 4-Mic Array USB.

Test en deux temps : enregistrement audio brut via ALSA/PipeWire, puis lecture de l'angle de provenance du son (*DOA*) via le registre interne du microcontrôleur XMOS du ReSpeaker.

```
# 1. Enregistrer 5 s en mono 16 kHz S16_LE (format attendu par Vosk/
  Azure STT)
arecord -D pipewire -f S16_LE -r 16000 -c 1 -d 5 /tmp/test_mic.wav
aplay /tmp/test_mic.wav # écoute de controle
```

```
# 2. Detection de la Direction of Arrival (DOA) via tuning.py de Seeed
python3 -c "from usb_4_mic_array.tuning import Tuning; import usb.core;
\
dev=usb.core.find(idVendor=0x2886, idProduct=0x0018); \
t=Tuning(dev); print('DOA =', t.direction, 'deg')"
```

Listing V.2 – Enregistrement 5 s et lecture du DOA

V.7.1.0.3 Sortie jack + amplificateur PAM8403.

On lit un signal sinusoïdal stéréo connu pour vérifier les deux canaux (gauche/droite) de l'amplificateur et des haut-parleurs.

```
# Son systeme livre avec alsa-utils : "Front Center" annonce L puis R
aplay /usr/share/sounds/alsa/Front_Center.wav
aplay /usr/share/sounds/alsa/Front_Left.wav
aplay /usr/share/sounds/alsa/Front_Right.wav
```

Listing V.3 – Lecture d'un son test stereo

V.7.1.0.4 Sortie Bluetooth.

Appairage du haut-parleur Bluetooth puis lecture d'un WAV via le serveur audio PipeWire (qui route vers le sink bluez_output.<MAC>.a2dp-sink).

```
# Appairage initial (a faire une seule fois)
bluetoothctl power on
bluetoothctl pair AD:1C:99:E7:9B:78
bluetoothctl trust AD:1C:99:E7:9B:78
bluetoothctl connect AD:1C:99:E7:9B:78

# Lecture vers le sink Bluetooth
paplay --device=bluez_output.AD_1C_99_E7_9B_78.a2dp-sink /tmp/test_mic.
wav
```

Listing V.4 – Appairage et lecture sur HP Bluetooth

V.7.1.0.5 Caméra Raspberry Pi (CSI).

Capture d'une image fixe avec l'utilitaire `rpicas-still` et contrôle visuel du fichier produit.

```
# Capture une photo en 1920x1080 apres 2 s d'autofocus
rpicas-still --width 1920 --height 1080 -t 2000 -o /tmp/test_cam.jpg
```

```
# Verifier le fichier (taille typique : > 200 Ko si l'image est nette)
ls -lh /tmp/test_cam.jpg
```

Listing V.5 – Capture d’une image avec rpicalm-still

V.7.1.0.6 Écran Nextion (UART).

Envoi d’une commande page via la liaison série /dev/serial0 (GPIO 14/15 du Pi). Chaque commande Nextion se termine par trois octets 0xFF.

```
import serial
ser = serial.Serial("/dev/serial0", baudrate=9600, timeout=1)
# Trois 0xFF marquent la fin de chaque instruction Nextion
END = b"\xff\xff\xff"
ser.write(b"page 2" + END)          # bascule sur l'expression "joyeux"
ser.write(b"t0.txt=\"Hello\"" + END)
ser.close()
```

Listing V.6 – Changer la page affichée par le Nextion

V.7.1.0.7 Arduino + Shield L293D (commande série USB).

Le sketch Arduino lit un octet sur le port série; le Pi envoie un caractère pour déclencher un mouvement (F = forward, B = backward, S = stop). On vérifie ici le canal de communication uniquement.

```
import serial, time
ard = serial.Serial("/dev/ttyUSB0", baudrate=9600, timeout=2)
time.sleep(2)          # laisser l'Arduino rebooter apres
                        ouverture du port
ard.write(b"F")        # forward 1 s puis stop
time.sleep(1)
ard.write(b"S")
print(ard.readline().decode(errors="replace").strip())  # ACK eventuel
ard.close()
```

Listing V.7 – Envoi d’une commande au sketch Arduino

V.7.1.0.8 Moteurs DC (M1, M3).

Extrait du sketch Arduino utilisant la bibliothèque AFMotor (Adafruit Motor Shield v1, équivalent L293D). Ce *loop* fait avancer puis reculer le robot pendant une seconde dans chaque sens.

```
#include <AFMotor.h>
AF_DCMotor m1(1);    // borne M1
AF_DCMotor m3(3);    // borne M3
void setup() {
    m1.setSpeed(200); m3.setSpeed(200);    // 0-255
}
void loop() {
    m1.run(FORWARD);  m3.run(FORWARD);  delay(1000);
    m1.run(BACKWARD); m3.run(BACKWARD); delay(1000);
    m1.run(RELEASE);  m3.run(RELEASE);  delay(2000);
}
```

Listing V.8 – Test des moteurs M1 et M3 (Arduino + L293D)

V.7.1.0.9 Servomoteurs (S1, S2, S3).

Balayage de l'angle minimum au maximum pour les trois servomoteurs branchés sur les broches Servo_1, Servo_2 et Servo_3 du shield L293D.

```
#include <Servo.h>
Servo s1, s2, s3;    // bras droit, bras gauche, tete (Nextion)
void setup() {
    s1.attach(9);  s2.attach(10); s3.attach(11);
}
void loop() {
    for (int a = 0; a <= 180; a += 5) {
        s1.write(a); s2.write(180 - a); s3.write(a / 2);
        delay(30);
    }
}
```

Listing V.9 – Balayage des trois servomoteurs

V.7.1.0.10 Alimentation 3 piles 3,7 V (Vin shield).

Mesure au multimètre sur les bornes Vin du Shield L293D, sous charge (moteurs en rotation). La tension doit rester stable autour de 11 V.

```
# Aucune commande logicielle : mesure physique au multimetre.
# - A vide : ~12.4 V sur Vin du shield
# - Sous charge : ~11.0 V (chute typique d'environ 1 V due au courant
    moteurs)
# Si la tension chute sous 9 V -> recharger ou remplacer les piles.
```

Listing V.10 – Procedure de mesure (notes oscilloscope/multimetre)

V.7.2 Test d'intégration matérielle

Le **test complet** consiste à alimenter l'ensemble, vérifier que le Pi détecte simultanément micro, caméra et écran, que l'Arduino répond toujours sur USB, et qu'un scénario court combine : capture vocale, mouvement des servos, rotation des roues, changement d'expression Nextion, sans coupure d'alimentation ni conflit USB. Ce test valide la **plateforme prête pour le Cycle 4** (logiciel embarqué et liaison réseau avec le backend).

V.8 Conclusion

Ce chapitre a présenté l'intégralité de la plateforme matérielle du robot KODA. L'architecture biprocesseur retenue — Raspberry Pi pour l'intelligence et l'Arduino pour le contrôle moteur — garantit à la fois la puissance de calcul nécessaire aux traitements IA et la précision temporelle requise pour le pilotage des actionneurs.

Chaque composant a été sélectionné et intégré pour répondre aux exigences fonctionnelles définies au chapitre II : mobilité, interaction vocale bidirectionnelle, reconnaissance visuelle et expression émotionnelle. La plateforme matérielle est désormais prête à accueillir la couche logicielle embarquée, objet du **Cycle 4** présenté dans le chapitre suivant.

Points clés du Cycle 3

- Architecture biprocesseur : Raspberry Pi 4 + Arduino Uno / L293D
- 5 actionneurs : M1/M3 (DC) + S1/S2 (bras) + S3 (Nextion)
- Audio : ReSpeaker (entrée); sortie jack/PAM8403 ou Bluetooth
- Nextion NX4832T035_011 (UART GPIO 8/10, alim. 5 V Pi)
- Tests unitaires puis intégration; logiciel au Cycle 4

———— CHAPITRE VI ————

CYCLE 4 : SYSTÈME EMBARQUÉ — PARTIE LOGICIELLE RASPBERRY PI

VI.1 Introduction

Le **Cycle 3** a validé la plateforme matérielle de KODA : Raspberry Pi, Arduino, ReSpeaker, écran Nextion, caméra, sortie audio et alimentation. Le **Cycle 4** transforme cette plateforme physique en un robot capable de réagir en continu, de dialoguer et d'exécuter les décisions produites par le Distributeur de Services et le moteur n8n.

Ce chapitre présente donc la **partie logicielle embarquée Raspberry Pi**. Le programme principal est lancé par la commande `python -m app.main` et repose sur une architecture Python **asynchrone**, organisée autour de services de haut niveau et d'adaptateurs matériels. Le Raspberry Pi n'est pas le cerveau conversationnel complet de KODA ; il joue plutôt le rôle de **corps opérationnel** : il écoute, s'oriente vers la voix, capture la question, communique avec le backend, affiche une expression, prononce la réponse et transmet les mouvements à l'Arduino.

Objectif du Cycle 4

Implémenter et valider le logiciel embarqué du Raspberry Pi afin de relier les composants matériels du robot au backend FastAPI : écoute vocale, mot de réveil, orientation par DOA, capture de question, affichage Nextion, lecture audio, commandes Arduino USB et arrêt propre du système.

VI.2 Architecture logicielle globale

Le logiciel Raspberry Pi est conçu comme une couche d'orchestration locale. Il démarre les adaptateurs matériels, vérifie les dépendances au boot, puis exécute une boucle principale qui alterne entre un mode **passif** (attente du mot de réveil) et un mode **actif** (conversation avec l'utilisateur). Les traitements lourds de compréhension, de décision et de synthèse vocale restent délégués au **Distributeur FastAPI**, qui relaie l'information vers n8n, Supabase et les services cloud.

L'architecture suit quatre niveaux :

- **Point d'entrée** : `app.main`, responsable du démarrage, du diagnostic, de l'instanciation des services, de la boucle principale et de l'arrêt propre.
- **Configuration** : `app.config`, qui centralise les variables dépendantes du robot (`BACKEND_URL`, `NEXTION_PORT`, port Arduino, paramètres Vosk, Bluetooth, calibration de rotation).
- **Services** : modules métier indépendants du détail matériel (conversation, mot de réveil, affichage, mouvement, audio, musique).
- **Adaptateurs** : interfaces concrètes vers le monde externe : HTTP FastAPI, ReSpeaker, Nextion,

Arduino USB et sortie audio.

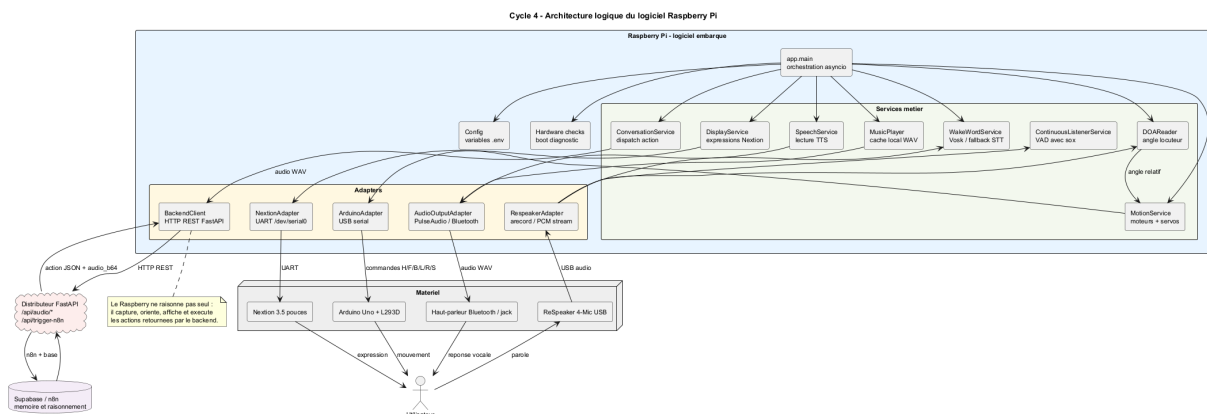


FIGURE VI.1 – Cycle 4 — architecture logique du logiciel Raspberry Pi

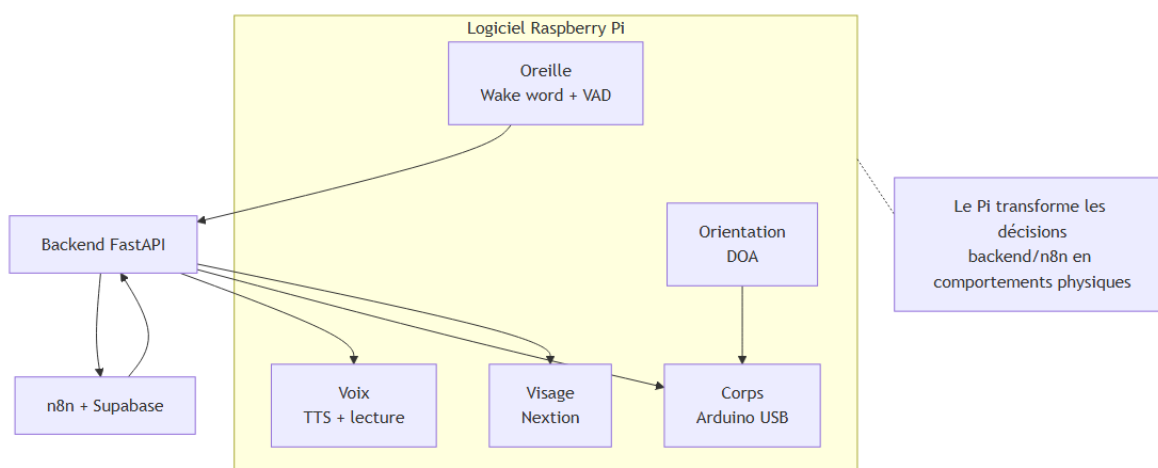


FIGURE VI.2 – Cycle 4 — le Raspberry Pi comme corps opérationnel de KODA

VI.3 Architecture du code

Le projet codeRaspberry est volontairement séparé en dossiers courts et lisibles. Cette organisation facilite le test d'un composant sans lancer l'ensemble du robot et évite que le programme principal manipule directement les détails série, audio ou HTTP.

- `app/` contient le point d'entrée et la configuration.
- `services/` regroupe les comportements de haut niveau : conversation, wake word, écoute continue, affichage, mouvement, audio, synthèse vocale, musique et diagnostic matériel.
- `adapters/` encapsule les interfaces techniques : HTTP FastAPI, ReSpeaker, Nextion, Arduino USB et sortie audio.
- `utils/` fournit les traces d'état et les logs.
- `tests/` vérifie la configuration, les imports et une partie des checks matériels.

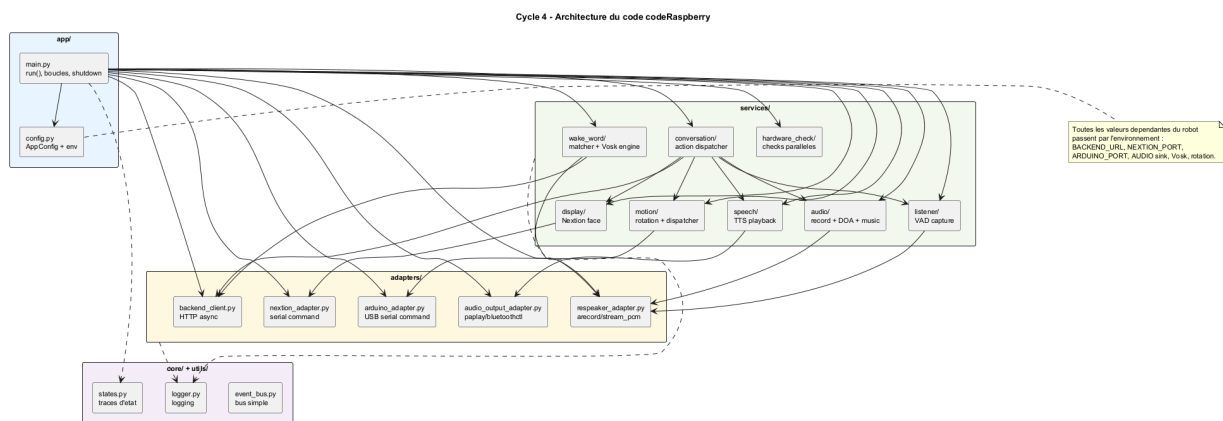


FIGURE VI.3 – Cycle 4 — architecture du code codeRaspberry

Le diagramme de classes ci-dessous présente les principaux objets logiciels qui coopèrent pendant une interaction vocale. ConversationService est l'orchestrateur fonctionnel d'un tour de parole : il récupère l'audio, appelle le backend, puis déclenche l'action appropriée. Les services DisplayService, MotionService, SpeechService et MusicPlayer traduisent ensuite la décision en comportement physique.

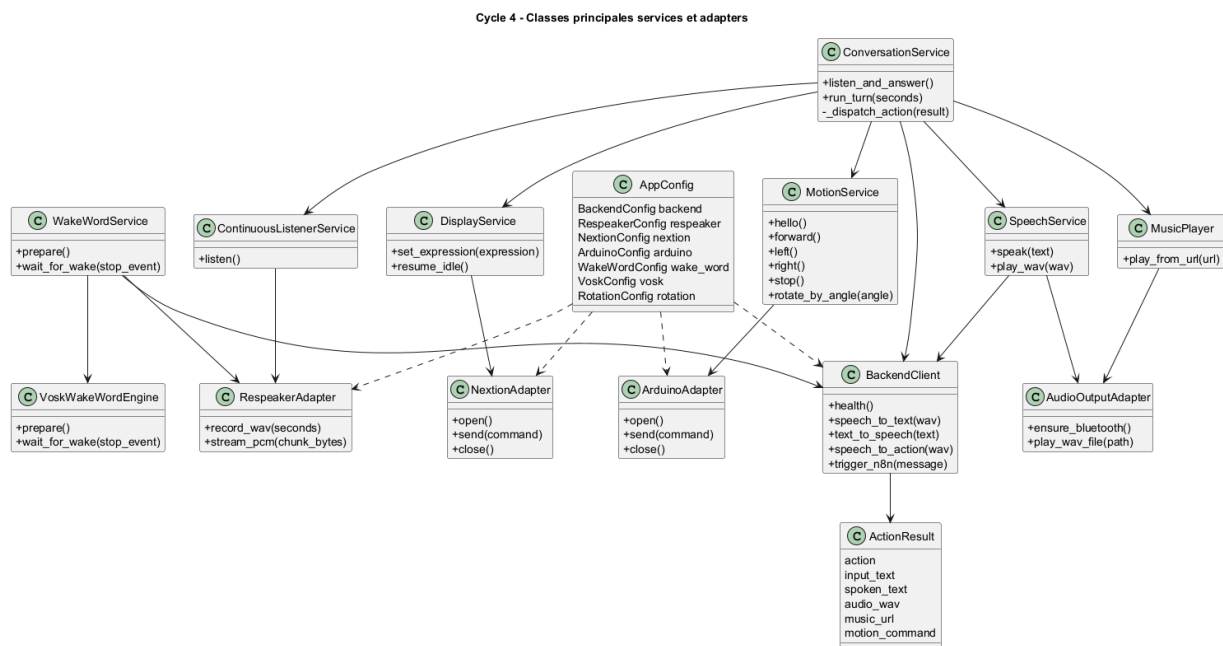


FIGURE VI.4 – Cycle 4 — classes principales, services et adaptateurs

VI.4 Configuration et lancement

L'exécution se fait depuis le dossier codeRaspberry. Les variables d'environnement permettent d'adapter le même code à un robot donné sans modifier les sources Python. La commande de lancement retenue est :

```
source .venv/bin/activate
```

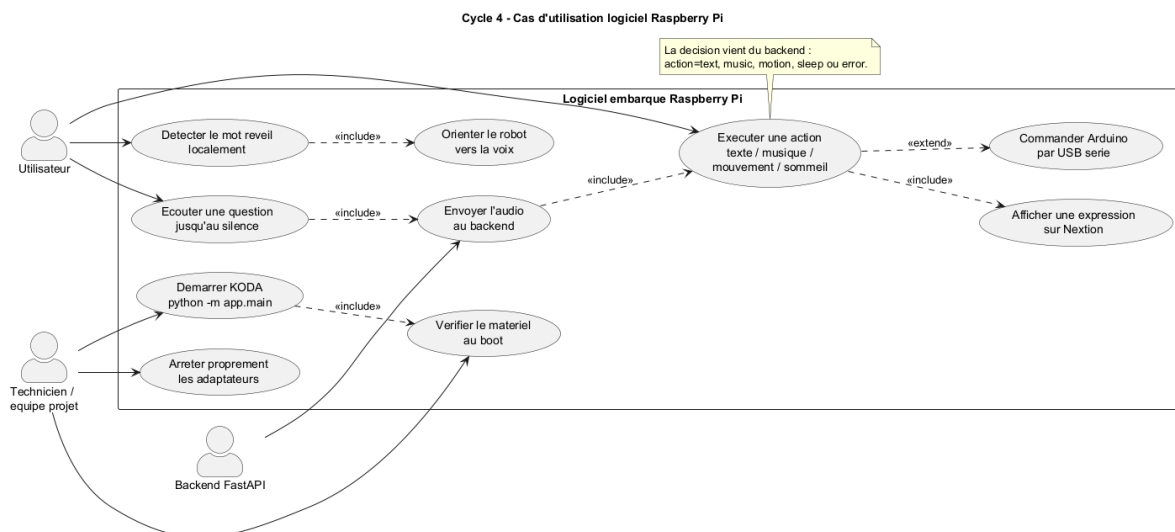


FIGURE VI.5 – Cycle 4 — cas d'utilisation du logiciel embarqué Raspberry Pi

```

export BACKEND_URL=http://192.168.69.185:8765
export NEXTION_PORT=/dev/serial0
export NEXTION_REQUIRE_GPIO_UART=1
export AUDIO_CARD_INDEX=1
python -m app.main

```

Listing VI.1 – Lancement du logiciel embarqué Raspberry Pi

Les paramètres essentiels sont :

- **BACKEND_URL** : adresse du Distributeur FastAPI sur le même réseau Wi-Fi que le Raspberry Pi.
- **NEXTION_PORT** : port série UART utilisé par l'écran, généralement /dev/serial0 avec TX/RX sur GPIO 14/15.
- **Arduino USB** : port série détecté parmi /dev/ttyUSB* ou /dev/ttyACM*, utilisé pour envoyer les commandes moteurs et servos.
- **Audio** : ReSpeaker en entrée, sortie Bluetooth ou jack selon la configuration PulseAudio/ALSA.
- **Vosk** : modèle local de reconnaissance utilisé pour détecter le mot de réveil sans interroger le cloud en continu.

VI.5 Démarrage et diagnostic matériel

Au démarrage, `app.main` charge la configuration, lance les checks matériels, ouvre les adaptateurs disponibles, initialise le client backend, puis construit les services applicatifs. Les checks matériels sont exécutés avant la boucle principale afin de produire un diagnostic lisible : micro, caméra, Nextion, Arduino USB, sortie audio, Bluetooth et système.

Cette étape n'arrête pas nécessairement le programme dès qu'un composant est absent. Le système privilégie un **mode dégradé contrôlé** : il loggue le problème, continue lorsque c'est possible, puis évite d'appeler un composant non ouvert. Par exemple, si l'Arduino n'est pas disponible, les réponses vocales et expressions peuvent encore fonctionner, mais les mouvements sont ignorés.

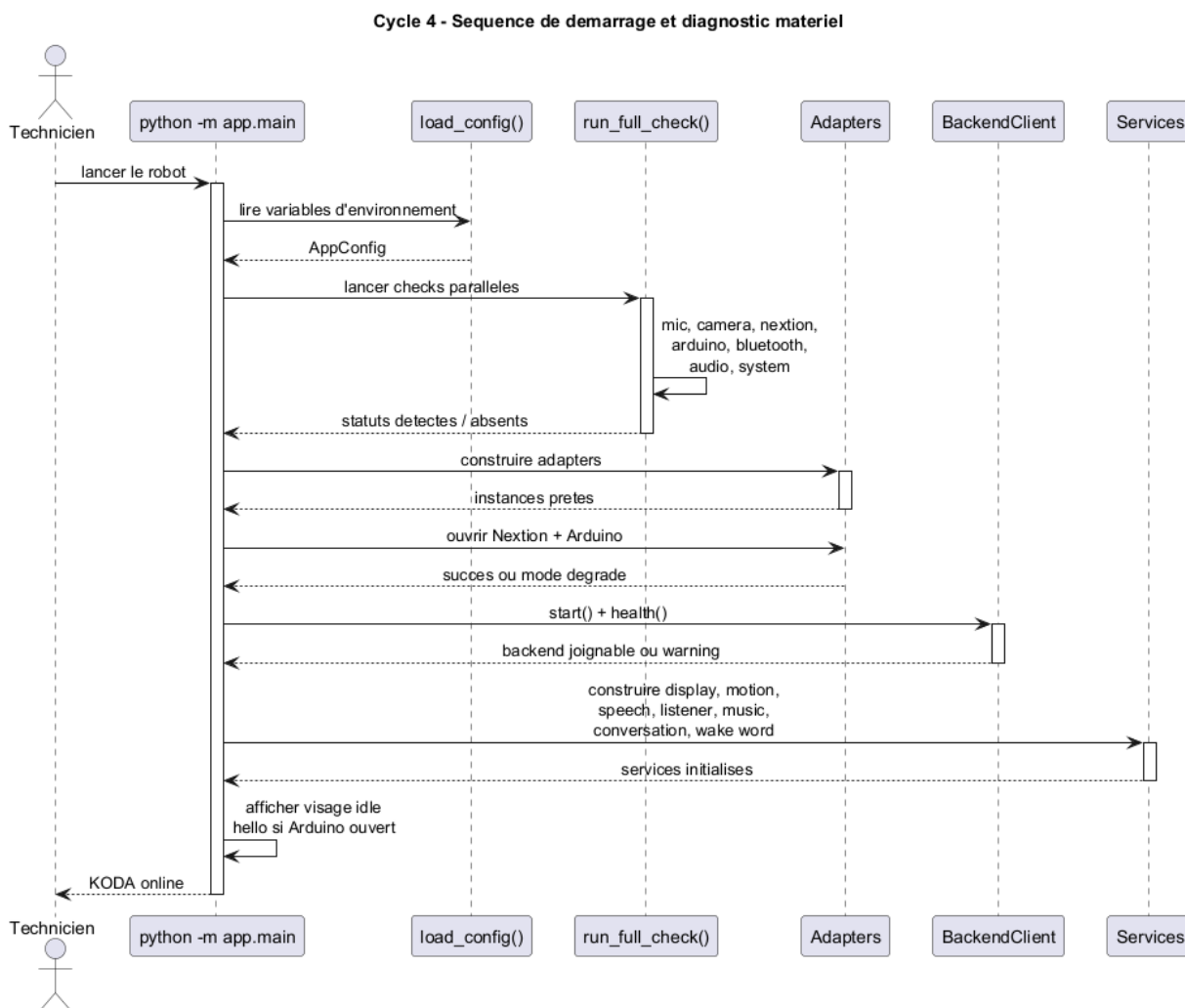


FIGURE VI.6 – Cycle 4 — séquence de démarrage et diagnostic matériel

VI.6 Boucle principale et machine d'états

Après l'initialisation, le robot entre dans une boucle asynchrone. En mode passif, il écoute le mot de réveil. Lorsqu'il le détecte, il affiche une expression de surprise, s'oriente vers l'utilisateur grâce au DOA, prononce une salutation, puis lance une capture de question jusqu'au silence. Le retour du backend décide ensuite du comportement : réponse vocale, musique, mouvement, sommeil ou erreur.

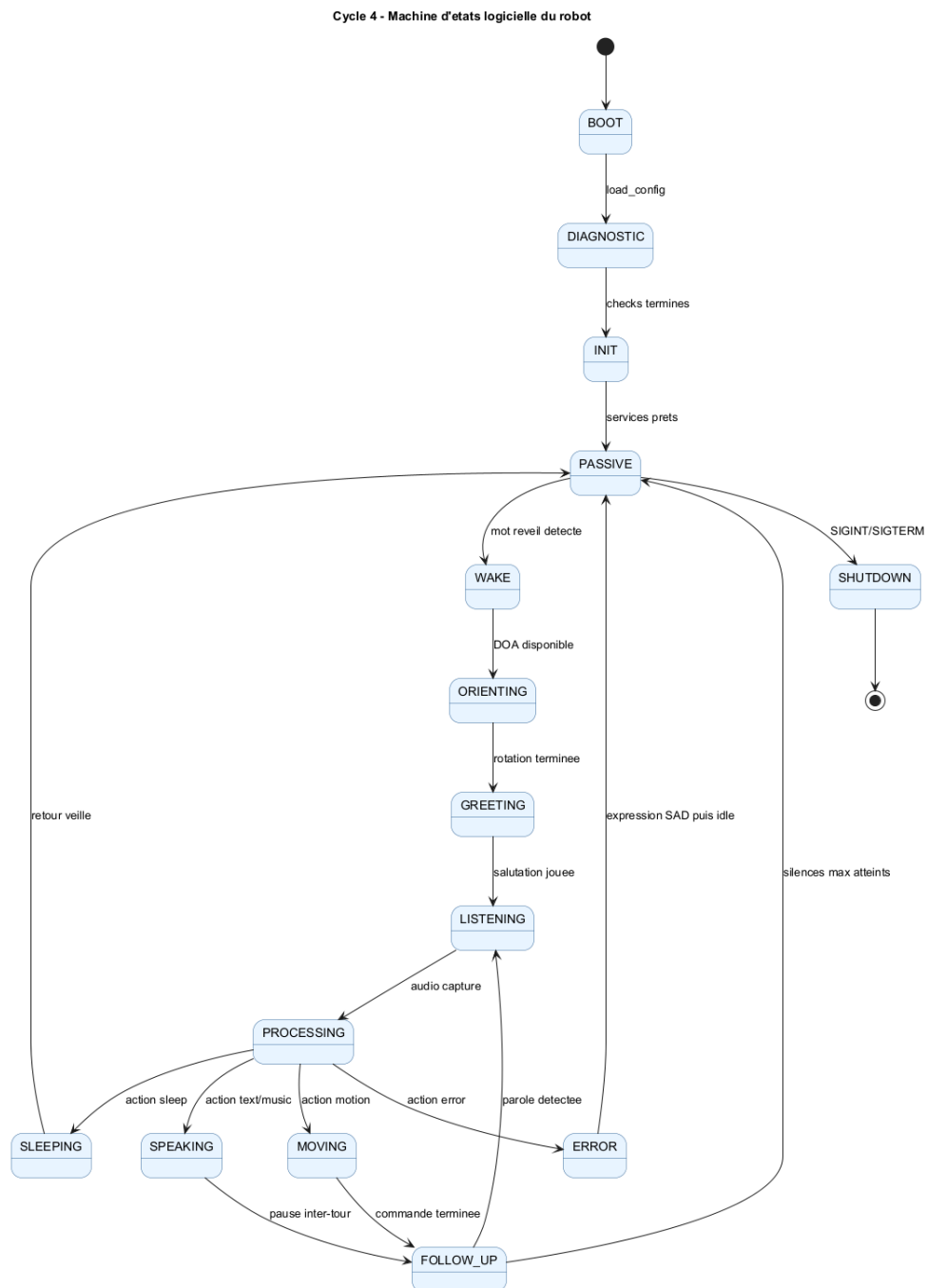


FIGURE VI.7 – Cycle 4 — machine d'états logicielle du robot

VI.7 Pipeline vocal : mot de réveil, écoute et réponse

Le pipeline vocal constitue le cœur de l'expérience utilisateur. Il combine trois traitements complémentaires :

- **Détection locale du mot de réveil** : le service de réveil utilise préférentiellement le moteur Vosk local, qui consomme un flux PCM continu du ReSpeaker. Ce choix réduit la latence et évite d'envoyer tous les sons ambiants au backend.
- **Écoute active par VAD** : après le réveil, le service `ContinuousListenerService` capture la question complète jusqu'à une période de silence, au lieu d'imposer une durée fixe.
- **Traitement distant** : l'audio WAV est envoyé au backend par `BackendClient` sur l'endpoint `/api/audio/speech-to-action`. Le backend renvoie un objet `ActionResult` contenant le type d'action, le texte reconnu, le texte à prononcer, l'audio TTS encodé et éventuellement une URL musique ou une commande de mouvement.

VI.8 Gestion des actions retournées par le backend

Le backend ne renvoie pas seulement un texte. Il interprète la réponse n8n et la transforme en **action structurée**. Côté Raspberry, `ConversationService` applique la logique suivante :

- `text` : lecture directe de l'audio TTS et animation faciale.
- `music` : annonce vocale, téléchargement ou récupération en cache local, puis lecture du morceau.
- `motion` : annonce vocale puis exécution d'une commande `forward`, `backward`, `left`, `right` ou `stop`.
- `sleep` : lecture d'un message de fin, expression `SLEEPING`, puis retour en attente du mot de réveil.
- `error` : expression triste et journalisation de l'erreur.

VI.9 Affichage Nextion et expressions

L'écran Nextion n'est pas piloté pixel par pixel depuis le Raspberry Pi. Les images et animations sont préparées dans l'interface Nextion, puis le Pi envoie des commandes série simples. Le service `DisplayService` sérialise ces commandes avec un verrou asynchrone afin d'éviter les conflits lorsque plusieurs actions tentent de changer l'expression au même moment.

Les expressions utilisées par le logiciel sont : neutre, clignement, heureux, amour, colère, triste, sommeil, surprise, chant et réflexion. Pendant une réponse, le service peut désactiver temporairement

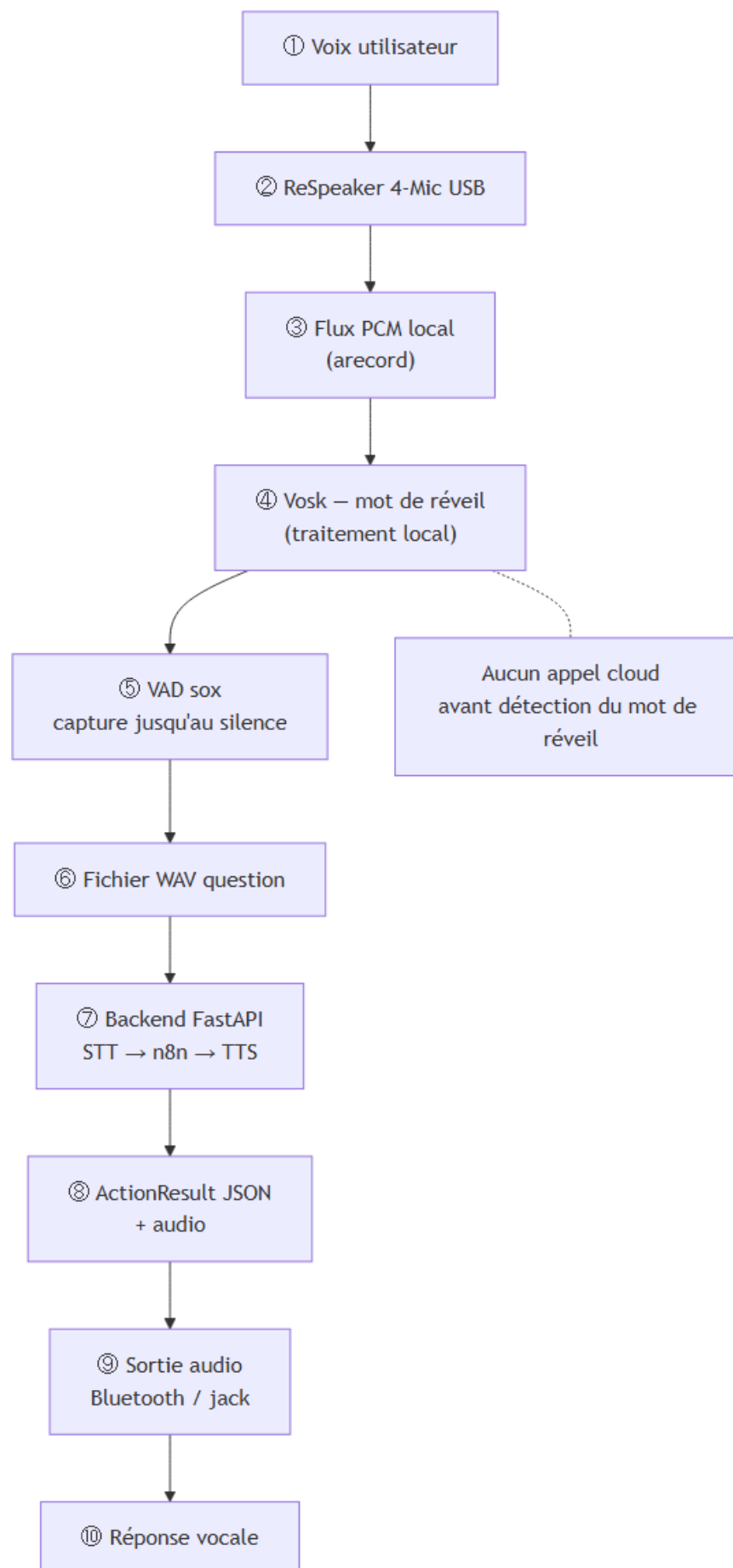


FIGURE VI.8 – Cycle 4 — pipeline audio logiciel du Raspberry Pi

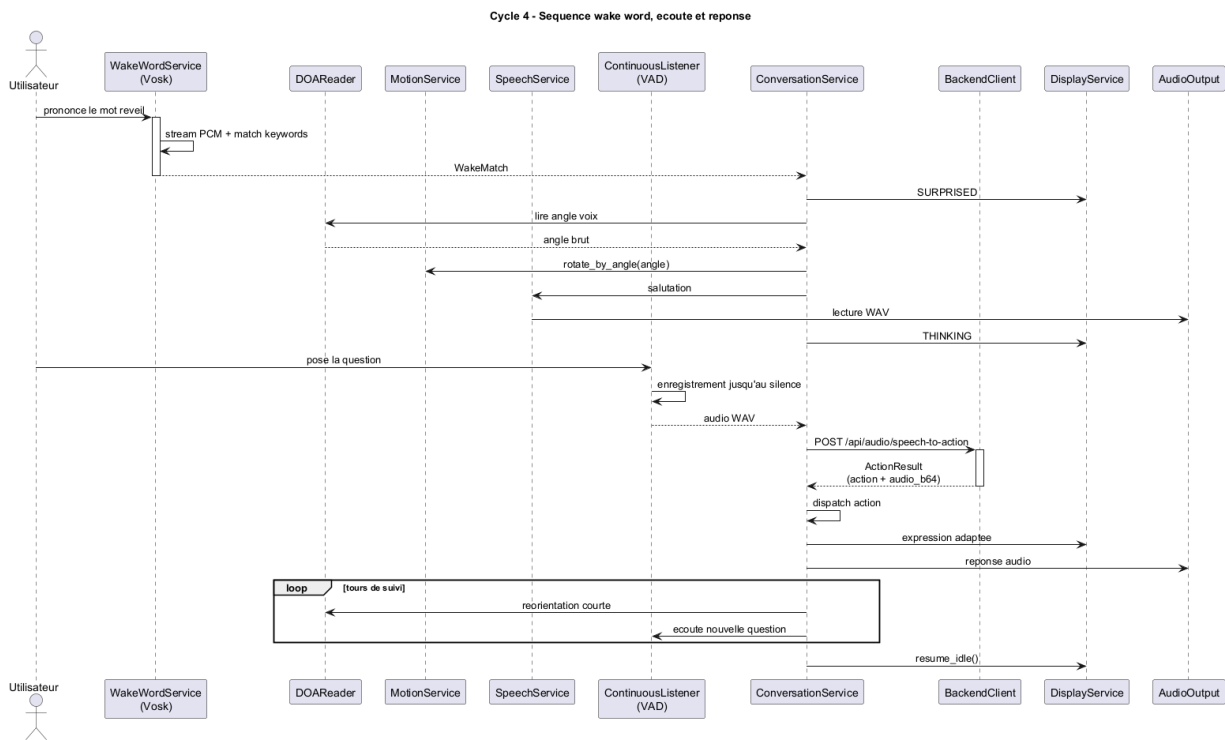


FIGURE VI.9 – Cycle 4 — séquence mot de réveil, écoute et réponse

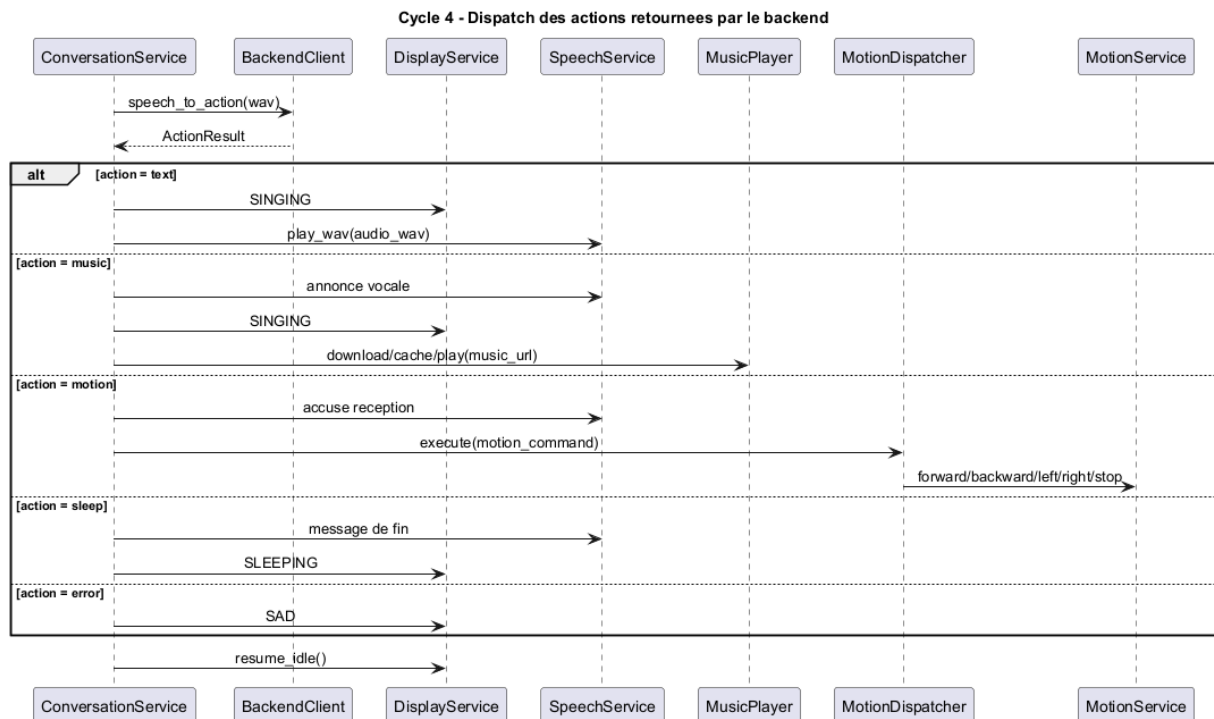


FIGURE VI.10 – Cycle 4 — dispatch des actions retournées par le backend

le timer de clignement ($tm0.en=0$), afficher une expression fixe, puis restaurer l'état idle.

TABLE VI.1 – Expressions logicielles envoyées à l'écran Nextion

Expression	Usage principal
NEUTRAL	Attente et retour idle
SURPRISED	Mot de réveil détecté
THINKING	Capture ou traitement de la question
SINGING	Réponse vocale ou musique
SAD	Erreur ou échec de traitement
SLEEPING	Passage en sommeil

VI.10 Mouvement, Arduino USB et orientation DOA

Le Raspberry Pi délègue le pilotage temps réel des moteurs et servomoteurs à l'Arduino Uno via une liaison **USB série**. Le protocole reste volontairement simple : chaque mouvement correspond à une commande courte, par exemple F pour avancer, B pour reculer, L et R pour tourner, S pour arrêter. Cette simplicité rend le protocole robuste et facile à tester depuis un terminal série.

Le service MotionService ajoute une couche de sécurité logicielle : les commandes sont protégées par un verrou, les rotations ouvertes sont temporisées, puis un STOP est envoyé après la durée calculée. Pour l'orientation vers l'utilisateur, DOAReader lit l'angle fourni par le ReSpeaker. Cet angle est converti en rotation relative dans l'intervalle $[-180^\circ, +180^\circ]$, puis envoyé au robot sous forme de rotation à gauche ou à droite.

Deux modes de calibration sont prévus :

- **Modèle linéaire** : durée calculée à partir d'une vitesse angulaire moyenne, d'un offset et d'une durée minimale.
- **Table LUT** : interpolation entre mesures réelles pour compenser l'inertie, la friction et la variation de tension batterie.

VI.11 Robustesse et arrêt propre

La robustesse du logiciel embarqué repose sur plusieurs choix :

- **Asynchronisme** : les lectures audio, appels HTTP, commandes série et lectures audio sont orchestrés par `asyncio`.

- **Verrous locaux** : `DisplayService` et `MotionService` protègent les ressources qui ne supportent pas les accès concurrents.
- **Fallback wake word** : si Vosk n'est pas disponible, le système peut revenir à une détection par morceaux audio envoyés au STT backend.
- **Diagnostics explicites** : les composants absents sont loggués avec leur nom et leur message d'erreur.
- **Arrêt propre** : sur `SIGINT` ou `SIGTERM`, le robot affiche l'expression sommeil, arrête les moteurs, ferme le client HTTP et libère les ports série.

L'arrêt propre est essentiel pour éviter de laisser le robot dans un état incohérent, par exemple avec les moteurs actifs ou un port série bloqué.

VI.12 Tests et validation logicielle

La validation du Cycle 4 combine des tests automatisés et des essais réels sur le Raspberry Pi.

TABLE VI.2 – Tests de validation du logiciel Raspberry Pi

Type de test	Procédure	Critère OK
Configuration	Exécuter les tests <code>test_config.py</code> avec variables par défaut et overrides	Valeurs chargées sans erreur
Imports	Lancer <code>test_imports.py</code>	Tous les modules principaux importables
Hardware checks	Lancer <code>run_full_check()</code> sur le Pi	Rapport lisible par composant
Wake word	Prononcer le mot de réveil devant le ReSpeaker	Passage de PASSIVE à WAKE
VAD question	Poser une question puis se taire	WAV complet capturé jusqu’au silence
Backend action	Envoyer une question vocale au backend	ActionResult exploitable reçu
Nextion	Changer d’expression depuis le service display	Expression visible à l’écran
Motion	Envoyer une commande directionnelle	Arduino reçoit et exécute le mouvement
Shutdown	Interrompre le programme par Ctrl+C	Moteurs arrêtés et ports fermés

Les commandes minimales de validation logicielle sont :

```
pytest tests/
python -m app.main
```

Listing VI.2 – Commandes de validation du Cycle 4

VI.13 Limites et perspectives

Le logiciel embarqué actuel assure le lien principal entre l’utilisateur, le mouvement, l’affichage et le backend. Certaines améliorations restent toutefois possibles :

- renforcer la reconnaissance faciale côté Raspberry en appelant le service `/api/identify-face`

depuis un module caméra intégré ;

- ajouter un service `systemd` pour démarrer KODA automatiquement au boot ;
- enrichir la télémétrie pour suivre les erreurs audio, backend et mouvement pendant les démonstrations ;
- stocker localement quelques réponses de secours si le backend devient temporairement indisponible.

VI.14 Conclusion

Ce chapitre a présenté le logiciel embarqué Raspberry Pi qui donne vie au prototype KODA. Le programme `app.main` coordonne les services et adaptateurs nécessaires pour passer d’une plateforme matérielle assemblée à un compagnon capable d’écouter, de s’orienter, de parler, d’afficher des émotions et d’exécuter des mouvements.

Le choix d’une architecture asynchrone, séparée en services et adaptateurs, rend le système plus maintenable et plus robuste face aux contraintes du réel : matériel parfois absent, latence réseau, capture audio continue, ports série et arrêt propre. Le Cycle 4 complète ainsi le Cycle 3 en transformant le corps matériel en plateforme interactive connectée au cerveau n8n et au Distributeur de Services.

Points clés du Cycle 4

- Démarrage par `python -m app.main` et configuration par variables d’environnement
- Architecture Python asynchrone : services métier + adaptateurs matériels
- Wake word local avec Vosk, écoute VAD et pipeline `/api/audio/speech-to-action`
- Nextion via UART, Arduino via USB série, audio via Bluetooth ou jack
- Diagnostic matériel, mode dégradé, logs et arrêt propre

Le chapitre suivant, **Cycle 5**, sera consacré à l’application mobile cross-plateforme KodaMate, qui permet à l’utilisateur de configurer, contrôler et superviser son compagnon.

CHAPITRE VII

CYCLE 5 : APPLICATION MOBILE CROSS-PLATEFORME KODAMATE

VII.1 Introduction : pourquoi une application compagnon ?

KODA a été conçu comme un **compagnon de vie** : présence, dialogue, mémoire et personnalisation (chapitre I). Sur le plan matériel et cognitif, les cycles précédents ont donné au robot une voix, un corps et une capacité de réflexion. Il reste pourtant une attente essentielle côté humain : pouvoir **habiter la relation** avec son compagnon au-delà du salon — le configurer, le suivre, lui écrire, retrouver ce qui a été dit, même lorsque l'on n'est pas face au robot.

C'est le **besoin** que répond **KodaMate**. L'application n'est pas le « cerveau » de KODA : elle est le **prolongement relationnel** de l'écosystème, accessible depuis un téléphone ou un ordinateur. Elle permet à l'utilisateur de rester acteur de la vie de son compagnon : lui donner une identité, vérifier qu'il est disponible, échanger par écrit lorsque la voix n'est pas adaptée, et consulter la mémoire des conversations comme on relirait un carnet de bord partagé.

Rôle de KodaMate dans le projet :

- **Continuité** : maintenir le lien utilisateur-compagnon entre deux présences physiques du robot ;
- **Personnalisation** : traduire l'intention « mon KODA à moi » en paramètres concrets (nom, langue, caractère) ;
- **Supervision bienveillante** : savoir si le compagnon est actif ou au repos, sans confondre affichage local et réalité du système ;
- **Dialogue écrit** : offrir un canal complémentaire à la parole, aligné sur la même mémoire que le dialogue vocal ;
- **Confiance d'accès** : n'ouvrir les fonctions sensibles qu'après identification de l'utilisateur.

Ces besoins reprennent explicitement ceux formulés au chapitre II (*pilotage à distance, dialogue par message, personnalisation*). KodaMate en est la réponse concrète du Cycle 5.

Objectif du Cycle 5

Offrir une expérience mobile et desktop simple, continue et sécurisée, pour configurer, superviser et converser avec KODA — sans remplacer le robot ni dupliquer son intelligence, mais en prolongeant la relation déjà construite dans les cycles précédents.

La figure VII.1 résume les capacités attendues du point de vue de l'utilisateur ; la plupart supposent une authentification préalable, ce qui traduit l'exigence de confiance énoncée ci-dessus.

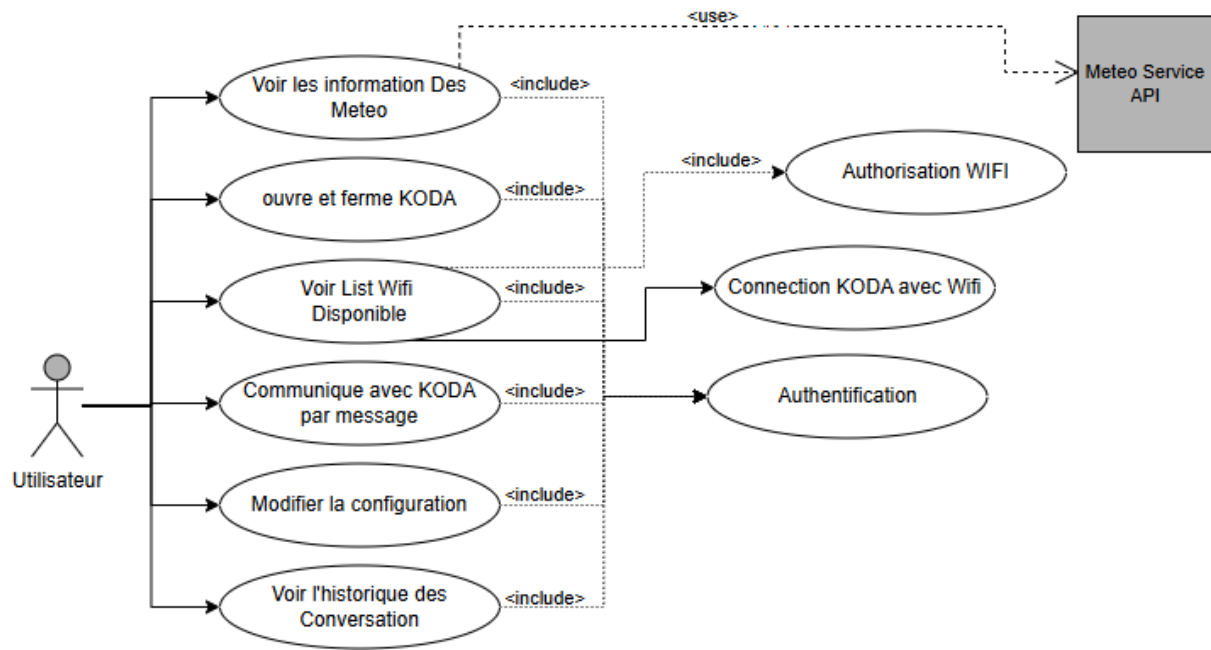


FIGURE VII.1 – Cycle 5 — cas d'utilisation de KodaMate (vue utilisateur)

VII.2 Place de KodaMate dans l'écosystème

Dans la vision globale du projet, KodaMate occupe la place du **interlocuteur à distance** : l'utilisateur exprime une intention (lire l'état, modifier un réglage, poser une question), et le système central — déjà présenté au chapitre IV — se charge de la réaliser en s'appuyant sur la mémoire et le raisonnement du chapitre III. Le robot physique reste le lieu de la présence ; l'application reste le lieu du **pilotage et du suivi**.

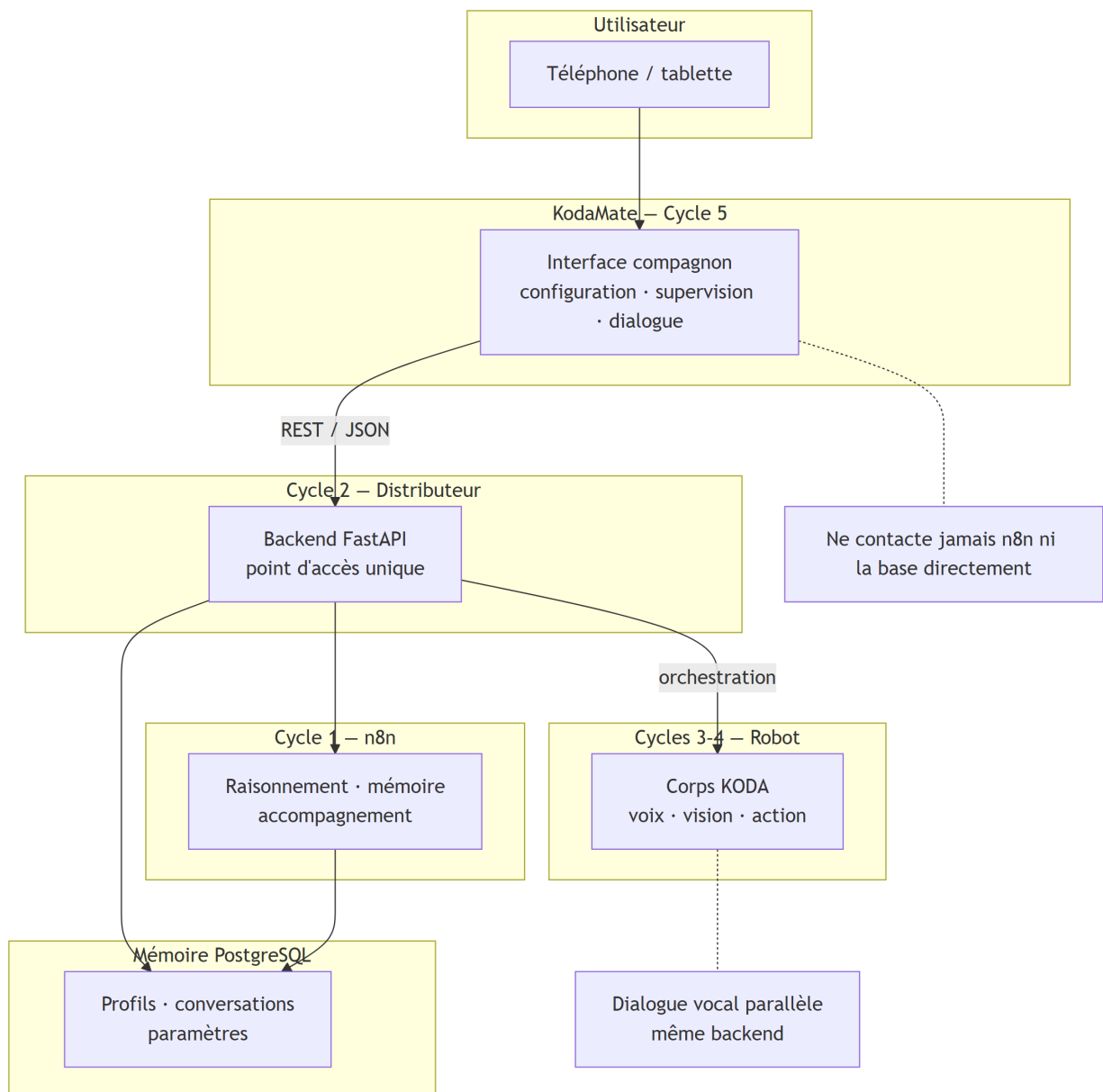


FIGURE VII.2 — Cycle 5 — place de KodaMate entre l'utilisateur, le Distributeur, la cognition n8n et le corps du robot

Cette répartition évite de fragmenter la relation : un même profil, une même histoire de conversations, que l'échange soit oral sur le robot ou écrit depuis le téléphone.

VII.3 Parcours et principes d'expérience

Avant d'aborder chaque écran, il est utile de décrire le **parcours vécu** par l'utilisateur. Celui-ci commence par une phase d'entrée (connexion, éventuelle première configuration), puis bascule vers un usage quotidien en quatre espaces : accueil, dialogue, historique et paramètres (figure VII.3).

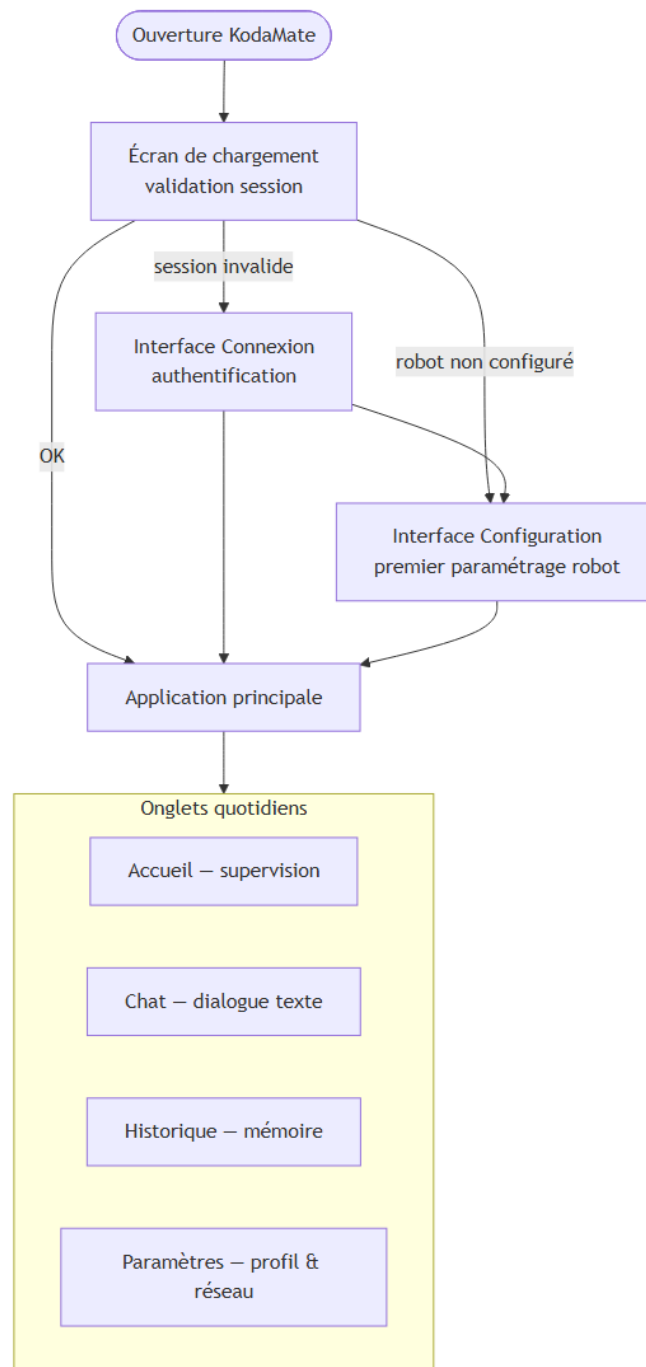
**FIGURE VII.3** – Cycle 5 — parcours utilisateur de KodaMate

TABLE VII.1 – Interfaces KodaMate et rôles utilisateur

Interface	Rôle pour l'utilisateur
Chargement	Accueillir l'utilisateur et orienter vers la bonne étape (connexion, configuration ou usage).
Connexion	Lier le compte utilisateur au Distributeur et mémoriser la session.
Configuration initiale	Personnaliser KODA (identité, langue, caractère) avant le premier usage.
Accueil	Superviser l'état du robot, la météo locale et la commande marche/arrêt.
Chat	Dialoguer par texte avec KODA, sur le même fond mémoriel que la voix.
Historique	Retrouver et filtrer les conversations passées.
Paramètres	Ajuster le profil, le réseau (simulation Wi-Fi) et quitter la session.

Chaque onglet de la phase quotidienne recharge ses informations lorsqu'il redevient visible, afin de refléter l'état réel du compagnon sans effort supplémentaire de l'utilisateur.

VII.4 Les interfaces utilisateur

Cette section décrit chaque écran selon le même fil : *rôle*, *fonctionnement*, puis *illustration* (capture d'écran lorsque le fichier est disponible dans public/).

VII.4.1 Interface de chargement

Rôle. C'est la première impression de KodaMate : un écran sobre avec le logo KODA, qui laisse le temps de vérifier si l'utilisateur peut entrer directement dans l'application ou doit se reconnecter.

Fonctionnement. Au lancement, l'application contrôle la session enregistrée localement, teste la disponibilité du serveur, puis interroge les paramètres du robot. Si la chaîne de vérification échoue, la session est effacée et l'utilisateur est renvoyé vers la connexion ; si le robot n'a jamais été configuré, vers l'écran de configuration ; sinon, vers les onglets principaux. L'adresse du serveur peut être saisie ou mémorisée ; en développement sur réseau local, elle est normalisée automatiquement (port, émulateur Android).

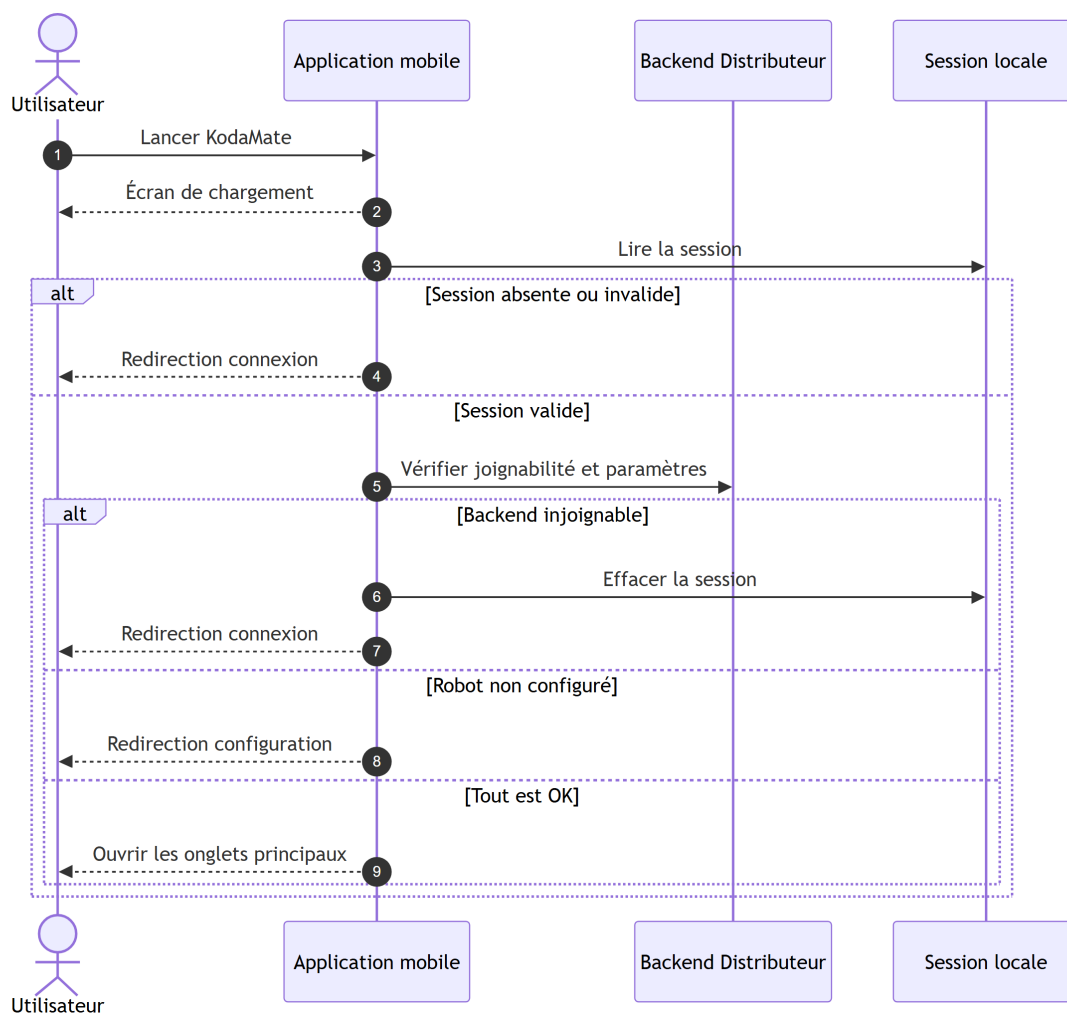


FIGURE VII.4 – Cycle 5 — séquence de démarrage et orientation de l'utilisateur

VII.4.2 Interface de connexion

Rôle. Établir le lien de confiance entre l'utilisateur et le Distributeur : sans connexion valide, aucune autre fonction n'est accessible.

Fonctionnement. L'utilisateur saisit l'adresse du Distributeur sur le réseau local (IP du poste de développement), peut *tester le serveur* avant de se connecter, puis s'authentifie par email et mot de passe. En cas d'échec réseau, un message explicite guide la vérification (backend actif, même Wi-Fi, adresse IP). Après connexion réussie, l'application mémorise les identifiants nécessaires aux appels suivants.



FIGURE VII.5 – Cycle 5 — connexion et test du serveur Distributeur

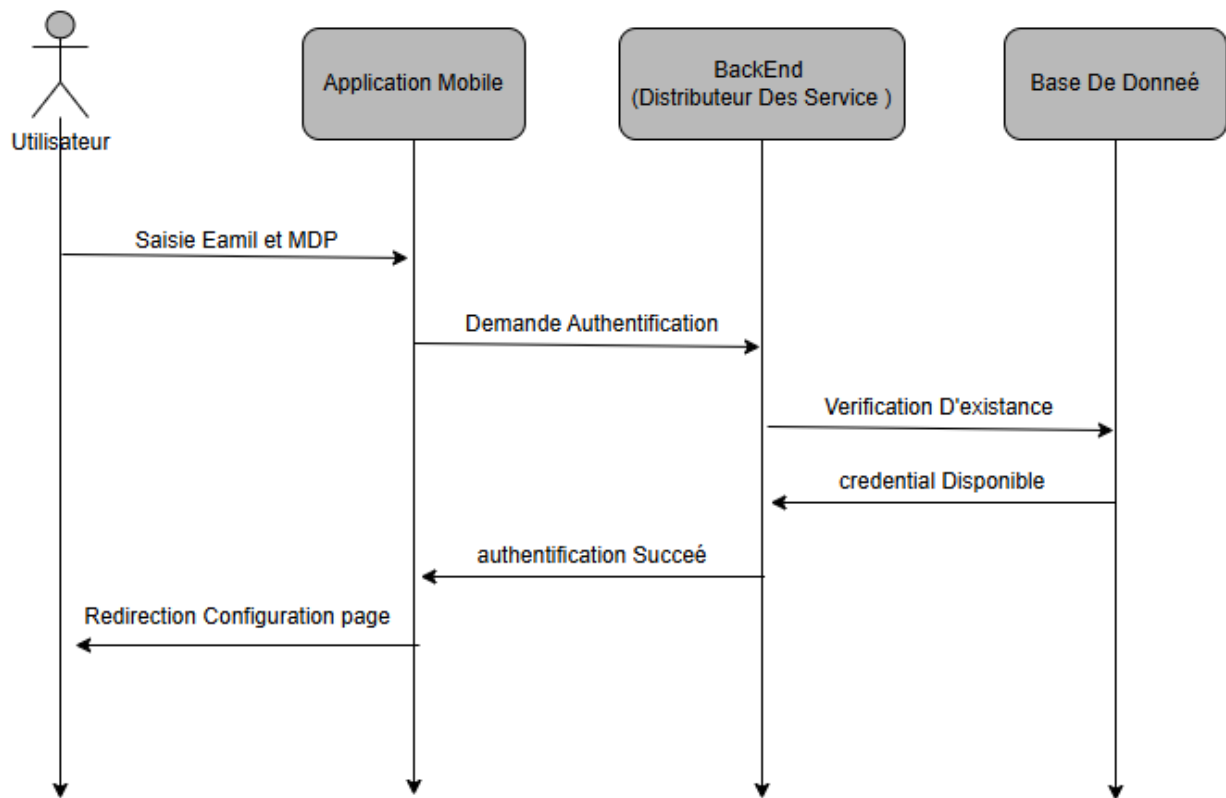


FIGURE VII.6 – Cycle 5 — séquence d’authentification utilisateur

VII.4.3 Interface de configuration du robot

Rôle. Transformer un compte technique en *compagnon nommé* : nom, langue, caractère, date de naissance symbolique, etc. Cette étape matérialise la personnalisation évoquée au Cycle 2 et prépare le dialogue n8n.

Fonctionnement. L’écran charge les paramètres existants depuis le backend, permet de les modifier, puis enregistre l’ensemble en une seule sauvegarde. Une fois validée, l’application marque le robot comme configuré et autorise l’accès aux onglets quotidiens. Les mêmes champs restent modifiables ultérieurement depuis l’écran Paramètres.

The figure displays two screenshots of a mobile application titled "Configuration du robot".

Left Screenshot (Initial Configuration):

- Header: "Configuration du robot" with a KODA logo.
- Text: "Renseignez les paramètres (table setting du backend)."
- Form fields:
 - "Nom du robot *": Input field containing "mohsen".
 - "Sexe (personnage)": Input field containing "male".
 - "Date": Input field containing "2026-04-12".
 - "Mot de passe (setting)": Input field with masked characters ".....".
 - "Caractère / personnalité": Input field containing "que, qui parle avec en ar-SA-HamedNeural".

Right Screenshot (Configuration with Values):

- Header: "Configuration du robot" with a KODA logo.
- Form fields (values entered):
 - Date: "2026-04-12"
 - Mot de passe (setting): "....."
 - Caractère / personnalité: "que, qui parle avec en ar-SA-HamedNeural"
 - Langue (locale BCP-47): "ar-SA-HamedNeural"
- Toggle: "Robot allumé (etat)" is turned on.
- Button: "Enregistrer et continuer" (highlighted in red).
- Footer: "PUT /api/settings"

FIGURE VII.7 – Cycle 5 — configuration initiale : identité du robot (gauche) et voix, langue et état d'allumage (droite)

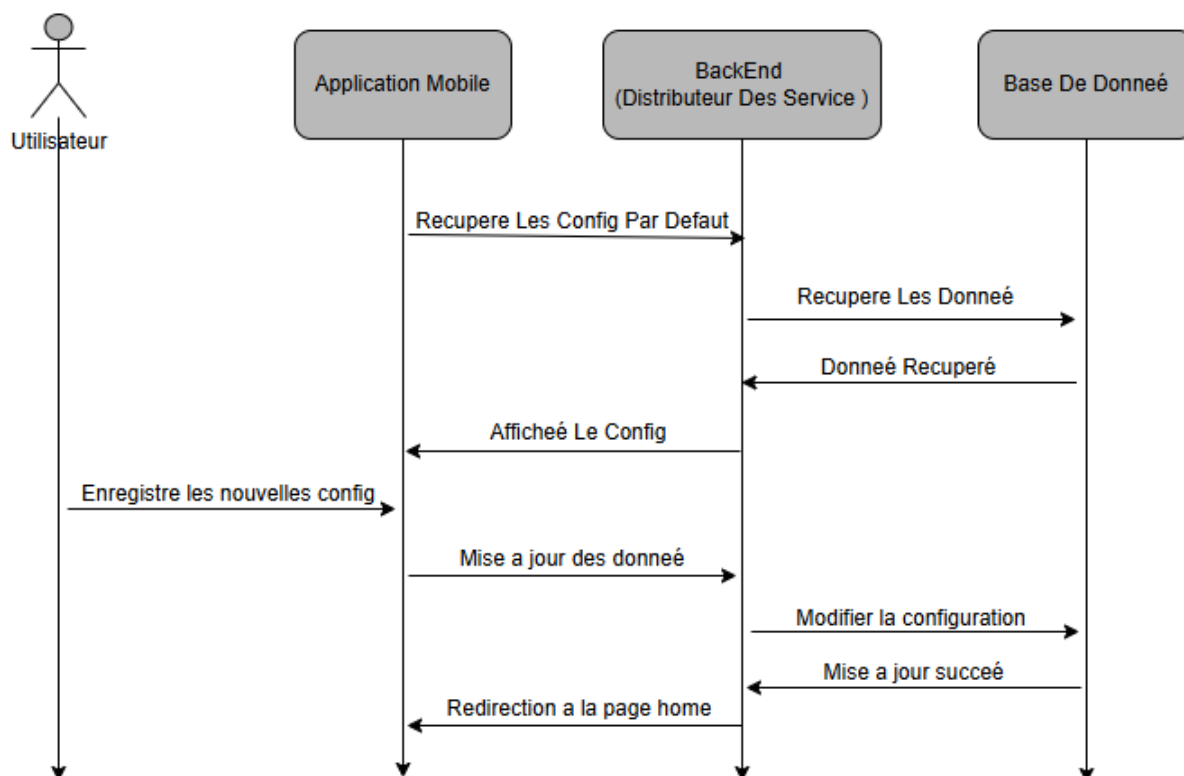


FIGURE VII.8 – Cycle 5 — séquence de lecture et mise à jour de la configuration

VII.4.4 Interface d'accueil (Home)

Rôle. Constituer le **tableau de bord** du compagnon : en un coup d'œil, l'utilisateur voit si KODA est joignable, dans quel état il se trouve, et peut agir sur l'alimentation logique (marche/arrêt). La météo d'Alger apporte un contexte ambiant léger, distinct du cœur métier mais utile pour l'expérience d'accueil.

Fonctionnement. L'écran combine deux sources : le Distributeur pour l'état réel du robot (lecture des paramètres, commandes d'allumage/extinction confirmées par relecture serveur) et l'API Open-Meteo pour la météo. En cas d'indisponibilité réseau, l'interface passe en mode hors ligne plutôt que d'afficher un état fictif. Le rafraîchissement intervient à chaque retour sur l'onglet.

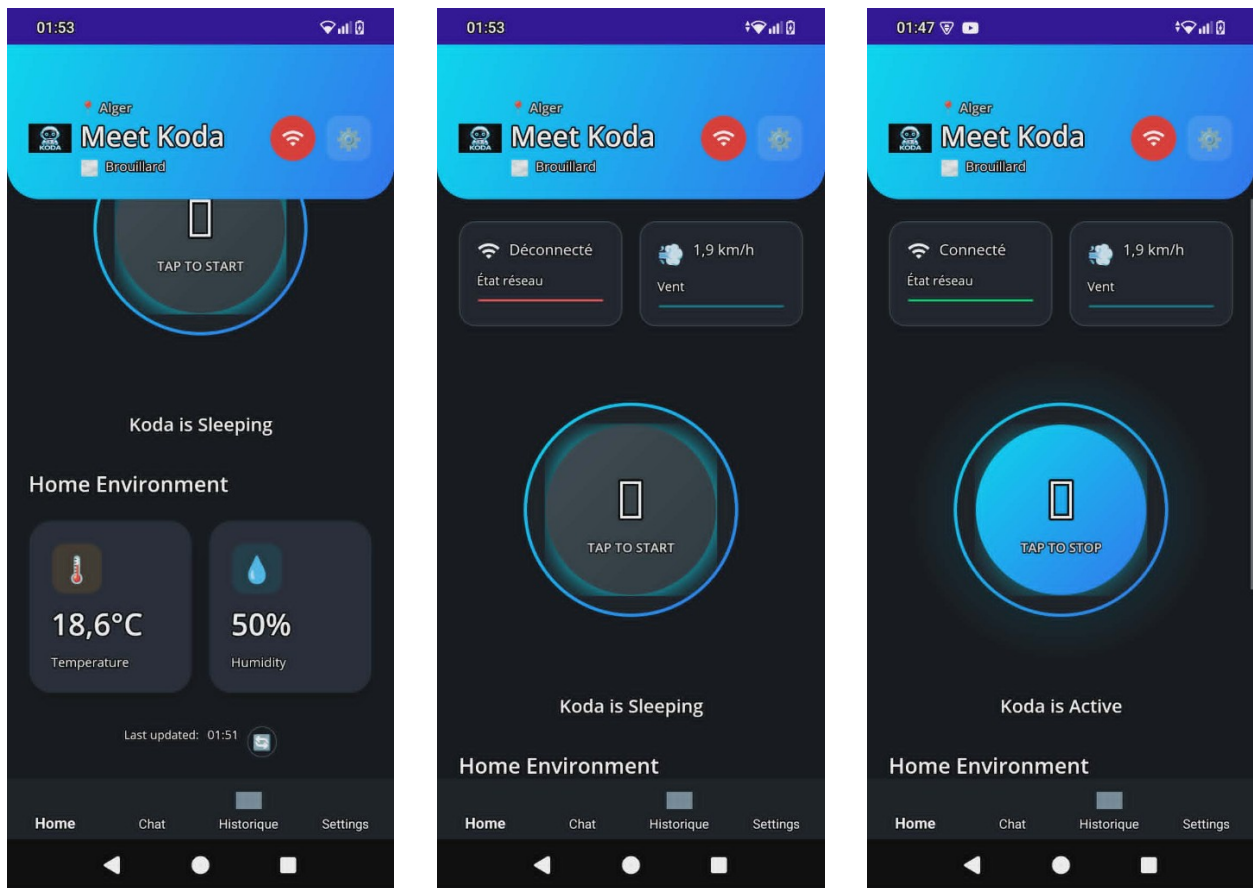


FIGURE VII.9 – Cycle 5 — accueil : veille et environnement (gauche), réseau déconnecté (centre), robot actif (droite)

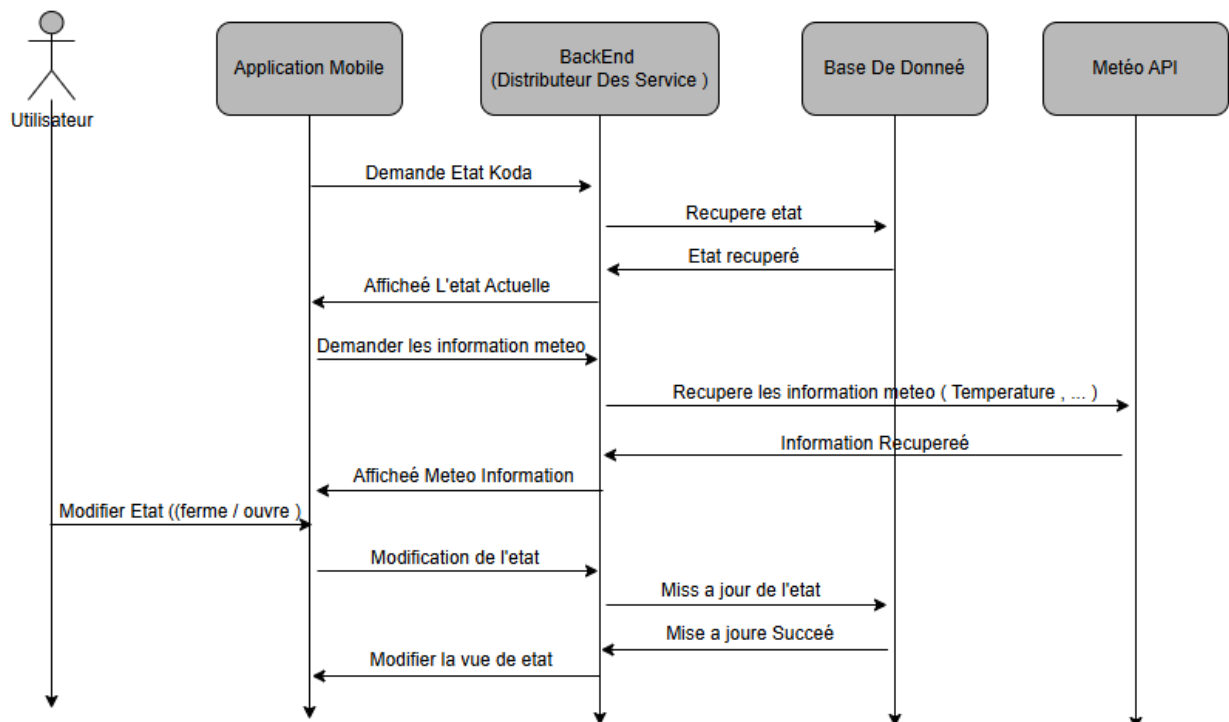


FIGURE VII.10 – Cycle 5 — séquence de supervision : état KODA, météo et marche/arrêt

VII.4.5 Interface de dialogue (Chat)

Rôle. Offrir un canal textuel avec KODA, complémentaire de la voix. L'utilisateur formule une question ; la réponse s'appuie sur le même raisonnement n8n que le Cycle 1, orchestré par le Distributeur.

Fonctionnement. Le message de l'utilisateur apparaît immédiatement dans la conversation ; l'application transmet la question au système central, qui déclenche l'analyse et la réflexion (chapitre III), enregistre l'échange en mémoire, puis renvoie une réponse formulée pour l'affichage. La capture photo prépare l'envoi multimodal ; l'intégration complète avec la reconnaissance faciale reste une évolution prévue.

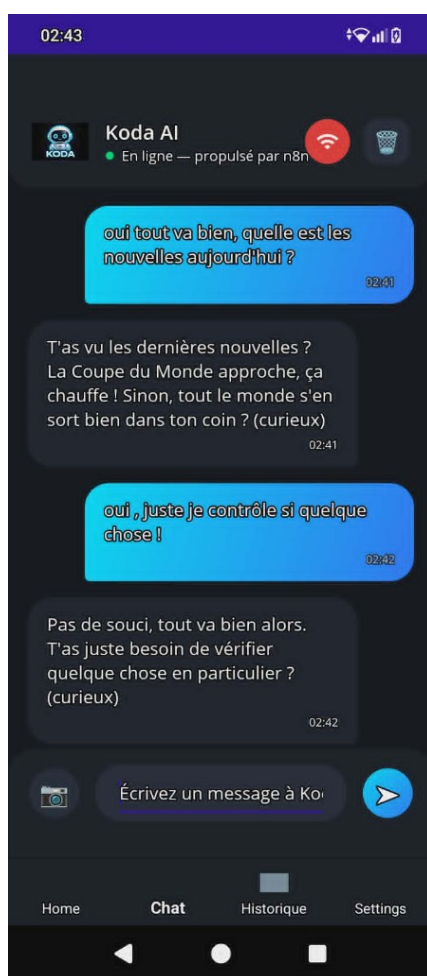


FIGURE VII.11 – Cycle 5 — dialogue texte avec KODA

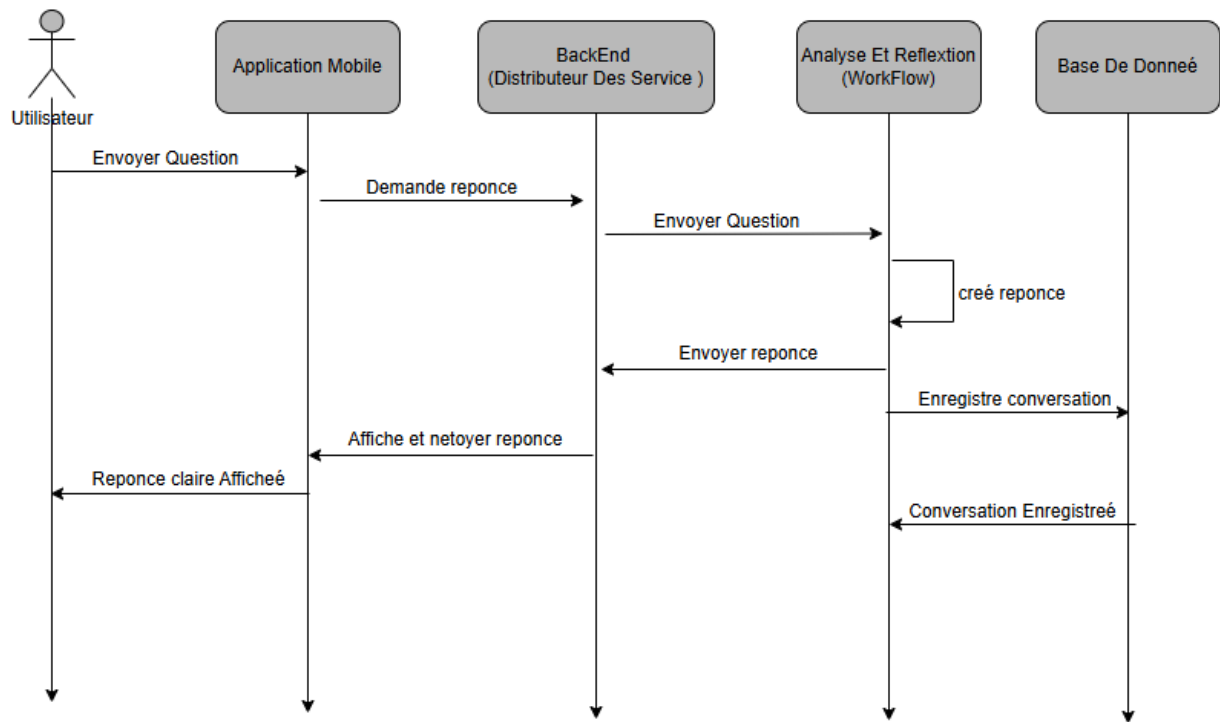


FIGURE VII.12 – Cycle 5 — séquence d’envoi d’une question et affichage de la réponse

VII.4.6 Interface d’historique

Rôle. Donner de la **continuité mémorielle** à l’usage : retrouver ce qui a été dit, filtrer par période, rechercher un mot-clé. Cette fonction s’aligne sur la mémoire PostgreSQL alimentée par le Cycle 1 et exposée par le Distributeur.

Fonctionnement. L’écran charge les dernières conversations depuis le backend, propose des filtres temporels (*aujourd’hui, hier, cette semaine*, date au choix), puis affine localement par recherche textuelle sur la question, la réponse et le type d’échange. Les résultats sont regroupés par jour pour faciliter la lecture.



FIGURE VII.13 – Cycle 5 — historique des conversations avec filtre et recherche

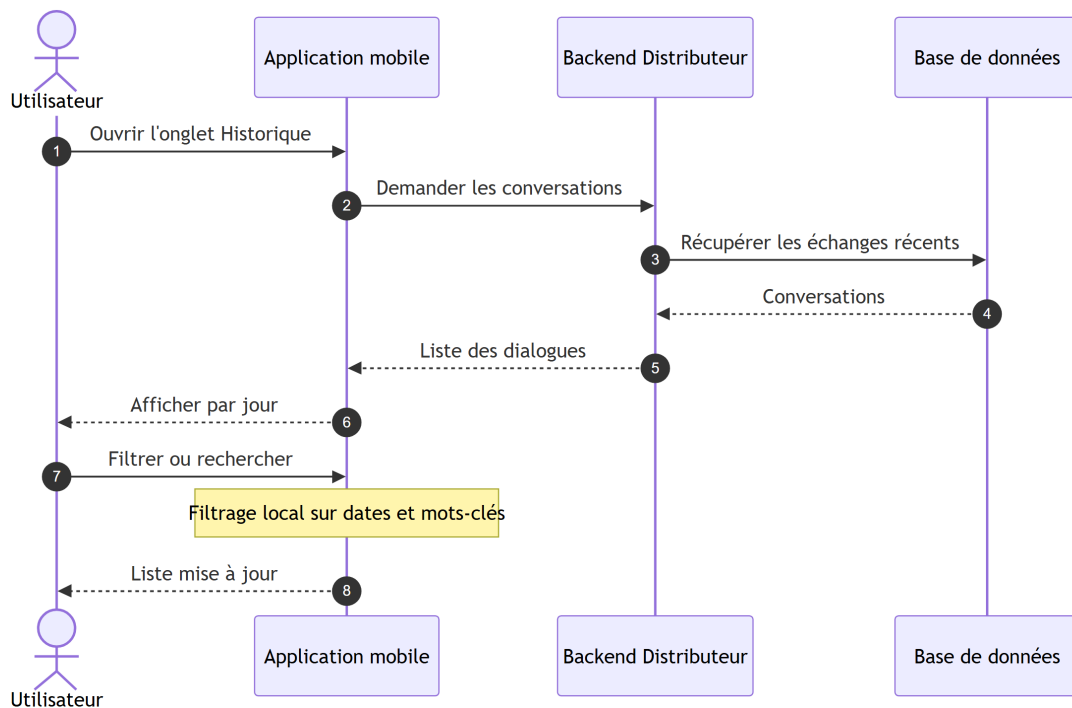


FIGURE VII.14 – Cycle 5 — séquence de consultation de l'historique

VII.4.7 Interface de paramètres

Rôle. Centraliser les réglages durables : profil du robot, préférences d’interface, réseau Wi-Fi (simulation ou scan Android), notifications de démonstration et déconnexion.

Fonctionnement. Les champs liés au robot sont synchronisés avec le backend comme lors de la configuration initiale. Le choix d’un SSID Wi-Fi est enregistré localement pour l’indicateur visuel ; sur Android, un scan peut lister les réseaux visibles après autorisation de localisation. Certaines options (voix, notifications avancées) sont visibles dans l’interface mais encore partiellement déconnectées du backend. La déconnexion efface la session et le SSID mémorisé pour repartir d’un état neutre.

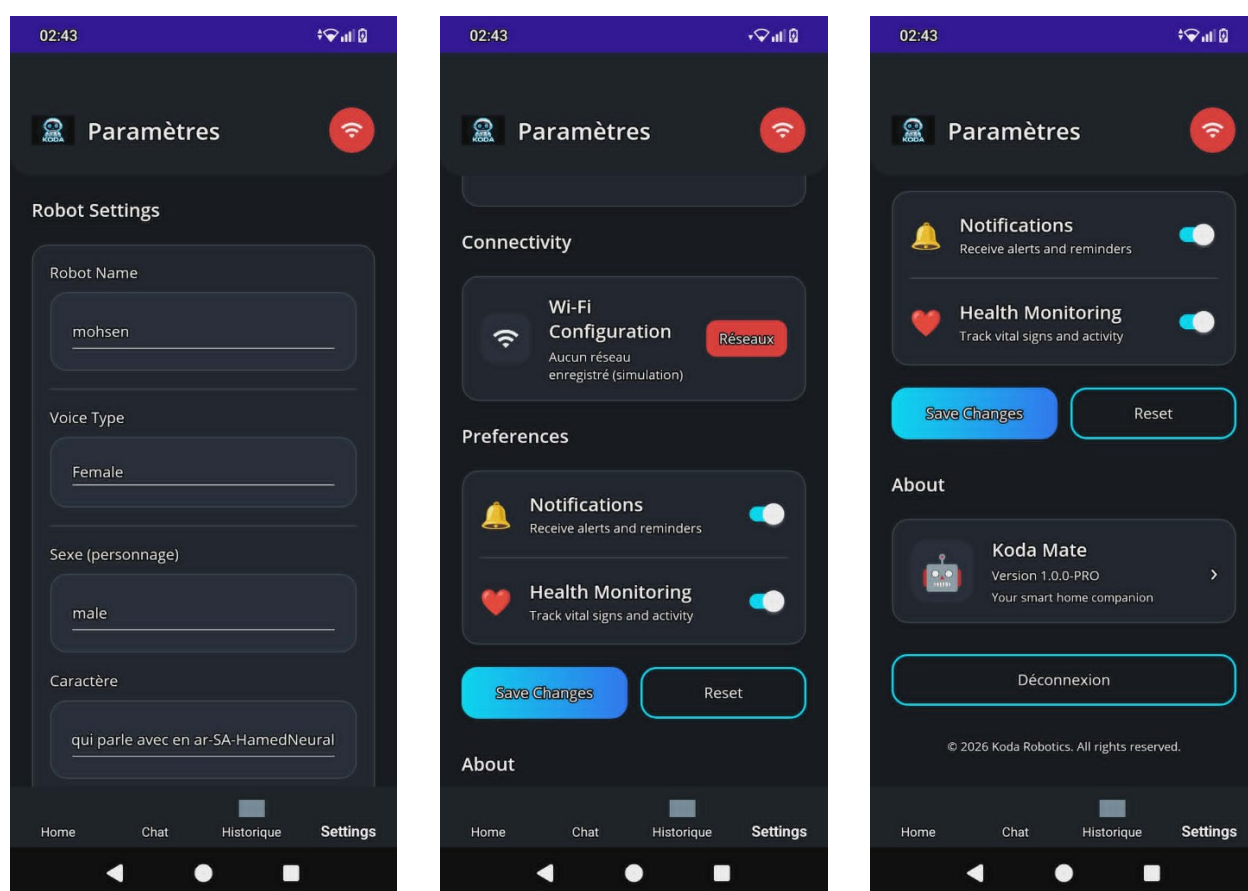


FIGURE VII.15 – Cycle 5 — paramètres : profil robot (gauche), Wi-Fi et préférences (centre), informations et déconnexion (droite)

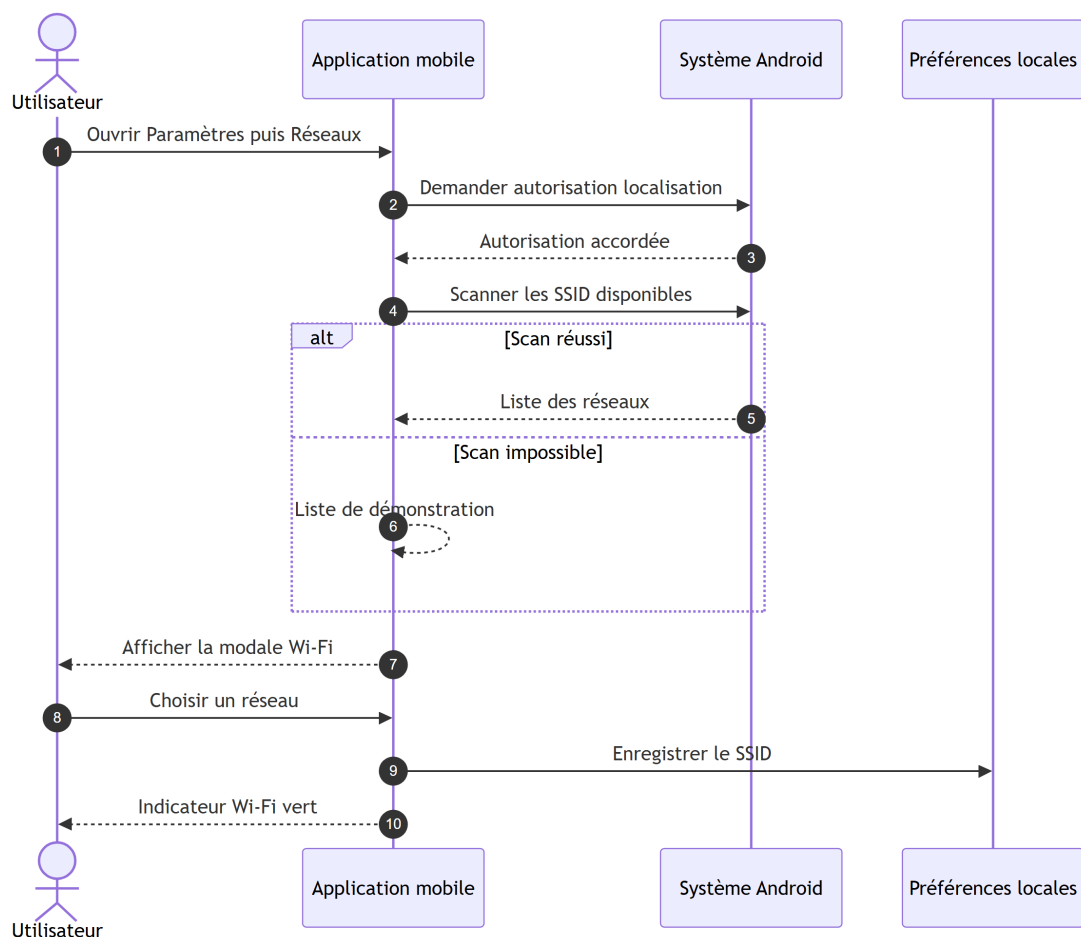


FIGURE VII.16 – Cycle 5 — séquence de sélection d’un réseau Wi-Fi (simulation locale)

VII.5 Organisation technique du projet

Au-delà des écrans, le code respecte une séparation **Views / ViewModels / Services / Models**. Le bootstrap (`App.xaml.cs`) choisit la page initiale ; `AppShell.xaml` porte la navigation par onglets ; `DistributeurApiService` regroupe les appels REST ; `SessionService` et les préférences MAUI conservent un minimum local (session, URL serveur, indicateur de configuration, SSID Wi-Fi).

TABLE VII.2 – Correspondance services principaux et besoins utilisateur

Composant	Usage dans KodaMate
<code>DistributeurApiService</code>	Authentification, paramètres, chat, historique, marche/arrêt.
<code>SessionService</code>	Persistance de la session et du statut « robot configuré ».
<code>WeatherService</code>	Météo d’accueil via Open-Meteo (Alger).
<code>WifiNetworkService</code>	Scan ou liste de démonstration des réseaux.
<code>MockFirebaseService</code>	Données de démonstration pour les notifications.

Sur Android, le manifeste autorise réseau, Wi-Fi, localisation et vibration ; le trafic HTTP clair facilite les tests en LAN et devra être remplacé par HTTPS en déploiement public.

VII.6 Fiabilité et session

La robustesse repose sur des règles simples mais strictes :

- aucun accès aux onglets sans session valide vérifiée au démarrage ;
- effacement automatique de la session si le backend ne répond plus ;
- délais d'attente sur les appels HTTP (backend et météo) ;
- messages d'erreur explicites sur chaque écran ;
- confirmation serveur après une commande marche/arrêt ;
- normalisation de l'URL en environnement de développement.

Les données locales restent volontairement limitées : elles ne remplacent pas la base PostgreSQL, elles permettent seulement de reprendre l'expérience entre deux ouvertures de l'application.

VII.7 Tests et validation

La validation du Cycle 5 porte à la fois sur la compilation multi-plateforme et sur les scénarios utilisateur. Les tests manuels restent importants car une partie du comportement dépend du réseau local, de la plateforme d'exécution et de la disponibilité du backend.

TABLE VII.3 – Tests de validation par interface

Interface	Procédure	Critère OK
Projet MAUI	Compiler sur la plateforme disponible	Build sans erreur
Connexion	Tester l'URL LAN puis se connecter	Backend joignable, session créée
Chargement	Redémarrer avec session existante	Orientation correcte (config. ou onglets)
Configuration	Modifier le profil puis enregistrer	Paramètres visibles après rechargement
Accueil	Ouvrir l'onglet Home	État robot et météo affichés
Accueil	Action marche/arrêt	État confirmé après rafraîchissement
Chat	Envoyer un message à KODA	Réponse affichée ou erreur lisible
Historique	Filtrer et rechercher	Liste cohérente par jour
Paramètres	Choisir un SSID Wi-Fi	Indicateur vert, SSID mémorisé
Paramètres	Déconnexion	Retour à l'écran de connexion

Les commandes minimales de validation côté développement sont :

```
dotnet restore KodaMate/KodaMate/KodaMate.csproj
dotnet build KodaMate/KodaMate/KodaMate.csproj
```

Listing VII.1 – Commandes de validation du Cycle 5

VII.8 Limites et perspectives

L'application KodaMate fournit déjà une base cross-plateforme fonctionnelle. Certaines limites restent toutefois identifiées dans le code actuel :

- remplacer `MockFirebaseService` par un service temps réel pour les notifications ;
- harmoniser définitivement le payload du chat avec le backend si le workflow attend un champ `message` au lieu de `question` ;
- ajouter l'envoi réel de l'image capturée vers un endpoint multimodal ou vers `/api/identify-face` ;
- passer les tests réseau locaux en HTTPS pour une version distribuable ;
- ajouter des tests unitaires sur les `ViewModels` et des tests d'intégration sur les services REST ;
- remplacer l'adresse IP LAN fixe par une découverte ou une configuration plus guidée.

VII.9 Conclusion

Le Cycle 5 achève la boucle utilisateur de l'écosystème KODA : après la cognition (Cycle 1), l'orchestration (Cycle 2) et l'incarnation physique (Cycles 3–4), **KodaMate** offre une présence numérique quotidienne. Chaque interface répond à un besoin précis — entrer, configurer, superviser, dialoguer, se souvenir, régler — sans court-circuiter le Distributeur.

Structurée par écran et fondée sur une base MAUI unique, l'application traduit en expérience tangible la continuité promise par les chapitres précédents : un même compagnon, une même mémoire, plusieurs modalités d'interaction.

Points clés du Cycle 5

- **Compagnon utilisateur** : canal mobile/desktop complémentaire de la voix
- **Sept interfaces** : chargement, connexion, configuration, accueil, chat, historique, paramètres
- **Passage unique** par le Distributeur ; n8n et PostgreSQL restent centralisés
- **Supervision et dialogue** : état du robot, marche/arrêt, chat texte, mémoire consultable
- **Évolutions identifiées** : notifications temps réel, multimodal photo, HTTPS en production

_____ CHAPITRE VIII _____

CONCLUSION GÉNÉRALE ET SYNTHÈSE DU PROJET

VIII.1 Du constat initial à la solution réalisée

Ce rapport a documenté la conception et la réalisation de **KODA**, un compagnon robotique intelligent visant à dépasser les limites des assistants génériques : absence de personnalité durable, froideur des réponses, dépendance à des clouds fermés et faible intégration entre la voix, le corps du robot et le pilotage utilisateur (chapitre I).

La réponse apportée repose sur une **architecture distribuée** articulée autour de cinq briques complémentaires, développées selon un **modèle en spirale** (chapitre II) : à chaque tour, une partie du système est précisée, construite, testée et raccordée au reste, jusqu'à former un écosystème cohérent. Le présent chapitre en donne une **lecture d'ensemble**, pour que le lecteur puisse comprendre, en un seul passage, le rôle de chaque cycle et la manière dont les composants coopèrent.

VIII.2 Vue d'ensemble : les cinq cycles et leurs rôles

Le tableau VIII.1 résume la contribution de chaque cycle. En dessous, chaque pilier est rappelé dans le langage du besoin utilisateur, puis dans celui de la réalisation technique.

TABLE VIII.1 – Synthèse des cycles KODA — besoin couvert et composant principal

N°	Cycle	Besoin adressé	Composant
0	Cadre & préparation (ch. I–II)	Cadrer le projet, formaliser les exigences et la méthode	Besoins, architecture cible, environnements
1	Analyseur et réflexion	Comprendre, mémoriser et prendre l'initiative de parler	Workflows n8n (3 webhooks)
2	Distributeur de services	Unifier les accès données, voix, mobile et robot	Backend FastAPI (pivot central)
3	Embarqué matériel	Donner un corps mobile, expressif et sensoriel au compagnon	Raspberry Pi + Arduino + capteurs/actionneurs
4	Embarqué logiciel	Écouter, parler, afficher, bouger et dialoguer avec le backend	Logiciel Python sur Raspberry Pi
5	Application KodaMate	Configurer, superviser et converser à distance	Application .NET MAUI

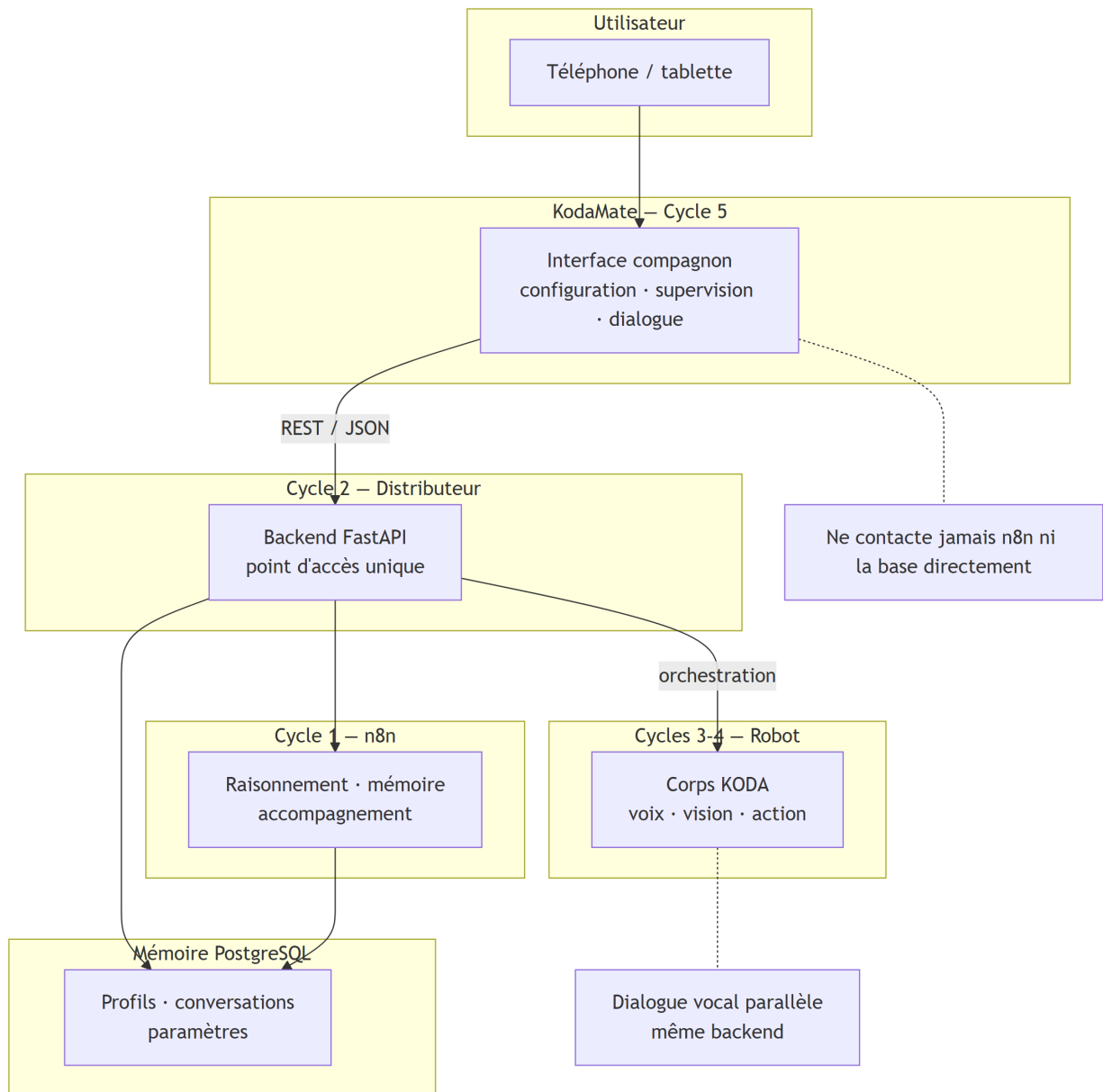


FIGURE VIII.1 – Vue synthétique de l'écosystème KODA au terme du projet

VIII.3 Les cinq piliers du système KODA

VIII.3.1 Le cerveau conversationnel (Cycle 1)

Le **Cycle 1** répond à la question : *comment KODA réfléchit-il et se souvient-il ?* Trois workflows n8n structurent la cognition :

- **Webhook 1** : traitement des questions en temps réel (classification sémantique, confrontation à l'historique, réponse contextualisée, notes) ;
- **Webhook 2** : consolidation mémorielle journalière (synthèses d'accompagnement) ;
- **Webhook 3** : spontanéité — KODA peut *prendre la parole* en s'appuyant sur la mémoire consolidée.

Ce cycle incarne le passage d'un assistant qui « répond » à un compagnon qui « réfléchit », en s'appuyant sur PostgreSQL et des services d'IA (OpenRouter, SerpAPI, etc.). Sans lui, le robot n'aurait ni personnalité évolutive ni continuité dans la relation.

VIII.3.2 Le système nerveux central (Cycle 2)

Le **Cycle 2** répond à la question : *comment tout le monde communique-t-il sans se perdre ?* Le **Distributeur FastAPI** est le **seul pivot** entre le robot, l'application mobile, la base de données, n8n et les services cloud (Azure pour la voix et la vision, Supabase/PostgreSQL pour la mémoire). Il expose un catalogue d'endpoints REST et WebSocket, gère l'authentification, la reconnaissance faciale, le pipeline vocal et le déclenchement des workflows.

À **retenir** : ni le téléphone ni le Raspberry Pi n'appellent n8n directement ; ils passent par le Distributeur, ce qui garantit cohérence des données, sécurité et évolutivité.

VIII.3.3 Le corps du compagnon (Cycles 3 et 4)

Les **Cycles 3 et 4** répondent à la question : *comment KODA est-il présent physiquement ?*

Le **Cycle 3** a posé la plateforme matérielle : architecture biprocesseur (Raspberry Pi pour l'intelligence, Arduino pour le temps réel moteur), mobilité, capture audio/vidéo, écran Nextion pour l'expression émotionnelle, alimentation et tests d'intégration mécanique.

Le **Cycle 4** a animé ce corps par un logiciel Python asynchrone : mot de réveil (Vosk), écoute par détection d'activité vocale, envoi des requêtes au backend, synthèse et lecture audio, commandes de mouvement vers l'Arduino, affichage des émotions sur Nextion, diagnostics matériels et arrêt propre. C'est le cycle qui transforme l'assemblage électronique en **compagnon interactif sur site**.

VIII.3.4 La relation à distance (Cycle 5)

Le **Cycle 5** répond à la question : *comment l'utilisateur habite-t-il sa relation avec KODA au quotidien, même loin du robot ?* L'application **KodaMate** (.NET MAUI) offre sept espaces fonctionnels (chargement, connexion, configuration, accueil, chat, historique, paramètres). Elle prolonge les besoins du chapitre II — pilotage à distance, dialogue par message, personnalisation — sans dupliquer l'intelligence du Cycle 1.

VIII.4 Comment lire l'ensemble : un scénario type

Pour **comprendre le rapport dans sa globalité**, imaginez un utilisateur qui revient chez lui le soir :

1. Sur son téléphone, il ouvre **KodaMate**, consulte l'**accueil** (état du robot, météo) et envoie un

message dans le **chat**.

2. **KodaMate** transmet la demande au **Distributeur**, qui enrichit le contexte depuis **PostgreSQL** et déclenche le **workflow n8n**.
3. La réponse est enregistrée en mémoire et renvoyée à l'application ; elle sera visible plus tard dans l'**historique**.
4. Sur place, le **Raspberry Pi** détecte le mot de réveil, capture la voix, obtient la même logique via le backend, fait parler KODA, affiche une émotion sur **Nextion** et peut actionner les **moteurs** via **Arduino**.

Ce scénario montre que KODA n'est pas une juxtaposition de modules, mais une **chaîne de présence** : cognition centralisée, orchestration unique, corps local, lien mobile — chaque cycle couvre un maillon, aucun ne le remplace.

VIII.5 Apports du projet

Synthèse — ce qu'il faut retenir du rapport

1. **Problème** : concevoir un compagnon personnalisé, mémoriel et supervisable, au-delà des assistants génériques.
2. **Méthode** : développement en spirale, six cycles thématiques, intégration incrémentale et tests à chaque tour.
3. **Cognition (C1)** : trois webhooks n8n — réponse, mémoire, proactivité.
4. **Orchestration (C2)** : Distributeur FastAPI = point d'accès unique (robot, mobile, BDD, n8n, cloud).
5. **Incarnation (C3–C4)** : matériel biprocesseur + logiciel embarqué vocal et expressif.
6. **Relation utilisateur (C5)** : KodaMate pour configurer, superviser et dialoguer à distance.
7. **Fil rouge** : une seule mémoire, une seule personnalité, plusieurs modalités d'interaction (voix, texte, présence physique).

Sur le plan **ingénierie**, le projet démontre la faisabilité d'un système robotique distribué à coût maîtrisé (Raspberry Pi, APIs ouvertes, workflows modifiables) tout en respectant une séparation claire des responsabilités. Sur le plan **humain**, il explore une relation d'accompagnement où l'utilisateur reste maître de la configuration de son compagnon — nom, langue, caractère — et peut suivre son activité sans expertise technique.

VIII.6 Limites et enseignements

Comme tout prototype de fin d'études, KODA présente des limites assumées, déjà identifiées dans les chapitres de cycles :

- **Environnement de démonstration** : tests souvent réalisés en réseau local (HTTP, adresse IP manuelle dans KodaMate), ce qui diffère d'un déploiement grand public sécurisé (HTTPS, découverte automatique des services).
- **Maturité mobile** : certaines fonctions (notifications temps réel, envoi photo vers la reconnaissance faciale) restent partielles ou simulées.
- **Robustesse embarquée** : dépendance au backend pour la cognition ; modes dégradés limités en cas de coupure prolongée.
- **Validation globale** : le **Cycle 6** (tests et validation bout-en-bout formalisés) est présent dans la méthodologie du chapitre II mais reste à renforcer par une campagne de recette utilisateur plus systématique.

Ces limites ne remettent pas en cause la **cohérence architecturale** du projet ; elles définissent une feuille de route réaliste pour une industrialisation future.

VIII.7 Perspectives

Les évolutions prioritaires découlent naturellement de la synthèse ci-dessus :

- **Production** : HTTPS, durcissement sécurité (JWT, rate limiting), déploiement conteneurisé du backend et observabilité (métriques, traces n8n).
- **Expérience utilisateur** : notifications push depuis le Distributeur vers KodaMate, multimodalité complète (photo → identification faciale), parcours de configuration guidé sans saisie d'IP.
- **Compagnon physique** : démarrage automatique au boot du Raspberry, télémétrie embarquée, réponses de secours hors ligne.
- **Cognition** : affinage des prompts n8n, évaluation qualitative des synthèses mémorielles, tests A/B sur la spontanéité (Webhook 3).

VIII.8 Conclusion finale

Ce rapport de stage a présenté **KODA** dans sa dimension complète : une ambition (compagnon de vie intelligent et personnalisé), une méthode (spirale et cycles thématiques), une architecture (n8n, Distributeur, embarqué, mobile, PostgreSQL) et une réalisation incrémentale validée par

prototypes, tests et captures d'interface.

En relisant les chapitres dans l'ordre — du cadre général (I) à la préparation (II), puis des Cycles 1 à 5 — le lecteur suit le cheminement d'une idée à un système opérationnel. **Le Cycle 1** pense et mémorise ; **le Cycle 2** orchestre ; **les Cycles 3 et 4** incarnent ; **le Cycle 5** relie l'utilisateur à son compagnon au quotidien. Ensemble, ils composent une réponse structurée à la problématique initiale : offrir un accompagnement plus humain, plus personnel et plus maîtrisé que les assistants du marché, tout en gardant l'utilisateur au centre de la relation avec son robot.

Le travail réalisé constitue une **base solide** pour poursuivre le projet vers une version démontrable en conditions réelles ; les cycles documentés fournissent à la fois une mémoire technique pour l'équipe et une carte lisible pour toute personne souhaitant comprendre, en une lecture, l'ensemble de l'écosystème KODA.